

sesion-de-clase-01/README.md

Proyecto de Minería Web - Sesión de clase 01

Este proyecto corresponde al **primer ejercicio del curso de Minería Web**. En este proyecto se realiza un **web scraping inicial** con Python y BeautifulSoup, usando como fuente de datos la aplicación “**Mi Tienda Virtual**” (NextJS) del mismo repositorio.

Estructura del proyecto

- `/tienda_scraper/` → Paquete Python con la lógica de scraping (`config` , `scraper` , `parser` , `writer`).
 - `/scripts/` → Un script por ejercicio.
 - `/data/` → Archivos CSV generados.
 - `requirements.txt` → Dependencias necesarias para el proyecto.
 - `README.md` → Este archivo con la documentación del proyecto.
-

Instalación de dependencias

Antes de ejecutar el proyecto, asegúrate de tener **Python 3.8+** instalado. Luego instala las dependencias usando:

```
pip install -r requirements.txt
```

Paso previo: levantar “Mi Tienda Virtual”

Los ejercicios de esta sesión hacen scraping sobre la app `tienda-virtual` corriendo en `http://localhost:3000` . En una terminal aparte, dentro de la carpeta `tienda-virtual` :

```
npm run dev
```

Deja ese servidor corriendo mientras ejecutas los scripts de Python de esta carpeta.

Ejercicios propuestos

Ejercicio 1 (`scripts/ejercicio1_productos.py`): extrae código, nombre y precio de cada producto listado en `/productos` .

Ejercicio 2 (`scripts/ejercicio2_clientes.py`): extrae nombre, apellidos y ciudad de cada cliente listado en `/clientes` .

Ejercicio 3 (`scripts/ejercicio3_resenas_entrega.py`): extrae, de una orden en `/ordenes/{id}` , las reseñas de entrega: autor, producto, calificación y comentario. El ID de la orden se configura en `tienda_scraper/config.py` (`ORDEN_ID`); si esa orden no tiene reseñas, revisa la sección Órdenes de la app y prueba con otro ID.

Cada script imprime los resultados en consola y los guarda como CSV en `data/` .

El scraping en profundidad de todo el sitio se abordará en la siguiente clase; esta actividad busca una primera práctica guiada con datos reales.

s01_scripts

Sesión 01 – scripts de scraping (tienda_scraper) y su salida

```
===== tienda_scraper/config.py =====
```

```
BASE_URL = "http://localhost:3000"
```

```
URL_PRODUCTOS = f"{BASE_URL}/productos"
```

```
URL_CLIENTES = f"{BASE_URL}/clientes"
```

```
ORDEN_ID = 1
```

```
URL_ORDEN = f"{BASE_URL}/ordenes/{ORDEN_ID}"
```

```
OUTPUT_PRODUCTOS = "productos.csv"
```

```
OUTPUT_CLIENTES = "clientes.csv"
```

```
OUTPUT_RESENAS_ENTREGA = "resenas_entrega.csv"
```

```
===== tienda_scraper/scrapper.py =====
```

```
import requests
from bs4 import BeautifulSoup
```

```
def obtener_soup(url):
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        return BeautifulSoup(response.text, "html.parser")
    except requests.RequestException as e:
        print(f"Hubo un error al obtener la pagina {url}: {e}")
        print("Verifica que la app 'Mi Tienda Virtual' este corriendo (npm run dev).")
        return None
```

```
===== tienda_scraper/parser.py =====
```

```
from bs4 import BeautifulSoup
```

```
def parse_productos(soup: BeautifulSoup):
    """Ejercicio 1: codigo, nombre y precio de cada producto listado en /productos."""
    productos = []

    for fila in soup.select("table tbody tr"):
        codigo_tag = fila.select_one("td:nth-child(1)")
        nombre_tag = fila.select_one("td:nth-child(2)")
        precio_tag = fila.select_one("td:nth-child(4)")

        productos.append({
            "codigo": codigo_tag.get_text(strip=True) if codigo_tag else None,
            "nombre": nombre_tag.get_text(strip=True) if nombre_tag else None,
            "precio": precio_tag.get_text(strip=True) if precio_tag else None,
        })

    return productos

def parse_clientes(soup: BeautifulSoup):
    """Ejercicio 2: nombre, apellidos y ciudad de cada cliente listado en /clientes."""
    clientes = []

    for fila in soup.select("table tbody tr"):
        nombre_tag = fila.select_one("td:nth-child(1)")
        apellidos_tag = fila.select_one("td:nth-child(2)")
        ciudad_tag = fila.select_one("td:nth-child(5)")

        clientes.append({
            "nombre": nombre_tag.get_text(strip=True) if nombre_tag else None,
            "apellidos": apellidos_tag.get_text(strip=True) if apellidos_tag else None,
            "ciudad": ciudad_tag.get_text(strip=True) if ciudad_tag else None,
        })

    return clientes

def parse_resenas_entrega(soup: BeautifulSoup):
    """Ejercicio 3: autor, producto, calificacion y comentario de las resenas
```

```

de entrega de una orden (/ordenes/{id})."""
resenas = []

# Cada resena de entrega es un <li itemprop="review" data-calificacion="...">
for resena in soup.select('li[itemprop="review"]'):
    autor_tag = resena.select_one('span[itemprop="name"]')
    producto_tag = resena.select_one("p span.font-normal")
    comentario_tag = resena.select_one('p[itemprop="reviewBody"]')

    producto = None
    if producto_tag:
        # el texto viene como " - Nombre del producto"
        producto = producto_tag.get_text(strip=True).lstrip("-").strip()

    resenas.append({
        "autor": autor_tag.get_text(strip=True) if autor_tag else None,
        "producto": producto,
        "calificacion": resena.get("data-calificacion"),
        "comentario": comentario_tag.get_text(strip=True) if comentario_tag else None,
    })

return resenas

===== tienda_scraper/writer.py =====

import os
import csv

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA_DIR = os.path.join(BASE_DIR, "data")
os.makedirs(DATA_DIR, exist_ok=True)

def write_to_csv(data, filename):
    if not data:
        print("No hay datos disponibles.")
        return

    fieldnames = data[0].keys()
    output_file = os.path.join(DATA_DIR, filename)
    with open(output_file, mode="w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(data)

    print(f"Datos guardados en {output_file}")

===== scripts/ejercicio1_productos.py =====

from tienda_scraper.scrapers import obtener_soup
from tienda_scraper.parsers import parse_productos
from tienda_scraper.writers import write_to_csv
from tienda_scraper.config import URL_PRODUCTOS, OUTPUT_PRODUCTOS

if __name__ == "__main__":
    soup = obtener_soup(URL_PRODUCTOS)

    if soup:
        productos = parse_productos(soup)

        for producto in productos:
            print(producto["codigo"], producto["nombre"], producto["precio"])

        write_to_csv(productos, OUTPUT_PRODUCTOS)

    print("El scraping de productos ha finalizado.")

===== scripts/ejercicio2_clientes.py =====

from tienda_scraper.scrapers import obtener_soup
from tienda_scraper.parsers import parse_clientes
from tienda_scraper.writers import write_to_csv
from tienda_scraper.config import URL_CLIENTES, OUTPUT_CLIENTES

if __name__ == "__main__":
    soup = obtener_soup(URL_CLIENTES)

    if soup:
        clientes = parse_clientes(soup)

```

```

        for cliente in clientes:
            print(cliente["nombre"], cliente["apellidos"], cliente["ciudad"])

        write_to_csv(clientes, OUTPUT_CLIENTES)

    print("El scraping de clientes ha finalizado.")

```

===== scripts/ejercicio3_resenas_entrega.py =====

```

from tienda_scraper.scrapers import obtener_soup
from tienda_scraper.parsers import parse_resenas_entrega
from tienda_scraper.writers import write_to_csv
from tienda_scraper.config import URL_ORDEN, OUTPUT_RESENAS_ENTREGA

if __name__ == "__main__":
    soup = obtener_soup(URL_ORDEN)

    if soup:
        resenas = parse_resenas_entrega(soup)

        if not resenas:
            print(f"La orden en {URL_ORDEN} no tiene resenas de entrega. "
                  "Prueba con otro ORDEN_ID en tienda_scraper/config.py.")

        for resena in resenas:
            print(resena["autor"], "-", resena["producto"], "-", resena["calificacion"])
            print(" ", resena["comentario"])

        write_to_csv(resenas, OUTPUT_RESENAS_ENTREGA)

    print("El scraping de resenas de entrega ha finalizado.")

```

===== Salida de ejecución =====

```

$ python -m scripts.ejercicio1_productos
PROD-001 Apple iPhone 15 Pro Max 256GB S/5599.00
PROD-002 Apple iPhone 15 128GB S/3799.00
PROD-003 Samsung Galaxy S24 Ultra 512GB S/5299.00
PROD-004 Samsung Galaxy A55 128GB S/1399.00
PROD-005 Xiaomi Redmi Note 13 Pro 256GB S/999.00
PROD-006 Xiaomi Poco X6 Pro 512GB S/1299.00
PROD-007 Google Pixel 8 128GB S/2899.00
PROD-008 Motorola Moto G84 256GB S/899.00
PROD-009 Samsung Galaxy A15 128GB S/699.00
PROD-010 OnePlus 12 256GB S/3299.00
PROD-011 Apple MacBook Air M2 13 pulgadas 256GB S/4999.00
PROD-012 Apple MacBook Pro 14 pulgadas M3 512GB S/8999.00
PROD-013 Dell XPS 13 9340 Intel Core i7 S/6499.00
PROD-014 HP Pavilion 15 Intel Core i5 S/2799.00
PROD-015 Lenovo IdeaPad Slim 3 Ryzen 5 S/1999.00
PROD-016 ASUS ROG Strix G16 RTX 4060 S/7499.00
PROD-017 Lenovo Legion 5 Pro RTX 4070 S/8299.00
PROD-018 Acer Aspire 5 Intel Core i5 S/2299.00
PROD-019 HP Omen 16 RTX 4060 S/6999.00
PROD-020 Lenovo ThinkPad E14 Gen 5 S/3499.00
Datos guardados en /private/tmp/claude-501/Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-01/data/productos.csv
El scraping de productos ha finalizado.

```

```

$ python -m scripts.ejercicio2_clientes
No hay datos disponibles.
El scraping de clientes ha finalizado.

```

```

$ python -m scripts.ejercicio3_resenas_entrega
La orden en http://localhost:3000/ordenes/1 no tiene resenas de entrega. Prueba con otro ORDEN_ID en
tienda_scraper/config.py.
No hay datos disponibles.
El scraping de resenas de entrega ha finalizado.

```

sesion-de-clase-02/README.md

Proyecto de Minería Web - Sesión de clase 02

Este proyecto corresponde a los ejercicios del curso de **Minería Web** sobre técnicas de web scraping. Todos los ejercicios apuntan a la aplicación **tienda-virtual** (Next.js) corriendo en `http://localhost:3000`, así que esa aplicación debe estar levantada (`npm run dev` en el proyecto `tienda-virtual`) antes de ejecutar los notebooks.

Instalación de dependencias

Antes de ejecutar el proyecto, asegúrate de tener **Python 3.8+** instalado.

Instala las dependencias con:

```
pip install -r requirements.txt
playwright install chromium
```

Ejercicios (notebooks/)

Notebook	Técnica	Qué hace
<code>a_robots.ipynb</code>	<code>urllib.robotparser</code> + verificación manual de comodines	Lee <code>robots.txt</code> y determina qué rutas puede visitar un agente. Muestra además una limitación real de <code>robotparser</code> con patrones que usan <code>*</code> .
<code>b_productos_sitemap.ipynb</code>	<code>requests</code> + BeautifulSoup, guiado por el sitemap	Descubre todas las URLs de producto en <code>sitemap-productos.xml</code> y descarga el detalle completo de cada uno a <code>data/productos.csv</code> .
<code>c_clientes_sitemap.ipynb</code>	<code>requests</code> + BeautifulSoup, sitemap + paginación	El sitemap de clientes solo expone el listado (no hay página de detalle por cliente), así que se recorre la paginación de <code>/clientes</code> hasta agotarla, guardando todo en <code>data/clientes.csv</code> .
<code>d_comentarios_beautifulsoup.ipynb</code>	<code>requests</code> + BeautifulSoup, paginación manual	Recorre la paginación de <code>/testimonios</code> (comentarios generales de clientes) y guarda todos los comentarios en <code>data/comentarios.csv</code> .
<code>e_resenas_entrega_playwright.ipynb</code>	Playwright (API asíncrona)	Automatiza un navegador para recorrer el listado paginado de <code>/ordenes</code> y extraer todas las reseñas de entrega (<code>post_compra</code>) a <code>data/resenas_entrega.csv</code> .

Cada notebook es autocontenido: se puede ejecutar de principio a fin (`Run All`) y regenera su CSV correspondiente en `data/`.

Otros archivos

`pucp_scraper/` y `scripts/run_scraper.py` corresponden a un ejercicio previo (scraping del repositorio institucional de la PUCP con Selenium) y no forman parte de los ejercicios a)-e) descritos arriba.

a) Lectura de robots.txt: ¿puede un agente acceder al sitio?

Este ejercicio usa `urllib.robotparser.RobotFileParser`, de la librería estándar de Python, para interpretar el archivo `robots.txt` de la tienda virtual publicada en `http://localhost:3000` y determinar si un *user-agent* determinado tiene permiso para acceder a una URL concreta.

Flujo de trabajo:

- `set_url(url)` : define la ubicación del `robots.txt`.
- `read()` : descarga y procesa el archivo, cargando las reglas en memoria.
- `can_fetch(user_agent, url)` : devuelve `True / False` según las directivas `Allow / Disallow`.

Documentación oficial: <https://docs.python.org/3/library/urllib.robotparser.html>

```
from urllib.robotparser import RobotFileParser

BASE_URL = "http://localhost:3000"
USER_AGENT = "MineriaWeb-2026-2/1.0"

parser = RobotFileParser()
parser.set_url(f"{BASE_URL}/robots.txt")
parser.read()

print(f"Sitemaps declarados: {parser.site_maps()}")
```

Sitemaps declarados: ['http://localhost:3000/sitemap.xml']

Verificando el acceso a distintas rutas

El `robots.txt` de la tienda virtual (`app/robots.txt`) permite todo el sitio (`Allow: /`) salvo dos rutas de edición:

```
User-Agent: *
Allow: /
Disallow: /productos/*/editar
Disallow: /clientes/*/editar
```

Probemos `can_fetch` contra rutas que deberían estar permitidas y contra las dos que deberían estar bloqueadas.

```
rutas_a_verificar = [
    "/",
    "/productos",
    "/productos/1",
    "/productos/1/editar",
```

```

"/clientes",
"/clientes/1/editar",
"/testimonios",
"/ordenes",
]

for ruta in rutas_a_verificar:
    permitido = parser.can_fetch(USER_AGENT, f"{BASE_URL}{ruta}")
    print(f"{ruta:25s} -> {'PERMITIDO' if permitido else 'BLOQUEADO'}")

/ -> PERMITIDO
/productos -> PERMITIDO
/productos/1 -> PERMITIDO
/productos/1/editar -> BLOQUEADO
/clientes -> PERMITIDO
/clientes/1/editar -> BLOQUEADO
/testimonios -> PERMITIDO
/ordenes -> PERMITIDO

```

Una limitación importante de `urllib.robotparser`

Las rutas `/productos/1/editar` y `/clientes/1/editar` deberían aparecer como **BLOQUEADO**, pero `RobotFileParser` las marca como **PERMITIDO**.

La razón es que `urllib.robotparser` (a diferencia de librerías como `protego` o de los rastreadores de Google/Bing) **no interpreta el comodín `*` dentro de una ruta**: compara la URL contra el patrón `/productos/*/editar` como si el asterisco fuera un carácter literal, y como ninguna URL real contiene un `*`, la regla nunca coincide.

Esto es una lección práctica valiosa: la librería estándar es útil para reglas simples, pero para *robots.txt* con comodines conviene interpretar las reglas manualmente o usar una librería especializada. A continuación implementamos una verificación manual que sí soporta comodines, traduciendo cada patrón a una expresión regular.

```

import re

DISALLOW = ["/productos/*/editar", "/clientes/*/editar"]

def coincide_con_patron(patron: str, path: str) -> bool:
    """Traduce un patron de robots.txt (con * y $) a una regex y verifica
    coincidencia."""
    regex = re.escape(patron).replace(r"\*", ".*")
    if regex.endswith(r"\$"):
        regex = regex[:-2] + "$"
    return re.match(regex, path) is not None

def accesible_para_agente(path: str) -> bool:
    return not any(coincide_con_patron(patron, path) for patron in DISALLOW)

for ruta in rutas_a_verificar:
    permitido = accesible_para_agente(ruta)
    print(f"{ruta:25s} -> {'PERMITIDO' if permitido else 'BLOQUEADO'}")

```

/	-> PERMITIDO
/productos	-> PERMITIDO
/productos/1	-> PERMITIDO
/productos/1/editar	-> BLOQUEADO
/clientes	-> PERMITIDO
/clientes/1/editar	-> BLOQUEADO
/testimonios	-> PERMITIDO
/ordenes	-> PERMITIDO

Con la verificación manual, `/productos/1/editar` y `/clientes/1/editar` sí aparecen como **BLOQUEADO**, coincidiendo con la intención real del archivo `robots.txt`. El resto de ejercicios de esta sesión (b–e) sólo visitan rutas de lectura (`/productos/...`, `/clientes`, `/testimonios`, `/ordenes/...`), por lo que están permitidas para nuestro agente.

b) Scraping completo de productos usando el sitemap

Objetivo: descargar la información completa de **todos** los productos de la tienda virtual (`http://localhost:3000`) y guardarla en `data/productos.csv` .

Estrategia:

1. Leer el sitemap índice (`/sitemap.xml`) para descubrir el sitemap específico de productos.
2. Leer `sitemap-productos.xml` , que expone la URL de detalle de **cada** producto (`/productos/{id}`) — a diferencia de clientes, el sitemap de productos sí enumera cada elemento individualmente, por lo que no se necesita paginación.
3. Visitar cada URL de producto y extraer sus datos con BeautifulSoup, usando los atributos `data-*` y las propiedades `itemprop` (microdatos `schema.org/Product`) que ya expone la página.

```
import csv
import os
import re
import requests
from bs4 import BeautifulSoup

BASE_URL = "http://localhost:3000"
HEADERS = {"User-Agent": "MineriaWeb-2026-2/1.0 (+scraper-tienda-virtual)"}

def get_soup(url: str, parser: str = "html.parser") -> BeautifulSoup:
    respuesta = requests.get(url, headers=HEADERS, timeout=10)
    respuesta.raise_for_status()
    return BeautifulSoup(respuesta.content, parser)
```

1. Descubrir el sitemap de productos a partir del sitemap índice

```
sitemap_index = get_soup(f"{BASE_URL}/sitemap.xml", "xml")
sub_sitemaps = [loc.text for loc in sitemap_index.find_all("loc")]
print("Sub-sitemaps encontrados:")
for url in sub_sitemaps:
    print(" -", url)

productos_sitemap_url = next(url for url in sub_sitemaps if "sitemap-productos" in url)
print("\nSitemap de productos:", productos_sitemap_url)
```

Sub-sitemaps encontrados:

- <http://localhost:3000/sitemap-general.xml>
- <http://localhost:3000/sitemap-productos.xml>
- <http://localhost:3000/sitemap-clientes.xml>
- <http://localhost:3000/sitemap-ordenes.xml>
- <http://localhost:3000/sitemap-testimonios.xml>

Sitemap de productos: <http://localhost:3000/sitemap-productos.xml>

2. Extraer las URLs de cada producto

El sitemap de productos también incluye `/productos` y `/productos/nuevo` (páginas de listado y de creación), así que filtramos únicamente las URLs con formato `/productos/{id numérico}`.

```
sitemap_productos = get_soup(productos_sitemap_url, "xml")
producto_urls = [
    loc.text for loc in sitemap_productos.find_all("loc") if
    re.search(r"/productos/\d+$", loc.text)
]
print(f"Total de productos encontrados en el sitemap: {len(producto_urls)}")
print(producto_urls[:5])
```

```
Total de productos encontrados en el sitemap: 90
['http://localhost:3000/productos/1', 'http://localhost:3000/productos/2',
'http://localhost:3000/productos/3', 'http://localhost:3000/productos/4',
'http://localhost:3000/productos/5']
```

3. Scraping del detalle de cada producto

Cada página `/productos/{id}` renderiza un `<section itemscope itemtype="https://schema.org/Product">` con atributos `data-*` (id, código, categoría, subcategoría, precio, stock) y microdatos `itemprop` para nombre, descripción y la calificación agregada de reseñas.

```
def scrape_producto(url: str) -> dict:
    soup = get_soup(url)
    section = soup.select_one("section[data-producto-id]")

    nombre = section.select_one("h2[itemprop=name]").get_text(strip=True)
    descripcion = section.select_one("p[itemprop=description]").get_text(strip=True)

    rating_el = section.select_one("[itemprop=aggregateRating]")
    if rating_el:
        calificacion_promedio = rating_el.select_one("[itemprop=ratingValue]").get_text(strip=True)
        total_resenas = rating_el.select_one("[itemprop=reviewCount]").get_text(strip=True)
    else:
        calificacion_promedio, total_resenas = "", "0"

    return {
        "id": section["data-producto-id"],
        "codigo": section["data-producto-codigo"],
        "nombre": nombre,
        "descripcion": descripcion,
        "categoria": section["data-categoria"],
```

```

        "subcategoria": section["data-subcategoria"],
        "precio": section["data-precio"],
        "stock": section["data-stock"],
        "calificacion_promedio": calificacion_promedio,
        "total_resenas": total_resenas,
        "url": url,
    }

```

```

productos = []
for url in producto_urls:
    try:
        productos.append(scrape_producto(url))
    except Exception as error:
        print(f"Error al procesar {url}: {error}")

print(f"Productos scrapeados correctamente: {len(productos)}")
productos[:2]

```

Productos scrapeados correctamente: 90

```

[{'id': '1',
  'codigo': 'PROD-001',
  'nombre': 'Apple iPhone 15 Pro Max 256GB',
  'descripcion': 'Smartphone premium con chip A17 Pro fabricado en 3nm, camara triple de 48MP con teleobjetivo periscopico de 5x y pantalla Super Retina XDR de 6.7 pulgadas con ProMotion a 120Hz. Cuenta con marco de titanio, Dynamic Island, puerto USB-C y resistencia al agua IP68.',
  'categoria': 'Smartphones',
  'subcategoria': 'Gama Alta',
  'precio': '5599',
  'stock': '18',
  'calificacion_promedio': '3.5',
  'total_resenas': '4',
  'url': 'http://localhost:3000/productos/1'},
 {'id': '2',
  'codigo': 'PROD-002',
  'nombre': 'Apple iPhone 15 128GB',
  'descripcion': 'Smartphone con chip A16 Bionic, camara dual de 48MP con modo retrato mejorado y Dynamic Island para notificaciones interactivas. Incluye pantalla Super Retina XDR de 6.1 pulgadas, puerto USB-C y proteccion Ceramic Shield.',
  'categoria': 'Smartphones',
  'subcategoria': 'Gama Alta',
  'precio': '3799',
  'stock': '25',
  'calificacion_promedio': '3.3',
  'total_resenas': '4',
  'url': 'http://localhost:3000/productos/2'}]

```

4. Guardar los datos en data/productos.csv

```

DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)
OUTPUT_PATH = os.path.join(DATA_DIR, "productos.csv")

with open(OUTPUT_PATH, mode="w", newline="", encoding="utf-8") as archivo:
    writer = csv.DictWriter(archivo, fieldnames=productos[0].keys())
    writer.writeheader()
    writer.writerows(productos)

```

```
print(f"Se guardaron {len(productos)} productos en {OUTPUT_PATH}")
```

Se guardaron 90 productos en /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-02/notebooks/./data/productos.csv

c) Scraping completo de clientes usando el sitemap (con paginación)

Objetivo: descargar la información completa de **todos** los clientes registrados y guardarla en `data/clientes.csv`.

A diferencia de productos, `sitemap-clientes.xml` **no** enumera cada cliente individualmente (no existe una página de detalle pública `/clientes/{id}`, solo un listado en `/clientes` y un formulario en `/clientes/nuevo`; la ruta `/clientes/{id}/editar` existe pero está bloqueada en `robots.txt`). Por eso la estrategia combina dos técnicas:

1. Usar el sitemap para descubrir de forma programática la URL del listado de clientes (en vez de asumirla/escribirla a mano).
2. Recorrer la **paginación** de ese listado (`/clientes?page=1`, `?page=2`, ...) hasta agotar las páginas, extrayendo los datos de cada fila con BeautifulSoup.

```
import csv
import os
import requests
from bs4 import BeautifulSoup

BASE_URL = "http://localhost:3000"
HEADERS = {"User-Agent": "MineriaWeb-2026-2/1.0 (+scraper-tienda-virtual)"}

def get_soup(url: str, parser: str = "html.parser") -> BeautifulSoup:
    respuesta = requests.get(url, headers=HEADERS, timeout=10)
    respuesta.raise_for_status()
    return BeautifulSoup(respuesta.content, parser)
```

1. Descubrir la URL del listado de clientes a partir del sitemap

```
sitemap_index = get_soup(f"{BASE_URL}/sitemap.xml", "xml")
sub_sitemaps = [loc.text for loc in sitemap_index.find_all("loc")]
clientes_sitemap_url = next(url for url in sub_sitemaps if "sitemap-clientes" in url)

sitemap_clientes = get_soup(clientes_sitemap_url, "xml")
urls_clientes_sitemap = [loc.text for loc in sitemap_clientes.find_all("loc")]
print("URLs declaradas en sitemap-clientes.xml:", urls_clientes_sitemap)

clientes_base_url = next(url for url in urls_clientes_sitemap if
url.rstrip("/").endswith("/clientes"))
print("\nListado base de clientes:", clientes_base_url)
```

```
URLs declaradas en sitemap-clientes.xml: ['http://localhost:3000/clientes',
'http://localhost:3000/clientes/nuevo']
```

```
Listado base de clientes: http://localhost:3000/clientes
```

2. Función de paginación

El componente `Pagination` de la tienda virtual deshabilita el enlace **Siguiente** (lo convierte de `<a>` a `<span>`) cuando ya no hay más páginas. Aprovechamos esa señal del HTML para saber cuándo detenernos, sin tener que calcular el total de páginas de antemano.

```
def hay_pagina_siguiente(soup: BeautifulSoup) -> bool:
    nav = soup.select_one('nav[aria-label="Paginacion"]')
    if nav is None:
        return False
    return any(enlace.get_text(strip=True) == "Siguiente" for enlace in
nav.select("a"))
```

3. Scraping de cada fila de la tabla de clientes

```
def scrape_fila_cliente(fila) -> dict:
    celdas = fila.find_all("td")
    return {
        "id": fila["data-cliente-id"],
        "dni": fila["data-dni"],
        "nombre": celdas[0].get_text(strip=True),
        "apellidos": celdas[1].get_text(strip=True),
        "email": celdas[3].get_text(strip=True),
        "ciudad": fila["data-ciudad"],
        "pais": fila["data-pais"],
    }
```

```
clientes = []
numero_pagina = 1

while True:
    soup = get_soup(f"{clientes_base_url}?page={numero_pagina}")
    filas = soup.select("tr[data-cliente-id]")
    if not filas:
        break

    clientes.extend(scrape_fila_cliente(fila) for fila in filas)
    print(f"Pagina {numero_pagina}: {len(filas)} clientes")

    if not hay_pagina_siguiente(soup):
        break
    numero_pagina += 1

print(f"\nTotal de clientes scrapeados: {len(clientes)} en {numero_pagina} paginas")
```

Total de clientes scrapeados: 0 en 1 paginas

4. Guardar los datos en `data/clientes.csv`

```
DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)
OUTPUT_PATH = os.path.join(DATA_DIR, "clientes.csv")

with open(OUTPUT_PATH, mode="w", newline="", encoding="utf-8") as archivo:
```

```
writer = csv.DictWriter(archivo, fieldnames=clientes[0].keys())
writer.writeheader()
writer.writerows(clientes)

print(f"Se guardaron {len(clientes)} clientes en {OUTPUT_PATH}")
```

```
-----
IndexError                                Traceback (most recent call last)
Cell In[6], line 6
      2 os.makedirs(DATA_DIR, exist_ok=True)
      3 OUTPUT_PATH = os.path.join(DATA_DIR, "clientes.csv")
      4
      5 with open(OUTPUT_PATH, mode="w", newline="", encoding="utf-8") as archivo:
----> 6     writer = csv.DictWriter(archivo, fieldnames=clientes[0].keys())
      7     writer.writeheader()
      8     writer.writerows(clientes)
      9
```

```
IndexError: list index out of range
```

d) Scraping completo de comentarios (testimonios) con BeautifulSoup y paginación

Objetivo: descargar **todos** los testimonios generales de clientes (/testimonios) y guardarlos en data/comentarios.csv .

sitemap-testimonios.xml solo declara la URL del listado (/testimonios), no cada comentario por separado, y ese listado está paginado (10 comentarios por página). El ejercicio es igual en espíritu al de clientes (c), pero aquí el foco está puesto en usar BeautifulSoup para **recorrer manualmente la paginación** hasta agotar todas las páginas de comentarios.

```
import csv
import os
import requests
from bs4 import BeautifulSoup

BASE_URL = "http://localhost:3000"
HEADERS = {"User-Agent": "MineriaWeb-2026-2/1.0 (+scraper-tienda-virtual)"}

def get_soup(url: str, parser: str = "html.parser") -> BeautifulSoup:
    respuesta = requests.get(url, headers=HEADERS, timeout=10)
    respuesta.raise_for_status()
    return BeautifulSoup(respuesta.content, parser)

def hay_pagina_siguiente(soup: BeautifulSoup) -> bool:
    nav = soup.select_one('nav[aria-label="Paginacion"]')
    if nav is None:
        return False
    return any(enlace.get_text(strip=True) == "Siguiente" for enlace in
nav.select("a"))
```

1. Descubrir la URL del listado de testimonios a partir del sitemap

```
sitemap_index = get_soup(f"{BASE_URL}/sitemap.xml", "xml")
sub_sitemaps = [loc.text for loc in sitemap_index.find_all("loc")]
testimonios_sitemap_url = next(url for url in sub_sitemaps if "sitemap-testimonios"
in url)

sitemap_testimonios = get_soup(testimonios_sitemap_url, "xml")
testimonios_base_url = sitemap_testimonios.find("loc").text
print("Listado base de testimonios:", testimonios_base_url)
```

Listado base de testimonios: http://localhost:3000/testimonios

2. Scraping de cada comentario

Cada testimonio es un `<li data-comentario-id itemscope itemtype="https://schema.org/Review">` con atributos `data-*` para el cliente, la calificación y la fecha, y microdatos `itemprop` para el texto de la reseña.

Detalle a tener en cuenta: el nombre completo del cliente se renderiza como `{nombre} {apellidos}` dentro de un mismo elemento `itemprop="name"`, es decir, React separa ambos valores con un nodo de texto que es un único espacio. Si se usa `get_text(strip=True)` sin especificar un separador, BeautifulSoup termina eliminando ese espacio (porque `strip=True` vacía los nodos de texto que solo contienen espacios antes de unirlos), y el resultado queda pegado (por ejemplo `"UrsulaCastaneda"`). La forma correcta es pasar un separador explícito: `get_text(" ", strip=True)`.

```
def scrape_comentario(li) -> dict:
    autor = li.select_one('[itemprop=author]')
    nombre_completo = autor.select_one('[itemprop=name]').get_text(" ", strip=True)
    return {
        "id": li["data-comentario-id"],
        "cliente_id": li["data-cliente-id"],
        "cliente_nombre_completo": nombre_completo,
        "cliente_ciudad": li.select_one('[data-ciudad]')['data-ciudad'],
        "cliente_pais": li.select_one('[data-pais]')['data-pais'],
        "calificacion": li["data-calificacion"],
        "fecha": li["data-fecha"],
        "texto": li.select_one('[itemprop=reviewBody]').get_text(strip=True),
    }
```

3. Recorrer todas las páginas hasta que no haya un enlace "Siguiente"

```
comentarios = []
numero_pagina = 1

while True:
    soup = get_soup(f"{testimonios_base_url}?page={numero_pagina}")
    items = soup.select("li[data-comentario-id]")
    if not items:
        break

    comentarios.extend(scrape_comentario(item) for item in items)
    print(f"Pagina {numero_pagina}: {len(items)} comentarios")

    if not hay_pagina_siguiente(soup):
        break
    numero_pagina += 1

print(f"\nTotal de comentarios scrapeados: {len(comentarios)} en {numero_pagina} paginas")
```

Pagina 1: 10 comentarios
Pagina 2: 10 comentarios
Pagina 3: 10 comentarios
Pagina 4: 10 comentarios
Pagina 5: 10 comentarios
Pagina 6: 10 comentarios
Pagina 7: 10 comentarios
Pagina 8: 10 comentarios
Pagina 9: 10 comentarios
Pagina 10: 10 comentarios
Pagina 11: 10 comentarios
Pagina 12: 10 comentarios
Pagina 13: 10 comentarios
Pagina 14: 5 comentarios

Total de comentarios scrapeados: 135 en 14 paginas

4. Guardar los datos en data/comentarios.csv

```
DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)
OUTPUT_PATH = os.path.join(DATA_DIR, "comentarios.csv")

with open(OUTPUT_PATH, mode="w", newline="", encoding="utf-8") as archivo:
    writer = csv.DictWriter(archivo, fieldnames=comentarios[0].keys())
    writer.writeheader()
    writer.writerows(comentarios)

print(f"Se guardaron {len(comentarios)} comentarios en {OUTPUT_PATH}")
```

Se guardaron 135 comentarios en /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-02/notebooks/./data/comentarios.csv

e) Scraping de reseñas de entrega de órdenes usando Playwright

Objetivo: descargar **todas** las reseñas sobre la entrega de los pedidos (`tipo = "post_compra"`) y guardarlas en `data/resenas_entrega.csv` .

Estas reseñas aparecen incrustadas dentro del listado paginado de órdenes (`/ordenes`), una por cada pedido que las tiene. A diferencia de los ejercicios b–d (que usan `requests` + BeautifulSoup sobre el HTML ya renderizado por el servidor), aquí usamos **Playwright** para automatizar un navegador real: se abre cada página del listado en un navegador headless y se localizan los elementos con su propia API de *locators*, en vez de descargar y parsear el HTML manualmente.

Esta técnica es la que se necesitaría en sitios donde el contenido se genera dinámicamente en el cliente (JavaScript puro, SPA); aquí la aplicamos igualmente sobre la tienda virtual para practicar el flujo de trabajo de Playwright: lanzar el navegador, navegar, esperar a que el contenido esté listo y extraer datos con locators.

```
import csv
import os
from playwright.async_api import async_playwright

BASE_URL = "http://localhost:3000"
USER_AGENT = "MineriaWeb-2026-2/1.0 (+scraper-tienda-virtual)"
```

Recorrido paginado con Playwright

Igual que en los ejercicios c) y d), detenemos la paginación cuando el enlace **Siguiente** del componente de paginación deja de ser un `<a>` (se vuelve `<span>` deshabilitado). Con Playwright esto se resuelve contando cuántos elementos coinciden con el selector `nav[aria-label="Paginacion"] a:has-text("Siguiente")` .

```
resenas_entrega = []

playwright = await async_playwright().start()
navegador = await playwright.chromium.launch(headless=True)
pagina = await navegador.new_page(user_agent=USER_AGENT)

numero_pagina = 1
while True:
    await pagina.goto(f"{BASE_URL}/ordenes?page={numero_pagina}",
wait_until="networkidle")

    ordenes = pagina.locator("article[data-orden-id]")
    total_ordenes = await ordenes.count()
    if total_ordenes == 0:
        break
```

```

for i in range(total_ordenes):
    orden = ordenes.nth(i)
    orden_id = await orden.get_attribute("data-orden-id")

    resenas_de_la_orden = orden.locator("li[data-tipo='post_compra']")
    total_resenas_orden = await resenas_de_la_orden.count()
    for j in range(total_resenas_orden):
        resena = resenas_de_la_orden.nth(j)
        resenas_entrega.append({
            "id": await resena.get_attribute("data-comentario-id"),
            "orden_id": orden_id,
            "cliente_id": await resena.get_attribute("data-cliente-id"),
            "producto_id": await resena.get_attribute("data-producto-id"),
            "calificacion": await resena.get_attribute("data-calificacion"),
            "fecha": await resena.get_attribute("data-fecha"),
            "texto": await resena.locator("[itemprop='reviewBody']").inner_text(),
        })

    print(f"Pagina {numero_pagina}: {total_ordenes} ordenes revisadas")

    siguiente = pagina.locator('nav[aria-label="Paginacion"] a:has-text("Siguiente")')
    if await siguiente.count() == 0:
        break
    numero_pagina += 1

await navegador.close()
await playwright.stop()

print(f"\nTotal de resenas de entrega scrapeadas: {len(resenas_entrega)} en {numero_pagina} paginas de ordenes")

```

Total de resenas de entrega scrapeadas: 0 en 1 paginas de ordenes

Guardar los datos en data/resenas_entrega.csv

```

DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)
OUTPUT_PATH = os.path.join(DATA_DIR, "resenas_entrega.csv")

with open(OUTPUT_PATH, mode="w", newline="", encoding="utf-8") as archivo:
    writer = csv.DictWriter(archivo, fieldnames=resenas_entrega[0].keys())
    writer.writeheader()
    writer.writerows(resenas_entrega)

print(f"Se guardaron {len(resenas_entrega)} resenas de entrega en {OUTPUT_PATH}")

```

IndexError

Traceback (most recent call last)

Cell In[3], line 6

```
2 os.makedirs(DATA_DIR, exist_ok=True)
3 OUTPUT_PATH = os.path.join(DATA_DIR, "resenas_entrega.csv")
4
5 with open(OUTPUT_PATH, mode="w", newline="", encoding="utf-8") as archivo:
----> 6     writer = csv.DictWriter(archivo, fieldnames=resenas_entrega[0].keys())
7     writer.writeheader()
8     writer.writerows(resenas_entrega)
9
```

IndexError: list index out of range

sesion-de-clase-03/README.md

Proyecto de Minería Web - Sesión de clase 03

Esta sesión pasa del *scraping de HTML* (sesión 02) al **scraping de APIs**: REST, REST con autenticación, GraphQL y, cuando no hay API, un navegador real con login (Playwright). Los notebooks están numerados y van de menor a mayor complejidad.

Todos los ejercicios apuntan a la aplicación **tienda-virtual** (Next.js) corriendo en `http://localhost:3000`, así que debe estar levantada antes de ejecutarlos:

```
cd ../tienda-virtual
cp .env.example .env.local    # solo la primera vez
npm install                   # solo la primera vez
npm run dev
```

Documentación de las APIs de la tienda: [tienda-virtual/docs/API.md](#) (REST) y [tienda-virtual/docs/GRAPHQL.md](#).

Instalación de dependencias

Requiere **Python 3.8+**.

```
pip install -r requirements.txt
playwright install chromium
```

Ejercicios (notebooks/)

Se ejecutan en orden; cada notebook agrega una capa sobre el anterior.

Notebook	Técnica	Qué hace
<code>01_rest_recurso_y_coleccion.ipynb</code>	<code>requests</code> contra la API REST de la tienda	Primer contacto: <code>GET /api/health</code> (sin credencial), un recurso individual (<code>/api/productos/1</code>) y una página de la colección (<code>/api/productos</code>) con el header <code>x-api-key</code> . Guarda <code>data/rest_muestra_productos.csv</code> .
<code>02_rest_paginacion.ipynb</code>	Paginación por número de página (<code>pageInfo</code>)	Recorre <code>/api/productos</code> y <code>/api/clientes</code> completos (bucle <code>page</code> hasta <code>totalPages</code> , y variante calculando <code>totalPages</code> de antemano); muestra que <code>/api/comentarios</code> no pagina. Guarda <code>data/rest_productos.csv</code> y <code>data/rest_clientes.csv</code> .

Notebook	Técnica	Qué hace
03_rest_autenticado_api_key.ipynb	La API key a fondo	401 unauthorized sin credencial vs 401 invalid_api_key con una inválida; scopes read / write (alta y baja de un producto de prueba); revocación desde /admin/api-keys . Scrapea todos los testimonios a data/rest_testimonios.csv .
04_rest_autenticado_oauth.ipynb	REST con OAuth2 client_credentials	Obtiene un access_token en POST /api/oauth/token , lo renueva solo antes de que expire (ClienteOAuth) y pagina /api/ordenes con Authorization: Bearer . Guarda data/rest_ordenes.csv .
05_graphql_consultas_basicas.ipynb	GraphQL: primera query	POST /api/graphql con x-api-key ; query plana de productos, manejo del array errors (llega con HTTP 200), introspección del esquema y paginación con variables. Guarda data/graphql_productos.csv .
06_graphql_consultas_anidadadas.ipynb	GraphQL: variables + anidado + OAuth	Una sola query trae cada orden con su cliente y sus items (y el producto de cada ítem); autentica con OAuth Bearer y consulta las reseñas de entrega (comentarios(tipo: "post_compra")) por orden. Guarda data/graphql_ordenes.csv y data/graphql_resenas_entrega.csv .
07_playwright_paginas_con_login.ipynb	Playwright con login	/ordenes redirige (307) a /login sin sesión. Playwright completa el formulario (admin@tienda.local / admin123), guarda el storage_state , recorre el listado paginado protegido extrayendo las reseñas de entrega embebidas y reutiliza la sesión para entrar a /admin/api-keys . Guarda data/playwright_resenas_entrega.csv .

Cada notebook es autocontenido: se puede ejecutar de principio a fin (Run All) y regenera sus CSV en data/ .

Credenciales demo de tienda-virtual

Se re-siembran en cada arranque de la app (ver tienda-virtual/.env.example):

Qué	Valor
Admin (login del back-office)	admin@tienda.local / admin123
API key	sk_demo_00000000000000000000000000000000
Cliente OAuth (client_credentials)	scraper-demo / scraper-demo-secret

El archivo data/playwright_storage_state.json que genera el notebook 07 contiene un JWT de sesión local (caduca en 8 h) y está en .gitignore .

01 · REST: primer contacto con la API

Scrapear una **API REST** es casi siempre mejor que parsear HTML: los datos llegan ya estructurados (JSON), no se rompen cuando cambia la maquetación de la web y se transfiere mucho menos volumen.

Toda la sesión trabaja contra la API de **tienda-virtual** (`http://localhost:3000/api`). La app debe estar corriendo:

```
cd ../tienda-virtual && npm run dev
```

Un solo endpoint, `GET /api/health`, responde **sin credenciales**; todos los demás `/api/*` piden el header `x-api-key`. En este notebook lo usamos sin más como "el header que pide esta API"; su gestión a fondo (claves inválidas, scopes, revocación) es el notebook 03.

Progresión de la sesión:

Notebook	Qué se agrega
Este	Primer contacto: <code>health</code> , un recurso y una colección
<code>02_rest_paginacion</code>	Recorrer una colección completa con paginación
<code>03_rest_autenticado_api_key</code>	La API key a fondo: 401, claves inválidas, scopes, revocación
<code>04_rest_autenticado_oauth</code>	OAuth2 (<code>client_credentials</code>): token Bearer que expira
<code>05_graphql_consultas_basicas</code>	Lo mismo con GraphQL : primera query
<code>06_graphql_consultas_anidadadas</code>	GraphQL con variables, campos anidados y OAuth
<code>07_playwright_paginas_con_login</code>	Sin API: Playwright con login para páginas protegidas

```
import csv
import os

import requests

BASE = "http://localhost:3000"
API_KEY = "sk_demo_00000000000000000000000000000000" # key demo, se re-siembr en cada
arranque
TIMEOUT = 30

# Una Session reutiliza la conexion TCP y aplica cabeceras comunes a cada request.
sesion = requests.Session()
sesion.headers.update({"x-api-key": API_KEY, "User-Agent": "MineriaWeb-2026-2/1.0"})
```

El "hola mundo": `GET /api/health` (sin credenciales)

`raise_for_status()` convierte un 4xx/5xx en excepción; `resp.json()` decodifica el cuerpo JSON a un `dict` de Python.

```

resp = requests.get(f"{BASE}/api/health", timeout=TIMEOUT) # sin header: este endpoint es abierto
resp.raise_for_status()

print("Status:", resp.status_code)
print("Content-Type:", resp.headers["Content-Type"])
print("Cuerpo:", resp.json())

```

```

Status: 200
Content-Type: application/json
Cuerpo: {'status': 'ok', 'service': 'tienda-virtual-api', 'time': '2026-10-07T01:42:19.824Z'}

```

Un solo recurso: GET /api/productos/1

Este endpoint sí exige el header `x-api-key` (ya cargado en la `Session`). El detalle de un producto trae, además de sus campos, sus `especificaciones`, sus `resenas` y el rating agregado.

```

resp = sesion.get(f"{BASE}/api/productos/1", timeout=TIMEOUT)
resp.raise_for_status()
producto = resp.json()

print("Claves del recurso:", list(producto.keys()))
print()
for campo in ("id", "codigo", "nombre", "categoria", "subcategoria", "precio", "stock"):
    print(f" {campo:12s} = {producto[campo]}")
print(f" especificaciones = {len(producto['especificaciones'])} filas")
print(f" resenas           = {len(producto['resenas'])}")
print(f" aggregateRating   = {producto['aggregateRating']}")

```

```

Claves del recurso: ['id', 'codigo', 'nombre', 'descripcion', 'categoria', 'subcategoria', 'precio', 'stock', 'especificaciones', 'aggregateRating', 'resenas']

```

```

id           = 1
codigo       = PROD-001
nombre       = Apple iPhone 15 Pro Max 256GB
categoria    = Smartphones
subcategoria = Gama Alta
precio       = 5599
stock        = 18
especificaciones = 5 filas
resenas      = 4
aggregateRating = {'ratingValue': 3.5, 'reviewCount': 4}

```

Una colección: GET /api/productos

Las APIs REST no devuelven "toda la tabla" de golpe: entregan **una página** y unos metadatos (`pageInfo`) para pedir el resto. Aquí pedimos solo la primera página de 5.

```

resp = sesion.get(f"{BASE}/api/productos", params={"page": 1, "pageSize": 5},
timeout=TIMEOUT)
resp.raise_for_status()
data = resp.json()

```

```

print("pageInfo:", data["pageInfo"])
print(f"La respuesta trae {len(data['items'])} de {data['pageInfo']['total']} productos\n")
for p in data["items"]:
    ar = p["aggregateRating"]
    print(f"    [{p['id']:>2}] {p['nombre'][:38]:38s} {p['categoria'][:22s]} S/ {p['precio']})")

```

pageInfo: {'page': 1, 'pageSize': 5, 'total': 90, 'totalPages': 18}
 La respuesta trae 5 de 90 productos

[1]	Apple iPhone 15 Pro Max 256GB	Smartphones	S/ 5599
[2]	Apple iPhone 15 128GB	Smartphones	S/ 3799
[3]	Samsung Galaxy S24 Ultra 512GB	Smartphones	S/ 5299
[4]	Samsung Galaxy A55 128GB	Smartphones	S/ 1399
[5]	Xiaomi Redmi Note 13 Pro 256GB	Smartphones	S/ 999

Guardar la muestra en data/rest_muestra_productos.csv

```

DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)
OUTPUT = os.path.join(DATA_DIR, "rest_muestra_productos.csv")

filas = []
for p in data["items"]:
    ar = p["aggregateRating"] or {}
    filas.append({
        "id": p["id"],
        "codigo": p["codigo"],
        "nombre": p["nombre"],
        "categoria": p["categoria"],
        "precio": p["precio"],
        "stock": p["stock"],
        "rating_promedio": ar.get("ratingValue", ""),
        "total_resenas": ar.get("reviewCount", ""),
    })

with open(OUTPUT, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=filas[0].keys())
    writer.writeheader()
    writer.writerows(filas)

print(f"Guardadas {len(filas)} filas en {OUTPUT}")

```

Guardadas 5 filas en /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-03/notebooks/./data/rest_muestra_productos.csv

Aquí pedimos una sola página a mano. En el siguiente notebook recorremos la colección **completa** automatizando la paginación.

02 · REST: paginación (descargar la colección completa)

Ninguna API entrega todos los registros en una sola respuesta. La API de tienda-virtual pagina **por número de página**: cada lista devuelve `pageInfo { page, pageSize, total, totalPages }`. Hay dos formas de recorrerla entera:

- **Forma A** — pedir `page` 1, 2, 3... y parar cuando `page == totalPages`.
- **Forma B** — leer `total` de la primera respuesta y calcular `totalPages = ceil(total / pageSize)` de antemano.

Otras APIs pagan **por cursor**: en vez de un número de página te devuelven un puntero (`next`) a la página siguiente y se itera hasta que vale `null`. El bucle es el mismo; solo cambia "incrementa `page` " por "sigue el enlace `next` ".

```
import csv
import math
import os

import requests

BASE = "http://localhost:3000"
API_KEY = "sk_demo_00000000000000000000000000000000"
TIMEOUT = 30

sesion = requests.Session()
sesion.headers.update({"x-api-key": API_KEY, "User-Agent": "MineriaWeb-2026-2/1.0"})

DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)
```

Forma A — bucle por número de página

`descargar_todo` sirve para cualquier listado paginado de esta API (`productos`, `clientes`, `ordenes`, `testimonios`).

```
def descargar_todo(path, page_size=50, **filtros):
    items = []
    page = 1
    while True:
        r = sesion.get(
            f"{BASE}{path}",
            params={**filtros, "page": page, "pageSize": page_size},
            timeout=TIMEOUT,
        )
        r.raise_for_status()
        payload = r.json()
        items.extend(payload["items"])
```

```

        info = payload["pageInfo"]
        print(f"  {path}  página {info['page']:>2}/{info['totalPages']}  (+
{len(payload['items'])}, total {len(items)}/{info['total']})")
        if info["page"] >= info["totalPages"]:
            return items
        page += 1

```

```

productos = descargar_todo("/api/productos")
clientes = descargar_todo("/api/clientes", page_size=40)
print(f"\n{len(productos)} productos, {len(clientes)} clientes")

```

```

/api/productos  página  1/2  (+50, total 50/90)
/api/productos  página  2/2  (+40, total 90/90)
/api/clientes   página  1/3  (+40, total 40/120)
/api/clientes   página  2/3  (+40, total 80/120)
/api/clientes   página  3/3  (+40, total 120/120)

```

90 productos, 120 clientes

Forma B — calcular las páginas de antemano

Se pide la primera página solo para leer `total`, y con eso ya sabemos cuántas peticiones faltan (útil para barras de progreso o para paralelizar).

```

PAGE_SIZE = 25
primera = sesion.get(
    f"{BASE}/api/productos", params={"page": 1, "pageSize": PAGE_SIZE},
    timeout=TIMEOUT
).json()

total = primera["pageInfo"]["total"]
total_paginas = math.ceil(total / PAGE_SIZE)
print(f"total={total}  pageSize={PAGE_SIZE}  ->  {total_paginas} páginas")

todos = list(primera["items"])
for page in range(2, total_paginas + 1):
    todos.extend(
        sesion.get(
            f"{BASE}/api/productos", params={"page": page, "pageSize": PAGE_SIZE},
            timeout=TIMEOUT
        ).json()["items"]
    )
print(f"descargados {len(todos)} productos en {total_paginas} peticiones")

```

```

total=90  pageSize=25  ->  4 páginas
descargados 90 productos en 4 peticiones

```

No todo se pagina: GET /api/comentarios

Algunos endpoints devuelven **todos** los registros que coinciden con el filtro, sin `pageInfo`. Aquí, todos los comentarios de un producto.

```

r = sesion.get(
    f"{BASE}/api/comentarios",
    params={"tipo": "producto", "productoId": 1},

```

```

        timeout=TIMEOUT,
    )
    r.raise_for_status()
    data = r.json()
    print("Claves de la respuesta:", list(data.keys()), "(sin pageInfo)")
    print(f"{data['total']} comentarios del producto 1")
    for c in data["items"][:3]:
        print(f"    {c['calificacion']}*   {c['clienteNombre']} {c['clienteApellidos']}: {c['texto'][:60]}...")

```

Claves de la respuesta: ['items', 'total'] (sin pageInfo)

4 comentarios del producto 1

4* Nicolas David Perez Salazar: Compre el iPhone 15 Pro Max para trabajo y la fluidez de iOS...

3* Daniela Fiorella Ochoa Vidal: Excelente camara pero el precio sigue siendo dificil de just...

2* Luciana Valentina Torres Mejia: El precio me parece excesivo para el salto de funciones fren...

Guardar en data/rest_productos.csv y data/rest_clientes.csv

```

ruta_productos = os.path.join(DATA_DIR, "rest_productos.csv")
with open(ruta_productos, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=["id", "codigo", "nombre", "categoria", "subcategoria", "precio", "stock", "rating_promedio", "total_resenas"],
    )
    writer.writeheader()
    for p in productos:
        ar = p["aggregateRating"] or {}
        writer.writerow({
            "id": p["id"], "codigo": p["codigo"], "nombre": p["nombre"],
            "categoria": p["categoria"], "subcategoria": p["subcategoria"],
            "precio": p["precio"], "stock": p["stock"],
            "rating_promedio": ar.get("ratingValue", ""),
            "total_resenas": ar.get("reviewCount", ""),
        })

ruta_clientes = os.path.join(DATA_DIR, "rest_clientes.csv")
with open(ruta_clientes, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(
        f, fieldnames=["id", "dni", "nombre", "apellidos", "email", "ciudad", "pais", "fecha_registro"]
    )
    writer.writeheader()
    for c in clientes:
        writer.writerow({k: c.get(k, "") for k in writer.fieldnames})

print("Guardado:", ruta_productos)
print("Guardado:", ruta_clientes)

```

Guardado: /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-03/notebooks/./data/rest_productos.csv

Guardado: /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-03/notebooks/./data/rest_clientes.csv

Hasta aquí la API key fue "un header más". En el siguiente notebook la **seguridad de la API** es el tema: qué responde ante credenciales ausentes o inválidas, qué son los *scopes* y cómo se revoca una clave.

03 · REST autenticado con API Key

Los notebooks 01 y 02 ya usaron el header `x-api-key`. Ahora la **autenticación es el tema**: qué pasa sin credencial o con una inválida, qué son los *scopes* y cómo se gestiona una clave.

La key demo `sk_demo_0000...` se re-siembra en cada arranque (`tienda-virtual/.env.example` → `DEMO_API_KEY`). Se generan y revocan más en el panel `/admin/api-keys` (requiere login; eso se ve en el notebook 07).

```
import csv
import os

import requests

BASE = "http://localhost:3000"
API_KEY = "sk_demo_00000000000000000000000000000000"
TIMEOUT = 30
```

Sin credencial: 401 unauthorized

El servidor rechaza la petición y explica en el cuerpo qué header espera.

```
r = requests.get(f"{BASE}/api/productos", timeout=TIMEOUT)
print("Sin API key ->", r.status_code, r.json())
```

Sin API key -> 401 {'error': 'unauthorized', 'message': "Envia el header 'x-api-key: <key>' o 'Authorization: Bearer <token OAuth>'."}

Con una key inválida: 401 invalid_api_key

Un `error` distinto: el servidor recibió una credencial pero no la reconoce (o fue revocada).

```
r = requests.get(
    f"{BASE}/api/productos",
    headers={"x-api-key": "sk_live_clave_que_no_existe_000000000000"},
    timeout=TIMEOUT,
)
print("Con key inválida ->", r.status_code, r.json())
```

Con key inválida -> 401 {'error': 'invalid_api_key', 'message': 'La API key no es valida o fue revocada.'}

Con la key demo válida

La cargamos en la `Session` y la hereda cada request.

```

sesion = requests.Session()
sesion.headers.update({"x-api-key": API_KEY, "User-Agent": "MineriaWeb-2026-2/1.0"})

r = sesion.get(f"{BASE}/api/testimonios", params={"page": 1, "pageSize": 3},
timeout=TIMEOUT)
r.raise_for_status()
print("OK ->", r.status_code, "| pageInfo:", r.json()["pageInfo"])

```

OK -> 200 | pageInfo: {'page': 1, 'pageSize': 3, 'total': 135, 'totalPages': 45}

Scopes: read vs write

Cada credencial lleva scopes. Los GET necesitan read ; las mutaciones (POST / PUT / DELETE) necesitan write . La key demo tiene **ambos**, así que puede crear y borrar. Una credencial de solo lectura recibiría 403 insufficient_scope en un POST . Para scraping basta read .

(La base de datos es SQLite en memoria y se re-siembr en cada arranque, así que este alta/baja de prueba no deja rastro.)

```

nuevo = sesion.post(
    f"{BASE}/api/productos",
    json={"codigo": "TMP-03", "nombre": "Producto de prueba (scope write)",
        "descripcion": "alta temporal", "precio": 1.0, "stock": 1},
    timeout=TIMEOUT,
)
print("POST ->", nuevo.status_code, "(201 = creado; el scope 'write' lo permite)")
creado_id = nuevo.json()["id"]

borrado = sesion.delete(f"{BASE}/api/productos/{creado_id}", timeout=TIMEOUT)
print("DELETE ->", borrado.status_code, borrado.json())

```

POST -> 201 (201 = creado; el scope 'write' lo permite)
DELETE -> 200 {'eliminado': True, 'id': 91}

Scraping con la key: todos los testimonios

Recorremos el listado paginado completo de /api/testimonios (comentarios generales de clientes).

```

def descargar_todo(path, page_size=50, **filtros):
    items, page = [], 1
    while True:
        payload = sesion.get(
            f"{BASE}/{path}", params={**filtros, "page": page, "pageSize": page_size},
            timeout=TIMEOUT
        ).json()
        items.extend(payload["items"])
        info = payload["pageInfo"]
        print(f"  página {info['page']}>2}/{info['totalPages']} (total
{len(items)}/{info['total']})")
        if info["page"] >= info["totalPages"]:
            return items
        page += 1

```

```
testimonios = descargar_todo("/api/testimonios")
print(f"\n{len(testimonios)} testimonios")
```

```
página 1/3 (total 50/135)
página 2/3 (total 100/135)
página 3/3 (total 135/135)
```

135 testimonios

Guardar en data/rest_testimonios.csv

```
DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)
OUTPUT = os.path.join(DATA_DIR, "rest_testimonios.csv")

with open(OUTPUT, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=["id", "cliente_id", "cliente_nombre", "cliente_ciudad",
"cliente_pais", "calificacion", "fecha", "texto"],
    )
    writer.writeheader()
    for t in testimonios:
        writer.writerow({
            "id": t["id"],
            "cliente_id": t["clienteId"],
            "cliente_nombre": f"{t['clienteNombre']} {t['clienteApellidos']}",
            "cliente_ciudad": t["clienteCiudad"],
            "cliente_pais": t["clientePais"],
            "calificacion": t["calificacion"],
            "fecha": t["fecha"],
            "texto": t["texto"],
        })

print(f"Guardadas {len(testimonios)} filas en {OUTPUT}")
```

Guardadas 135 filas en /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-03/notebooks/./data/rest_testimonios.csv

La API key es un secreto de **larga vida**: si se filtra, hay que revocarla a mano en el panel. Para integraciones máquina-a-máquina se prefiere OAuth2, con tokens de vida corta que caducan solos. Eso es el siguiente notebook.

04 · REST autenticado con OAuth2 (client_credentials)

OAuth2 `client_credentials` es el flujo **máquina-a-máquina** (sin usuario que autorice nada en un navegador):

1. `POST /api/oauth/token` con `client_id` + `client_secret` -> devuelve un `access_token` (tipo *Bearer*) que **expira** (`expires_in: 3600` segundos).
2. Cada petición a la API lleva `Authorization: Bearer <access_token>`.
3. Cuando el token caduca, se vuelve a pedir otro.

Cliente demo: `scraper-demo` / `scraper-demo-secret` (ver `tienda-virtual/.env.example`; se crean más en `/admin/oauth-clients`). Los scopes son `read` (para los `GET`) y `write` (para mutaciones); el cliente demo tiene ambos.

```
import csv
import os
import time

import requests

BASE = "http://localhost:3000"
CLIENT_ID = "scraper-demo"
CLIENT_SECRET = "scraper-demo-secret"
TIMEOUT = 30
```

Pedir un token

El endpoint acepta `application/x-www-form-urlencoded` (lo que hace `requests` con `data=`), `JSON` o `Authorization: Basic`.

```
r = requests.post(
    f"{BASE}/api/oauth/token",
    data={
        "grant_type": "client_credentials",
        "client_id": CLIENT_ID,
        "client_secret": CLIENT_SECRET,
    },
    timeout=TIMEOUT,
)
r.raise_for_status()
token_resp = r.json()
print({k: (v[:28] + "...") if k == "access_token" else v for k, v in
      token_resp.items()})

{'access_token': 'eyJhbGciOiJIUzI1NiJ9.eyJzY29...', 'token_type': 'Bearer',
 'expires_in': 3600, 'scope': 'read write'}
```

Credencial equivocada: 401 invalid_client

```
r = requests.post(
    f"{BASE}/api/oauth/token",
    data={
        "grant_type": "client_credentials",
        "client_id": CLIENT_ID,
        "client_secret": "secreto-incorrecto",
    },
    timeout=TIMEOUT,
)
print(r.status_code, r.json())
```

```
401 {'error': 'invalid_client', 'error_description': 'client_id o client_secret
invalidos.'}
```

Un cliente que renueva el token solo

`ClienteOAuth` pide el token la primera vez, lo cachea y lo renueva automáticamente unos segundos antes de que caduque. Así el bucle de scraping no se preocupa por la expiración.

```
class ClienteOAuth:
    def __init__(self, base, client_id, client_secret):
        self.base = base
        self.client_id = client_id
        self.client_secret = client_secret
        self.sesion = requests.Session()
        self.sesion.headers["User-Agent"] = "MineriaWeb-2026-2/1.0"
        self._token = None
        self._expira_en = 0.0

    def _renovar(self):
        r = requests.post(
            f"{self.base}/api/oauth/token",
            data={
                "grant_type": "client_credentials",
                "client_id": self.client_id,
                "client_secret": self.client_secret,
            },
            timeout=TIMEOUT,
        )
        r.raise_for_status()
        d = r.json()
        self._token = d["access_token"]
        # renovamos 60 s antes del vencimiento real, con margen de seguridad
        self._expira_en = time.monotonic() + d["expires_in"] - 60
        print(f" [token renovado, válido ~{d['expires_in']} s]")

    def _cabecera_auth(self):
        if self._token is None or time.monotonic() >= self._expira_en:
            self._renovar()
        return {"Authorization": f"Bearer {self._token}"}

    def get(self, path, **params):
        r = self.sesion.get(
            f"{self.base}{path}",
```

```

        headers=self._cabecera_auth(),
        params=params,
        timeout=TIMEOUT,
    )
    r.raise_for_status()
    return r.json()

```

```

api = ClienteOAuth(BASE, CLIENT_ID, CLIENT_SECRET)
print("Prueba:", api.get("/api/productos", page=1, pageSize=1)["pageInfo"])

```

[token renovado, válido ~3600 s]
 Prueba: {'page': 1, 'pageSize': 1, 'total': 90, 'totalPages': 90}

Scraping paginado con el token Bearer

Bajamos el listado de **órdenes**. Cada orden ya trae su **cliente** y sus **items** embebidos (con el **producto** de cada ítem).

```

def descargar_todo(path, page_size=50, **filtros):
    items = []
    page = 1
    while True:
        payload = api.get(path, page=page, pageSize=page_size, **filtros)
        items.extend(payload["items"])
        info = payload["pageInfo"]
        print(f" {path} página {info['page']}>2}/{info['totalPages']} (total
{len(items)}/{info['total']})")
        if info["page"] >= info["totalPages"]:
            return items
        page += 1

ordenes = descargar_todo("/api/ordenes")
print(f"\n{len(ordenes)} órdenes")

```

```

/api/ordenes página 1/6 (total 50/300)
/api/ordenes página 2/6 (total 100/300)
/api/ordenes página 3/6 (total 150/300)
/api/ordenes página 4/6 (total 200/300)
/api/ordenes página 5/6 (total 250/300)
/api/ordenes página 6/6 (total 300/300)

```

300 órdenes

Guardar en data/rest_ordenes.csv

```

DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)

ruta_ordenes = os.path.join(DATA_DIR, "rest_ordenes.csv")
with open(ruta_ordenes, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=["id", "numero", "fecha", "cliente_id", "cliente_nombre",
"cliente_pais", "num_items", "sub_total", "igv", "total"],
    )

```

```

writer.writeheader()
for o in ordenes:
    cli = o["cliente"]
    writer.writerow({
        "id": o["id"],
        "numero": o["numero"],
        "fecha": o["fecha"],
        "cliente_id": cli["id"],
        "cliente_nombre": f"{cli['nombre']} {cli['apellidos']}",
        "cliente_pais": cli["pais"],
        "num_items": len(o["items"]),
        "sub_total": o["subTotal"],
        "igv": o["igv"],
        "total": o["total"],
    })

print("Guardado:", ruta_ordenes)

```

Guardado: /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-03/notebooks/./data/rest_ordenes.csv

Recapitulando la parte REST: **01** primer contacto, **02** paginación, **03** la API key a fondo, **04** OAuth2. En los dos notebooks siguientes cambiamos de estilo de API: GraphQL.

05 · GraphQL: consultas básicas

GraphQL cambia el trato con el servidor:

- **Un solo endpoint:** `POST /api/graphql` (no una URL por recurso).
- **Tú describes la forma exacta** de la respuesta: pides los campos que quieres y los recibes anidados igual que en la query.
- En `tienda-virtual` es **solo lectura** (las escrituras van por REST) y usa la **misma autenticación dual** que el REST: `x-api-key` o `Authorization: Bearer`.

Detalle importante: en GraphQL un error de consulta llega con **HTTP 200** y un array `errors` en el cuerpo. Hay que revisarlo siempre.

```
import csv
import os

import requests

GQL = "http://localhost:3000/api/graphql"
HEADERS = {"x-api-key": "sk_demo_00000000000000000000000000000000"}
TIMEOUT = 30

def gql(query, variables=None):
    r = requests.post(
        GQL,
        json={"query": query, "variables": variables or {}},
        headers=HEADERS,
        timeout=TIMEOUT,
    )
    r.raise_for_status() # errores de transporte / auth (401)
    cuerpo = r.json()
    if "errors" in cuerpo: # errores de la propia query (llegan con HTTP 200)
        raise RuntimeError(cuerpo["errors"])
    return cuerpo["data"]
```

Una query plana: productos con su rating agregado

```
QUERY = """
{
  productos(page: 1, pageSize: 5) {
    items {
      id codigo nombre precio stock
      aggregateRating { ratingValue reviewCount }
    }
    pageInfo { page pageSize total totalPages }
  }
}
```

```
data = gql(QUERY)
print("pageInfo:", data["productos"]["pageInfo"])
for p in data["productos"]["items"]:
    ar = p["aggregateRating"]
    valor = f"{{ar['ratingValue']}} ({{ar['reviewCount']}})" if ar else "-"
    print(f"  [{p['id']:>2}] {p['nombre'][:38]:38s} S/ {p['precio']:>7} {valor}")
```

```
pageInfo: {'page': 1, 'pageSize': 5, 'total': 90, 'totalPages': 18}
[ 1] Apple iPhone 15 Pro Max 256GB      S/      5599  3.5 (4)
[ 2] Apple iPhone 15 128GB             S/      3799  3.3 (4)
[ 3] Samsung Galaxy S24 Ultra 512GB     S/      5299  3.3 (4)
[ 4] Samsung Galaxy A55 128GB          S/      1399  3 (4)
[ 5] Xiaomi Redmi Note 13 Pro 256GB     S/       999  2.8 (4)
```

Un campo que no existe: `errors` con HTTP 200

```
try:
    gql("{ productos { items { campoInexistente } } }")
except RuntimeError as e:
    print("GraphQL devolvió errores (y aun así HTTP 200):")
    for err in e.args[0]:
        print("  -", err["message"])
```

```
GraphQL devolvió errores (y aun así HTTP 200):
- Cannot query field "campoInexistente" on type "Producto".
```

Introspección: preguntarle al servidor qué queries ofrece

GraphQL se describe a sí mismo. Con `__schema` obtenemos la lista de consultas disponibles y sus argumentos (útil para explorar una API nueva).

```
INTROSPECCION = """
{
  __schema {
    queryType {
      fields { name args { name } }
    }
  }
}
"""

for f in gql(INTROSPECCION)["__schema"]["queryType"]["fields"]:
    args = ", ".join(a["name"] for a in f["args"])
    print(f"  {f['name']}({args})")
```

```
productos(page, pageSize)
producto(id)
clientes(page, pageSize)
cliente(id)
ordenes(page, pageSize, fecha, clienteId)
orden(id)
testimonios(page, pageSize)
comentarios(tipo, productoId, ordenId, clienteId)
```

Paginación con variables y guardado en `data/graphql_productos.csv`

Las **variables** (`$page` , `$pageSize`) evitan concatenar strings en la query.

```
QUERY_PAGINA = """
query Catalogo($page: Int!, $pageSize: Int!) {
  productos(page: $page, pageSize: $pageSize) {
    items {
      id codigo nombre categoria subcategoria precio stock
      aggregateRating { ratingValue reviewCount }
    }
    pageInfo { page totalPages total }
  }
}
"""

def todas_las_paginas(page_size=50):
    items = []
    page = 1
    while True:
        d = gql(QUERY_PAGINA, {"page": page, "pageSize": page_size})["productos"]
        items.extend(d["items"])
        info = d["pageInfo"]
        print(f"  página {info['page']}>2}/{info['totalPages']} (total
{len(items)}/{info['total']})")
        if info["page"] >= info["totalPages"]:
            return items
        page += 1

productos = todas_las_paginas()

DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)
OUTPUT = os.path.join(DATA_DIR, "graphql_productos.csv")
with open(OUTPUT, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=["id", "codigo", "nombre", "categoria", "subcategoria", "precio",
"stock", "rating_promedio", "total_resenas"],
    )
    writer.writeheader()
    for p in productos:
        ar = p["aggregateRating"] or {}
        writer.writerow({
            "id": p["id"], "codigo": p["codigo"], "nombre": p["nombre"],
            "categoria": p["categoria"], "subcategoria": p["subcategoria"],
            "precio": p["precio"], "stock": p["stock"],
            "rating_promedio": ar.get("ratingValue", ""),
            "total_resenas": ar.get("reviewCount", ""),
        })

print(f"\nGuardadas {len(productos)} filas en {OUTPUT}")
```

```
página 1/2 (total 50/90)
página 2/2 (total 90/90)
```

Guardadas 90 filas en /private/tmp/claude-501/Users/ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-03/notebooks/./data/graphql_productos.csv

En el siguiente notebook aprovechamos de verdad GraphQL: una sola query con datos **anidados** (órdenes -> cliente -> items -> producto) y autenticación por OAuth.

06 · GraphQL: consultas anidadas, variables y OAuth

Sumamos cuatro cosas sobre c1:

1. **Variables** tipadas (`$page: Int!`) en lugar de interpolar strings.
2. **Campos anidados**: cada orden con su `cliente` y sus `items` , y de cada ítem su `producto` .
En REST esto serían varias llamadas; aquí es **una sola** por página.
3. Autenticación con **OAuth Bearer** (reutilizamos el flujo de b2) en vez de API key.
4. Las **reseñas de entrega** (`comentarios(tipo: "post_compra")`) consultadas por orden.

```
import csv
import os

import requests

BASE = "http://localhost:3000"
GQL = f"{BASE}/api/graphql"
TIMEOUT = 30

# --- OAuth: obtener un access token (igual que en b2) ---
tok = requests.post(
    f"{BASE}/api/oauth/token",
    data={
        "grant_type": "client_credentials",
        "client_id": "scraper-demo",
        "client_secret": "scraper-demo-secret",
    },
    timeout=TIMEOUT,
).json()["access_token"]

HEADERS = {"Authorization": f"Bearer {tok}"}
print("Token OAuth:", tok[:28], "...")

def gql(query, variables=None):
    r = requests.post(
        GQL, json={"query": query, "variables": variables or {}}, headers=HEADERS,
        timeout=TIMEOUT
    )
    r.raise_for_status()
    cuerpo = r.json()
    if "errors" in cuerpo:
        raise RuntimeError(cuerpo["errors"])
    return cuerpo["data"]
```

Token OAuth: eyJhbGciOiJIUzI1NiJ9.eyJzY29 ...

Una query anidada con variables

ordenes -> items (lista) -> cada item con cliente e items { producto }. Traemos las 3 primeras para inspeccionar la forma de la respuesta.

```
QUERY_ORDENES = """
query PaginaOrdenes($page: Int!, $pageSize: Int!) {
  ordenes(page: $page, pageSize: $pageSize) {
    items {
      id numero fecha subTotal igv total
      cliente { id nombre apellidos ciudad pais }
      items {
        productoId cantidad subTotal
        producto { codigo nombre categoria }
      }
    }
  }
  pageInfo { page totalPages total }
}
"""

primera = gql(QUERY_ORDENES, {"page": 1, "pageSize": 3})["ordenes"]
print("pageInfo:", primera["pageInfo"])

o = primera["items"][0]
print(f"\nOrden {o['numero']} ({o['fecha']})")
print(f" Cliente: {o['cliente']['nombre']} {o['cliente']['apellidos']} - {o['cliente']['ciudad']}, {o['cliente']['pais']}")
print(f" Total: S/ {o['total']}")
for it in o["items"]:
    print(f"      {it['cantidad']:>2} x {it['producto']['nombre'][:40]:40s} S/ {it['subTotal']}")
```

pageInfo: {'page': 1, 'totalPages': 100, 'total': 300}

Orden OR-00300 (2026-08-10)

Cliente: Mateo Sebastian Dominguez Silva - Ciudad de Mexico, Mexico

Total: S/ 70311.48

4 x DJI Air 3	S/ 27996
1 x Samsung T7 SSD Portatil 1TB	S/ 399
2 x Dell XPS 13 9340 Intel Core i7	S/ 12998
3 x LG OLED55C3 55 pulgadas	S/ 14997
4 x SteelSeries Arctis Nova 7	S/ 3196

Recorrer todas las páginas de órdenes

```
def todas_las_ordenes(page_size=50):
    items = []
    page = 1
    while True:
        d = gql(QUERY_ORDENES, {"page": page, "pageSize": page_size})["ordenes"]
        items.extend(d["items"])
        info = d["pageInfo"]
        print(f" página {info['page']:>2}/{info['totalPages']} (total {len(items)}/{info['total']})")
        if info["page"] >= info["totalPages"]:
            return items
        page += 1
```

```
ordenes = todas_las_ordenes()
print(f"\n{len(ordenes)} órdenes")
```

```
página 1/6 (total 50/300)
página 2/6 (total 100/300)
página 3/6 (total 150/300)
página 4/6 (total 200/300)
página 5/6 (total 250/300)
página 6/6 (total 300/300)
```

300 órdenes

Reseñas de entrega por orden

`comentarios(tipo: "post_compra", ordenId: N)` devuelve las reseñas sobre la entrega de esa orden. No todas las órdenes tienen: iteramos todas y nos quedamos con las que sí traen reseñas.

```
QUERY_RESENAS = """
query ResenasEntrega($ordenId: Int!) {
  comentarios(tipo: "post_compra", ordenId: $ordenId) {
    id clienteId productoId calificacion texto fecha
    clienteNombre clienteApellidos
  }
}
"""

resenas = []
for o in ordenes:
    for c in gql(QUERY_RESENAS, {"ordenId": o["id"]})["comentarios"]:
        resenas.append({
            "id": c["id"],
            "orden_id": o["id"],
            "orden_numero": o["numero"],
            "cliente_id": c["clienteId"],
            "cliente_nombre": f"{c['clienteNombre']} {c['clienteApellidos']}",
            "producto_id": c["productoId"],
            "calificacion": c["calificacion"],
            "fecha": c["fecha"],
            "texto": c["texto"],
        })

ordenes_con_resena = len({r["orden_id"] for r in resenas})
print(f"{len(resenas)} reseñas de entrega, repartidas en {ordenes_con_resena} de {len(ordenes)} órdenes")
```

110 reseñas de entrega, repartidas en 74 de 300 órdenes

Guardar en `data/graphql_ordenes.csv` y `data/graphql_resenas_entrega.csv`

```
DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)

ruta_ordenes = os.path.join(DATA_DIR, "graphql_ordenes.csv")
with open(ruta_ordenes, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(
```

```

        f,
        fieldnames=["id", "numero", "fecha", "cliente_id", "cliente_nombre",
"cliente_pais", "num_items", "sub_total", "igv", "total"],
    )
    writer.writeheader()
    for o in ordenes:
        cli = o["cliente"]
        writer.writerow({
            "id": o["id"], "numero": o["numero"], "fecha": o["fecha"],
            "cliente_id": cli["id"], "cliente_nombre": f"{cli['nombre']}
{cli['apellidos']}",
            "cliente_pais": cli["pais"], "num_items": len(o["items"]),
            "sub_total": o["subTotal"], "igv": o["igv"], "total": o["total"],
        })

ruta_resenas = os.path.join(DATA_DIR, "graphql_resenas_entrega.csv")
with open(ruta_resenas, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=list(resenas[0].keys()))
    writer.writeheader()
    writer.writerows(resenas)

print("Guardado:", ruta_ordenes)
print("Guardado:", ruta_resenas)

```

Guardado: /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-03/notebooks/./data/graphql_ordenes.csv

Guardado: /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-03/notebooks/./data/graphql_resenas_entrega.csv

Una sola query anidada reemplazó decenas de llamadas REST. Falta el caso en que **no hay API** y la página exige iniciar sesión: eso es el último notebook, con Playwright.

07 · Playwright: páginas protegidas con login

A veces no hay API y la información vive en páginas que **exigen login**. En `tienda-virtual`, `proxy.ts` protege `/clientes`, `/ordenes`, `/carrito` y `/admin/*`: sin cookie de sesión válida, cualquier petición redirige (**HTTP 307**) a `/login`.

Con `Playwright` automatizamos un navegador real: abrimos `/login`, completamos el formulario (`admin@tienda.local` / `admin123`) y a partir de ahí el navegador conserva la cookie de sesión. Guardamos el `storage_state` para reutilizar la sesión sin volver a logear.

Requisitos: `tienda-virtual` corriendo y `playwright install chromium` ya ejecutado.

```
import csv
import os

import requests
from playwright.async_api import async_playwright

BASE = "http://localhost:3000"
USUARIO = "admin@tienda.local"
CLAVE = "admin123"
UA = "MineriaWeb-2026-2/1.0 (scraping con login)"

# Sin sesión, /ordenes redirige a /login (no seguimos el redirect para verlo):
r = requests.get(f"{BASE}/ordenes", allow_redirects=False, timeout=15)
print("GET /ordenes sin sesión ->", r.status_code, "->", r.headers.get("location"))
```

GET /ordenes sin sesión -> 307 -> /login?next=%2Fordenes

Iniciar sesión con Playwright

Rellenamos el formulario por el atributo `name` de cada input y esperamos a que el `submit` nos deje en la home ya autenticados.

```
playwright = await async_playwright().start()
navegador = await playwright.chromium.launch(headless=True)
contexto = await navegador.new_context(user_agent=UA)
pagina = await contexto.new_page()

await pagina.goto(f"{BASE}/login", wait_until="networkidle")
await pagina.fill("input[name='email']", USUARIO)
await pagina.fill("input[name='password']", CLAVE)
await pagina.click("button[type='submit']")
await pagina.wait_for_url(f"{BASE}/", wait_until="networkidle")

print("URL tras el login:", pagina.url)
cookies = await contexto.cookies()
print("Cookies presentes:", [c["name"] for c in cookies])
```

URL tras el login: `http://localhost:3000/`
Cookies presentes: `['tienda_session']`

Guardar la sesión (`storage_state`) para reutilizarla

```
DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)
STATE_PATH = os.path.join(DATA_DIR, "playwright_storage_state.json")
```

```
await contexto.storage_state(path=STATE_PATH)
print("Sesión guardada en", STATE_PATH)
```

Sesión guardada en /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-03/notebooks/./data/playwright_storage_state.json

Recorrer el listado paginado protegido `/ordenes`

Ya con sesión, `/ordenes` responde con HTML. Cada `<article data-orden-id>` puede traer reseñas de entrega embebidas como `<li data-tipo="post_compra">` con sus datos en atributos `data-*`. Paramos cuando el enlace **Siguiente** deja de ser un `<a>`.

```
resenas = []
numero_pagina = 1
while True:
    await pagina.goto(f"{BASE}/ordenes?page={numero_pagina}",
wait_until="networkidle")

    articulos = pagina.locator("article[data-orden-id]")
    n = await articulos.count()
    if n == 0:
        break

    for i in range(n):
        art = articulos.nth(i)
        orden_id = await art.get_attribute("data-orden-id")
        li = art.locator("li[data-tipo='post_compra']")
        for j in range(await li.count()):
            fila = li.nth(j)
            resenas.append({
                "id": await fila.get_attribute("data-comentario-id"),
                "orden_id": orden_id,
                "cliente_id": await fila.get_attribute("data-cliente-id"),
                "producto_id": await fila.get_attribute("data-producto-id"),
                "calificacion": await fila.get_attribute("data-calificacion"),
                "fecha": await fila.get_attribute("data-fecha"),
                "texto": (await fila.locator("
[itemprop='reviewBody']").inner_text()).strip(),
            })

        print(f"  página {numero_pagina:>2}: {n} órdenes")
        siguiente = pagina.locator('nav[aria-label="Paginacion"] a:has-
text("Siguiente")')
        if await siguiente.count() == 0:
            break
        numero_pagina += 1

print(f"\n{len(resenas)} reseñas de entrega en {numero_pagina} páginas de órdenes")
```

página 1: 10 órdenes
página 2: 10 órdenes
página 3: 10 órdenes
página 4: 10 órdenes
página 5: 10 órdenes
página 6: 10 órdenes
página 7: 10 órdenes
página 8: 10 órdenes
página 9: 10 órdenes
página 10: 10 órdenes
página 11: 10 órdenes
página 12: 10 órdenes
página 13: 10 órdenes
página 14: 10 órdenes
página 15: 10 órdenes
página 16: 10 órdenes
página 17: 10 órdenes
página 18: 10 órdenes
página 19: 10 órdenes
página 20: 10 órdenes
página 21: 10 órdenes
página 22: 10 órdenes
página 23: 10 órdenes
página 24: 10 órdenes
página 25: 10 órdenes
página 26: 10 órdenes
página 27: 10 órdenes
página 28: 10 órdenes
página 29: 10 órdenes
página 30: 10 órdenes

110 reseñas de entrega en 30 páginas de órdenes

Reutilizar la sesión guardada

Un contexto nuevo creado con `storage_state=STATE_PATH` entra directo a una página protegida (`/admin/api-keys`) sin volver a pasar por el login.

```
contexto2 = await navegador.new_context(storage_state=STATE_PATH, user_agent=UA)
pagina2 = await contexto2.new_page()
await pagina2.goto(f"{BASE}/admin/api-keys", wait_until="networkidle")

print("URL:", pagina2.url, "(no redirige a /login)")
print("Título de la página protegida:", await
pagina2.locator("h2").first.inner_text())
print("Filas en la tabla de API keys:", await pagina2.locator("table tbody
tr").count())

await contexto2.close()
```

URL: `http://localhost:3000/admin/api-keys` (no redirige a `/login`)

Título de la página protegida: API keys

Filas en la tabla de API keys: 1

Guardar en `data/playwright_resenas_entrega.csv`
y cerrar el navegador

```
OUTPUT = os.path.join(DATA_DIR, "playwright_resenas_entrega.csv")
with open(OUTPUT, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=list(resenas[0].keys()))
    writer.writeheader()
    writer.writerows(resenas)

await navegador.close()
await playwright.stop()

print(f"Guardadas {len(resenas)} filas en {OUTPUT}")
```

Guardadas 110 filas en /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-03/notebooks/./data/playwright_resenas_entrega.csv

Resumen de la sesión: **a)** REST público (recurso, colección, paginación) -> **b)** REST con seguridad (API key, luego OAuth2) -> **c)** GraphQL (query simple, luego variables + anidado + OAuth) -> **d)** sin API, navegador real con login.

sesion-de-clase-04/README.md

Proyecto de Minería Web - Sesión de clase 04

Esta sesión trata la **minería de texto y el Procesamiento del Lenguaje Natural**: tokenización, normalización, n-gramas y las representaciones Bag of Words y TF-IDF.

Los notebooks no scrapean: trabajan sobre los CSV de texto de la tienda-virtual que ya están **copiados en la carpeta `datos/`** de este proyecto.

Instalación de dependencias

Requiere **Python 3.8+**.

```
pip install -r requirements.txt
python -m spacy download es_core_news_sm
```

Los recursos de NLTK (`punkt` , `punkt_tab` , `stopwords`) se descargan solos la primera vez desde cada notebook con `nltk.download(...)` .

Datos (`datos/`)

Archivo	Contenido	Uso
<code>resenas_entrega.csv</code>	Reseñas de entrega (<code>post_compra</code>): <code>texto</code> , <code>calificacion</code> , <code>producto_id</code> , ...	Corpus para tokenización, n-gramas y el pipeline completo
<code>comentarios.csv</code>	Testimonios / comentarios de clientes: <code>texto</code> , <code>calificacion</code> , <code>ciudad</code> , <code>país</code> , ...	Corpus para el notebook de normalización
<code>productos.csv</code>	Catálogo: <code>nombre</code> , <code>descripcion</code> , <code>categoria</code> , ...	Las descripciones son los "documentos" para Bag of Words y TF-IDF

Ejercicios (`notebooks/`)

Se recomienda ejecutarlos en este orden; cada uno reutiliza ideas del anterior.

Notebook	Técnica	Qué hace
<code>01_tokenizacion.ipynb</code>	NLTK (<code>sent_tokenize</code> , <code>word_tokenize</code>) + <code>spacy</code>	Divide las reseñas de entrega en frases y palabras; muestra el vocabulario crudo y los atributos lingüísticos por token (POS, lema, stopwords). Introduce la tokenización en subpalabras (BPE / WordPiece).

Notebook	Técnica	Qué hace
02_normalizacion.ipynb	re, NLTK stopwords, SnowballStemmer, spaCy	Sobre los comentarios de clientes: minúsculas, quita puntuación y stopwords, y compara stemming (Snowball) con lematización (spaCy). Deja una función <code>normalizar()</code> reutilizable.
03_ngram.ipynb	nltk.ngrams, CountVectorizer(ngram_range=...)	Unigramas, bigramas y trigramas de las reseñas; n-gramas de contenido más frecuentes; estimación de probabilidad $P(w_i \setminus w_{i-1})$ y un pequeño modelo generativo de n-gramas.
04_bow.ipynb	sklearn.CountVectorizer	Bag of Words sobre las 90 descripciones de producto: vocabulario, matriz de conteos, dispersión (<i>sparsity</i>), palabras frecuentes por categoría y la limitación de ignorar el orden.
05_tf-idf.ipynb	sklearn.TfidfVectorizer	TF, IDF y TF-IDF sobre las mismas descripciones: cálculo de IDF a mano (incluido el caso $IDF = 0$), términos más distintivos por producto y comparación común vs. distintivo frente a BoW.
06_mineria-de-texto.ipynb	Pipeline completo + cosine_similarity	De extremo a extremo sobre las reseñas de entrega: exploración → tokenización + normalización → TF-IDF → análisis (términos que distinguen reseñas negativas de positivas, reseñas similares por coseno, bigramas por sentimiento). Cierra con un ejercicio de clasificación propuesto.

Cada notebook es autocontenido: se puede ejecutar de principio a fin (`Run All`).

01 · Tokenización

Primer paso del pipeline de minería de texto: dividir un texto en unidades más pequeñas llamadas *tokens* (frases, palabras o subpalabras). Todo lo que viene después —normalización, Bag of Words, TF-IDF— opera sobre estos tokens.

Corpus: las **reseñas de entrega** (`post_compra`) de la tienda-virtual, en `datos/resenas_entrega.csv` .

```
from pathlib import Path

import pandas as pd

# El texto ya minado de la tienda-virtual está copiado en la carpeta datos/ de
# este mismo proyecto:
# datos/resenas_entrega.csv   reseñas de entrega (post_compra)
# datos/comentarios.csv      testimonios / comentarios de clientes
# datos/productos.csv        catálogo con la descripción de cada producto
DATOS = Path("../datos")
```

```
def cargar(nombre, **kwargs):
    """Lee un CSV de la carpeta datos/ y lo devuelve como DataFrame."""
    ruta = DATOS / nombre
    if not ruta.exists():
        raise FileNotFoundError(f"No se encontró {ruta.resolve()}")
    print(f"Leyendo {ruta} ({ruta.stat().st_size / 1024:.1f} KB)")
    return pd.read_csv(ruta, **kwargs)
```

```
resenas = cargar("resenas_entrega.csv")
print(f"{len(resenas)} reseñas")
resenas[["texto"]].head(5)
```

```
Leyendo ../datos/resenas_entrega.csv (15.0 KB)
110 reseñas
```

texto

- | | texto |
|---|---|
| 0 | Recibi el pedido incompleto, faltaba el cable ... |
| 1 | El segundo cargador Belkin lleo sin contratie... |
| 2 | La segunda Switch Lite que regale lleo perfec... |
| 3 | El segundo television TCL lleo perfecto, bien... |
| 4 | Me confirmaron por correo la garantia de un an... |

```
# El corpus con el que trabajaremos: la columna de texto libre
corpus = resenas[["texto"]].dropna().astype(str).tolist()
print(f"{len(corpus)} documentos")
print("Ejemplo:", corpus[0])
```

110 documentos

Ejemplo: Recibi el pedido incompleto, faltaba el cable de carga original del OnePlus 12 dentro de la caja. Tuve que esperar un envio adicional solo para recibir ese accesorio.

Tokenización por frases (NLTK)

`nltk.sent_tokenize` parte un texto en oraciones usando un modelo entrenado para el idioma (puntos, signos, abreviaturas). Tomamos la reseña más larga del corpus para ver varias frases.

```
import nltk

nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)

texto_largo = max(corpus, key=len)
frases = nltk.sent_tokenize(texto_largo, language="spanish")

print(texto_largo, "\n")
for i, f in enumerate(frases, 1):
    print(f"Frase {i}: {f}")
```

El SSD Samsung T7 llego con el cable USB-C incorrecto dentro de la caja, uno que no era compatible con el puerto del disco. Tuve que comprar un cable aparte mientras resolvian el reclamo.

Frase 1: El SSD Samsung T7 llego con el cable USB-C incorrecto dentro de la caja, uno que no era compatible con el puerto del disco.

Frase 2: Tuve que comprar un cable aparte mientras resolvian el reclamo.

Tokenización por palabras (NLTK)

`nltk.word_tokenize` separa cada frase en palabras y signos de puntuación como tokens independientes.

```
tokens_por_frase = [nltk.word_tokenize(f, language="spanish") for f in frases]
for i, toks in enumerate(tokens_por_frase, 1):
    print(f"Frase {i}: {toks}")
```

Frase 1: ['El', 'SSD', 'Samsung', 'T7', 'llego', 'con', 'el', 'cable', 'USB-C', 'incorrecto', 'dentro', 'de', 'la', 'caja', ',', 'uno', 'que', 'no', 'era', 'compatible', 'con', 'el', 'puerto', 'del', 'disco', '.']

Frase 2: ['Tuve', 'que', 'comprar', 'un', 'cable', 'aparte', 'mientras', 'resolvian', 'el', 'reclamo', '.']

```
from collections import Counter

# Tokenizamos TODO el corpus y miramos el vocabulario
todos = [t.lower() for texto in corpus for t in nltk.word_tokenize(texto,
language="spanish")]
vocabulario = sorted(set(todos))

print(f"Tokens totales : {len(todos)}")
print(f"Vocabulario : {len(vocabulario)} tokens únicos")
print(f"\n20 tokens más frecuentes (todavía sin normalizar):")
for palabra, n in Counter(todos).most_common(20):
    print(f" {palabra!r:15} {n}")
```

Tokens totales : 2322
Vocabulario : 609 tokens únicos

20 tokens más frecuentes (todavía sin normalizar):

'.'	146
'el'	136
'la'	108
'de'	105
'llego'	64
'con'	59
'en'	53
','	48
'que'	47
'un'	34
'a'	27
'sin'	25
'y'	25
'del'	22
'se'	22
'caja'	18
'por'	18
'bien'	17
'fue'	16
'no'	16

Casi todos los tokens más frecuentes son signos de puntuación y *stopwords* (`el` , `de` , `la` , `y` ...).
Ese ruido es exactamente lo que elimina el paso de **normalización** (`02_normalizacion.ipynb`).

spaCy: tokenización con información lingüística

spaCy tokeniza y, en la misma pasada, anota cada token: categoría gramatical (`pos_`), lema, si es puntuación, si es *stopword*, etc.

```
import spacy

nlp = spacy.load("es_core_news_sm")

doc = nlp(corpus[0])
tabla = pd.DataFrame(
    {
        "token": [t.text for t in doc],
        "lema": [t.lemma_ for t in doc],
        "pos": [t.pos_ for t in doc],
        "es_stopword": [t.is_stop for t in doc],
        "es_puntuacion": [t.is_punct for t in doc],
    }
)
tabla.head(20)
```

	token	lema	pos	es_stopword	es_puntuacion
0	Recibi	Recibi	PROPN	False	False
1	el	el	DET	True	False
2	pedido	pedido	NOUN	False	False
3	incompleto	incompleto	NOUN	False	False
4	,	,	PUNCT	False	True
5	faltaba	faltar	VERB	False	False
6	el	el	DET	True	False
7	cable	cable	NOUN	False	False
8	de	de	ADP	True	False
9	carga	carga	NOUN	False	False
10	original	original	ADJ	False	False
11	del	del	ADP	True	False
12	OnePlus	OnePlus	PROPN	False	False
13	12	12	NUM	False	False
14	dentro	dentro	ADV	True	False
15	de	de	ADP	True	False
16	la	el	DET	True	False
17	caja	caja	NOUN	False	False
18	.	.	PUNCT	False	True
19	Tuve	tener	VERB	False	False

Subpalabras (BPE / WordPiece / SentencePiece)

Los modelos basados en Transformers (GPT, BERT, T5) no tokenizan por palabra sino por **subpalabra**: fragmentos frecuentes que permiten representar términos fuera de vocabulario combinando piezas conocidas. Un tokenizador de palabras trata "OnePlus", "iPhone" o "repuesto" como unidades atómicas; uno de subpalabras las parte en One, Plus, ... y nunca se queda sin representación.

En este curso trabajamos a nivel de palabra (NLTK / spaCy), suficiente para Bag of Words y TF-IDF.

Siguiente: `02_normalizacion.ipynb` — reducir estos tokens a una forma canónica (minúsculas, sin signos, sin stopwords, lematizados).

02 · Normalización

Normalizar es uniformizar el texto para reducir variaciones que no aportan significado:

1. *lowercasing* → todo a minúsculas
2. eliminación de puntuación y símbolos
3. *stopwords removal* → quitar palabras muy comunes
4. *stemming* o *lematización* → llevar cada palabra a su forma base

Corpus: los **testimonios / comentarios** de clientes de la tienda-virtual, en `datos/comentarios.csv`.

```
from pathlib import Path

import pandas as pd

# El texto ya minado de la tienda-virtual está copiado en la carpeta datos/ de
# este mismo proyecto:
#  datos/resenas_entrega.csv   reseñas de entrega (post_compra)
#  datos/comentarios.csv      testimonios / comentarios de clientes
#  datos/productos.csv        catálogo con la descripción de cada producto
DATOS = Path("../datos")

def cargar(nombre, **kwargs):
    """Lee un CSV de la carpeta datos/ y lo devuelve como DataFrame."""
    ruta = DATOS / nombre
    if not ruta.exists():
        raise FileNotFoundError(f"No se encontró {ruta.resolve()}")
    print(f"Leyendo {ruta} ({ruta.stat().st_size / 1024:.1f} KB)")
    return pd.read_csv(ruta, **kwargs)
```

```
comentarios = cargar("comentarios.csv")
corpus = comentarios["texto"].dropna().astype(str).tolist()
print(f"{len(corpus)} comentarios")
comentarios[["calificacion", "texto"]].head(5)
```

Leyendo ../datos/comentarios.csv (22.4 KB)
135 comentarios

	calificacion	texto
0	5	El proceso de facturacion para empresa fue mas...
1	3	En general cumple lo que promete, ni mejor ni ...
2	4	Buena tienda en general, seguire comprando aqu...
3	3	El envio internacional tardo mas dias de los i...
4	3	El proceso de devolucion fue sencillo de inici...

Paso a paso sobre un comentario

```
import re

original = corpus[3]
print("0. original      :", original)

minuscula = original.lower()
print("1. minúsculas    :", minuscula)

sin_signos = re.sub(r"^[^\w\s]", " ", minuscula)
sin_signos = re.sub(r"\s+", " ", sin_signos).strip()
print("2. sin puntuación :", sin_signos)
```

0. original : El envío internacional tardo mas dias de los indicados originalmente.
1. minúsculas : el envío internacional tardo mas dias de los indicados originalmente.
2. sin puntuación : el envío internacional tardo mas dias de los indicados originalmente

Stopwords

NLTK trae una lista de *stopwords* en español. Quitarlas reduce el ruido, aunque en tareas de estilo o búsqueda exacta de frases conviene conservarlas.

```
import nltk

nltk.download("stopwords", quiet=True)
stop_es = set(nltk.corpus.stopwords.words("spanish"))
print(f"{len(stop_es)} stopwords en español. Ejemplos: {sorted(list(stop_es))[:15]}")

tokens = sin_signos.split()
sin_stop = [t for t in tokens if t not in stop_es]
quitadas = [t for t in tokens if t in stop_es]

print("\n3. sin stopwords      :", sin_stop)
print("    (se quitaron)      :", quitadas)
```

313 stopwords en español. Ejemplos: ['a', 'al', 'algo', 'algunas', 'algunos', 'ante', 'antes', 'como', 'con', 'contra', 'cual', 'cuando', 'de', 'del', 'desde']

3. sin stopwords : ['envío', 'internacional', 'tardo', 'mas', 'dias', 'indicados', 'originalmente']
 (se quitaron) : ['el', 'de', 'los']

Stemming (Snowball) vs Lematización (spaCy)

- **Stemming**: recorta la palabra a una *raíz* con reglas mecánicas. Rápido, pero la raíz puede no ser una palabra real (entrega → entreg).
- **Lematización**: devuelve el *lema* (forma de diccionario) usando contexto y categoría gramatical (entregados → entregar).

```

import spacy
from nltk.stem import SnowballStemmer

nlp = spacy.load("es_core_news_sm")
stemmer = SnowballStemmer("spanish")

palabras = ["llegó", "llegaron", "llegando", "entrega", "entregado", "entregaron",
            "rápido", "rápidamente", "comprando", "compré", "productos",
            "devolución"]

comparacion = pd.DataFrame(
    {
        "palabra": palabras,
        "stem (Snowball)": [stemmer.stem(p) for p in palabras],
        "lema (spaCy)": [nlp(p)[0].lemma_ for p in palabras],
    }
)
comparacion

```

	palabra	stem (Snowball)	lema (spaCy)
0	llegó	lleg	llegar
1	llegaron	lleg	llegar
2	llegando	lleg	llegar
3	entrega	entreg	entrega
4	entregado	entreg	entregado
5	entregaron	entreg	entregar
6	rápido	rap	rápido
7	rápidamente	rapid	rápidamente
8	comprando	compr	comprar
9	compré	compr	comprar
10	productos	product	producto
11	devolución	devolu	devolución

Pipeline de normalización completo

Combinamos todos los pasos en una función reutilizable. La usaremos también en los notebooks de Bag of Words, TF-IDF y minería de texto.

```

import re
import unicodedata

import nltk
import spacy

nlp = spacy.load("es_core_news_sm")
STOPWORDS = set(nltk.corpus.stopwords.words("spanish"))

```

```

def _limpiar(texto):
    # NFKC junta tildes combinantes sueltas; luego minúsculas y solo letras/espacios
    texto = unicodedata.normalize("NFKC", str(texto)).lower()
    return re.sub(r"^\w\s]", " ", texto)

def _lemas(doc):
    return [
        t.lemma_.lower()
        for t in doc
        if t.is_alpha and not t.is_stop and t.lemma_.lower() not in STOPWORDS and
len(t.lemma_) > 2
    ]

def normalizar(texto):
    """minúsculas -> sin signos -> sin stopwords -> lematizado. Devuelve un str."""
    return " ".join(_lemas(nlp(_limpiar(texto))))

def normalizar_muchos(textos):
    """Igual que normalizar() pero en lote con nlp.pipe (más rápido para un
corpus)."""
    return [" ".join(_lemas(doc)) for doc in nlp.pipe([_limpiar(t) for t in textos],
batch_size=64)]

```

```

muestra = pd.DataFrame(
    {
        "original": corpus[:6],
        "normalizado": normalizar_muchos(corpus[:6]),
    }
)
pd.set_option("display.max_colwidth", 90)
print(muestra)

```

```

original \
0          El proceso de facturacion para empresa fue mas simple de lo que
esperaba.
1  En general cumple lo que promete, ni mejor ni peor que otras tiendas de tecnologia
que...
2          Buena tienda en general, seguire comprando aqui mis proximos
equipos.
3          El envio internacional tardo mas dias de los indicados
originalmente.
4  El proceso de devolucion fue sencillo de iniciar aunque tomo unos dias en
confirmarse.
5  Muy buena atencion personalizada cuando pregunte por disponibilidad de un modelo
agotado.

```

```

                                normalizado
0          proceso facturacion empresa simple esperar
1          general cumplir prometer tienda tecnologia probar
2          tienda general seguire comprar proximo equipo
3          envio internacional tardo indicado originalmente
4          proceso devolucion sencillo iniciar tomo confirmar él
5  atencion personalizado pregunte disponibilidad modelo agotado

```

Siguiente: `03_ngram.ipynb` — antes de descartar el orden por completo (Bag of Words), veamos cómo los n-gramas capturan el contexto local.

03 · N-gramas

Un **n-grama** es una secuencia de n palabras consecutivas. A diferencia de Bag of Words, los n-gramas **sí conservan el orden local**:

- **unigrama** ($n=1$): palabras sueltas
- **bigrama** ($n=2$): pares consecutivos
- **trigrama** ($n=3$): tríos consecutivos

Corpus: las **reseñas de entrega**, en `datos/resenas_entrega.csv`.

```
from pathlib import Path

import pandas as pd

# El texto ya minado de la tienda-virtual está copiado en la carpeta datos/ de
# este mismo proyecto:
# datos/resenas_entrega.csv   reseñas de entrega (post_compra)
# datos/comentarios.csv      testimonios / comentarios de clientes
# datos/productos.csv        catálogo con la descripción de cada producto
DATOS = Path("../datos")

def cargar(nombre, **kwargs):
    """Lee un CSV de la carpeta datos/ y lo devuelve como DataFrame."""
    ruta = DATOS / nombre
    if not ruta.exists():
        raise FileNotFoundError(f"No se encontró {ruta.resolve()}")
    print(f"Leyendo {ruta} ({ruta.stat().st_size / 1024:.1f} KB)")
    return pd.read_csv(ruta, **kwargs)
```

```
import re

resenas = cargar("resenas_entrega.csv")
corpus = resenas["texto"].dropna().astype(str).tolist()

def a_tokens(texto):
    # normalización ligera: minúsculas + solo palabras. Conservamos stopwords
    # porque para n-gramas el contexto ("llegó a tiempo") es justamente lo valioso.
    return re.findall(r"[a-záéíóúñü]+", texto.lower())

print(f"{len(corpus)} reseñas")
print("Ejemplo de tokens:", a_tokens(corpus[0])[:15])
```

```
Leyendo ../datos/resenas_entrega.csv (15.0 KB)
110 reseñas
Ejemplo de tokens: ['recibi', 'el', 'pedido', 'incompleto', 'faltaba', 'el', 'cable',
'de', 'carga', 'original', 'del', 'oneplus', 'dentro', 'de', 'la']
```

Extraer n-gramas con NLTK

`nltk.ngrams` genera las secuencias. Lo vemos sobre una frase real del corpus.

```
from nltk import ngrams

frase = a_tokens(corpus[2])
print("frase:", " ".join(frase), "\n")
for n in (1, 2, 3):
    grams = list(ngrams(frase, n))
    print(f"n={n} -> {len(grams)} {'unigramas', 'bigramas', 'trigramas'}[n-1]}")
    print("    ", grams[:6])
```

frase: la segunda switch lite que regale llego perfecta y justo a tiempo para el cumpleaños que la necesitaba toda la experiencia de compra fue muy fluida

```
n=1 -> 26 unigramas
[('la',), ('segunda',), ('switch',), ('lite',), ('que',), ('regale',)]]
n=2 -> 25 bigramas
[('la', 'segunda'), ('segunda', 'switch'), ('switch', 'lite'), ('lite', 'que'), ('que', 'regale'), ('regale', 'llego')]
n=3 -> 24 trigramas
[('la', 'segunda', 'switch'), ('segunda', 'switch', 'lite'), ('switch', 'lite', 'que'), ('lite', 'que', 'regale'), ('que', 'regale', 'llego'), ('regale', 'llego', 'perfecta')]
```

N-gramas más frecuentes en el corpus de reseñas

Con *stopwords* dentro, los bigramas más comunes son de relleno (`de la` , `con el`). Filtrando los n-gramas que contienen alguna *stopword* aparece el vocabulario **informativo** del corpus (`fecha estimada` , `dia retraso` , `cable carga`).

```
from collections import Counter

import nltk

nltk.download("stopwords", quiet=True)
stop_es = set(nltk.corpus.stopwords.words("spanish"))

def top_ngramas(n, solo_contenido, k=12):
    cont = Counter()
    for texto in corpus:
        for g in ngrams(a_tokens(texto), n):
            if solo_contenido and any(w in stop_es for w in g):
                continue
            cont[g] += 1
    return cont.most_common(k)

print("Bigramas crudos (con stopwords):")
for g, c in top_ngramas(2, solo_contenido=False):
    print(f"    {' '.join(g):28} {c}")

print("\nBigramas de contenido (sin stopwords):")
for g, c in top_ngramas(2, solo_contenido=True):
    print(f"    {' '.join(g):28} {c}")

print("\nTrigramas de contenido (sin stopwords):")
```

```
for g, c in top_ngramas(3, solo_contenido=True):
    print(f" {' '.join(g):34} {c}")
```

Bigramas crudos (con stopwords):

de la	20
llego con	19
la caja	16
con la	14
tuve que	13
con el	10
en la	10
sin ningun	9
a tiempo	8
despues de	7
el equipo	7
con un	7

Bigramas de contenido (sin stopwords):

llego completo	7
llego bien	6
fecha estimada	6
ningun problema	4
galaxy s	4
robot aspiradora	4
bien empacado	3
cable anker	3
llego completa	3
compre llego	3
canon rebel	3
s ultra	3

Trigramas de contenido (sin stopwords):

galaxy s ultra	3
disco wd blue	3
cargador belkin llego	2
television tcl llego	2
camara instantanea llego	2
parlante jbl llego	2
llego bien empacado	2
ssd kingston llego	2
robot aspiradora llego	2
samsung the frame	2
ninguna notificacion previa	2
llego bien protegido	2

N-gramas con scikit-learn

`CountVectorizer(ngram_range=(2, 2))` construye directamente una matriz de bigramas; con `stop_words` descarta los de relleno. Sumando por columna se obtiene la frecuencia total de cada bigrama.

```
from sklearn.feature_extraction.text import CountVectorizer

vec = CountVectorizer(ngram_range=(2, 2), stop_words=list(stop_es))
X = vec.fit_transform(corpus)
frecuencias = X.sum(axis=0).A1
top = sorted(zip(vec.get_feature_names_out(), frecuencias), key=lambda x: -x[1])[:12]
```

```
pd.DataFrame(top, columns=["bigrama", "frecuencia"])
```

	bigrama	frecuencia
0	llego bien	7
1	llego completo	7
2	fecha estimada	6
3	dentro caja	5
4	llego dia	5
5	llego tiempo	5
6	dia retraso	4
7	galaxy s24	4
8	meses uso	4
9	ningun problema	4
10	robot aspiradora	4
11	acer aspire	3

Estimación de probabilidad (modelo de bigramas)

Un modelo de bigramas aproxima la probabilidad de una palabra a partir de la anterior:

$$P(w_i | w_{i-1}) \approx \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

Es decir: dado que apareció una palabra, ¿cuál es la más probable a continuación?

```
from collections import defaultdict

siguientes = defaultdict(Counter)
for texto in corpus:
    toks = a_tokens(texto)
    for a, b in ngrams(toks, 2):
        siguientes[a][b] += 1

def continuaciones(palabra, k=6):
    total = sum(siguientes[palabra].values())
    filas = [(w, n, n / total) for w, n in siguientes[palabra].most_common(k)]
    return pd.DataFrame(filas, columns=["siguiente", "count", "P(siguiete|'%s')"%
palabra])

continuaciones("llego") # el texto minado viene sin tildes
```

	siguiente	count	P(siguiete 'llego')
0	con	19	0.296875
1	completo	7	0.109375
2	bien	6	0.093750
3	sin	5	0.078125
4	a	5	0.078125
5	en	5	0.078125

Modelo generativo de n-gramas

Con las mismas probabilidades podemos *generar* texto: elegimos la siguiente palabra al azar según lo observado tras el prefijo de n-1 palabras. Es una versión minúscula de lo que, llevado al extremo con atención y Transformers, hacen los LLM actuales.

```
import random

class ModeloNgram:
    def __init__(self, n=3):
        self.n = n
        self.tabla = defaultdict(list)
        self.tokens = []

    def entrenar(self, textos):
        for texto in textos:
            toks = a_tokens(texto)
            self.tokens.extend(toks)
            for i in range(len(toks) - self.n + 1):
                prefijo = tuple(toks[i : i + self.n - 1])
                self.tabla[prefijo].append(toks[i + self.n - 1])
        return self

    def generar(self, inicio, largo=25):
        prefijo = tuple([inicio] * (self.n - 1))
        salida = [inicio]
        for _ in range(largo - 1):
            opciones = self.tabla.get(prefijo) or self.tokens
            salida.append(random.choice(opciones))
            prefijo = tuple(salida[-(self.n - 1):])
        return " ".join(salida)

random.seed(7)
modelo = ModeloNgram(n=3).entrenar(corpus)
for palabra in ("el", "llego", "producto"):
    print(f"[{palabra}] ->", modelo.generar(palabra, largo=25), "\n")
```

[el] -> el vuelto a diaria un de me sin un mis aspiradora sin experiencia día que sin leve completo retraso por un problema logístico que la

[llego] -> llevo durante llevo para un producto nuevo estoy esperando la respuesta del reclamo por garantía como parlante escribi con la caja sin defectos original este

[producto] -> producto veia muy fuera fue la mejor compra de tecnología que hice este año de la caja visiblemente golpeada en una esquina de la pantalla

El texto generado es localmente coherente (pares y tríos de palabras reales del corpus) pero se desvía sin rumbo: el modelo solo "recuerda" $n-1$ palabras. Word2Vec y los Transformers resuelven esa falta de contexto global.

Siguiente: 04_bow.ipynb .

04 · Bag of Words (BoW)

Bag of Words representa cada documento como un vector con la **frecuencia** de cada palabra del vocabulario, **ignorando el orden**.

Como "documentos" usamos las **descripciones de producto** del catálogo (`datos/productos.csv`): 90 descripciones repartidas en 9 categorías, un corpus con vocabulario técnico variado.

```
from pathlib import Path

import pandas as pd

# El texto ya minado de la tienda-virtual está copiado en la carpeta datos/ de
# este mismo proyecto:
# datos/resenas_entrega.csv   reseñas de entrega (post_compra)
# datos/comentarios.csv      testimonios / comentarios de clientes
# datos/productos.csv        catálogo con la descripción de cada producto
DATOS = Path("../datos")
```

```
def cargar(nombre, **kwargs):
    """Lee un CSV de la carpeta datos/ y lo devuelve como DataFrame."""
    ruta = DATOS / nombre
    if not ruta.exists():
        raise FileNotFoundError(f"No se encontró {ruta.resolve()}")
    print(f"Leyendo {ruta} ({ruta.stat().st_size / 1024:.1f} KB)")
    return pd.read_csv(ruta, **kwargs)
```

```
productos = cargar("productos.csv")
print(productos.shape)
productos[["nombre", "categoria", "descripcion"]].head(3)
```

```
Leyendo ../datos/productos.csv (27.5 KB)
(90, 11)
```

	nombre	categoria	descripcion
0	Apple iPhone 15 Pro Max 256GB	Smartphones	Smartphone premium con chip A17 Pro fabricado ...
1	Apple iPhone 15 128GB	Smartphones	Smartphone con chip A16 Bionic, camara dual de...
2	Samsung Galaxy S24 Ultra 512GB	Smartphones	Smartphone con S Pen integrado, camara cuadруп...

Normalización (mismo pipeline de 02_normalizacion.ipynb)

```
import re
import unicodedata
```

```

import nltk
import spacy

nlp = spacy.load("es_core_news_sm")
STOPWORDS = set(nltk.corpus.stopwords.words("spanish"))

def _limpiar(texto):
    # NFKC junta tildes combinantes sueltas; luego minúsculas y solo letras/espacios
    texto = unicodedata.normalize("NFKC", str(texto)).lower()
    return re.sub(r"^\w\s", " ", texto)

def _lemas(doc):
    return [
        t.lemma_.lower()
        for t in doc
        if t.is_alpha and not t.is_stop and t.lemma_.lower() not in STOPWORDS and
len(t.lemma_) > 2
    ]

def normalizar(texto):
    """minúsculas -> sin signos -> sin stopwords -> lematizado. Devuelve un str."""
    return " ".join(_lemas(nlp(_limpiar(texto))))

def normalizar_muchos(textos):
    """Igual que normalizar() pero en lote con nlp.pipe (más rápido para un corpus)."""
    return [" ".join(_lemas(doc)) for doc in nlp.pipe(_limpiar(t) for t in textos),
batch_size=64)]

```

```

docs = productos["descripcion"].fillna("").tolist()
docs_norm = normalizar_muchos(docs)
print("original  :", docs[0][:110], "...")
print("normalizado:", docs_norm[0][:110], "...")

```

original : Smartphone premium con chip A17 Pro fabricado en 3nm, camara triple de 48MP con teleobjetivo periscopico de 5x ...
normalizado: smartphonir premium chip pro fabricado camamar triple teleobjetivo periscopico pantalla super retín xdr pulgad ...

Nota sobre el ruido del preprocesamiento. El modelo `es_core_news_sm` a veces produce lemas extraños en términos de dominio o en texto sin tildes (`smartphone` → `smartphonir`, `retina` → `retín`). Es un recordatorio de que ningún pipeline es perfecto: para producción se usan modelos más grandes (`es_core_news_md/lg`) o listas de excepciones.

Vocabulario y matriz de conteos

`CountVectorizer` construye el vocabulario y cuenta. Con `min_df=2` descartamos palabras que aparecen en un solo documento (ruido).

```

from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer(min_df=2)
X = vectorizer.fit_transform(docs_norm)
vocab = vectorizer.get_feature_names_out()

print(f"Matriz BoW: {X.shape[0]} documentos x {X.shape[1]} términos del vocabulario")

df_bow = pd.DataFrame(X.toarray(), columns=vocab, index=productos["codigo"])
df_bow.iloc[:5, :14]

```

Matriz BoW: 90 documentos x 253 términos del vocabulario

	abatible	acceso	accion	actividad	adaptativo	adicional	agua	ajustar	alexa	almacena
codigo										
PROD-001	0	0	0	0	0	0	1	0	0	
PROD-002	0	0	0	0	0	0	0	0	0	
PROD-003	0	0	0	0	0	0	0	0	0	
PROD-004	0	0	0	0	0	0	0	0	0	
PROD-005	0	0	0	0	0	0	0	0	0	

Dimensionalidad y dispersión (*sparsity*)

```

total = X.shape[0] * X.shape[1]
no_cero = X.nnz
print(f"Celdas totales      : {total}")
print(f"Celdas distintas de 0: {no_cero}  ({100 * no_cero / total:.1f} %)")
print(f"Dispersión (sparsity): {100 * (1 - no_cero / total):.1f} % de la matriz son ceros")
print(f"Términos por documento: {no_cero / X.shape[0]:.1f} de {X.shape[1]} en promedio")

```

Celdas totales : 22770
 Celdas distintas de 0: 1057 (4.6 %)
 Dispersión (sparsity): 95.4 % de la matriz son ceros
 Términos por documento: 11.7 de 253 en promedio

Palabras más frecuentes por categoría

Sumando las filas BoW de cada categoría vemos que el vocabulario dominante es **temático**: BoW captura de qué trata un texto aunque pierda el orden.

```

df_cat = df_bow.copy()
df_cat["categoria"] = productos["categoria"].values
sumas = df_cat.groupby("categoria").sum()

for cat in ["Smartphones", "Audio", "Fotografia y Video"]:

```

```
top = sumas.loc[cat].sort_values(ascending=False).head(8)
print(f"\n{cat}:")
print("  " + ", ".join(f"{w}({int(n)})" for w, n in top.items()))
```

Smartphones:

pantalla(11), smartphonir(10), pulgada(9), carga(6), amoled(6), camara(5), super(4), rapido(4)

Audio:

hora(10), ear(6), bateria(5), cancelacion(5), ruido(5), portatil(4), reproduccion(4), parlante(4)

Fotografia y Video:

camarar(5), pantalla(4), aps(4), camara(4), autoenfoco(3), ideal(3), incluir(3), video(3)

Limitación: BoW ignora el orden

Dos frases con las mismas palabras en distinto orden producen **el mismo vector**.

```
par = ["el cargador daña la batería", "la batería daña el cargador"]
vec_par = CountVectorizer().fit(par)
Xpar = vec_par.transform(par).toarray()
demo = pd.DataFrame(Xpar, columns=vec_par.get_feature_names_out(), index=["frase A",
"frase B"])
print(demo)
print("\n¿Vectores idénticos?", bool((Xpar[0] == Xpar[1]).all()))
```

	batería	cargador	daña	el	la
frase A	1	1	1	1	1
frase B	1	1	1	1	1

¿Vectores idénticos? True

Siguiente: `05_tf-idf.ipynb` — pondera cada término por lo distintivo que es, no solo por cuántas veces aparece.

05 · TF-IDF

TF-IDF combina:

- **TF** (*Term Frequency*): $TF(t, d) = f(t, d) / N(d)$ — proporción del documento que ocupa el término.
- **IDF** (*Inverse Document Frequency*): $IDF(t) = \log(N/n(t))$ — cuán raro es el término en el corpus.
- **TF-IDF** = $TF \times IDF$: alto cuando el término es frecuente en ese documento y raro en los demás → **término distintivo**.

Mismo corpus que Bag of Words: las **descripciones de producto** (`datos/productos.csv`).

```
from pathlib import Path

import pandas as pd

# El texto ya minado de la tienda-virtual está copiado en la carpeta datos/ de
# este mismo proyecto:
# datos/resenas_entrega.csv   reseñas de entrega (post_compra)
# datos/comentarios.csv      testimonios / comentarios de clientes
# datos/productos.csv        catálogo con la descripción de cada producto
DATOS = Path("../datos")

def cargar(nombre, **kwargs):
    """Lee un CSV de la carpeta datos/ y lo devuelve como DataFrame."""
    ruta = DATOS / nombre
    if not ruta.exists():
        raise FileNotFoundError(f"No se encontró {ruta.resolve()}")
    print(f"Leyendo {ruta} ({ruta.stat().st_size / 1024:.1f} KB)")
    return pd.read_csv(ruta, **kwargs)
```

```
import re
import unicodedata

import nltk
import spacy

nlp = spacy.load("es_core_news_sm")
STOPWORDS = set(nltk.corpus.stopwords.words("spanish"))

def _limpiar(texto):
    # NFKC junta tildes combinantes sueltas; luego minúsculas y solo letras/espacios
    texto = unicodedata.normalize("NFKC", str(texto)).lower()
    return re.sub(r"^[^\w\s]", " ", texto)

def _lemas(doc):
    return [
        t.lemma_.lower()
```

```

        for t in doc
            if t.is_alpha and not t.is_stop and t.lemma_.lower() not in STOPWORDS and
len(t.lemma_) > 2
        ]

def normalizar(texto):
    """minúsculas -> sin signos -> sin stopwords -> lematizado. Devuelve un str."""
    return " ".join(_lemas(nlp(_limpiar(texto))))

def normalizar_muchos(textos):
    """Igual que normalizar() pero en lote con nlp.pipe (más rápido para un
corpus)."""
    return [" ".join(_lemas(doc)) for doc in nlp.pipe([_limpiar(t) for t in textos],
batch_size=64)]

```

```

productos = cargar("productos.csv")
docs = productos["descripcion"].fillna("").tolist()
docs_norm = normalizar_muchos(docs)
print(f"{len(docs_norm)} documentos normalizados")

```

Leyendo ../datos/productos.csv (27.5 KB)
90 documentos normalizados

IDF a mano: verificar la intuición

$IDF(t) = \log(N/n(t))$. Un término que aparece en casi todas las descripciones (pantalla) tiene IDF bajo; uno raro (titanio) IDF alto; y uno que aparece en **todos** los documentos tiene IDF = $\log(1) = 0$.

```

import numpy as np

docs_norm_demo = [d + " marcavisible" for d in docs_norm] # término presente en
TODOS
N = len(docs_norm_demo)
for termino in ["pantalla", "camara", "bateria", "titanio", "hdr", "marcavisible"]:
    n_t = sum(1 for d in docs_norm_demo if termino in d.split())
    idf = np.log(N / n_t) if n_t else float("nan")
    print(f" {termino:13} aparece en {n_t:2}/{N} docs    IDF = {idf:.3f}")

```

pantalla	aparece en 35/90 docs	IDF = 0.944
camara	aparece en 13/90 docs	IDF = 1.935
bateria	aparece en 15/90 docs	IDF = 1.792
titanio	aparece en 3/90 docs	IDF = 3.401
hdr	aparece en 2/90 docs	IDF = 3.807
marcavisible	aparece en 90/90 docs	IDF = 0.000

En **scikit-learn** el IDF lleva un +1 de suavizado ($idf = \ln((1+N)/(1+n(t))) + 1$), así que un término omnipresente **no llega a 0** exactamente, pero sí obtiene el peso más bajo y casi constante de toda la matriz.

TF-IDF con scikit-learn

```

from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(min_df=2)
X = vectorizer.fit_transform(docs_norm)
vocab = vectorizer.get_feature_names_out()
print(f"Matriz TF-IDF: {X.shape[0]} x {X.shape[1]}")

df_tfidf = pd.DataFrame(X.toarray(), columns=vocab, index=productos["codigo"])
df_tfidf.iloc[:5, :12]

```

Matriz TF-IDF: 90 x 253

	abatible	acceso	accion	actividad	adaptativo	adicional	agua	ajustar	alexa	alma
codigo										
PROD-001	0.0	0.0	0.0	0.0	0.0	0.0	0.170191	0.0	0.0	
PROD-002	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	
PROD-003	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	
PROD-004	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	
PROD-005	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	

Términos más distintivos de cada producto

```

for i in [0, 2, 20, 45]:
    fila = df_tfidf.iloc[i].sort_values(ascending=False).head(6)
    print(f"\n[{productos.iloc[i]['codigo']}] {productos.iloc[i]['nombre']}")
    print(" " + ", ".join(f"{w} ({v:.2f})" for w, v in fila.items()))

```

[PROD-001] Apple iPhone 15 Pro Max 256GB
 island (0.27), teleobjetivo (0.27), triple (0.27), pro (0.25), premium (0.25),
 dynamic (0.25)

[PROD-003] Samsung Galaxy S24 Ultra 512GB
 optico (0.31), dynamic (0.31), incorporo (0.31), titanio (0.31), marco (0.30),
 artificial (0.30)

[PROD-021] Sony WH-1000XM5
 hora (0.37), dedicado (0.29), over (0.29), minuto (0.29), microfono (0.28),
 inalambrico (0.26)

[PROD-046] Logitech G502 Hero
 perfil (0.42), dpi (0.39), mouse (0.39), juego (0.37), gamer (0.33), sensor (0.33)

Común vs distintivo: BoW vs TF-IDF

El mismo término pesa poco en TF-IDF si es común en el corpus y mucho si es raro, aunque su conteo bruto (BoW) sea alto.

```

from sklearn.feature_extraction.text import CountVectorizer

cv = CountVectorizer(min_df=2).fit(docs_norm)
conteo = cv.transform(docs_norm)
idx = {w: k for k, w in enumerate(cv.get_feature_names_out())}

filas = []
for termino in ["pantalla", "camara", "titanio", "bluetooth", "periscopico"]:
    if termino in idx and termino in vocab:
        docs_con = int((df_tfidf[termino] > 0).sum())
        filas.append({
            "termino": termino,
            "docs en que aparece": docs_con,
            "conteo total BoW": int(conteo[:, idx[termino]].sum()),
            "TF-IDF medio (>0)": round(df_tfidf[termino][df_tfidf[termino] >
0].mean(), 3),
        })
pd.DataFrame(filas)

```

	termino	docs en que aparece	conteo total BoW	TF-IDF medio (>0)
0	pantalla	35	38	0.167
1	camara	13	13	0.228
2	titanio	3	3	0.284
3	bluetooth	3	3	0.315

pantalla aparece en muchos documentos: conteo alto pero TF-IDF bajo. **titanio** o **periscopico** aparecen en pocos: aunque su conteo sea 1, su TF-IDF es alto porque son **distintivos**.

Siguiente: `06_mineria-de-texto.ipynb` — el pipeline completo de principio a fin sobre las reseñas de entrega.

06 · Minería de texto de extremo a extremo

Juntamos todo el pipeline sobre un caso real:

Tokenización → Normalización → Representación (TF-IDF) → Análisis

Corpus: las **reseñas de entrega** (`post_compra`), en `datos/resenas_entrega.csv` ,
enriquecidas con el nombre del producto desde `datos/productos.csv` .

```
from pathlib import Path

import pandas as pd

# El texto ya minado de la tienda-virtual está copiado en la carpeta datos/ de
# este mismo proyecto:
# datos/resenas_entrega.csv   reseñas de entrega (post_compra)
# datos/comentarios.csv      testimonios / comentarios de clientes
# datos/productos.csv        catálogo con la descripción de cada producto
DATOS = Path("../datos")

def cargar(nombre, **kwargs):
    """Lee un CSV de la carpeta datos/ y lo devuelve como DataFrame."""
    ruta = DATOS / nombre
    if not ruta.exists():
        raise FileNotFoundError(f"No se encontró {ruta.resolve()}")
    print(f"Leyendo {ruta} ({ruta.stat().st_size / 1024:.1f} KB)")
    return pd.read_csv(ruta, **kwargs)

import re
import unicodedata

import nltk
import spacy

nlp = spacy.load("es_core_news_sm")
STOPWORDS = set(nltk.corpus.stopwords.words("spanish"))

def _limpiar(texto):
    # NFKC junta tildes combinantes sueltas; luego minúsculas y solo letras/espacios
    texto = unicodedata.normalize("NFKC", str(texto)).lower()
    return re.sub(r"^[^\w\s]", " ", texto)

def _lemas(doc):
    return [
        t.lemma_.lower()
        for t in doc
        if t.is_alpha and not t.is_stop and t.lemma_.lower() not in STOPWORDS and
        len(t.lemma_) > 2
    ]
```

```
def normalizar(texto):
    """minúsculas -> sin signos -> sin stopwords -> lematizado. Devuelve un str."""
    return " ".join(_lemas(nlp(_limpiar(texto))))

def normalizar_muchos(textos):
    """Igual que normalizar() pero en lote con nlp.pipe (más rápido para un corpus)."""
    return [" ".join(_lemas(doc)) for doc in nlp.pipe([_limpiar(t) for t in textos], batch_size=64)]
```

```
resenas = cargar("resenas_entrega.csv")
productos = cargar("productos.csv")

# Enriquecer con el nombre del producto (join por id) si la columna existe
if "producto_id" in resenas.columns and "id" in productos.columns:
    resenas = resenas.merge(
        productos[["id", "nombre", "categoria"]],
        left_on="producto_id", right_on="id", how="left", suffixes=("", "_prod"),
    )

resenas = resenas.dropna(subset=["texto"]).reset_index(drop=True)
print(resenas.shape)
resenas[[c for c in ["calificacion", "nombre", "texto"] if c in
resenas.columns]].head(5)
```

Leyendo ../datos/resenas_entrega.csv (15.0 KB)

Leyendo ../datos/productos.csv (27.5 KB)

(110, 10)

	calificacion	nombre	texto
0	1	OnePlus 12 256GB	Recibi el pedido incompleto, faltaba el cable ...
1	3	Belkin Cargador USB-C 30W	El segundo cargador Belkin llego sin contratie...
2	5	Nintendo Switch Lite	La segunda Switch Lite que regale llego perfec...
3	5	TCL 50 pulgadas 4K Google TV	El segundo television TCL llego perfecto, bien...
4	4	Anker PowerLine III Cable USB-C	Me confirmaron por correo la garantia de un an...

1. Exploración

```
resenas["n_palabras"] = resenas["texto"].str.split().str.len()
print("Reseñas          :", len(resenas))
print("Calificación media :", round(resenas["calificacion"].mean(), 2))
print("Palabras por reseña :", round(resenas["n_palabras"].mean(), 1), "en promedio")
print("\nDistribución de calificaciones:")
print(resenas["calificacion"].value_counts().sort_index())
```

Reseñas : 110
Calificación media : 3.58
Palabras por reseña : 19.3 en promedio

Distribución de calificaciones:

calificacion

1 10
2 15
3 21
4 29
5 35

Name: count, dtype: int64

2. Tokenización + normalización (pipeline reusable)

```
resenas["texto_norm"] = normalizar_muchos(resenas["texto"].tolist())  
resenas[["texto", "texto_norm"]].head(3)
```

	texto	texto_norm
0	Recibi el pedido incompleto, faltaba el cable ...	recibi pedido incompleto faltar cable carga or...
1	El segundo cargador Belkin llego sin contratie...	cargador belkin llego contratiempo tiempo esti...
2	La segunda Switch Lite que regale llego perfec...	switch lite regalar llego perfecto justo tiemp...

3. Representación TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer  
  
vectorizer = TfidfVectorizer(min_df=3)  
X = vectorizer.fit_transform(resenas["texto_norm"])  
vocab = vectorizer.get_feature_names_out()  
print(f"Matriz TF-IDF: {X.shape[0]} reseñas x {X.shape[1]} términos")
```

Matriz TF-IDF: 110 reseñas x 112 términos

4. Análisis A — ¿qué palabras caracterizan a una reseña negativa vs una positiva?

Separamos por calificación (≤ 2 negativas, ≥ 4 positivas) y, para quedarnos con lo **distintivo** de cada grupo (y no con `llego`, que está en casi todas), ordenamos por la **diferencia** de TF-IDF medio entre grupos.

```
import numpy as np  
  
neg = (resenas["calificacion"] <= 2).values  
pos = (resenas["calificacion"] >= 4).values  
  
media_neg = np.asarray(X[neg].mean(axis=0)).ravel()  
media_pos = np.asarray(X[pos].mean(axis=0)).ravel()  
dif = pd.Series(media_neg - media_pos, index=vocab)  
  
print(f"NEGATIVAS ({neg.sum()} reseñas) - términos que más las distinguen:")  
print(" " + ", ".join(dif.sort_values(ascending=False).head(12).index))
```

```
print(f"\nPOSITIVAS ({pos.sum()} reseñas):")
print(" " + ", ".join(dif.sort_values().head(12).index))
```

NEGATIVAS (25 reseñas) – términos que más las distinguen:

cambio, equipo, tener, pantalla, retraso, pedido, samsung, iniciar, laptop, reclamo, comprar, entrega

POSITIVAS (64 reseñas):

galaxy, xiaomi, perfecto, ningun, configurar, rapido, completo, acer, mes, amazfit, empacado, compra

5. Análisis B — reseñas similares (similitud del coseno)

Sobre los vectores TF-IDF, la similitud del coseno mide cuán parecidas son dos reseñas. Tomamos una y buscamos las 3 más cercanas.

```
from sklearn.metrics.pairwise import cosine_similarity

base = 0
sims = cosine_similarity(X[base], X).ravel()
orden = sims.argsort()[::-1][1:4] # saltamos la propia reseña

print("RESEÑA BASE:", resenas.loc[base, "texto"], "\n")
for j in orden:
    print(f" sim={sims[j]:.2f} | {resenas.loc[j, 'texto']}")
```

RESEÑA BASE: Recibi el pedido incompleto, faltaba el cable de carga original del OnePlus 12 dentro de la caja. Tuve que esperar un envío adicional solo para recibir ese accesorio.

sim=0.48 | El OnePlus 12 que recibí no coincidía con el color que había pedido en el pedido original. Tuve que devolverlo y esperar el cambio por el modelo correcto.

sim=0.44 | El Huawei Watch GT 4 llegó a tiempo pero sin el cable de carga dentro de la caja. Tuve que reclamar por separado para que me lo enviaran.

sim=0.42 | El parlante JBL llegó sin el cable de carga incluido en la caja. Tuve que reclamar por separado para que me lo enviaran.

6. Análisis C — bigramas más frecuentes en las reseñas negativas

```
from sklearn.feature_extraction.text import CountVectorizer

cv = CountVectorizer(ngram_range=(2, 2), min_df=2)
Xn = cv.fit_transform(resenas.loc[neg, "texto_norm"])
frec = Xn.sum(axis=0).A1
top = sorted(zip(cv.get_feature_names_out(), frec), key=lambda x: -x[1][:10])
pd.DataFrame(top, columns=["bigrama (reseñas negativas)", "frecuencia"])
```

	bigrama (reseñas negativas)	frecuencia
0	equipo tecnico	3
1	llego caja	3
2	llego pantalla	3
3	llego retraso	3
4	alto demanda	2
5	cable carga	2
6	caja tener	2
7	caja visiblemente	2
8	color distinto	2
9	comprar llego	2

Cierre y ejercicio propuesto

Con TF-IDF + scikit-learn ya se puede **clasificar automáticamente** una reseña nueva como positiva o negativa:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

y = (resenas["calificacion"] >= 4).astype(int)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0)
clf = LogisticRegression(max_iter=1000).fit(Xtr, ytr)
print("accuracy:", clf.score(Xte, yte))
```

Los siguientes pasos del temario —**Word2Vec** y los modelos contextuales tipo **BERT / GPT**— sustituyen estos vectores dispersos por *embeddings* densos que además capturan relaciones semánticas.

sesion-de-clase-05/README.md

Proyecto de Minería Web - Sesión de clase 05

Esta sesión trata la **representación vectorial de texto**: de contar palabras a que el vector de una palabra cambie según la frase en la que aparece. Cuatro métodos, cada uno corrigiendo una limitación concreta del anterior — Bag-of-Words, TF-IDF, Word2Vec y Transformers/SBERT.

Los notebooks no scrapean: trabajan sobre los CSV de la tienda-virtual que ya están **copiados en la carpeta `data/`** de este proyecto, más un corpus de juguete de tres frases que se reutiliza en todo el recorrido para que los resultados de cada método sean comparables entre sí.

Instalación de dependencias

Requiere **Python 3.11+**.

```
pip install -r requirements.txt
python -m spacy download es_core_news_sm
```

El primer notebook que usa `gensim.downloader` (`04_word2vec.ipynb`) y el primero que usa `sentence-transformers` (`05_transformers-sbert.ipynb`) descargan sus modelos preentrenados solos la primera vez que se ejecutan (unos 130 MB y 470 MB respectivamente) y quedan cacheados localmente.

Datos (`data/`)

Archivo	Contenido	Uso
<code>productos.csv</code>	Catálogo: <code>nombre_producto</code> , <code>descripcion</code> , <code>categoria</code> , <code>precio</code>	Documentos para TF-IDF, el buscador semántico y el sistema de recomendación
<code>comentarios.csv</code>	Comentarios de clientes con calificación	Corpus para Bag-of-Words, TF-IDF y el Word2Vec propio
<code>clientes.csv</code>	Datos de cliente: ciudad, país, fecha de registro	Sistema de recomendación
<code>historial_de_compras.csv</code>	Compras por cliente (<code>id_producto</code> , <code>cantidad</code>)	Perfil de cliente para el sistema de recomendación

Notebooks (`notebooks/`)

Se recomienda ejecutarlos en este orden; cada uno reutiliza ideas del anterior y todos son autocontenidos (`Run All` funciona de principio a fin).

Notebook	Técnica	Qué hace
<code>01_fundamentos-vectores-similitud.ipynb</code>	<code>numpy</code>	Qué es un vector, similitud coseno a mano, y la ley de Zipf / contenido informativo en bits — sobre un ejemplo de juguete y sobre los comentarios reales.
<code>02_bag-of-words.ipynb</code>	<code>CountVectorizer</code>	Vocabulario, matriz de conteos, similitud entre documentos y la limitación de perder el orden de las palabras.
<code>03_tfidf.ipynb</code>	<code>TfidfVectorizer</code>	IDF a mano, matriz TF-IDF, comparación contra Bag-of-Words. Incluye el Ejercicio 01 (dispersión, ranking por coseno, <code>ngram_range</code>) sobre los 619 comentarios reales.
<code>04_word2vec.ipynb</code>	<code>gensim.Word2Vec</code> , GloVe preentrenado	Hipótesis distribucional, embeddings propios sobre los comentarios, y el Ejercicio 02 (analogías vectoriales, sinónimos/antónimos) con un modelo preentrenado.
<code>05_transformers-sbert.ipynb</code>	<code>transformers</code> , <code>sentence-transformers</code>	Polisemia resuelta con vectores contextuales (demo con un Transformer real), self-attention implementada desde cero, codificación posicional, y el Ejercicio 03 (buscador semántico vs. TF-IDF) sobre el catálogo de productos.
<code>06_sistema-recomendacion.ipynb</code>	<code>sentence-transformers</code>	Práctica integradora: perfil de cliente por promedio de embeddings, top-N de recomendaciones, y el problema real de agregar compras repetidas.
<code>07_sintesis-comparativa.ipynb</code>	—	Cierre: las tres representaciones sobre la misma frase, criterios de selección, errores frecuentes y glosario.

01 · Fundamentos: vectores y similitud coseno

Esta sesión recorre cuatro formas de convertir texto en números, cada una corrigiendo una limitación concreta de la anterior:

#	Método	Idea	Limitación que deja pendiente
1	Bag-of-Words	contar palabras	"el" pesa igual que "epicentro"
2	TF-IDF	pesar por rareza	sigue sin saber que rey $\approx$ monarca
3	Word2Vec	aprender del contexto	un solo vector para "banco"
4	Transformers	contextualizar	resuelve el contexto, pero cuesta caro

La pregunta que guía todo el recorrido: **¿cómo logramos que la distancia entre dos vectores signifique algo lingüístico?** Antes de vectorizar nada hace falta fijar dos herramientas: qué es un vector de texto y cómo se mide el parecido entre dos vectores.

Corpus de trabajo

Tres frases muy simples, usadas en toda la sesión para que los resultados de cada método sean comparables entre sí.

```
corpus = {
    "D1": "el gato duerme en el sofá",
    "D2": "el perro duerme en la alfombra",
    "D3": "el sismo sacudió la costa",
}
for doc_id, texto in corpus.items():
    print(f"{doc_id}: {texto!r}")
```

```
D1: 'el gato duerme en el sofá'
D2: 'el perro duerme en la alfombra'
D3: 'el sismo sacudió la costa'
```

Resultado esperable: D1 y D2 hablan de animales durmiendo, deberían salir parecidos; D3 habla de otra cosa y debería quedar lejos de ambos.

Observación previa: D1 y D2 comparten las palabras *el · duerme · en*; D1 y D3 comparten solo *el*. Queda por determinar si contar palabras compartidas basta para estimar el parecido — ese es, exactamente, el problema que resuelven los métodos de esta sesión.

Qué es un vector

Un ejemplo fuera del lenguaje ayuda a fijar la idea: describir canciones mediante atributos numéricos.

```
import pandas as pd

canciones = pd.DataFrame(
    {
        "energía": [0.9, 0.8, 0.2],
        "baile": [0.8, 0.9, 0.1],
        "acústica": [0.1, 0.2, 0.9],
        "ánimo": [0.9, 0.8, 0.3],
    },
    index=["Canción A", "Canción B", "Canción C"],
)
canciones
```

	energía	baile	acústica	ánimo
Canción A	0.9	0.8	0.1	0.9
Canción B	0.8	0.9	0.2	0.8
Canción C	0.2	0.1	0.9	0.3

A y B tienen números parecidos en las cuatro columnas → deberían "sonar parecido". C es distinta en casi todo → suena a otra cosa. Más abajo se confirma esta intuición con un número.

Idea central: si cada palabra o texto se describe con una lista de números, el parecido deja de ser una apreciación subjetiva y pasa a ser una operación calculable. Los métodos que siguen solo difieren en cómo se obtienen esos números.

Medir el parecido entre dos vectores

Se usa el **ángulo** que forman los vectores, no la distancia entre sus extremos: un texto de 200 palabras y otro de 2000 sobre el mismo tema apuntan en la misma dirección aunque uno sea mucho "más largo". El coseno ignora el tamaño y se queda con el tema.

$$\cos(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

- $A \cdot B$: multiplicar las dos listas posición por posición y sumar.
- $\|A\|$: longitud del vector, raíz de la suma de sus cuadrados.
- El resultado va de 1 (misma dirección, muy parecidos) a 0 (sin relación).

```
import numpy as np

def similitud_coseno(a, b):
    a, b = np.asarray(a, dtype=float), np.asarray(b, dtype=float)
    return a.dot(b) / (np.linalg.norm(a) * np.linalg.norm(b))

# Ejemplo de referencia: A=[2,1,0], B=[1,1,1] -> se espera ≈ 0.77
A, B = [2, 1, 0], [1, 1, 1]
print(f"A · B = {np.dot(A, B)}")
print(f"||A|| = {np.linalg.norm(A):.2f}    ||B|| = {np.linalg.norm(B):.2f}")
print(f"cos(A, B) = {similitud_coseno(A, B):.2f}")
```

$A \cdot B = 3$
 $\|A\| = 2.24 \quad \|B\| = 1.73$
 $\cos(A, B) = 0.77$

```
# Misma fórmula sobre las canciones: confirma la intuición visual de la tabla
pares = [("Canción A", "Canción B"), ("Canción A", "Canción C"), ("Canción B", "Canción C")]
```

```
for x, y in pares:
    sim = similitud_coseno(canciones.loc[x], canciones.loc[y])
    print(f"cos({x}, {y}) = {sim:.3f}")
```

$\cos(\text{Canción A}, \text{Canción B}) = 0.991$

$\cos(\text{Canción A}, \text{Canción C}) = 0.422$

$\cos(\text{Canción B}, \text{Canción C}) = 0.471$

A y B quedan cerca de 1 (suenan parecido); cualquier par con C cae mucho más bajo.

`similitud_coseno` es la única función que se reutiliza sin cambios en el resto de la sesión — lo que cambia de un método a otro es **cómo se construye el vector**, no cómo se comparan dos vectores.

Frecuencia de palabras y contenido informativo

No todas las palabras valen lo mismo. "el", "de", "que" son casi universales y no informan sobre el tema de un texto; "sismo" o "epicentro" son raras y lo delimitan de inmediato. La **ley de Zipf** describe esa desigualdad: la palabra más frecuente de un idioma aparece el doble que la segunda, el triple que la tercera, y así sucesivamente — unas pocas palabras ocupan casi todo el texto y no distinguen nada. De ahí nacerá el IDF en el notebook `03_tfidf.ipynb`.

La cantidad de información que aporta observar una palabra se mide en bits:

$$I(x) = -\log_2 p(x)$$

donde $p(x)$ es la probabilidad de que una palabra cualquiera del texto sea justamente esa. $p(x)$ nunca es negativa, pero su logaritmo sí (es un número entre 0 y 1); el signo `-` convierte ese resultado en una cantidad positiva de bits.

```
frecuencias = pd.DataFrame(
    {
        "palabra": ["el", "de", "sismo", "epicentro"],
        "aparece_en": ["6 de cada 100 palabras", "5 de cada 100", "2 de cada 10 000", "2 de cada 100 000"],
        "p(x)": [0.062, 0.049, 0.0002, 0.00002],
    }
)
frecuencias["bits I(x) = -log2 p(x)"] = -np.log2(frecuencias["p(x)"])
frecuencias.round(3)
```

	palabra	aparece_en	p(x)	bits $I(x) = -\log_2 p(x)$
0	el	6 de cada 100 palabras	0.062	4.012
1	de	5 de cada 100	0.049	4.351
2	sismo	2 de cada 10 000	0.000	12.288
3	epicentro	2 de cada 100 000	0.000	15.610

Saber que un texto contiene "el" casi no aporta información porque lo contienen todos; "epicentro" delimita el tema de inmediato — por eso pesa 15.6 bits frente a los 4.0 de "el".

La ley de Zipf en un corpus real

La misma desigualdad aparece en cualquier texto real, no solo en el ejemplo de juguete. Se comprueba contando palabras en los 619 comentarios de clientes de la tienda virtual (`../data/comentarios.csv`) y graficando frecuencia contra rango en escala log-log: si Zipf se cumple, los puntos caen aproximadamente sobre una recta.

```
import re
from collections import Counter

import matplotlib.pyplot as plt

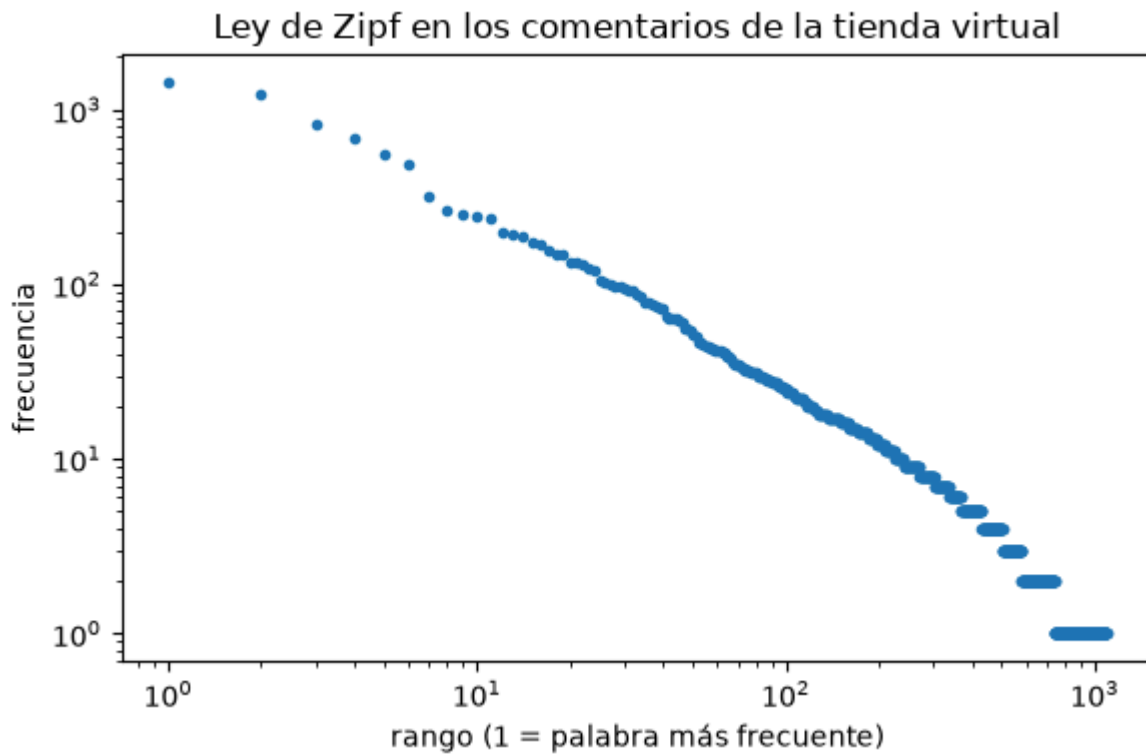
comentarios = pd.read_csv("../data/comentarios.csv")
palabras = re.findall(r"[a-záéíóúñü]+", "
".join(comentarios["texto_comentario"]).lower())

conteo = Counter(palabras)
mas_comunes = conteo.most_common()
rangos = np.arange(1, len(mas_comunes) + 1)
frecuencias_reales = np.array([f for _, f in mas_comunes])

print("Top 10 más frecuentes:", mas_comunes[:10])

plt.figure(figsize=(6, 4))
plt.loglog(rangos, frecuencias_reales, marker=".", linestyle="none")
plt.xlabel("rango (1 = palabra más frecuente)")
plt.ylabel("frecuencia")
plt.title("Ley de Zipf en los comentarios de la tienda virtual")
plt.tight_layout()
plt.show()
```

Top 10 más frecuentes: [('es', 1434), ('la', 1214), ('el', 834), ('de', 682), ('y', 557), ('muy', 487), ('para', 318), ('se', 266), ('un', 251), ('pero', 248)]



Un puñado de palabras funcionales domina el corpus y el resto forma una cola larga de términos raros: exactamente el patrón que hace que "contar palabras" (`02_bag-of-words.ipynb`) no baste por sí solo, y que motiva ponderar por rareza en `03_tfidf.ipynb` .

Siguiente: `02_bag-of-words.ipynb` — construir el primer vector real a partir del corpus de trabajo.

02 · Bag-of-Words

Parte 2 — representaciones dispersas. La primera forma de vectorizar texto es la más literal posible: contar cuántas veces aparece cada palabra. Se trabaja primero sobre el corpus de juguete (01_fundamentos...ipynb) y luego sobre los comentarios reales de clientes de la tienda virtual.

```
corpus = {
    "D1": "el gato duerme en el sofá",
    "D2": "el perro duerme en la alfombra",
    "D3": "el sismo sacudió la costa",
}
```

Paso 1 — construir el vocabulario

Se juntan todas las palabras distintas del corpus y se ordenan. Ese orden ya no cambia: define las posiciones del vector.

```
vocabulario = sorted(set(" ".join(corpus.values()).split()))
print(f"Vocabulario V ({len(vocabulario)} palabras distintas):")
for posicion, palabra in enumerate(vocabulario):
    print(f" {posicion:>2} {palabra}")
```

```
Vocabulario V (11 palabras distintas):
 0  alfombra
 1  costa
 2  duerme
 3  el
 4  en
 5  gato
 6  la
 7  perro
 8  sacudió
 9  sismo
10  sofá
```

En un corpus de noticias real el vocabulario no tendría 11 entradas sino unas 50 000. La escala del problema aparece más abajo, con datos reales.

Paso 2 — matriz de conteos

`CountVectorizer` hace exactamente esto: fija un vocabulario y cuenta, por documento, cuántas veces aparece cada palabra.

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

count_vectorizer = CountVectorizer(token_pattern=r"(?u)\b\w+\b")
X_counts = count_vectorizer.fit_transform(corpus.values())
```

```
matriz_bow = pd.DataFrame(
    X_counts.toarray(),
    columns=count_vectorizer.get_feature_names_out(),
    index=corpus.keys(),
)
matriz_bow
```

	alfombra	costa	duerme	el	en	gato	la	perro	sacudió	sismo	sofá
D1	0	0	1	2	1	1	0	0	0	0	1
D2	1	0	1	1	1	0	1	1	0	0	0
D3	0	1	0	1	0	0	1	0	1	1	0

En D1 la palabra "el" aparece dos veces, y por eso su columna marca 2. Cada frase es ahora una lista de 11 números: ya se pueden calcular cosenos.

Limitación: casi todo son ceros. El nombre "bolsa de palabras" alude a que el orden no se conserva — cuánto pesa esto se mide más abajo, sobre el corpus real.

Similitud entre documentos con Bag-of-Words

```
import numpy as np

def similitud_coseno(a, b):
    a, b = np.asarray(a, dtype=float), np.asarray(b, dtype=float)
    return a.dot(b) / (np.linalg.norm(a) * np.linalg.norm(b))

cos_d1_d2 = similitud_coseno(matriz_bow.loc["D1"], matriz_bow.loc["D2"])
cos_d1_d3 = similitud_coseno(matriz_bow.loc["D1"], matriz_bow.loc["D3"])
print(f"cos(D1, D2) = {cos_d1_d2:.3f} (gato duerme vs perro duerme)")
print(f"cos(D1, D3) = {cos_d1_d3:.3f} (gato duerme vs sismo costa)")

cos(D1, D2) = 0.577 (gato duerme vs perro duerme)
cos(D1, D3) = 0.316 (gato duerme vs sismo costa)
```

El orden es correcto (D1-D2 > D1-D3), pero 0.316 resulta excesivo para dos frases sin ninguna relación temática: se debe a que ambas contienen "el". Corresponde reducir el peso de palabras tan comunes — el motivo de ser de TF-IDF, en el siguiente notebook.

Limitación: la pérdida del orden

Bag-of-Words no distingue sujeto de objeto: solo importa qué palabras aparecen, no en qué posición.

```
frases_opuestas = ["El perro mordió al hombre", "El hombre mordió al perro"]
X_opuestas = CountVectorizer(token_pattern=r"(?u)\b\w+\b",
    lowercase=True).fit_transform(frases_opuestas)

print("Vectores idénticos:", (X_opuestas[0].toarray() ==
    X_opuestas[1].toarray()).all())
```

```
print(f"cos(frase 1, frase 2) = {similitud_coseno(X_opuestas[0].toarray()[0],
X_opuestas[1].toarray()[0]):.3f}")
```

Vectores idénticos: True
cos(frase 1, frase 2) = 1.000

Para el modelo, ambas frases son idénticas aunque signifiquen lo contrario. Contar pares de palabras (bigramas) recupera algo de orden, pero dispara el vocabulario — se explora el costo real en el ejercicio de `03_tfidf.ipynb`. El orden se recupera de verdad recién con los Transformers (`05_...ipynb`).

Aplicación al corpus real: comentarios de clientes

Antes de vectorizar hace falta normalizar el texto: minúsculas, sin puntuación, sin *stopwords* y lematizado (para que "duermo" y "durmiendo" cuenten como la misma palabra). Se usa spaCy en español.

```
import spacy

nlp = spacy.load("es_core_news_sm")

def normalizar(texto):
    doc = nlp(str(texto).lower())
    lemas = [token.lemma_ for token in doc if token.is_alpha and not token.is_stop]
    return " ".join(lemas)

def normalizar_muchos(textos):
    return [
        " ".join(t.lemma_ for t in doc if t.is_alpha and not t.is_stop)
        for doc in nlp.pipe((str(x).lower() for x in textos), batch_size=64)
    ]
```

```
comentarios = pd.read_csv("../data/comentarios.csv")
comentarios["texto_normalizado"] = normalizar_muchos(comentarios["texto_comentario"])
comentarios[["texto_comentario", "texto_normalizado"]].head(3)
```

	texto_comentario	texto_normalizado
0	El Smartphone Nexus 5G es un cambio de juego. ...	smartphonir nexus g cambio juego pantalla oled...
1	La Camiseta Deportiva Ultralight es muy cómoda...	camiseta deportivo ultralight cómodo entrenami...
2	La Laptop Gamer Pro es una bestia de rendimien...	laptop gamer pro bestia rendimiento superar ex...

```
cv_real = CountVectorizer(min_df=2)
X_real = cv_real.fit_transform(comentarios["texto_normalizado"])

n_docs, n_vocab = X_real.shape
dispersidad = 1 - X_real.nnz / (n_docs * n_vocab)
print(f"Matriz de conteos: {n_docs} comentarios x {n_vocab} palabras")
print(f"Dispersidad (proporción de ceros): {dispersidad:.1%}")
```

Matriz de conteos: 620 comentarios x 514 palabras
Dispersidad (proporción de ceros): 97.9%

Con 619 comentarios reales el vocabulario ya crece a varios cientos de palabras y la matriz es abrumadoramente cero — la limitación de la slide teórica ("con 50 000 palabras, 99.9% de cada fila sería cero") se confirma aquí con datos propios, aunque a menor escala.

Siguiente: `03_tfidf.ipynb` — pesar cada palabra por su rareza en el corpus, y el Ejercicio 01 completo sobre este mismo corpus de comentarios.

03 · TF-IDF

Bag-of-Words (02_bag-of-words.ipynb) infló la similitud entre frases sin relación porque ambas compartían palabras muy comunes como "el". **TF-IDF** corrige eso: un término importa si aparece mucho **en este documento** y poco **en todo lo demás**.

- **TF** (frecuencia en el documento): sube el peso de lo que el texto repite.
- **IDF** (rareza en el corpus): hunde el peso de lo que aparece en todos.
- **peso final = TF × IDF**.

```
corpus = {  
    "D1": "el gato duerme en el sofá",  
    "D2": "el perro duerme en la alfombra",  
    "D3": "el sismo sacudió la costa",  
}
```

IDF a mano sobre el corpus (N = 3)

$$IDF(t) = \log\left(\frac{1 + N}{1 + df(t)}\right) + 1$$

- N : total de documentos (3, así que $1 + N = 4$ en todas las filas).
- $df(t)$: en cuántos de esos documentos aparece el término t .
- El logaritmo comprime la escala; el **+1** evita que un término quede en cero (el mismo suavizado que usa `TfidfVectorizer` por defecto).

Lo único que cambia de una fila a otra es df : cuantos más documentos contienen el término, mayor el denominador y menor el resultado.

```
import numpy as np  
import pandas as pd  
  
tokens_por_doc = {doc_id: set(texto.split()) for doc_id, texto in corpus.items()}  
N = len(corpus)  
  
def idf(termino):  
    df = sum(termino in tokens for tokens in tokens_por_doc.values())  
    return np.log((1 + N) / (1 + df)) + 1, df  
  
filas = []  
for termino in ["el", "duerme", "gato"]:  
    valor, df = idf(termino)  
    filas.append({"término": termino, "df": df, "IDF": round(valor, 3)})  
pd.DataFrame(filas)
```

	término	df	IDF
0	el	3	1.000
1	duerme	2	1.288
2	gato	1	1.693

"el" está en los 3 documentos → IDF mínimo posible (1.000), no distingue. "gato" está en 1 solo → IDF máximo en este corpus (1.693), muy informativa.

Matriz TF-IDF con scikit-learn

`TfidfVectorizer` combina TF × IDF y normaliza cada fila para que todos los vectores midan 1 (por eso el resultado ya no son conteos enteros).

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vectorizer = TfidfVectorizer(token_pattern=r"(?u)\b\w+\b")
X_tfidf = tfidf_vectorizer.fit_transform(corpus.values())

matriz_tfidf = pd.DataFrame(
    X_tfidf.toarray(),
    columns=tfidf_vectorizer.get_feature_names_out(),
    index=corpus.keys(),
).round(2)
matriz_tfidf
```

	alfombra	costa	duerme	el	en	gato	la	perro	sacudió	sismo	sofá
D1	0.00	0.0	0.36	0.55	0.36	0.47	0.00	0.00	0.0	0.0	0.47
D2	0.49	0.0	0.38	0.29	0.38	0.00	0.38	0.49	0.0	0.0	0.00
D3	0.00	0.5	0.00	0.30	0.00	0.00	0.38	0.00	0.5	0.5	0.00

Efecto del IDF dentro de D1: "el" aparece 2 veces y obtiene 0.55; "gato" aparece solo 1 vez pero obtiene 0.47 — repetirse el doble apenas compensa ser una palabra común.

Bag-of-Words frente a TF-IDF

Misma fórmula de coseno, dos representaciones distintas del mismo corpus.

```
from sklearn.feature_extraction.text import CountVectorizer

def similitud_coseno(a, b):
    a, b = np.asarray(a, dtype=float), np.asarray(b, dtype=float)
    return a.dot(b) / (np.linalg.norm(a) * np.linalg.norm(b))

count_vectorizer = CountVectorizer(token_pattern=r"(?u)\b\w+\b")
X_counts = count_vectorizer.fit_transform(corpus.values()).toarray()
X_tfidf_arr = X_tfidf.toarray()
```

```
cos_bow = {
    "D1-D2": similitud_coseno(X_counts[0], X_counts[1]),
    "D1-D3": similitud_coseno(X_counts[0], X_counts[2]),
}
cos_tfidf = {
    "D1-D2": similitud_coseno(X_tfidf_arr[0], X_tfidf_arr[1]),
    "D1-D3": similitud_coseno(X_tfidf_arr[0], X_tfidf_arr[2]),
}

pd.DataFrame({"Bow": cos_bow, "TF-IDF": cos_tfidf}).round(3)
```

	Bow	TF-IDF
D1-D2	0.577	0.430
D1-D3	0.316	0.165

```
sep_bow = cos_bow["D1-D2"] / cos_bow["D1-D3"]
sep_tfidf = cos_tfidf["D1-D2"] / cos_tfidf["D1-D3"]
print(f"Separación con BoW: {cos_bow['D1-D2']:.3f} / {cos_bow['D1-D3']:.3f} = {sep_bow:.1f}x")
print(f"Separación con TF-IDF: {cos_tfidf['D1-D2']:.3f} / {cos_tfidf['D1-D3']:.3f} = {sep_tfidf:.1f}x")
```

Separación con BoW: 0.577 / 0.316 = 1.8x
 Separación con TF-IDF: 0.430 / 0.165 = 2.6x

Con conteos, D1 y D3 (temas sin relación) todavía comparten 0.316 de similitud solo por culpa de "el". Con TF-IDF esa similitud cae a 0.165, mientras que el par relacionado (D1-D2) se mantiene alto: la separación entre "parecido" y "no parecido" casi se duplica. Esa distancia es la que sirve en un buscador real.

Ejercicio 01 — Vectorización dispersa

Objetivo: ¿cuánta información se pierde al ignorar el orden de las palabras, y cuánto ayuda TF-IDF a rankear por relevancia? Se trabaja sobre los 619 comentarios reales de clientes.

```
import spacy

nlp = spacy.load("es_core_news_sm")

def normalizar_muchos(textos):
    return [
        " ".join(t.lemma_ for t in doc if t.is_alpha and not t.is_stop)
        for doc in nlp.pipe((str(x).lower() for x in textos), batch_size=64)
    ]

comentarios = pd.read_csv("../data/comentarios.csv")
comentarios["texto_normalizado"] = normalizar_muchos(comentarios["texto_comentario"])
print(f"{len(comentarios)} comentarios normalizados")
```

620 comentarios normalizados

1-2. Matriz término-documento y dispersidad real

```

cv = CountVectorizer(min_df=2)
X_cv = cv.fit_transform(comentarios["texto_normalizado"])

n_docs, n_vocab = X_cv.shape
dispersidad = 1 - X_cv.nnz / (n_docs * n_vocab)
print(f"CountVectorizer: {n_docs} documentos x {n_vocab} términos")
print(f"Dispersidad: {dispersidad:.1%} de la matriz son ceros")

```

CountVectorizer: 620 documentos x 514 términos
 Dispersidad: 97.9% de la matriz son ceros

3-4. Documentos más parecidos: BoW vs TF-IDF

Se elige un comentario cualquiera y se buscan los 3 más parecidos con cada representación.

```

from sklearn.metrics.pairwise import cosine_similarity

tfidf = TfidfVectorizer(min_df=2)
X_tf = tfidf.fit_transform(comentarios["texto_normalizado"])

idx_consulta = 0
print("Consulta:", comentarios.loc[idx_consulta, "texto_comentario"][:100], "...\\n")

for nombre, matriz in [("BoW", X_cv), ("TF-IDF", X_tf)]:
    sims = cosine_similarity(matriz[idx_consulta], matriz).ravel()
    top = sims.argsort()[::-1][1:4] # excluye el propio documento
    print(f"-- Top 3 más parecidos según {nombre} --")
    for i in top:
        print(f"    [{sims[i]:.3f}] {comentarios.loc[i, 'texto_comentario'][:90]}...")
    print()

```

Consulta: El Smartphone Nexus 5G es un cambio de juego. La pantalla OLED es vívida, la cámara de 108MP captura ...

-- Top 3 más parecidos según BoW --

[0.503] El Smartphone Nexus 5G es rápido y la pantalla es brillante, pero la batería se drena un p...

[0.470] El Smartphone Nexus es muy rápido y la pantalla es de una calidad impresionante. La cámara...

[0.450] El Smartphone Nexus es bueno en general. Sin embargo, la batería se agota rápidamente con ...

-- Top 3 más parecidos según TF-IDF --

[0.415] El Smartphone Nexus es muy rápido y la pantalla es de una calidad impresionante. La cámara...

[0.361] El Smartphone Nexus 5G es rápido y la pantalla es brillante, pero la batería se drena un p...

[0.348] El Smartphone Nexus es bueno en general. Sin embargo, la batería se agota rápidamente con ...

Reto — ngram_range=(1, 2)

Contar también pares de palabras (bigramas) recupera algo de orden, al costo de un vocabulario mucho más grande.

```
import time

for ngram_range in [(1, 1), (1, 2)]:
    inicio = time.perf_counter()
    vect = CountVectorizer(min_df=2, ngram_range=ngram_range)
    X_ng = vect.fit_transform(comentarios["texto_normalizado"])
    duracion = time.perf_counter() - inicio
    print(f"ngram_range={ngram_range}: |V| = {X_ng.shape[1]:>5}
    ({duracion*1000:.0f} ms)")
```

```
ngram_range=(1, 1): |V| = 514 (4 ms)
ngram_range=(1, 2): |V| = 1532 (6 ms)
```

El vocabulario crece varias veces al incluir bigramas, y la mayoría de esos pares nuevos aparece una sola vez en todo el corpus (son demasiado específicos para generalizar). En un corpus de 619 comentarios el costo en tiempo todavía es insignificante; con miles de documentos deja de serlo. El ranking de similitud tiende a moverse poco: la mayoría de la señal ya la aportan las palabras sueltas.

Siguiente: `04_word2vec.ipynb` — vectores densos que aprenden del contexto en lugar de contar.

04 · Word2Vec

Parte 3 — embeddings densos. TF-IDF sigue sin saber que "rey" y "monarca" significan casi lo mismo: son sinónimos, pero ocupan posiciones distintas del vector y su coseno da 0. Además, con 50 000 columnas casi todas las palabras terminan pareciendo igual de "lejanas" entre sí (la maldición de la dimensionalidad). Word2Vec resuelve ambos problemas **aprendiendo** los números en vez de contarlos.

Hipótesis distribucional

"Una palabra se conoce por la compañía que mantiene." — J. R. Firth, 1957

"café" y "té" casi nunca aparecen en la misma frase, pero comparten vecinos ("bebí", "taza", "caliente"). Si los vectores se construyen a partir de esos vecinos compartidos, "café" y "té" terminan próximos aunque nunca hayan coincidido en un texto — el significado se deriva de la estadística de uso, no de un diccionario.

Disperso vs. denso: la misma palabra, dos formatos

Un vector disperso (Bag-of-Words) enciende una sola posición por palabra; uno denso tiene un valor en cada posición y muchas menos dimensiones.

```
import numpy as np
import pandas as pd

def similitud_coseno(a, b):
    a, b = np.asarray(a, dtype=float), np.asarray(b, dtype=float)
    return a.dot(b) / (np.linalg.norm(a) * np.linalg.norm(b))

# Embeddings de juguete (4 dimensiones) solo para ilustrar la mecánica
embeddings_juguete = pd.DataFrame(
    {
        "gato": [0.91, 0.12, 0.83, 0.05],
        "perro": [0.88, 0.19, 0.79, 0.09],
        "sismo": [0.07, 0.94, 0.11, 0.88],
    }
).T
embeddings_juguete.columns = ["dim1", "dim2", "dim3", "dim4"]
embeddings_juguete
```

	dim1	dim2	dim3	dim4
gato	0.91	0.12	0.83	0.05
perro	0.88	0.19	0.79	0.09
sismo	0.07	0.94	0.11	0.88

```
cos_gato_perro = similitud_coseno(embeddings_juguete.loc["gato"],
embeddings_juguete.loc["perro"])
cos_gato_sismo = similitud_coseno(embeddings_juguete.loc["gato"],
embeddings_juguete.loc["sismo"])
print(f"cos(gato, perro) = {cos_gato_perro:.3f}")
print(f"cos(gato, sismo) = {cos_gato_sismo:.3f}")
```

```
cos(gato, perro) = 0.997
cos(gato, sismo) = 0.195
```

Con TF-IDF, "gato" y "perro" daban coseno 0 al no compartir ningún término del vocabulario. Con embeddings alcanzan 0.997: comparten vecinos, aunque no compartan ninguna letra.

Cómo se consiguen esos números

Nadie escribe a mano los valores de un embedding. Se entrena una red con una tarea de mentira — "dada una palabra, adivina qué palabras la rodean" — y al terminar se descarta la tarea: lo que interesa no es la predicción, sino la tabla de pesos que la red ajustó por dentro. Como el propio texto contiene la respuesta, no hace falta etiquetar nada (**auto-supervisado**): se puede entrenar con millones de oraciones sin que nadie anote una sola línea.

Existen dos variantes de la tarea: **Skip-gram** (entrada: la palabra central → predice sus vecinos) y **CBOW** (entrada: los vecinos → predice la palabra central). Skip-gram es más lento de entrenar pero aprende mejor las palabras poco frecuentes, y es la variante que se usa por defecto en la mayoría de modelos publicados.

El obstáculo: para cada ejemplo de entrenamiento, predecir la palabra vecina exacta entre un vocabulario de un millón de palabras obliga a un softmax sobre esas mismas un millón de opciones — carísimo, y se repite miles de millones de veces. La solución, **negative sampling**, cambia la pregunta: en vez de "¿cuál de las 1 000 000 de palabras sigue a 'gato'?" se pregunta "¿este par de palabras salió del texto o me lo inventé?" — una pregunta de sí/no que solo toca un puñado de vectores (1 real + ~10 falsos) en cada paso, no el vocabulario entero. Levy & Goldberg (2014) probaron que entrenar así produce, en el límite, los mismos vectores que factorizar la enorme tabla de coocurrencias palabra-palabra del corpus — el entrenamiento la factoriza sin llegar a construirla.

Entrenar un Word2Vec propio

Se entrena sobre los 619 comentarios de clientes con `gensim`. Es un corpus minúsculo para este método (Word2Vec necesita millones de oraciones para generalizar bien), así que sirve para ver la mecánica, no para obtener vectores de calidad — eso se resuelve con un modelo preentrenado más abajo.

```

import spacy
from gensim.models import Word2Vec

nlp = spacy.load("es_core_news_sm")
comentarios = pd.read_csv("../data/comentarios.csv")

def tokenizar(textos):
    return [
        [t.lemma_ for t in doc if t.is_alpha and not t.is_stop]
        for doc in nlp.pipe((str(x).lower() for x in textos), batch_size=64)
    ]

oraciones = tokenizar(comentarios["texto_comentario"])

modelo_propio = Word2Vec(
    sentences=oraciones,
    vector_size=50, # dimensión pequeña, acorde al tamaño del corpus
    window=5,
    min_count=3,
    sg=1, # skip-gram
    workers=4,
    seed=42,
)

print(f"Vocabulario aprendido: {len(modelo_propio.wv):,} palabras")
print(modelo_propio.wv.most_similar("laptop", topn=5))

```

Vocabulario aprendido: 410 palabras
 [('rendimiento', 0.9952747821807861), ('manejar', 0.9951295256614685), ('juego', 0.9950977563858032), ('vibrante', 0.9948370456695557), ('frustrante', 0.9946812987327576)]

```

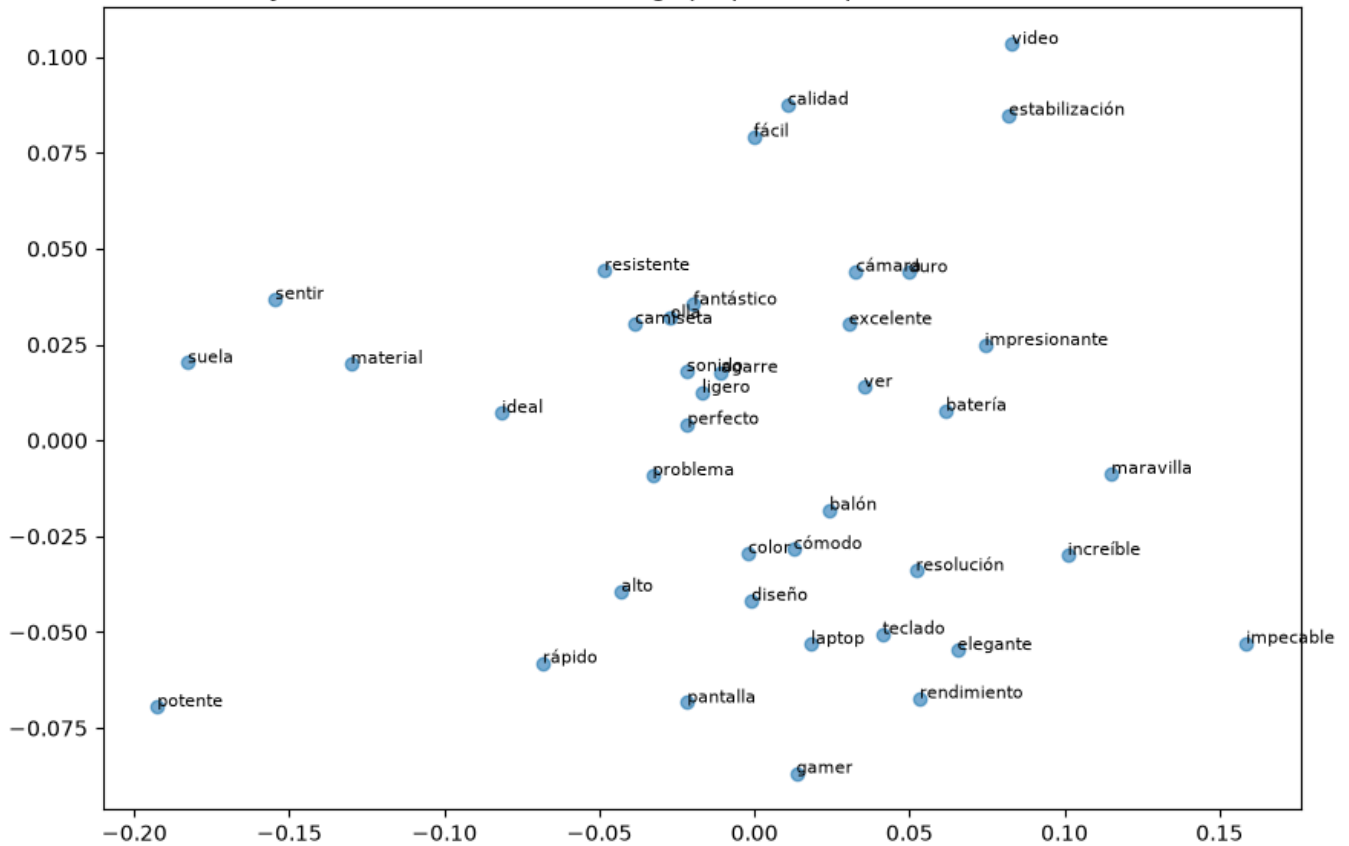
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

palabras_frecuentes = modelo_propio.wv.index_to_key[:40]
vectores = modelo_propio.wv[palabras_frecuentes]
coords = PCA(n_components=2, random_state=42).fit_transform(vectores)

plt.figure(figsize=(9, 6))
plt.scatter(coords[:, 0], coords[:, 1], alpha=0.6)
for (x, y), palabra in zip(coords, palabras_frecuentes):
    plt.annotate(palabra, (x, y), fontsize=8)
plt.title("Proyección PCA de los embeddings propios (40 palabras más frecuentes)")
plt.tight_layout()
plt.show()

```

Proyección PCA de los embeddings propios (40 palabras más frecuentes)



Palabras que aparecen en contextos parecidos (categorías de producto, adjetivos de opinión) tienden a agruparse, aunque con solo 619 comentarios el resultado es ruidoso.

Aritmética de significados

Si los vectores codifican significado, sumarlos y restarlos debería dar resultados con sentido:

$\vec{rey} - \vec{hombre} + \vec{mujer} \approx \vec{reina}$. El corpus propio es demasiado chico para que esto emerja con calidad, así que se usa un modelo preentrenado sobre miles de millones de palabras: **GloVe** (Pennington et al., 2014), una variante de la misma familia de embeddings densos.

Ejercicio 02 — Analogías vectoriales

Objetivo: ¿la analogía es una propiedad real del espacio vectorial, o un efecto del método de consulta?

```
import gensim.downloader as api

wv = api.load("glove-wiki-gigaword-100") # 400k palabras en inglés, 100 dimensiones
print(f"Vocabulario preentrenado: {len(wv):,} palabras")

[=====] 100.0% 128.1/128.1MB downloaded
Vocabulario preentrenado: 400,000 palabras
```

1. `most_similar` con términos del dominio

```
for termino in ["laptop", "phone", "camera"]:
    print(f"{termino}: {wv.most_similar(termino, topn=5)}")
```

```
laptop: [('laptops', 0.8518659472465515), ('computers', 0.7559927105903625),
('phones', 0.7229112386703491), ('portable', 0.7157840728759766), ('desktop',
0.7085692286491394)]
phone: [('telephone', 0.9113394021987915), ('cellphone', 0.8122149705886841),
('phones', 0.8030568957328796), ('mobile', 0.7307463884353638), ('mail',
0.7292180061340332)]
camera: [('cameras', 0.8474849462509155), ('screen', 0.754730224609375), ('video',
0.6977447867393494), ('screens', 0.678825855255127), ('microphone',
0.6621578335762024)]
```

2. Reproducir la analogía con la librería

```
resultado_libreria = wv.most_similar(positive=["king", "woman"], negative=["man"],
topn=5)
print("king - man + woman ≈")
for palabra, score in resultado_libreria:
    print(f" {palabra:12} {score:.3f}")
```

```
king - man + woman ≈
queen      0.770
monarch    0.684
throne     0.676
daughter   0.659
princess   0.652
```

3. Repetirla a mano con numpy, sin excluir los vectores de entrada

`most_similar` excluye por defecto las palabras que aparecen en la consulta (`king` , `woman` , `man`). Repitiendo el cálculo a mano y buscando entre **todo** el vocabulario se ve qué tan real es el efecto.

```
vector_resultado = wv["king"] - wv["man"] + wv["woman"]

matriz = wv.vectors # (vocab, dim)
normas = np.linalg.norm(matriz, axis=1) * np.linalg.norm(vector_resultado)
cosenos = matriz @ vector_resultado / normas

top_idx = np.argsort(cosenos)[::-1][:6]
print("Vecinos más cercanos a (king - man + woman), sin excluir nada:")
for i in top_idx:
    print(f" {wv.index_to_key[i]:12} {cosenos[i]:.3f}")
```

```
Vecinos más cercanos a (king - man + woman), sin excluir nada:
king      0.855
queen     0.783
monarch    0.693
throne     0.683
daughter   0.681
prince     0.671
```

Sin el filtro, "king" vuelve a aparecer entre los primeros resultados: la resta desplaza el vector hacia "queen", pero no lo suficiente para dejar atrás a la palabra original. El efecto es real (queen aparece muy arriba, con un coseno alto) aunque el filtro de `most_similar` lo exagera un poco.

4. Coseno entre sinónimos, antónimos y pares sin relación

```
pares = [  
    ("car", "automobile", "sinónimos"),  
    ("hot", "cold", "antónimos"),  
    ("happy", "sad", "antónimos"),  
    ("computer", "banana", "sin relación"),  
]  
  
for a, b, tipo in pares:  
    print(f"cos({a}, {b}) = {wv.similarity(a, b):.3f}    [{tipo}]")  
  
cos(car, automobile) = 0.683    [sinónimos]  
cos(hot, cold) = 0.725    [antónimos]  
cos(happy, sad) = 0.680    [antónimos]  
cos(computer, banana) = 0.121    [sin relación]
```

Los antónimos ("hot"/"cold") quedan sorprendentemente cerca — casi tan cerca como los sinónimos. Tiene sentido con la hipótesis distribucional: "the coffee is hot" y "the coffee is cold" comparten exactamente el mismo contexto, así que el modelo los coloca en la misma zona del espacio. La similitud vectorial captura **relación temática**, no necesariamente "significan lo mismo".

Siguiente: `05_transformers-sbert.ipynb` — un vector distinto para cada sentido de una palabra, según la frase en la que aparece.

05 · Transformers y SBERT

Parte 4 — vectores que cambian según la frase. Word2Vec guarda **un solo vector por palabra**, así que sus distintos sentidos tienen que compartir esa única casilla. El español está lleno de casos de polisemia: "banco" (asiento del parque / entidad financiera), "gato" (animal / herramienta del auto), "sierra" (cadena de montañas / herramienta de cortar). Para separar esos sentidos, el vector de una palabra tiene que construirse **mirando a las palabras que la acompañan** — a eso se le llama *self-attention*.

Comprobación: el mismo vector, dos contextos

Con un modelo Transformer real (el mismo `MiniLM` multilingüe que se usa más abajo para SBERT) se extrae el vector interno de "banco" en dos frases distintas y se compara.

```
import numpy as np
import torch
from transformers import AutoModel, AutoTokenizer

MODEL_NAME = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
modelo_base = AutoModel.from_pretrained(MODEL_NAME)
modelo_base.eval()

def vector_de_palabra(texto, palabra):
    # Vector contextual (último estado oculto) del primer token que cubre `palabra`
    codificado = tokenizer(texto, return_offsets_mapping=True, return_tensors="pt")
    offsets = codificado.pop("offset_mapping")[0]
    with torch.no_grad():
        estados = modelo_base(**codificado).last_hidden_state[0]

    inicio = texto.lower().index(palabra)
    fin = inicio + len(palabra)
    for i, (a, b) in enumerate(offsets.tolist()):
        if a < fin and b > inicio:
            return estados[i].numpy()
    raise ValueError(f"No se encontró {palabra!r} en {texto!r}")

def similitud_coseno(a, b):
    return a.dot(b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

```
/private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/venv311/lib/python3.11/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
Loading weights: 100%|██████████| 199/199 [00:00<00:00, 8616.53it/s]
```

```

frase_parque = "me senté en el banco del parque a descansar"
frase_dinero = "retiré dinero del banco esta mañana"

banco_parque = vector_de_palabra(frase_parque, "banco")
banco_dinero = vector_de_palabra(frase_dinero, "banco")
parque = vector_de_palabra(frase_parque, "parque")
dinero = vector_de_palabra(frase_dinero, "dinero")

print(f"cos(banco@parque, banco@dinero) = {similitud_coseno(banco_parque,
banco_dinero):.3f} (mismo token, dos frases)")
print(f"cos(banco@parque, 'parque') = {similitud_coseno(banco_parque,
parque):.3f} (mismo contexto)")
print(f"cos(banco@dinero, 'dinero') = {similitud_coseno(banco_dinero,
dinero):.3f} (mismo contexto)")
print(f"cos(banco@parque, 'dinero') = {similitud_coseno(banco_parque,
dinero):.3f} (contexto cruzado)")

cos(banco@parque, banco@dinero) = 0.208 (mismo token, dos frases)
cos(banco@parque, 'parque') = 0.712 (mismo contexto)
cos(banco@dinero, 'dinero') = 0.715 (mismo contexto)
cos(banco@parque, 'dinero') = 0.122 (contexto cruzado)

```

El vector de "banco" **no es el mismo** en las dos frases (el primer coseno queda lejos de 1) y cada versión se acerca más a las palabras de su propio contexto que al contexto de la otra frase. Con Word2Vec ese primer coseno habría sido exactamente 1.0, porque solo existe un vector por palabra.

Cómo lo logra: preguntar a los vecinos (self-attention)

Antes de fijar el vector de una palabra, el modelo revisa el resto de la frase y se queda con lo que le sirve. Cada palabra genera tres versiones de sí misma:

- **Query** — lo que esa palabra necesita saber ("¿de qué tipo de banco se habla?").
- **Key** — la etiqueta con la que se anuncia ("soy un lugar al aire libre").
- **Value** — la información que entrega si la eligen.

El cálculo tiene tres pasos: comparar la Query de una palabra contra la Key de cada palabra de la frase, convertir esas comparaciones en porcentajes con softmax, y mezclar los Value de todas usando esos porcentajes como pesos.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V$$

Dividir entre $\sqrt{d_k}$ evita que los productos se disparen y el softmax termine dando 1.0 a una sola palabra y 0.0 al resto.

```

def softmax(x):
    x = x - x.max()
    e = np.exp(x)
    return e / e.sum()

def self_attention(query, keys, values):
    d_k = query.shape[-1]
    puntajes = keys @ query / np.sqrt(d_k)

```

```

pesos = softmax(puntajes)
return pesos, pesos @ values

# Vectores de juguete (3 dimensiones) solo para ilustrar el mecanismo,
# igual que el ejemplo de la diapositiva: "la cámara tiene buena calidad de imagen"
palabras = ["la", "cámara", "tiene", "buena", "calidad", "de", "imagen"]
K = np.array([
    [0.00, 0.00, 1.00], # la      -> función gramatical, sin contenido
    [0.90, 0.70, 0.10], # cámara -> contenido visual
    [0.00, 0.10, 0.90], # tiene  -> función gramatical
    [0.50, 0.60, 0.20], # buena  -> valoración
    [1.00, 0.80, 0.00], # calidad -> es la propia consulta
    [0.00, 0.00, 1.00], # de      -> función gramatical
    [0.95, 0.75, 0.05], # imagen -> contenido visual
])
V = K.copy() # simplificación: en un Transformer real, V se aprende por separado

query_calidad = K[palabras.index("calidad")]
pesos, mezcla = self_attention(query_calidad, K, V)

for palabra, peso in zip(palabras, pesos):
    barra = "█" * int(peso * 40)
    print(f"{palabra:8} {peso:.3f} {barra}")

```

la	0.082	██
cámara	0.191	██████
tiene	0.086	██
buena	0.145	████
calidad	0.212	██████
de	0.082	██
imagen	0.201	██████

"cámara" e "imagen" concentran la mayor parte del peso, y "la", "tiene", "de" quedan casi en cero: el nuevo vector de "calidad" termina siendo una mezcla dominada por sus vecinos de contenido, no por las palabras gramaticales. Si "calidad" apareciera en otra frase (por ejemplo, "la calidad del servicio"), el reparto sería distinto y su vector también — ahí nace el vector contextual.

Una sola ronda de atención (una "cabeza") solo puede fijarse en un tipo de relación. Los Transformers reales repiten este cálculo varias veces en paralelo (**multi-head attention**, 12 cabezas en BERT-base, hasta 96 en GPT-3) y concatenan los resultados, de modo que cada cabeza puede especializarse en una relación distinta (quién, dónde, cuándo...) sin que el costo total crezca de forma proporcional.

Lo que la atención no ve: el orden

La atención compara todas las palabras entre sí, pero no registra en qué posición está cada una — sin más, "el perro mordió al hombre" y "el hombre mordió al perro" producirían el mismo conjunto de vectores. La solución es sumar a cada palabra una marca que depende de su posición, construida con senos y cosenos de distinta frecuencia por dimensión:

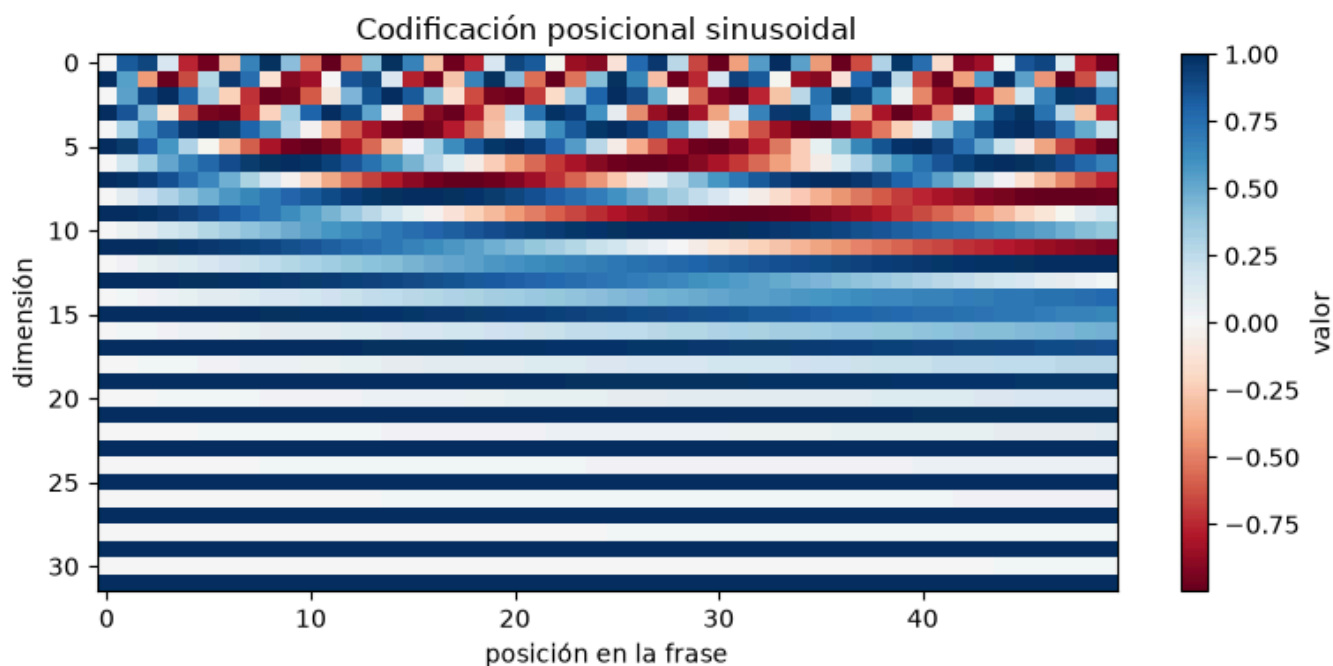
$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

```
import matplotlib.pyplot as plt

def codificacion_posicional(n_posiciones, dim):
    posiciones = np.arange(n_posiciones)[: , None]
    i = np.arange(dim)[None, :]
    angulos = posiciones / (10000 ** (2 * (i // 2) / dim))
    pe = np.zeros((n_posiciones, dim))
    pe[:, 0::2] = np.sin(angulos[:, 0::2])
    pe[:, 1::2] = np.cos(angulos[:, 1::2])
    return pe

pe = codificacion_posicional(n_posiciones=50, dim=32)

plt.figure(figsize=(8, 4))
plt.imshow(pe.T, aspect="auto", cmap="RdBu")
plt.xlabel("posición en la frase")
plt.ylabel("dimensión")
plt.title("Codificación posicional sinusoidal")
plt.colorbar(label="valor")
plt.tight_layout()
plt.show()
```



Las dimensiones bajas oscilan rápido (distinguen posiciones vecinas) y las altas oscilan lento (distinguen el inicio del final del texto). Los valores quedan acotados entre -1 y 1 , y como es una función matemática (no una tabla aprendida), el modelo puede extrapolar a textos más largos que los vistos en entrenamiento. Queda un costo: comparar todas las palabras con todas hace que el cómputo crezca con el cuadrado de la longitud del texto.

BERT y GPT: qué se les permite mirar

La diferencia entre las dos familias de Transformers está en qué palabras puede mirar cada una al calcular su atención:

	Dirección	Se entrena	Sirve para
BERT	ambos lados	tapar una palabra y adivinarla viendo antes y después	clasificar, buscar, extraer información
GPT	solo hacia atrás	predecir la siguiente palabra sin ver el final	generar, resumir, conversar

Esa restricción de GPT es justo lo que permite entrenar con cualquier texto de internet sin etiquetar nada — de ahí salen los modelos de lenguaje actuales.

SBERT: un vector por oración, no por palabra

BERT entrega un vector por palabra, no por documento. Comparar una consulta contra 10 000 documentos pasando cada par junto por BERT obligaría a correr el modelo 10 000 veces. **SBERT** (Reimers & Gurevych, 2019) resuelve esto entrenando el modelo para que la cercanía entre vectores de *oraciones completas* refleje su similitud de significado: cada documento se vectoriza **una sola vez** y las búsquedas se reducen a comparar vectores ya calculados con el coseno (arquitectura *bi-encoder*). Es lo que hace viable la búsqueda semántica y los sistemas RAG en producción.

Ejercicio 03 — Buscador semántico

Objetivo: construir un buscador que encuentre productos por significado y no por coincidencia de palabras, y comparar en qué casos supera a TF-IDF y en cuáles queda por debajo.

```
import pandas as pd
from sentence_transformers import SentenceTransformer

productos = pd.read_csv("../data/productos.csv")
sbert = SentenceTransformer(MODEL_NAME)

embeddings_productos = sbert.encode(
    productos["descripcion"].tolist(), convert_to_numpy=True,
    normalize_embeddings=True
)
print(f"Embeddings: {embeddings_productos.shape}")
```

```
Loading weights: 100%|██████████| 199/199 [00:00<00:00, 14117.22it/s]
Embeddings: (10, 384)
```

```
def buscar_semantico(consulta, top_k=3):
    vector_consulta = sbert.encode([consulta], normalize_embeddings=True)[0]
    similitudes = embeddings_productos @ vector_consulta
    top = similitudes.argsort()[::-1][:top_k]
    return [(productos.loc[i, "nombre_producto"], round(float(similitudes[i]), 3))
            for i in top]

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

tfidf_productos = TfidfVectorizer()
X_tfidf_productos = tfidf_productos.fit_transform(productos["descripcion"])
```

```
def buscar_tfidf(consulta, top_k=3):
    vector_consulta = tfidf_productos.transform([consulta])
    similitudes = cosine_similarity(vector_consulta, X_tfidf_productos).ravel()
    top = similitudes.argsort()[::-1][:top_k]
    return [(productos.loc[i, "nombre_producto"], round(float(similitudes[i]), 3))
            for i in top]
```

Tanda 1 — consultas parafraseadas (sin palabras en común con el producto)

```
consultas_parafraseadas = [
    "algo para grabar mis aventuras al aire libre",
    "un libro para perderme en un mundo de magia",
    "quiero jugar videojuegos exigentes con gráficos potentes",
]
for consulta in consultas_parafraseadas:
    print(f"\n'{consulta}'")
    print(" SBERT :", buscar_semantico(consulta))
    print(" TF-IDF:", buscar_tfidf(consulta))
```

```
'algo para grabar mis aventuras al aire libre'
  SBERT : [('Cámara de Acción 4K', 0.435), ('Zapatillas Urbanas Fit', 0.324),
('Camiseta Deportiva Ultralight', 0.298)]
  TF-IDF: [('Balón de Fútbol Pro', 0.183), ('Cámara de Acción 4K', 0.112), ('Mochila
Urbana Tech', 0.095)]
```

```
'un libro para perderme en un mundo de magia'
  SBERT : [('Libro: El Círculo Mágico', 0.524), ('Zapatillas Urbanas Fit', 0.122),
('Camiseta Deportiva Ultralight', 0.109)]
  TF-IDF: [('Libro: El Círculo Mágico', 0.275), ('Balón de Fútbol Pro', 0.268),
('Auriculares Inalámbricos X', 0.235)]
```

```
'quiero jugar videojuegos exigentes con gráficos potentes'
  SBERT : [('Cámara de Acción 4K', 0.452), ('Laptop Gamer Pro', 0.426), ('Balón de
Fútbol Pro', 0.255)]
  TF-IDF: [('Mochila Urbana Tech', 0.216), ('Smartphone Nexus 5G', 0.154), ('Laptop
Gamer Pro', 0.145)]
```

Tanda 2 — consultas con términos técnicos exactos

```
consultas_exactas = ["RTX 4080", "108MP", "DDR5"]
for consulta in consultas_exactas:
    print(f"\n'{consulta}'")
    print(" SBERT :", buscar_semantico(consulta))
    print(" TF-IDF:", buscar_tfidf(consulta))
```

'RTX 4080'

SBERT : [('Laptop Gamer Pro', 0.373), ('Cámara de Acción 4K', 0.292), ('Smartphone Nexus 5G', 0.259)]

TF-IDF: [('Laptop Gamer Pro', 0.169), ('Cámara de Acción 4K', 0.0), ('Mochila Urbana Tech', 0.0)]

'108MP'

SBERT : [('Laptop Gamer Pro', 0.389), ('Cámara de Acción 4K', 0.357), ('Auriculares Inalámbricos X', 0.317)]

TF-IDF: [('Smartphone Nexus 5G', 0.127), ('Cámara de Acción 4K', 0.0), ('Mochila Urbana Tech', 0.0)]

'DDR5'

SBERT : [('Laptop Gamer Pro', 0.392), ('Smartphone Nexus 5G', 0.325), ('Mochila Urbana Tech', 0.235)]

TF-IDF: [('Laptop Gamer Pro', 0.12), ('Cámara de Acción 4K', 0.0), ('Mochila Urbana Tech', 0.0)]

Entregable: recomendación razonada

Con solo 10 productos el catálogo es chico, pero el patrón ya se nota en los resultados de arriba:

Tanda	Producto correcto en el top-1	Producto correcto en el top-3
Parafraseada — SBERT	2 de 3	3 de 3
Parafraseada — TF-IDF	1 de 3	3 de 3
Términos exactos — SBERT	2 de 3	3 de 3
Términos exactos — TF-IDF	3 de 3	3 de 3

TF-IDF nunca falla con los códigos técnicos porque "108MP" o "DDR5" son casi nombres propios y el matcheo es literal. SBERT los sitúa peor — "108MP" termina más cerca de la laptop que del smartphone, porque el embedding entiende "especificación técnica de electrónica" pero no lee la sigla letra por letra. En las consultas parafraseadas ocurre lo contrario: SBERT ubica el producto correcto primero incluso sin compartir palabras con la descripción, mientras TF-IDF solo acierta el top-1 cuando hay alguna coincidencia literal de fondo (como "libro").

Ninguno de los dos gana siempre: en producción los buscadores casi nunca eligen un solo método, sino que combinan ambos enfoques (**búsqueda híbrida**) — primero recuperan candidatos con los dos y luego reordenan solo los mejores con un modelo más caro y preciso.

Siguiente: `06_sistema-recomendacion.ipynb` — usar estos mismos embeddings de producto para recomendar, y el problema de agregarlos en un perfil de cliente.

06 · Práctica integradora — Sistema de recomendación

Aplicación a un caso de negocio real de la tienda virtual: a partir del catálogo de productos y el historial de compras de cada cliente, generar un top-N de recomendaciones.

Productos --descripción--> embedding
Cliente --promedio de lo que compró--> perfil
Perfil --coseno contra lo NO comprado--> Top-N

El mismo `SentenceTransformer` de `05_transformers-sbert.ipynb` sirve para vectorizar productos; lo nuevo es cómo se agregan varios vectores de producto en **un** vector de cliente.

```
import numpy as np
import pandas as pd
from sentence_transformers import SentenceTransformer

productos = pd.read_csv("../data/productos.csv")
clientes = pd.read_csv("../data/clientes.csv")
historial = pd.read_csv("../data/historial_de_compras.csv")

MODEL_NAME = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
sbert = SentenceTransformer(MODEL_NAME)

texto_producto = productos["nombre_producto"] + ". " + productos["descripcion"]
embeddings_producto = sbert.encode(texto_producto.tolist(), convert_to_numpy=True,
normalize_embeddings=True)
id_a_indice = {pid: i for i, pid in enumerate(productos["id_producto"])}

print(f"{len(productos)} productos vectorizados en {embeddings_producto.shape[1]} dimensiones")
```

```
/private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/venv311/lib/python3.11/site-
packages/tqdm/auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and
ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
Loading weights: 100%|██████████| 199/199 [00:00<00:00, 42027.52it/s]
10 productos vectorizados en 384 dimensiones
```

Perfil de cliente: promedio de lo que compró

Cada compra aporta el embedding de su producto; el perfil del cliente es el promedio de esos vectores (una compra repetida pesa más, porque aparece más veces en el promedio).

```
def perfil_cliente(id_cliente):
    compras = historial.loc[historial["id_cliente"] == id_cliente, "id_producto"]
    indices = [id_a_indice[pid] for pid in compras]
    vector = embeddings_producto[indices].mean(axis=0)
    return vector / np.linalg.norm(vector), set(compras)
```

```
def recomendar(id_cliente, top_n=3, agregacion=perfil_cliente):
    vector_cliente, comprados = agregacion(id_cliente)
    similitudes = embeddings_producto @ vector_cliente
    ranking = [
        (productos.loc[i, "nombre_producto"], productos.loc[i, "categoria"],
        round(float(similitudes[i]), 3))
        for i in similitudes.argsort()[::-1]
        if productos.loc[i, "id_producto"] not in comprados
    ]
    return ranking[:top_n]
```

Dos clientes muy distintos

- **Cliente 1** compró casi solo electrónica: 5 laptops, 3 smartphones, 1 auriculares, más un par de prendas.
- **Cliente 3** compró en las 5 categorías del catálogo, pero con las zapatillas repetidas 5 de sus 11 compras.

```
for id_cliente in [1, 3]:
    compras = historial.loc[historial["id_cliente"] == id_cliente, "id_producto"]
    categorias = productos.set_index("id_producto").loc[compras, "categoria"]
    print(f"Cliente {id_cliente} compró: {sorted(categorias.unique())}")
    print("  Top-3 recomendado:", recomendar(id_cliente))
    print()
```

Cliente 1 compró: ['Electrónica', 'Ropa']

Top-3 recomendado: [('Mochila Urbana Tech', 'Ropa', 0.625), ('Cámara de Acción 4K', 'Electrónica', 0.575), ('Set de Ollas Premium', 'Hogar', 0.434)]

Cliente 3 compró: ['Deportes', 'Electrónica', 'Hogar', 'Libros', 'Ropa']

Top-3 recomendado: [('Camiseta Deportiva Ultralight', 'Ropa', 0.741), ('Auriculares Inalámbricos X', 'Electrónica', 0.399), ('Cámara de Acción 4K', 'Electrónica', 0.384)]

El resultado del cliente 1 sorprende a primera vista: el top-1 es "Mochila Urbana Tech", catalogada como **Ropa**, no como electrónica. Pero su descripción incluye "funda acolchada para laptops" — el embedding lee el contenido semántico, no la etiqueta de categoría, y encuentra una prenda genuinamente relevante para alguien que compró cinco laptops. La recomendación #2, una cámara de acción, sí es electrónica pura.

El problema de la agregación

La intuición sería que el cliente 3, al comprar en cinco categorías, tendría un perfil "diluido y sin dirección". Los números muestran algo distinto: el promedio queda dominado por lo que más se repite, no por la variedad.

```
categoria_de = productos.set_index("id_producto")["categoria"]
compras_cliente_3 = historial.loc[historial["id_cliente"] == 3, "id_producto"]
print(compras_cliente_3.map(categoria_de).value_counts())
```

```
id_producto
Ropa          6
Libros         2
Electrónica    1
Hogar          1
Deportes       1
Name: count, dtype: int64
```

```
vector_cliente_3, _ = perfil_cliente(3)
```

```
promedio_por_categoria = {}
for categoria in productos["categoria"].unique():
    indices = productos.index[productos["categoria"] == categoria]
    v = embeddings_producto[indices].mean(axis=0)
    promedio_por_categoria[categoria] = v / np.linalg.norm(v)

for categoria, v in promedio_por_categoria.items():
    print(f" cos(perfil cliente 3, centroide {categoria:12}) = {vector_cliente_3 @ v:.3f}")
```

```
cos(perfil cliente 3, centroide Electrónica ) = 0.513
cos(perfil cliente 3, centroide Ropa          ) = 0.916
cos(perfil cliente 3, centroide Libros        ) = 0.333
cos(perfil cliente 3, centroide Hogar         ) = 0.565
cos(perfil cliente 3, centroide Deportes     ) = 0.576
```

"Ropa" se despegua con 0.916, muy por encima de cualquier otra categoría — no porque el cliente sea sobre todo comprador de ropa, sino porque las zapatillas, repetidas 5 veces, pesan 5 veces más que el libro o el balón que compró una sola vez. El promedio trata cada **compra** como un voto, no cada **interés**: un cliente explorador con una compra repetida termina pareciéndose, en el vector, a un cliente enfocado en esa única categoría.

Una alternativa: similitud al producto más cercano, no al promedio

En vez de mezclar todos los vectores comprados en uno solo (donde la repetición pesa), se puede recomendar por el producto **individual** que más se parece a cada candidato — cada compra vota una sola vez, sin importar cuántas veces se repitió.

```
def perfil_por_item_mas_cercano(id_cliente):
    compras = historial.loc[historial["id_cliente"] == id_cliente, "id_producto"]
    indices = [id_a_indice[pid] for pid in compras]
    return embeddings_producto[indices], set(compras)

def recomendar_por_item(id_cliente, top_n=3):
    vectores_comprados, comprados = perfil_por_item_mas_cercano(id_cliente)
    similitudes = (embeddings_producto @ vectores_comprados.T).max(axis=1) # mejor
    coincidencia individual
    ranking = [
        (productos.loc[i, "nombre_producto"], productos.loc[i, "categoria"],
        round(float(similitudes[i]), 3))
        for i in similitudes.argsort()[::-1]
        if productos.loc[i, "id_producto"] not in comprados
    ]
    return ranking[:top_n]
```

```
print("Cliente 3 – promedio de perfil:", recomendar(3))
print("Cliente 3 – mejor coincidencia individual:", recomendar_por_item(3))
```

Cliente 3 – promedio de perfil: [('Camiseta Deportiva Ultralight', 'Ropa', 0.741), ('Auriculares Inalámbricos X', 'Electrónica', 0.399), ('Cámara de Acción 4K', 'Electrónica', 0.384)]

Cliente 3 – mejor coincidencia individual: [('Camiseta Deportiva Ultralight', 'Ropa', 0.718), ('Smartphone Nexus 5G', 'Electrónica', 0.509), ('Cámara de Acción 4K', 'Electrónica', 0.501)]

El top-1 no cambia (la camiseta deportiva sigue siendo, con cualquiera de los dos criterios, el producto más cercano a algo que el cliente ya compró), pero el top-2 sí: pasa de "Auriculares" a "Smartphone Nexus 5G", más alineado con el resto del historial. El criterio por máximo evita que una compra repetida "vote" varias veces, sin necesidad de descartar información.

Entregable: top-N por cliente, con el criterio justificado

Se usa **promedio de perfil** como criterio por defecto — es más simple y, en un catálogo con miles de productos, diluye mejor el ruido de una compra aislada. Pero el experimento anterior deja una advertencia concreta: el promedio no pondera por *interés*, pondera por *cantidad de compras*, así que un cliente que solo repitió un producto puede terminar con un perfil casi idéntico al de alguien enfocado en esa categoría desde el principio. **Mejor coincidencia individual** es preferible cuando el historial mezcla categorías y las compras repetidas no deben dominar el perfil.

Preguntas abiertas que quedan sin resolver por diseño (no tienen una respuesta única, dependen del negocio):

- ¿Deduplicar antes de promediar, para que cada producto distinto vote una sola vez sin importar cuántas veces se recompró?
- ¿Ponderar las compras recientes más que las antiguas?
- ¿Cómo evaluar la calidad de las recomendaciones sin datos futuros (¿A/B testing? ¿ocultar la última compra y ver si el modelo la recupera?)
- Un cliente **nuevo sin historial** no tiene perfil que promediar — el método no cubre ese caso (*cold start*); ahí se recurre típicamente a recomendar lo más vendido o pedir preferencias explícitas al registrarse.

```
tabla_final = []
for id_cliente in clientes["id_cliente"]:
    if (historial["id_cliente"] == id_cliente).sum() == 0:
        continue
    top1 = recomendar(id_cliente, top_n=1)
    if top1:
        nombre, categoria, score = top1[0]
        tabla_final.append({"id_cliente": id_cliente, "recomendación #1": nombre,
                           "categoría": categoria, "similitud": score})

pd.DataFrame(tabla_final).head(10)
```

	id_cliente	recomendación #1	categoría	similitud
0	1	Mochila Urbana Tech	Ropa	0.625
1	2	Mochila Urbana Tech	Ropa	0.558
2	3	Camiseta Deportiva Ultralight	Ropa	0.741
3	4	Camiseta Deportiva Ultralight	Ropa	0.696
4	5	Camiseta Deportiva Ultralight	Ropa	0.512
5	6	Mochila Urbana Tech	Ropa	0.637
6	7	Mochila Urbana Tech	Ropa	0.564
7	8	Camiseta Deportiva Ultralight	Ropa	0.749
8	9	Camiseta Deportiva Ultralight	Ropa	0.638
9	10	Camiseta Deportiva Ultralight	Ropa	0.493

Siguiente: `07_sintesis-comparativa.ipynb` — las tres representaciones de esta sesión, una al lado de la otra, y cómo elegir entre ellas.

07 · Síntesis: las tres representaciones, lado a lado

Cierre de la sesión: la misma frase del corpus de trabajo, vectorizada con los tres enfoques vistos, y los criterios para elegir uno u otro fuera del salón de clase.

```
import numpy as np
from sentence_transformers import SentenceTransformer
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

corpus = ["el gato duerme en el sofá", "el perro duerme en la alfombra", "el sismo sacudió la costa"]
frase = corpus[0]

bow = CountVectorizer(token_pattern=r"(?u)\b\w+\b").fit(corpus)
tfidf = TfidfVectorizer(token_pattern=r"(?u)\b\w+\b").fit(corpus)
sbert = SentenceTransformer("sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2")

vector_bow = bow.transform([frase]).toarray()[0]
vector_tfidf = tfidf.transform([frase]).toarray()[0]
vector_embedding = sbert.encode([frase])[0]

for nombre, vector in [("Bag-of-Words", vector_bow), ("TF-IDF", vector_tfidf), ("Embedding", vector_embedding)]:
    ceros = int((vector == 0).sum())
    print(f"{nombre:14} dim={len(vector):>4} ceros={ceros:>4} ({ceros/len(vector):.0%}) primeros valores: {np.round(vector[:6], 2)}")
```

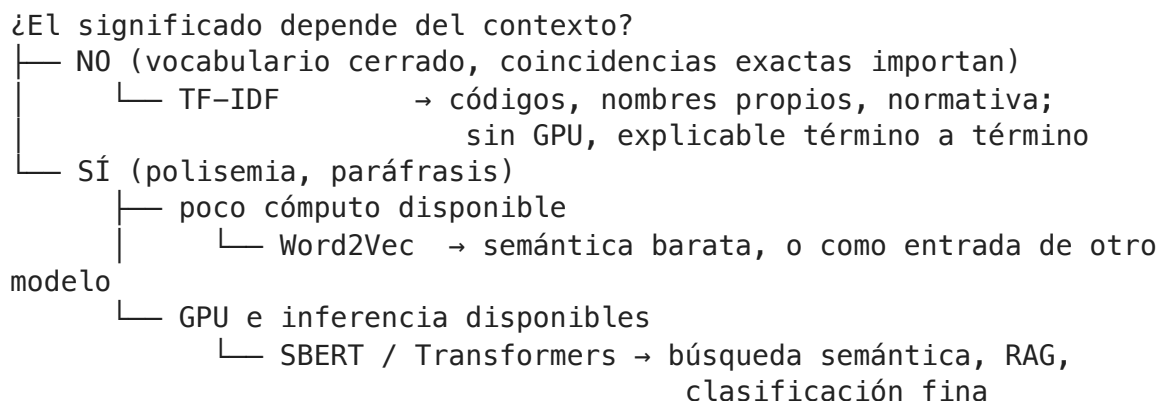
```
/private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/venv311/lib/python3.11/site-packages/tqdm/auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
Loading weights: 100%|██████████| 199/199 [00:00<00:00, 14520.99it/s]
Bag-of-Words    dim= 11   ceros= 6 (55%)   primeros valores: [0 0 1 2 1 1]
TF-IDF         dim= 11   ceros= 6 (55%)   primeros valores: [0.  0.  0.36 0.55 0.36 0.47]
Embedding      dim= 384   ceros= 0 (0%)    primeros valores: [ 0.48 -0.29 -0.13 0.18 0.07 0.3 ]
```

Representación	Dimensiones	Ceros	Qué determina los valores
Bag-of-Words	tamaño del vocabulario	casi todos	el conteo: cuántas veces aparece cada palabra
TF-IDF	tamaño del vocabulario	casi todos	el conteo, pesado por la rareza en el corpus
Embedding	fija (384 aquí; 300-1536 típico)	ninguno	un modelo entrenado sobre millones de oraciones

Registra apariciones sin ninguna noción de significado (BoW); distingue lo informativo de lo común, pero tampoco codifica significado (TF-IDF); codifica significado y contexto, a costa de no ser

interpretable (embedding). Un mismo texto, tres formas de convertirlo en números — la diferencia está en **qué determina esos valores**: el conteo, la estadística del corpus, o el modelo.

Criterios de selección



En producción casi nunca se elige uno solo: los buscadores combinan ambos enfoques (**búsqueda híbrida**) — primero recuperan candidatos con los dos métodos y luego reordenan solo los mejores con un modelo más caro y preciso. Así funcionó el buscador de `05_transformers-sbert.ipynb`.

Errores frecuentes en implementación

1. **Usar embeddings para todo.** Para buscar un código de factura, TF-IDF gana y cuesta mil veces menos cómputo.
2. **Promediar vectores sin pensar.** El promedio de un historial diverso —o dominado por una compra repetida, como se vio en `06_sistema-recomendacion.ipynb` — cae en un punto que no representa a nadie.
3. **Comparar cosenos entre modelos distintos.** 0.8 en un modelo y 0.8 en otro no significan lo mismo: los espacios vectoriales no son comparables entre sí.
4. **Olvidar el idioma del preentrenamiento.** Un modelo entrenado en inglés rinde mal en español técnico, aunque "funcione" en apariencia.
5. **No fijar una línea base.** Sin medir TF-IDF primero, no se sabe si el modelo caro aporta algo de verdad.

Glosario

Término	Definición
Vector disperso	Lista larga, casi toda de ceros. Bag-of-Words, TF-IDF.
Vector denso / embedding	Lista corta sin ceros, aprendida. Word2Vec, BERT.
Similitud coseno	Ángulo entre dos vectores. 1 = iguales, 0 = sin relación.
IDF	Dial que baja el peso de las palabras que salen en todos lados.
Negative sampling	Aproximación que evita recorrer todo el vocabulario en cada paso de Word2Vec.

Término	Definición
Self-attention	Cada palabra se recalcula mezclando a las demás según relevancia.
Contextual	El vector cambia según la frase. "Banco" ya no es uno solo.
Bi-encoder	Vectorizar por separado e indexar. Lo que hace viable buscar (SBERT).

Referencias

- Mikolov et al. (2013) — Word2Vec: la idea original.
- Levy & Goldberg (2014) — Word2Vec como factorización matricial.
- Pennington et al. (2014) — GloVe.
- Vaswani et al. (2017) — *Attention Is All You Need*.
- Devlin et al. (2019) — BERT.
- Reimers & Gurevych (2019) — Sentence-BERT.
- Jurafsky & Martin — *Speech and Language Processing*, cap. 6 (libre en línea).

Idea central de la sesión: vectorizar consiste en decidir qué relaciones queda en condiciones de detectar el modelo.

Sesión de clase 06 · Ejercicio 1: búsqueda semántica en comentarios.csv

Hasta la sesión anterior sabíamos convertir un texto en un vector y comparar dos vectores con similitud coseno. Hoy resolvemos el siguiente problema: dada una consulta, encontrar los k textos más parecidos entre **miles o millones** de comentarios, sin depender de que compartan las mismas palabras.

Este notebook reproduce el **Ejercicio 1** de la clase: comparar una búsqueda con `WHERE texto LIKE '%lento%'` contra una búsqueda semántica con **ChromaDB**, sobre los 135 comentarios reales de la tienda del curso.

Datos: `data/comentarios.csv` — `cliente_id`, `cliente_ciudad`, `cliente_pais`, `calificacion`, `fecha`, `texto`.

Modelo de embeddings: `paraphrase-multilingual-MiniLM-L12-v2` (384 dimensiones, multilingüe, funciona en CPU).

```
import pandas as pd
import chromadb
from chromadb.utils import embedding_functions

pd.set_option("display.max_colwidth", 100)
```

1. Cargar

Leemos `comentarios.csv` con pandas. Nos interesan sobre todo `texto`, `calificacion` y `cliente_ciudad`.

```
comentarios = pd.read_csv("../data/comentarios.csv")
print(comentarios.shape)
comentarios[["cliente_id", "cliente_ciudad", "cliente_pais", "calificacion",
"texto"]]
```

(135, 8)

	cliente_id	cliente_ciudad	cliente_pais	calificacion	texto
0	35	Trujillo	Peru	5	El proceso de facturacion para empresa fue mas simple de lo que esperaba.
1	90	Cuenca	Ecuador	3	En general cumple lo que promete, ni mejor ni peor que otras tiendas de tecnologia que he probado.
2	31	Arequipa	Peru	4	Buena tienda en general, seguire comprando aqui mis proximos equipos.
3	27	Lima	Peru	3	El envio internacional tardo mas dias de los indicados originalmente.
4	84	Rosario	Argentina	3	El proceso de devolucion fue sencillo de iniciar aunque tomo unos dias en confirmarse.
...	...	...	...	...	...
130	4	Arequipa	Peru	1	Me cobraron el envio dos veces por un error del sistema y tuve que escribir tres correos distint...
131	5	Arequipa	Peru	4	El proceso de pago con tarjeta fue rapido, solo me hubiera gustado mas metodos de pago.
132	39	Chiclayo	Peru	4	Me parece justo el precio de envio comparado con otras tiendas de electronica.
133	1	Lima	Peru	5	La pagina es muy facil de navegar, encuentre lo que buscaba en pocos minutos.
134	5	Arequipa	Peru	1	El sistema de puntos se reinicio sin explicacion despues de una actualizacion de la plataforma. ...

135 rows x 5 columns

¿Por qué no alcanza con LIKE ?

Necesidad: encontrar quejas sobre la lentitud del sitio web. La consulta SQL equivalente en pandas es `texto.str.contains("lento")`.

```
like_lento = comentarios[comentarios["texto"].str.contains("lento", case=False)]
print(f"{len(like_lento)} resultado(s) con LIKE '%lento%'")
like_lento[["calificacion", "texto"]]
```

3 resultado(s) con LIKE '%lento%'

	calificacion	texto
15	2	El proceso de cambio de producto por otro modelo fue mas lento de lo esperado.
41	3	El soporte responde en tiempos razonables, ni muy rapido ni muy lento comparado con otras tiendas.
117	1	La verificacion de identidad para mi primera compra tomo mas de tres dias habiles. Fue un proces...

Los comentarios que sí hablan de un sitio lento no siempre usan la palabra «lento»: dicen «tardan en cargar» o «podría cargar más rápido». `LIKE` no los encuentra porque busca la cadena exacta, no el significado. Y si ampliamos el patrón para intentar capturar variantes, aparecen falsos positivos:

```
like_lent = comentarios[comentarios["texto"].str.contains("lent", case=False)]
print(f"{len(like_lent)} resultado(s) con LIKE '%lent%'")
like_lent[["calificacion", "texto"]]
```

8 resultado(s) con LIKE '%lent%'

	calificacion	texto
15	2	El proceso de cambio de producto por otro modelo fue mas lento de lo esperado.
23	5	Excelente que muestren opiniones verificadas de compradores reales.
41	3	El soporte responde en tiempos razonables, ni muy rapido ni muy lento comparado con otras tiendas.
43	5	El servicio postventa fue excelente cuando necesite activar la garantia de mi laptop.
110	5	Excelente experiencia de principio a fin, desde la busqueda hasta la entrega del producto. Defin...
114	2	La pagina se puso lenta durante una oferta de temporada y perdi el producto que queria comprar. ...
117	1	La verificacion de identidad para mi primera compra tomo mas de tres dias habiles. Fue un proces...
123	5	Excelente variedad de marcas, encuentre opciones que no tenia en otras tiendas.

`%lent%` no solo trae los comentarios sobre lentitud: también atrapa «Excelente» porque la subcadena «lent» aparece ahí por coincidencia. Es el problema opuesto: ahora hay ruido. Esto es lo que motiva la búsqueda semántica.

2. Vectorizar y construir la colección de Chroma

Tres pasos: crear la colección (eligiendo el modelo de embedding y la métrica), agregar documentos (Chroma los vectoriza automáticamente) y consultar (la consulta se vectoriza con el mismo modelo).

Usamos `Client()` en memoria: alcanza para este ejercicio y se pierde al cerrar el proceso. La persistencia en disco (`PersistentClient`) se usa en el laboratorio.

```
ef = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="paraphrase-multilingual-MiniLM-L12-v2"
)
```

```

cliente = chromadb.Client()
col_comentarios = cliente.get_or_create_collection(
    name="comentarios",
    embedding_function=ef,
    metadata={"hnsw:space": "cosine"},
)

col_comentarios.add(
    ids=comentarios["id"].astype(str).tolist(),
    documents=comentarios["texto"].tolist(),
    metadatas=comentarios[["cliente_ciudad", "cliente_pais",
"calificacion"]].to_dict("records"),
)

print(col_comentarios.count(), "comentarios indexados")

```

Loading weights: 100%  199/199 [00:00<00:00, 9454.12it/s]
135 comentarios indexados

3. Consultar

`query()` devuelve **distancias**, no similitudes: con métrica coseno, `similitud = 1 - distancia`.

```

def buscar(coleccion, consulta, n_results=5):
    res = coleccion.query(query_texts=[consulta], n_results=n_results)
    filas = []
    for doc, dist, meta in zip(res["documents"][0], res["distances"][0],
res["metadatas"][0]):
        filas.append({
            "similitud": round(1 - dist, 3),
            "ciudad": meta.get("cliente_ciudad"),
            "calificacion": meta.get("calificacion"),
            "texto": doc,
        })
    return pd.DataFrame(filas)

buscar(col_comentarios, "el sitio web es lento")

```

	similitud	ciudad	calificacion	texto
0	0.651	Chimbote	2	La pagina se puso lenta durante una oferta de temporada y perdi el producto que queria comprar. ...
1	0.548	Cusco	3	El sitio funciona bien en general aunque algunas paginas tardan un poco mas de lo esperado en ca...
2	0.446	Trujillo	2	El sitio no guarda bien las preferencias de idioma y cada vez que entro tengo que cambiarlo de n...
3	0.437	Montevideo	3	El diseno del sitio en modo oscuro tiene algunos textos dificiles de leer.
4	0.414	Chiclayo	3	El diseno del sitio es agradable pero en modo oscuro algunos textos son dificiles de leer. Peque...

Comparado con `LIKE '%lento%'` (3 resultados, ninguno relevante) y `LIKE '%lent%'` (8 resultados, la mitad ruido), la búsqueda semántica recupera los comentarios sobre lentitud del sitio

aunque usen palabras distintas («tardan en cargar», «podría cargar más rápido»).

```
buscar(col_comentarios, "tuve problemas para pagar con mi tarjeta")
```

	similitud	ciudad	calificacion	texto
0	0.647	Trujillo	2	Tuve problemas para aplicar un cupon de descuento durante el checkout.
1	0.621	Cusco	2	La factura electronica llego con datos incorrectos y tuve que pedir que la corrigieran.
2	0.614	Ciudad de Mexico	1	Me cobraron dos veces por error y tuve que esperar varios días para que me reembolsaran.
3	0.602	Arequipa	4	El proceso de pago con tarjeta fue rapido, solo me hubiera gustado mas metodos de pago.
4	0.580	Chiclayo	1	Cancele un pedido y el reembolso tardo mas de diez días habiles en aparecer en mi tarjeta. El so...

```
buscar(col_comentarios, "quiero devolver un producto")
```

	similitud	ciudad	calificacion	texto
0	0.537	Santiago	3	El sitio se cayo un momento durante una oferta especial, tuve que reintentar la compra.
1	0.515	Manta	5	El programa de puntos por compras es un buen incentivo para volver a comprar.
2	0.511	Santiago	4	Muy buena atencion cuando pregunte por la disponibilidad de un modelo agotado. Me avisaron appena...
3	0.493	Medellin	1	Compre un producto marcado como disponible y luego me avisaron que no habia stock.
4	0.483	Chimbote	2	El precio de un producto cambio de un día para otro sin ninguna explicacion clara. Senti que per...

4. Umbral de similitud

`query()` siempre devuelve `n_results` documentos, aunque ninguno tenga relación con la consulta. El umbral es la similitud mínima para aceptar un resultado; con embeddings multilingües suele estar entre 0.35 y 0.50, y se ajusta probando con datos propios.

```
UMBRAL = 0.45
```

```
def buscar_con_umbral(coleccion, consulta, umbral=UMBRAL, n_results=10):  
    df = buscar(coleccion, consulta, n_results=n_results)  
    return df[df["similitud"] >= umbral]  
  
print("Resultados que superan el umbral (0.45):")  
buscar_con_umbral(col_comentarios, "el sitio web es lento")
```

Resultados que superan el umbral (0.45):

	similitud	ciudad	calificacion	texto
0	0.651	Chimbote	2	La pagina se puso lenta durante una oferta de temporada y perdi el producto que queria comprar. ...
1	0.548	Cusco	3	El sitio funciona bien en general aunque algunas paginas tardan un poco mas de lo esperado en ca...

Con un umbral más bajo entran más resultados, algunos menos relacionados; con uno más alto se descartan resultados dudosos pero también algunos relevantes. No hay un valor «correcto» - se fija probando consultas reales y viendo dónde empieza a aparecer ruido.

Discusión: similitud semántica no es lo mismo que sentimiento

La consulta sobre pagos con tarjeta trae comentarios sobre el **tema** pago/tarjeta, sin importar si la experiencia fue buena o mala.

```
resultado_pago = buscar(col_comentarios, "tuve problemas para pagar con mi tarjeta",
n_results=5)
resultado_pago
```

	similitud	ciudad	calificacion	texto
0	0.647	Trujillo	2	Tuve problemas para aplicar un cupon de descuento durante el checkout.
1	0.621	Cusco	2	La factura electronica llego con datos incorrectos y tuve que pedir que la corrigieran.
2	0.614	Ciudad de Mexico	1	Me cobraron dos veces por error y tuve que esperar varios días para que me reembolsaran.
3	0.602	Arequipa	4	El proceso de pago con tarjeta fue rapido, solo me hubiera gustado mas metodos de pago.
4	0.580	Chiclayo	1	Cancele un pedido y el reembolso tardo mas de diez días hábiles en aparecer en mi tarjeta. El so...

Preguntas de discusión

- ¿Por qué puede aparecer en el top-5 un comentario con calificación alta (5★) sobre pagos, aunque la consulta hable de un *problema*? (La similitud semántica mide de qué tema habla el texto, no si la experiencia fue positiva o negativa.)
- ¿Qué umbral usarían para descartar resultados poco relacionados? Prueben distintos valores en `buscar_con_umbral` y observen dónde el resultado deja de tener sentido.
- ¿Qué pasaría si compararan la consulta contra `resenas_entrega.csv` en vez de `comentarios.csv`? (Se explora en el laboratorio.)

Sesión de clase 06 · Ejercicio 2: clasificación de reseñas sin entrenamiento

Clasificación zero-shot: en vez de entrenar un clasificador con datos etiquetados, definimos cada categoría como una frase descriptiva, la vectorizamos igual que cualquier otro texto y asignamos a cada reseña la etiqueta cuyo vector sea más parecido al suyo. No hace falta ni un solo ejemplo etiquetado a mano.

Este notebook reproduce el **Ejercicio 2** de la clase sobre `data/resenas_entrega.csv` (110 reseñas de entrega, con su calificación de 1 a 5 estrellas).

```
import pandas as pd
import chromadb
from chromadb.utils import embedding_functions

pd.set_option("display.max_colwidth", 100)

resenas = pd.read_csv("../data/resenas_entrega.csv")
print(resenas.shape)
resenas[["cliente_id", "producto_id", "calificacion", "texto"]].head()
```

(110, 7)

	cliente_id	producto_id	calificacion	texto
0	73	10	1	Recibi el pedido incompleto, faltaba el cable de carga original del OnePlus 12 dentro de la caja...
1	88	89	3	El segundo cargador Belkin llego sin contratiempos, en el tiempo estimado y correctamente empackado.
2	28	45	5	La segunda Switch Lite que regale llego perfecta y justo a tiempo para el cumpleaños que la nece...
3	54	34	5	El segundo television TCL llego perfecto, bien empackado y sin ningun dano, muy distinto a la pri...
4	80	83	4	Me confirmaron por correo la garantia de un ano del cable Anker sin ningun problema.

1. Definir las etiquetas

Cinco categorías como frases descriptivas, no como palabras sueltas: el embedding representa mejor una frase completa que una sola palabra.

```
etiquetas = {
    "retraso": "Retraso en la entrega",
    "danado": "Producto dañado o defectuoso",
    "incompleto": "Pedido incompleto o accesorio equivocado",
    "buena_entrega": "Entrega rápida y en buen estado",
    "soporte": "Atención de soporte o posventa",
```

```
}  
etiquetas
```

```
{'retraso': 'Retraso en la entrega',  
 'danado': 'Producto dañado o defectuoso',  
 'incompleto': 'Pedido incompleto o accesorio equivocado',  
 'buena_entrega': 'Entrega rápida y en buen estado',  
 'soporte': 'Atención de soporte o posventa'}
```

2. Vectorizar etiquetas y reseñas

Guardamos las etiquetas como documentos de una colección de Chroma: así reutilizamos `query()` para encontrar, para cada reseña, la etiqueta más cercana (equivale a comparar la reseña contra cada una de las 5 frases y quedarnos con la de mayor similitud coseno).

```
ef = embedding_functions.SentenceTransformerEmbeddingFunction(  
    model_name="paraphrase-multilingual-MiniLM-L12-v2"  
)  
  
cliente = chromadb.Client()  
col_etiquetas = cliente.get_or_create_collection(  
    name="etiquetas_resenas",  
    embedding_function=ef,  
    metadata={"hnsw:space": "cosine"},  
)  
  
col_etiquetas.add(  
    ids=list(etiquetas.keys()),  
    documents=list(etiquetas.values()),  
)  
col_etiquetas.count()
```

Loading weights: 100%  199/199 [00:00<00:00, 8305.88it/s]

5

3. Clasificar cada reseña

Para cada texto de reseña, consultamos la colección de etiquetas con `n_results=1` : gana la etiqueta más parecida. Chroma acepta una lista de consultas en una sola llamada, así que clasificamos las 110 reseñas en un solo `query()` .

```
res = col_etiquetas.query(query_texts=resenas["texto"].tolist(), n_results=1)  
  
resenas = resenas.copy()  
resenas["etiqueta_id"] = [ids[0] for ids in res["ids"]]  
resenas["etiqueta"] = [docs[0] for docs in res["documents"]]  
resenas["similitud"] = [round(1 - dists[0], 3) for dists in res["distances"]]  
  
resenas[["calificacion", "etiqueta", "similitud", "texto"]].head(10)
```

	calificacion	etiqueta	similitud	texto
0	1	Pedido incompleto o accesorio equivocado	0.475	Recibi el pedido incompleto, faltaba el cable de carga original del OnePlus 12 dentro de la caja...
1	3	Entrega rápida y en buen estado	0.473	El segundo cargador Belkin llego sin contratiempos, en el tiempo estimado y correctamente empacado.
2	5	Entrega rápida y en buen estado	0.288	La segunda Switch Lite que regale llego perfecta y justo a tiempo para el cumpleaños que la neces...
3	5	Entrega rápida y en buen estado	0.208	El segundo television TCL llego perfecto, bien empacado y sin ningun dano, muy distinto a la pri...
4	4	Retraso en la entrega	0.249	Me confirmaron por correo la garantia de un ano del cable Anker sin ningun problema.
5	5	Entrega rápida y en buen estado	0.173	Los audifonos Bose se emparejaron al instante con mi telefono, cero complicaciones.
6	2	Retraso en la entrega	0.270	La instalacion de este segundo pedido de Family Hub se retraso otra vez, casi diez dias despues ...
7	3	Entrega rápida y en buen estado	0.455	La segunda camara instantanea llego completa y a tiempo, sin ningun inconveniente en esta ocasion.
8	5	Entrega rápida y en buen estado	0.326	Cambiar el color de mi Marshall por otro modelo fue un proceso rapido y sin costo.
9	4	Entrega rápida y en buen estado	0.203	El Amazfit se sincronizo sin problemas con la app apenas lo encendi.

```
ejemplo = resenas.iloc[0]
print("Reseña: ", ejemplo["texto"])
print("Etiqueta asignada:", ejemplo["etiqueta"], f"(similitud {ejemplo['similitud']})")
```

Reseña: Recibi el pedido incompleto, faltaba el cable de carga original del OnePlus 12 dentro de la caja. Tuve que esperar un envio adicional solo para recibir ese accesorio.

Etiqueta asignada: Pedido incompleto o accesorio equivocado (similitud 0.475)

4. Cruzar con la calificación (1–5★)

Si la clasificación captura bien el contenido de las reseñas, las categorías de problemas (retraso, dañado, incompleto) deberían concentrar las calificaciones bajas, y «entrega rápida y en buen estado» las calificaciones altas.

```
tabla = pd.crosstab(resenas["etiqueta"], resenas["calificacion"])
tabla
```

	calificacion	1	2	3	4	5
etiqueta						
Atención de soporte o posventa	0	0	0	1	0	
Entrega rápida y en buen estado	0	4	9	16	26	
Pedido incompleto o accesorio equivocado	8	4	9	3	1	
Producto dañado o defectuoso	1	2	1	2	4	
Retraso en la entrega	1	5	2	7	4	

```
resumen = (
    resenas.groupby("etiqueta")["calificacion"]
    .agg(n="count", calificacion_promedio="mean")
    .sort_values("calificacion_promedio")
    .round(2)
)
resumen
```

	n	calificacion_promedio
etiqueta		
Pedido incompleto o accesorio equivocado	25	2.40
Retraso en la entrega	19	3.42
Producto dañado o defectuoso	10	3.60
Atención de soporte o posventa	1	4.00
Entrega rápida y en buen estado	55	4.16

Discusión

- ¿Qué categoría concentra las calificaciones de 1★ y 2★? Revisen la tabla cruzada: ¿coincide con lo esperado (retraso, dañado, incompleto)?
- ¿Cómo cambia el resultado si reformulan alguna etiqueta? Por ejemplo, cambien "Producto dañado o defectuoso" por "El producto llegó roto" y vuelvan a ejecutar la celda de clasificación: ¿algunas reseñas cambian de categoría? ¿la similitud promedio sube o baja?
- Esta clasificación no usó ni un solo ejemplo etiquetado a mano: solo la descripción de cada categoría. ¿En qué casos esto sería insuficiente y haría falta un clasificador entrenado con ejemplos reales?

```
# Espacio para experimentar: reformulen una etiqueta y comparen resultados
# Usamos upsert() (no add()): si vuelven a ejecutar esta celda tras cambiar
# el texto de una etiqueta, add() ignoraría el id repetido y dejaría el
# embedding viejo sin actualizar. upsert() sí reemplaza el documento y
# vuelve a vectorizarlo.
etiquetas_v2 = dict(etiquetas)
etiquetas_v2["danado"] = "Todo mal"

col_etiquetas_v2 = cliente.get_or_create_collection(
    name="etiquetas_resenas_v2",
```

```

embedding_function=ef,
metadata={"hnsw:space": "cosine"},
)
col_etiquetas_v2.upsert(ids=list(etiquetas_v2.keys()),
documents=list(etiquetas_v2.values()))

res_v2 = col_etiquetas_v2.query(query_texts=resenas["texto"].tolist(), n_results=1)
etiqueta_v2 = pd.Series([docs[0] for docs in res_v2["documents"]],
index=resenas.index)

cambiaron = resenas["etiqueta"] != etiqueta_v2
print(f"{cambiaron.sum()} de {len(resenas)} reseñas cambiaron de categoría al reformular la etiqueta")
resenas.loc[cambiaron, ["texto", "etiqueta"]].assign(etiqueta_v2=etiqueta_v2[cambiaron])

```

10 de 110 reseñas cambiaron de categoría al reformular la etiqueta

	texto	etiqueta	etiqueta_v2
11	La segunda laptop HP que compre llego con la pantalla algo desalineada del chasis. Funciona bien...	Producto dañado o defectuoso	Pedido incompleto o accesorio equivocado
13	El Galaxy S24 Ultra ya venia con la mayoría del software actualizado desde fabrica.	Producto dañado o defectuoso	Retraso en la entrega
16	El soporte tardo en responder sobre un problema de bateria del Moto G84.	Producto dañado o defectuoso	Entrega rápida y en buen estado
22	El segundo Samsung The Frame que compre llego con un pixel muerto visible en la esquina superior...	Producto dañado o defectuoso	Pedido incompleto o accesorio equivocado
28	El equipo de soporte reemplazo rapido un cable defectuoso de mis audifonos Bose.	Producto dañado o defectuoso	Pedido incompleto o accesorio equivocado
33	El disco WD Blue llego bien protegido contra golpes con plastico de burbuja.	Producto dañado o defectuoso	Retraso en la entrega
62	Definitivamente el Galaxy S24 Ultra fue la mejor compra de tecnologia que hice este ano.	Producto dañado o defectuoso	Entrega rápida y en buen estado
84	Los Bose QuietComfort llegaron en caja resistente, se nota el cuidado en el empaque.	Producto dañado o defectuoso	Atención de soporte o posventa
100	El Redmi Note 13 Pro llego con la caja visiblemente aplastada y un golpe en una esquina de la pa...	Producto dañado o defectuoso	Pedido incompleto o accesorio equivocado
104	El robot Xiaomi X10+ llego a tiempo, aunque la caja venia algo maltratada por fuera.	Producto dañado o defectuoso	Pedido incompleto o accesorio equivocado

Sesión de clase 06 · Laboratorio: recomendador semántico de productos

Contexto: la tienda recibe quejas porque sus recomendaciones «parecen genéricas» (comentario real en `comentarios.csv`, calificación ★★☆☆☆). Construimos un recomendador que usa el **significado** de los productos y el **historial** de cada cliente, en vez de reglas fijas.

Datos:

- `data/productos.csv` — 90 productos, 9 categorías × 10.
- `data/resenas_entrega.csv` — 110 reseñas de entrega sobre 54 productos, de 63 clientes.

Etapas: A. Indexación · B. Búsqueda semántica · C. Perfil de cliente · D. Recomendar · E. Análisis.

Cierra con un reto opcional: agrupar `comentarios.csv` por tema con k-means.

```
import numpy as np
import pandas as pd
import chromadb
from chromadb.utils import embedding_functions

pd.set_option("display.max_colwidth", 100)

MODELO = "paraphrase-multilingual-MiniLM-L12-v2"
ef = embedding_functions.SentenceTransformerEmbeddingFunction(model_name=MODELO)
```

Loading weights: 100%  199/199 [00:00<00:00, 8595.42it/s]

Eta A — Indexación

Cargamos `productos.csv`, armamos el texto a vectorizar (nombre + categoría + descripción) e insertamos los 90 productos en una colección persistente de Chroma, con `categoria`, `subcategoria`, `precio` y `calificacion_promedio` como metadatos para poder filtrar después.

Usamos `PersistentClient` (guarda la base en disco, en `../bdv`) en vez de `Client()`: así la colección sobrevive entre ejecuciones del notebook y no hay que re-vectorizar los 90 productos cada vez.

```
productos = pd.read_csv("../data/productos.csv")
print(productos.shape)
productos[["id", "nombre", "categoria", "subcategoria", "precio",
"calificacion_promedio"]].head()
```

(90, 11)

	id	nombre	categoria	subcategoria	precio	calificacion_promedio
0	1	Apple iPhone 15 Pro Max 256GB	Smartphones	Gama Alta	5599	3.5
1	2	Apple iPhone 15 128GB	Smartphones	Gama Alta	3799	3.3
2	3	Samsung Galaxy S24 Ultra 512GB	Smartphones	Gama Alta	5299	3.3
3	4	Samsung Galaxy A55 128GB	Smartphones	Gama Media	1399	3.0
4	5	Xiaomi Redmi Note 13 Pro 256GB	Smartphones	Gama Media	999	2.8

```

cliente_bdv = chromadb.PersistentClient(path="../bdv")

col_productos = cliente_bdv.get_or_create_collection(
    name="productos",
    embedding_function=ef,
    metadata={"hnsw:space": "cosine"},
)

texto_producto = productos["nombre"] + ". " + productos["categoria"] + ". " +
productos["descripcion"]

if col_productos.count() == 0:
    col_productos.add(
        ids=productos["id"].astype(str).tolist(),
        documents=texto_producto.tolist(),
        metadatas=productos[["categoria", "subcategoria", "precio",
"calificacion_promedio"]].to_dict("records"),
    )

col_productos.count()

```

90

Etapas B — Búsqueda semántica

Tres consultas en lenguaje natural: una libre, una filtrada por precio (`where`) y otra filtrada por categoría.

```

def buscar_productos(consulta, n_results=5, where=None):
    res = col_productos.query(query_texts=[consulta], n_results=n_results,
where=where)
    filas = []
    for doc, dist, meta in zip(res["documents"][0], res["distances"][0],
res["metadatas"][0]):
        filas.append({
            "similitud": round(1 - dist, 3),
            "categoria": meta.get("categoria"),
            "precio": meta.get("precio"),
            "producto": doc.split(".")[0],
        })
    return pd.DataFrame(filas)

buscar_productos("algo para escuchar musica mientras hago deporte")

```

	similitud	categoria	precio	producto
0	0.393	Audio	189	JBL Tune 510BT
1	0.362	Televisores y Video	3299	Sony Bravia X75L 65 pulgadas
2	0.359	Gaming y Consolas	349	HyperX Cloud II
3	0.353	Gaming y Consolas	799	SteelSeries Arctis Nova 7
4	0.331	Audio	999	Apple AirPods Pro 2da Generacion

```

buscar_productos(
    "regalo para quien empieza en fotografia",
    where={"precio": {"$lt": 2000}},
)

```

	similitud	categoria	precio	producto
0	0.433	Fotografia y Video	349	Fujifilm Instax Mini 12
1	0.379	Fotografia y Video	1899	Canon EOS Rebel T7
2	0.322	Fotografia y Video	1699	DJI Osmo Action 4
3	0.271	Accesorios y Componentes	279	Logitech C920 Webcam HD Pro
4	0.267	Electrodomesticos Inteligentes	599	Philips Hue White and Color Kit de Inicio

```

buscar_productos(
    "equipo para jugar en casa con amigos",
    where={"categoria": "Gaming y Consolas"},
)

```

	similitud	categoria	precio	producto
0	0.256	Gaming y Consolas	399	Razer Kishi V2
1	0.243	Gaming y Consolas	189	Logitech G502 Hero
2	0.236	Gaming y Consolas	349	HyperX Cloud II
3	0.223	Gaming y Consolas	2699	Sony PlayStation 5 Slim
4	0.209	Gaming y Consolas	799	SteelSeries Arctis Nova 7

Etapas C — Perfil de cliente

$\text{perfil} = \sum w_i \cdot v_i / \sum w_i$, donde v_i es el embedding de cada producto reseñado por el cliente y w_i un peso según la calificación de su reseña. Recuperamos los embeddings ya guardados en Chroma con `col.get(..., include=["embeddings"])`, en vez de volver a vectorizar los productos.

Se calcula el perfil de los clientes 1, 85 y 22 (los que más reseñas tienen: 10, 5 y 4 respectivamente).

```

resenas = pd.read_csv("../data/resenas_entrega.csv")

resenas["cliente_id"].value_counts().head(5)

```

```

cliente_id
1         10
85         5
33         4
22         4
116        3
Name: count, dtype: int64

```

```

def perfil_cliente(cliente_id, resenas, col_productos, peso=lambda calificacion:
calificacion):
    hist = resenas[resenas["cliente_id"] == cliente_id]
    ids_productos = hist["producto_id"].astype(str).unique().tolist()

    embeddings = col_productos.get(ids=ids_productos, include=["embeddings"])
    vectores = {pid: emb for pid, emb in zip(embeddings["ids"],
embeddings["embeddings"])}

    pesos_totales = 0.0
    acumulado = np.zeros(len(next(iter(vectores.values()))))
    for _, fila in hist.iterrows():
        pid = str(fila["producto_id"])
        w = peso(fila["calificacion"])
        acumulado += w * np.array(vectores[pid])
        pesos_totales += w

    return acumulado / pesos_totales, ids_productos

clientes_objetivo = [1, 85, 22]
perfiles = {}
for cid in clientes_objetivo:
    vector, productos_comprados = perfil_cliente(cid, resenas, col_productos)
    perfiles[cid] = {"vector": vector, "comprados": productos_comprados}
    print(f"Cliente {cid}: perfil calculado a partir de {len(productos_comprados)}
producto(s) reseñado(s) -> {productos_comprados}")

```

Cliente 1: perfil calculado a partir de 3 producto(s) reseñado(s) -> ['63', '3', '33']

Cliente 85: perfil calculado a partir de 3 producto(s) reseñado(s) -> ['43', '73', '13']

Cliente 22: perfil calculado a partir de 4 producto(s) reseñado(s) -> ['44', '53', '83', '23']

Etapas D — Recomendar

Consultamos la colección con `query_embeddings` (el perfil ya es un vector, no hace falta re-vectorizarlo), pedimos más resultados de los necesarios y descartamos los productos que el cliente ya compró.

```

def recomendar(cliente_id, perfiles, col_productos, productos_df, top_n=5):
    perfil = perfiles[cliente_id]["vector"]
    comprados = set(perfiles[cliente_id]["comprados"])

    res = col_productos.query(
        query_embeddings=[perfil.tolist()],
        n_results=top_n + len(comprados),
    )

    filas = []

```

```

for pid, dist in zip(res["ids"][0], res["distances"][0]):
    if pid in comprados:
        continue
    nombre = productos_df.loc[productos_df["id"].astype(str) == pid,
"nombre"].values[0]
    categoria = productos_df.loc[productos_df["id"].astype(str) == pid,
"categoria"].values[0]
    filas.append({"producto_id": pid, "producto": nombre, "categoria": categoria,
"similitud": round(1 - dist, 3)})
    if len(filas) == top_n:
        break

return pd.DataFrame(filas)

for cid in clientes_objetivo:
    print(f"\nTop-5 recomendaciones para el cliente {cid}:")
    display(recomendar(cid, perfiles, col_productos, productos))

```

Top-5 recomendaciones para el cliente 1:

	producto_id	producto	categoria	similitud
0	31	LG OLED55C3 55 pulgadas	Televisores y Video	0.745
1	4	Samsung Galaxy A55 128GB	Smartphones	0.741
2	5	Xiaomi Redmi Note 13 Pro 256GB	Smartphones	0.709
3	9	Samsung Galaxy A15 128GB	Smartphones	0.699
4	39	Sony Bravia X75L 65 pulgadas	Televisores y Video	0.675

Top-5 recomendaciones para el cliente 85:

	producto_id	producto	categoria	similitud
0	6	Xiaomi Poco X6 Pro 512GB	Smartphones	0.642
1	37	LG 50 pulgadas UHD UR8000	Televisores y Video	0.635
2	19	HP Omen 16 RTX 4060	Laptops y Computadoras	0.633
3	87	Corsair Vengeance 16GB DDR4 3200MHz	Accesorios y Componentes	0.621
4	12	Apple MacBook Pro 14 pulgadas M3 512GB	Laptops y Computadoras	0.618

Top-5 recomendaciones para el cliente 22:

	producto_id	producto	categoria	similitud
0	58	Amazfit GTS 4 Mini	Wearables y Fitness	0.749
1	60	Huawei Watch GT 4	Wearables y Fitness	0.725
2	52	Apple Watch Ultra 2	Wearables y Fitness	0.682
3	59	Garmin Venu 3	Wearables y Fitness	0.672
4	4	Samsung Galaxy A55 128GB	Smartphones	0.666

Etapas E — Análisis

1. Arranque en frío. Un cliente con una sola reseña tiene un perfil igual al embedding de un único producto: las recomendaciones quedan muy pegadas a ese producto y no reflejan un "gusto"

general. Con cero reseñas no hay perfil que calcular - ahí se necesita otra estrategia (productos populares, o pedir preferencias explícitas).

2. Peso de una reseña de 1★. Hasta ahora usamos `peso = calificacion`, así que una reseña de 1★ sigue sumando en la misma dirección que una de 5★, solo que con menos peso - el perfil nunca se aleja de un producto que al cliente **no le gustó**. Probemos la alternativa sugerida en clase: `peso = calificacion - 3`, que vuelve negativas las reseñas de 1★ y 2★ (el perfil se aleja de esos productos) y positivas las de 4★ y 5★.

```
def perfil_cliente_centrado(cliente_id, resenas, col_productos):
    return perfil_cliente(cliente_id, resenas, col_productos, peso=lambda
calificacion: calificacion - 3)

for cid in clientes_objetivo:
    hist = resenas[resenas["cliente_id"] == cid][["producto_id", "calificacion"]]
    if (hist["calificacion"] <= 2).any() and hist["producto_id"].nunique() > 1:
        vector_centrado, comprados = perfil_cliente_centrado(cid, resenas,
col_productos)
        vector_original = perfiles[cid]["vector"]
        similitud_entre_perfiles = float(
            np.dot(vector_original, vector_centrado)
            / (np.linalg.norm(vector_original) * np.linalg.norm(vector_centrado))
        )
        print(f"Cliente {cid}: coseno entre el perfil con peso=calificacion y
peso=calificacion-3: ")
            f"{similitud_entre_perfiles:.3f}")
        perfiles_centrado = {cid: {"vector": vector_centrado, "comprados":
comprados}}
        print("Recomendaciones con el perfil centrado (peso = calificacion - 3):")
        display(recomendar(cid, perfiles_centrado, col_productos, productos))
        break
    else:
        print("Ninguno de los clientes objetivo tiene una reseña de 1-2 estrellas junto
con otras; ")
            "prueben con otro cliente_id de resenas_entrega.csv para ver el efecto.")
```

Cliente 22: coseno entre el perfil con peso=calificacion y peso=calificacion-3: 0.893
Recomendaciones con el perfil centrado (peso = calificacion - 3):

	producto_id	producto	categoria	similitud
0	60	Huawei Watch GT 4	Wearables y Fitness	0.666
1	58	Amazfit GTS 4 Mini	Wearables y Fitness	0.656
2	52	Apple Watch Ultra 2	Wearables y Fitness	0.629
3	27	Samsung Galaxy Buds2 Pro	Audio	0.616
4	51	Apple Watch Series 9 GPS 41mm	Wearables y Fitness	0.615

3. De Chroma embebido a un servidor con miles de usuarios. Con

`Client()` / `PersistentClient()` todo corre en el mismo proceso Python: no hay concurrencia real, no hay autenticación ni control de acceso, y no se puede escalar horizontalmente. Con miles de usuarios simultáneos haría falta el modo servidor de Chroma (o Milvus/Qdrant/un servicio gestionado), separar el proceso de ingesta del de consulta, y decidir el índice ANN (HNSW por defecto) y sus parámetros de recall/latencia según el tráfico esperado.

Reto opcional: agrupar comentarios.csv por tema

Indexamos `comentarios.csv` en una segunda colección y agrupamos sus embeddings con k-means (sin usar la BDV para el clustering en sí, solo para guardar y reutilizar los vectores ya calculados).

```
from sklearn.cluster import KMeans

comentarios = pd.read_csv("../data/comentarios.csv")

col_comentarios = cliente_bdv.get_or_create_collection(
    name="comentarios",
    embedding_function=ef,
    metadata={"hnsw:space": "cosine"},
)

if col_comentarios.count() == 0:
    col_comentarios.add(
        ids=comentarios["id"].astype(str).tolist(),
        documents=comentarios["texto"].tolist(),
    )

datos = col_comentarios.get(
    ids=comentarios["id"].astype(str).tolist(),
    include=["embeddings", "documents"],
)
X = np.array(datos["embeddings"])

N_CLUSTERS = 6
kmeans = KMeans(n_clusters=N_CLUSTERS, random_state=42, n_init="auto").fit(X)

for cluster_id in range(N_CLUSTERS):
    centro = kmeans.cluster_centers_[cluster_id]
    distancias = np.linalg.norm(X - centro, axis=1)
    idx_representativo = int(np.argmin(distancias))
    tamano = int((kmeans.labels_ == cluster_id).sum())
    print(f"Cluster {cluster_id} ({tamano} comentarios) – ejemplo más cercano al centroide:")
    print(" ", datos["documents"][idx_representativo])
    print()
```

Cluster 0 (18 comentarios) – ejemplo más cercano al centroide:

El sitio funciona bien en general pero el checkout como invitado a veces pide iniciar sesion de todas formas. Es un detalle menor pero confunde un poco.

Cluster 1 (24 comentarios) – ejemplo más cercano al centroide:

Un vendedor de soporte me dio informacion incorrecta sobre un plazo de entrega y tuve que reprogramar mis planes.

Cluster 2 (13 comentarios) – ejemplo más cercano al centroide:

Me encanto poder pagar en cuotas sin intereses, facilito mucho la compra.

Cluster 3 (46 comentarios) – ejemplo más cercano al centroide:

La seccion de resenas de otros clientes me ayudo bastante a decidir que comprar.

Cluster 4 (27 comentarios) – ejemplo más cercano al centroide:

El seguimiento del pedido no siempre se actualiza en tiempo real, a veces se queda igual por dos dias. Al final el paquete llego bien pero la informacion no era precisa.

Cluster 5 (7 comentarios) – ejemplo más cercano al centroide:

La comparacion de especificaciones tecnicas me ayudo muchisimo a elegir entre dos laptops similares. Es justo el tipo de informacion que uno busca antes de invertir en tecnologia.

Cada cluster agrupa comentarios que hablan de temas parecidos (envíos, pagos, soporte, calidad del sitio, etc.) sin haber definido esas categorías a mano — a diferencia del Ejercicio 2, donde las etiquetas se definieron antes de clasificar.

sesion-de-clase-07/README.md

Proyecto de Minería Web - Sesión de clase 07

Esta sesión trata el **análisis de sentimientos**: de los lexicones con reglas a los Transformers locales, cómo evaluarlos con métricas adecuadas (macro-F1, κ de Cohen, MAE) y cómo convertir las predicciones en indicadores de negocio (NSS por país, sentimiento por aspecto y matriz de riesgo de clientes).

Los notebooks no scrapean: trabajan sobre los CSV de la tienda-virtual que ya están **copiados en la carpeta `data/`** de este proyecto. Todos los modelos corren **localmente, en CPU**; no se usa ninguna API externa.

Instalación y kernel

Requiere **Python 3.11+**. Cada sesión tiene su propio entorno virtual y su propio kernel de Jupyter para evitar conflictos de versiones con las demás.

```
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python -m ipykernel install --user --name s07-venv --display-name "Python (sesion-de-clase-07)"
```

Al abrir los notebooks, seleccionar el kernel **Python (sesion-de-clase-07)**.

La primera ejecución descarga los modelos de Hugging Face a la caché local (unos 2.3 GB en total); después funcionan sin internet:

Modelo	Notebooks
<code>cardiffnlp/twitter-xlm-roberta-base-sentiment</code>	02, 04
<code>nlptown/bert-base-multilingual-uncased-sentiment</code>	03, 04
<code>MoritzLaurer/mDeBERTa-v3-base-mnli-xnli</code>	02, 04
<code>paraphrase-multilingual-MiniLM-L12-v2</code> (sentence-transformers)	03, 04

Datos (`data/`)

Archivo	Contenido	Uso
<code>comentarios.csv</code>	135 comentarios de 96 clientes, con calificación 1–5★ y fecha	Corpus de sentimiento

Archivo	Contenido	Uso
resenas_entrega.csv	110 reseñas de entrega de 63 clientes sobre 54 productos	Corpus de sentimiento; compras por cliente
productos.csv	90 productos, 9 categorías x 10, con precio	Valor del cliente en la matriz de riesgo
clientes.csv	120 clientes de 7 países (datos personales : DNI, nombre, correo)	País y ciudad de cada texto; se seudonimiza en el laboratorio

Notebooks (notebooks/)

Se recomienda ejecutarlos en este orden; todos son autocontenidos (Run All funciona de principio a fin).

Notebook	Técnica	Qué hace
01_ejercicio1_lexicon_vs_tfidf.ipynb	lexicón tipo VADER, TfidfVectorizer + LogisticRegression	Ejercicio 1: etiquetas desde las estrellas, actividad de puntaje a mano, lexicón de 40 términos con negación / «muy» / «pero», pipeline con validación cruzada estratificada, fuga de información, pesos aprendidos y comparación (matriz de confusión, macro-F1, κ , cinco errores de cada enfoque).
02_transformers_locales.ipynb	transformers.pipeline	Práctica: por qué la bolsa de palabras pierde el orden, tokenización por sub-palabras, text-classification con cardiffnlp sobre las reseñas (label , score , top_k=None) y zero-shot-classification con mDeBERTa para polaridad y aspectos.
03_ejercicio2_preentrenado_vs_zero_shot.ipynb	nlptown, ChromaDB	Ejercicio 2: modelo de 1–5★ frente a zero-shot con frases prototipo en ChromaDB; MAE, macro-F1, κ y los 10 desacuerdos más grandes.
04_laboratorio_observatorio.ipynb	todo lo anterior	Laboratorio (etapas A–F): seudonimización, JOIN con clientes.csv , línea base, Transformers locales, puntaje continuo, aspectos con ChromaDB, NSS por país, clientes silenciosos, tendencia trimestral, sentimiento por aspecto, matriz de riesgo, diez errores con diagnóstico, hallazgos y limitaciones. Incluye el reto opcional (mDeBERTa frente a ChromaDB).

Salidas (salidas/)

El laboratorio guarda aquí sus gráficos (.png) y tablas (.csv): comparación de enfoques, NSS por país, tendencia trimestral, sentimiento por aspecto, perfil de clientes, matriz de riesgo, diez errores y los textos con todas sus predicciones. Los archivos de clientes se guardan **seudonimizados** (sin DNI, nombre ni correo).

Sesión de clase 07 · Ejercicio 1: lexicon frente a TF-IDF + regresión logística

Dos formas clásicas de decidir si un texto es **negativo**, **neutro** o **positivo**:

- **Lexicón con reglas (tipo VADER)**: un diccionario de palabras con un puntaje de sentimiento (valencia) asignado a mano, más reglas fijas para la negación («no»), los intensificadores («muy») y el contraste («pero»). No aprende nada de los datos.
- **TF-IDF + regresión logística**: cada texto se convierte en un vector de pesos por palabra y un modelo lineal **aprende** de los ejemplos qué palabras empujan hacia cada clase.

Trabajamos con los **245 textos** del curso: `data/comentarios.csv` (135) y `data/resenas_entrega.csv` (110), cada uno con la calificación de 1 a 5 estrellas que eligió el cliente.

Etapas: 1. Etiquetar · 2. Lexicón · 3. Modelo supervisado · 4. Comparar · Discusión.

```
import math
import re
import unicodedata

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (ConfusionMatrixDisplay, accuracy_score,
                             classification_report, cohen_kappa_score,
                             confusion_matrix, f1_score)
from sklearn.model_selection import StratifiedKFold, cross_val_predict
from sklearn.pipeline import make_pipeline
from sklearn.utils.class_weight import compute_class_weight

pd.set_option("display.max_colwidth", 120)
CLASES = ["neg", "neu", "pos"]
```

1. Etiquetar: de estrellas a tres clases

Supervisión débil: las etiquetas salen de las estrellas que eligió el cliente, no de un experto. 1–2★ → neg, 3★ → neu, 4–5★ → pos.

`pd.cut` con los cortes `[0, 2, 3, 5]` arma los intervalos `(0, 2]`, `(2, 3]` y `(3, 5]`.

```
c = pd.read_csv("../data/comentarios.csv")
r = pd.read_csv("../data/resenas_entrega.csv")

df = pd.concat([c[["texto", "calificacion"]].assign(fuente="comentario"),
               r[["texto", "calificacion"]].assign(fuente="resena")],
```

```

        ignore_index=True)
df["y"] = pd.cut(df["calificacion"], [0, 2, 3, 5],
                 labels=CLASES).astype(str)

print(df.shape)
df.sample(5, random_state=7)

(245, 4)

```

	texto	calificacion	fuentes	y
3	El envio internacional tardo mas dias de los indicados originalmente.	3	comentario	neu
82	El registro de cuenta pide demasiados datos para algo tan simple como crear un perfil. Preferiria un proceso mas cor...	3	comentario	neu
238	La Xbox se actualizo sola apenas la conecte a internet, lista para jugar en poco tiempo.	5	resena	pos
80	El soporte por correo fue muy claro al explicarme las opciones de financiamiento.	5	comentario	pos
232	La Nikon Z50 cumplio todas mis expectativas para fotografia de viajes.	4	resena	pos

```

distribucion = pd.DataFrame({
    "textos": df["y"].value_counts(),
    "proporcion": df["y"].value_counts(normalize=True).round(3),
}).loc[["pos", "neg", "neu"]]
distribucion

```

	textos	proporcion
y		
pos	126	0.514
neg	67	0.273
neu	52	0.212

Desbalance: responder siempre «positivo» ya acierta la mitad de las veces sin leer nada. Esa es la línea base mínima que cualquier modelo debe superar, y la razón por la que no basta con mirar la exactitud.

```

siempre_pos = np.full(len(df), "pos")
print(f"Exactitud de «siempre positivo»: {accuracy_score(df['y'], siempre_pos):.3f}")
print(f"Macro-F1 de «siempre positivo»: {f1_score(df['y'], siempre_pos,
average='macro'):.3f}")

```

Exactitud de «siempre positivo»: 0.514
Macro-F1 de «siempre positivo»: 0.226

¿Qué revela una palabra? Probabilidad a partir de conteos

Antes de leer un texto, la probabilidad de que sea negativo es la proporción de negativos (27 %). Si miramos solo los textos que contienen una expresión, recalculamos esa proporción: cuánto se mueve dice cuánta evidencia aporta la expresión.

```
def normalizar(texto):
    """Minúsculas y sin tildes: los textos del curso casi no las usan."""
    t = unicodedata.normalize("NFKD", texto.lower())
    return "".join(ch for ch in t if not unicodedata.combining(ch))

df["texto_norm"] = df["texto"].map(normalizar)

def evidencia(expresion):
    contiene = df["texto_norm"].str.contains(rf"\b{expresion}\b", regex=True)
    sub = df[contiene]
    return {"expresion": expresion,
            "textos": len(sub),
            "negativos": (sub["y"] == "neg").sum(),
            "P(neg | expresion)": round((sub["y"] == "neg").mean(), 2)}

pd.DataFrame([
    {"expresion": "(todos los textos)", "textos": len(df),
     "negativos": (df["y"] == "neg").sum(),
     "P(neg | expresion)": round((df["y"] == "neg").mean(), 2)},
    evidencia("tuve que"),
    evidencia("no"),
    evidencia("rapido"),
])
```

	expresion	textos	negativos	P(neg expresion)
0	(todos los textos)	245	67	0.27
1	tuve que	27	20	0.74
2	no	37	18	0.49
3	rapido	16	0	0.00

«tuve que» es evidencia fuerte de insatisfacción (3 de cada 4 textos que la contienen son negativos). «no», en cambio, aparece en textos de todo tipo: **la mitad** de los textos con «no» son neutros o positivos. Por eso falla la regla «si contiene *no*, es negativo». «rapido» nunca aparece en un texto negativo.

Naive Bayes combina esta evidencia de todas las palabras del texto y elige la clase más probable.

2. Lexicón con reglas tipo VADER

Cuatro pasos: **tokenizar** → **buscar valencia** → **aplicar reglas** → **sumar y normalizar**.

2.1 Actividad de la clase: tres frases a mano

Primero reproducimos la actividad de la diapositiva con un lexicón de cuatro términos:

Regla	Efecto
«muy» antes del término	suma 0.293 al valor absoluto
«no» antes del término	multiplica por -0.74
«pero»	lo anterior × 0.5, lo posterior × 1.5

Regla

Efecto

Clase

suma > 0 positivo; < 0 negativo

```

NEGADORES = {"no", "nunca", "ni", "sin"}
INTENSIFICADORES = {"muy": 0.293, "bastante": 0.293, "super": 0.293,
                    "algo": -0.293}          # «algo» atenúa
FACTOR_NEGACION = -0.74
VENTANA_NEGACION = 4          # «no» hasta 3 palabras antes del término
ALPHA = 15                   # constante de normalización de VADER

def tokenizar(texto):
    return re.findall(r"\w+", normalizar(texto))

def puntaje(texto, lexicon, negadores=NEGADORES):
    """Devuelve (suma, compuesto en [-1, 1], detalle por término)."""
    tokens = tokenizar(texto)
    pos_pero = tokens.index("pero") if "pero" in tokens else None
    detalle = []
    for i, tok in enumerate(tokens):
        if tok not in lexicon:
            continue
        v = lexicon[tok]
        aporte, mods = v, []
        if i > 0 and tokens[i - 1] in INTENSIFICADORES:
            aporte += math.copysign(INTENSIFICADORES[tokens[i - 1]], v)
            mods.append(f"«{tokens[i - 1]}»")
        if negadores & set(tokens[max(0, i - VENTANA_NEGACION):i]):
            aporte *= FACTOR_NEGACION
            mods.append("negación × (-0.74)")
        if pos_pero is not None:
            factor = 0.5 if i < pos_pero else 1.5
            aporte *= factor
            mods.append(f"{'antes' if i < pos_pero else 'después'} de «pero» × {factor}")
        detalle.append({"termino": tok, "v": v,
                       "modificador": ", ".join(mods) or "-",
                       "aporte": round(aporte, 3)})
    suma = sum(d["aporte"] for d in detalle)
    compuesto = suma / math.sqrt(suma ** 2 + ALPHA)
    return suma, compuesto, detalle

```

```

lexicon_actividad = {"rapida": 1.5, "malo": -2.5, "buen": 1.9, "pesima": -3.0}

for frase in ["La entrega fue muy rápida",
              "El producto no es malo",
              "Buen precio, pero la atención fue pésima"]:
    suma, _, detalle = puntaje(frase, lexicon_actividad)
    clase = "positivo" if suma > 0 else ("negativo" if suma < 0 else "neutro")
    print(f"«{frase}» → suma = {suma:+.3f} → {clase}")
    display(pd.DataFrame(detalle))

```

«La entrega fue muy rápida» → suma = +1.793 → positivo

	termino	v	modificador	aporte
--	---------	---	-------------	--------

0	rapida	1.5	«muy»	1.793
---	--------	-----	-------	-------

«El producto no es malo» → suma = +1.850 → positivo

	termino	v	modificador	aporte
0	malo	-2.5	negación × (-0.74)	1.85

«Buen precio, pero la atención fue pésima» → suma = -3.550 → negativo

	termino	v	modificador	aporte
0	buen	1.9	antes de «pero» × 0.5	0.95
1	pesima	-3.0	después de «pero» × 1.5	-4.50

«no es malo» (+1.85) obtiene **más** puntaje que «muy rápida» (+1.79): la regla de negación convierte un término muy negativo en uno bastante positivo, aunque un lector humano lo leería como «aceptable» a lo sumo.

2.2 Lexicón de 40 términos para la tienda

Valencias en escala -4 a +4 (como VADER). Los textos del curso vienen casi sin tildes, por eso el lexicón y los textos se comparan ya normalizados.

```
LEXICON = {  
    # positivos (21)  
    "excelente": 3.2, "perfecto": 3.0, "perfecta": 3.0, "recomiendo": 2.0,  
    "satisfecho": 2.0, "buena": 1.9, "buen": 1.9, "bueno": 1.9,  
    "agradable": 1.9, "mejor": 1.8, "facil": 1.6, "bien": 1.5,  
    "rapida": 1.5, "conforme": 1.5, "fluida": 1.4, "ayudo": 1.4,  
    "util": 1.4, "sencillo": 1.3, "rapido": 1.2, "correctamente": 1.2,  
    "completo": 1.0,  
    # negativos (19)  
    "pesima": -3.0, "pesimo": -3.0, "malo": -2.5, "mala": -2.5,  
    "danado": -2.3, "fallo": -2.0, "retraso": -1.8, "error": -1.8,  
    "incompleto": -1.8, "rayada": -1.8, "problema": -1.7, "lento": -1.5,  
    "reclamo": -1.5, "dificil": -1.5, "dificiles": -1.5, "faltaba": -1.3,  
    "tardo": -1.2, "confunde": -1.2, "innecesarios": -1.1,  
}  
print(len(LEXICON), "términos")
```

40 términos

Comprobamos el cálculo con los dos ejemplos de la clase: contraste con «pero» y negación.

```
for frase in ["El diseño del sitio es agradable pero algunos textos son difíciles de  
leer",  
              "El registro de cuenta fue rápido y no me pidieron datos  
innecesarios"]:  
    suma, compuesto, detalle = puntaje(frase, LEXICON)  
    print(f"«{frase}»\n suma = {suma:+.2f}    compuesto = {compuesto:+.2f}")  
    display(pd.DataFrame(detalle))
```

«El diseño del sitio es agradable pero algunos textos son difíciles de leer»
suma = -1.30 compuesto = -0.32

	termino	v	modificador	aporte
0	agradable	1.9	antes de «pero» × 0.5	0.95
1	dificiles	-1.5	después de «pero» × 1.5	-2.25

«El registro de cuenta fue rápido y no me pidieron datos innecesarios»
suma = +2.01 compuesto = +0.46

	termino	v	modificador	aporte
0	rapido	1.2	—	1.200
1	innecesarios	-1.1	negación × (-0.74)	0.814

El **compuesto** lleva la suma a la escala -1 a +1 con $\text{suma} / \sqrt{(\text{suma}^2 + 15)}$. La clase sale de un umbral de ±0.05: por encima es positivo, por debajo negativo, y en medio (incluidos los textos sin ningún término del lexicón) neutro.

```
UMBRAL = 0.05
```

```
def clase_lexicon(compuesto):
    if compuesto >= UMBRAL:
        return "pos"
    if compuesto <= -UMBRAL:
        return "neg"
    return "neu"

df["lex_compuesto"] = [puntaje(t, LEXICON)[1] for t in df["texto"]]
df["pred_lex"] = df["lex_compuesto"].map(clase_lexicon)

sin_terminos = (df["lex_compuesto"] == 0).sum()
print(f"Textos sin ningún término del lexicón: {sin_terminos} de {len(df)}")
print(classification_report(df["y"], df["pred_lex"], digits=3))
```

Textos sin ningún término del lexicón: 97 de 245

	precision	recall	f1-score	support
neg	0.684	0.388	0.495	67
neu	0.227	0.423	0.295	52
pos	0.727	0.635	0.678	126
accuracy			0.522	245
macro avg	0.546	0.482	0.490	245
weighted avg	0.609	0.522	0.547	245

3. Modelo supervisado: TF-IDF + regresión logística

- `TfidfVectorizer(ngram_range=(1, 2))` : palabras y pares de palabras («tuve que»); `min_df=2` descarta términos que aparecen en un solo texto; `sublinear_tf=True` usa $1 + \log(\text{tf})$.
- `LogisticRegression(class_weight="balanced")` : cada clase pesa $\text{total} \div (3 \times \text{textos de la clase})$, así un error en un neutro cuenta más.
- **Validación cruzada estratificada de 5 particiones:** cada texto se evalúa exactamente una vez con un modelo que no lo vio al entrenar.

- **Pipeline:** el TF-IDF se ajusta **dentro** de cada partición, solo con los textos de entrenamiento.

```
pesos = compute_class_weight("balanced", classes=np.array(CLASES), y=df["y"])
pd.DataFrame({"textos": df["y"].value_counts()[CLASES],
              "peso": pesos.round(2)}, index=CLASES)
```

	textos	peso
neg	67	1.22
neu	52	1.57
pos	126	0.65

```
modelo = make_pipeline(
    TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True),
    LogisticRegression(C=5, max_iter=2000, class_weight="balanced"),
)
cv = StratifiedKFold(5, shuffle=True, random_state=42)

df["pred_rl"] = cross_val_predict(modelo, df["texto"], df["y"], cv=cv)
print(classification_report(df["y"], df["pred_rl"], digits=3))
print(f"κ de Cohen = {cohen_kappa_score(df['y'], df['pred_rl']):.3f}")
```

	precision	recall	f1-score	support
neg	0.708	0.687	0.697	67
neu	0.408	0.385	0.396	52
pos	0.809	0.841	0.825	126
accuracy			0.702	245
macro avg	0.642	0.637	0.639	245
weighted avg	0.696	0.702	0.699	245

κ de Cohen = 0.512

Fuga de información

Si el TF-IDF se ajusta con **los 245 textos** antes de dividir, el vocabulario y los IDF de los textos de prueba se filtran al entrenamiento. La métrica sube sin que el modelo sea mejor.

```
X_con_fuga = TfidfVectorizer(ngram_range=(1, 2), min_df=2,
                             sublinear_tf=True).fit_transform(df["texto"])
pred_fuga = cross_val_predict(
    LogisticRegression(C=5, max_iter=2000, class_weight="balanced"),
    X_con_fuga, df["y"], cv=cv)

print(f"Macro-F1 con Pipeline (correcto): {f1_score(df['y'], df['pred_rl'],
average='macro'):.3f}")
print(f"Macro-F1 con fuga de información: {f1_score(df['y'], pred_fuga,
average='macro'):.3f}")
```

Macro-F1 con Pipeline (correcto): 0.639

Macro-F1 con fuga de información: 0.687

¿Qué aprendió el modelo?

Ajustamos el pipeline con todos los textos solo para **inspeccionar** los pesos (la evaluación ya se hizo con validación cruzada). Cada clase tiene su propio vector de pesos: los más altos son las expresiones que más empujan hacia esa clase.

```
modelo.fit(df["texto"], df["y"])
tfidf, rl = modelo.named_steps.values()
vocab = tfidf.get_feature_names_out()

top = {}
for k, clase in enumerate(rl.classes_):
    orden = np.argsort(rl.coef_[k])[:, :-1][:10]
    top[clase] = [f"{vocab[j]} ({rl.coef_[k][j]:+.2f})" for j in orden]
pd.DataFrame(top)
```

	neg	neu	pos
0	que (+2.08)	aunque (+2.38)	muy (+1.63)
1	mas de (+1.43)	ni (+1.75)	rapido (+1.45)
2	tuve que (+1.39)	podria (+1.58)	buena (+1.32)
3	tuve (+1.24)	tiempo (+1.40)	excelente (+1.28)
4	no (+1.23)	funciona (+1.35)	fue (+1.25)
5	un (+1.21)	pero (+1.28)	xiaomi (+1.13)
6	una (+1.15)	actualizaciones (+1.24)	de mi (+0.98)
7	llego con (+1.15)	pero el (+1.24)	bose (+0.81)
8	nuevo (+1.08)	es (+1.22)	fue muy (+0.80)
9	espera (+1.08)	dias (+1.19)	facil (+0.80)

- «pero», «aunque» → **neutro**: el contraste suele indicar una opinión mixta.
- «tuve que» → **negativo**: el esfuerzo extra del cliente es la señal más clara de insatisfacción.
- **Marcas** → **positivo**: con solo 245 textos, una marca frecuente en reseñas positivas (p. ej., «xiaomi») se vuelve «positiva». Es una **correlación espuria**.

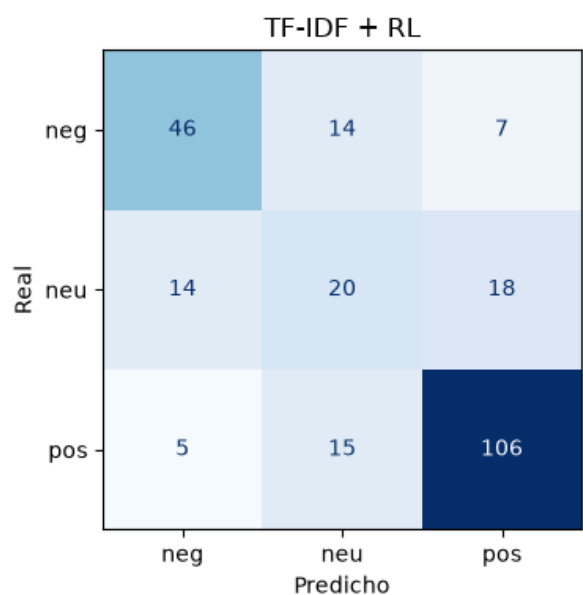
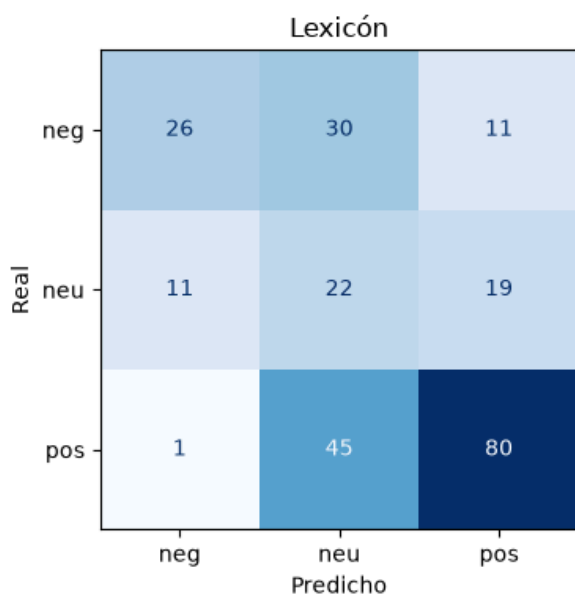
4. Comparar

```
def metricas(y, pred):
    return {"exactitud": accuracy_score(y, pred),
            "macro-F1": f1_score(y, pred, average="macro"),
            "k de Cohen": cohen_kappa_score(y, pred)}

pd.DataFrame({
    "siempre positivo": metricas(df["y"], siempre_pos),
    "lexicón (40 términos)": metricas(df["y"], df["pred_lex"]),
    "TF-IDF + RL": metricas(df["y"], df["pred_rl"]),
}).T.round(3)
```

	exactitud	macro-F1	κ de Cohen
siempre positivo	0.514	0.226	0.000
lexicón (40 términos)	0.522	0.490	0.257
TF-IDF + RL	0.702	0.639	0.512

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for ax, col, titulo in [(axes[0], "pred_lex", "Lexicón"),
                        (axes[1], "pred_rl", "TF-IDF + RL")]:
    ConfusionMatrixDisplay.from_predictions(
        df["y"], df[col], labels=CLASES, ax=ax, colorbar=False, cmap="Blues")
    ax.set_title(titulo)
    ax.set_xlabel("Predicho")
    ax.set_ylabel("Real")
plt.tight_layout()
plt.show()
```



Cinco errores de cada enfoque

Mostramos los errores **más graves** de cada uno: los que confunden positivo con negativo (o viceversa); si no alcanzan, se completan con los de mayor puntaje del lexicón en la dirección equivocada.

```
polaridad = (df["calificacion"] - 3) / 2 # -1 ... +1
distancia = lambda a, b: abs(CLASES.index(a) - CLASES.index(b))

errores_lex = df[df["y"] != df["pred_lex"]].assign(
    gravedad=lambda d: [distancia(a, b) for a, b in zip(d["y"], d["pred_lex"])],
    desvio=lambda d: (d["lex_compuesto"] - polaridad[d.index]).abs())
errores_lex.sort_values(["gravedad", "desvio", ascending=False]).head(5)[
    ["calificacion", "y", "pred_lex", "lex_compuesto", "texto"]].round(2)
```

	calificacion	y	pred_lex	lex_compuesto	texto
38	1	neg	pos	0.36	Pedi una factura por RUC y me la enviaron con datos incorrectos dos veces seguidas. Recien a la tercera solicitud lo...
119	1	neg	pos	0.36	Intente hacer una devolucion y el proceso fue confuso, nadie me explico bien los pasos.
66	1	neg	pos	0.25	El sitio me cerro sesion a mitad del pago y perdi el carrito completo que habia armado. Tuve que rehacer todo el ped...
69	1	neg	pos	0.25	El sitio se desconecto varias veces durante el checkout un dia de alta demanda, tuve que reiniciar el proceso completo.
204	2	neg	pos	0.44	El Moto G84 llego con la caja abierta, aunque el equipo en si estaba en buen estado.

```
proba = cross_val_predict(modelo, df["texto"], df["y"], cv=cv,
                          method="predict_proba")
df["rl_confianza"] = proba.max(axis=1)

errores_rl = df[df["y"] != df["pred_rl"]].assign(
    gravedad=lambda d: [distancia(a, b) for a, b in zip(d["y"], d["pred_rl"])]
)
errores_rl.sort_values(["gravedad", "rl_confianza"], ascending=False).head(5)[
    ["calificacion", "y", "pred_rl", "rl_confianza", "texto"]].round(2)
```

	calificacion	y	pred_rl	rl_confianza	texto
177	4	pos	neg	0.67	El television Hisense llego bien protegido con espuma adicional en las esquinas. Solo tuve que esperar un poco mas d...
74	2	neg	pos	0.63	El precio de envio internacional me parecia elevado comparado con tiendas similares de la region. El resto del proce...
119	1	neg	pos	0.61	Intente hacer una devolucion y el proceso fue confuso, nadie me explico bien los pasos.
97	1	neg	pos	0.60	Mi cupon de bienvenida expiro antes de poder usarlo, la vigencia era demasiado corta para el tramite que exigian.
78	4	pos	neg	0.54	Buena experiencia usando el asistente virtual para resolver dudas rapidas sobre garantia. Respondio en segundos y no...

¿En qué clase fallan ambos?

```
aciertos = pd.DataFrame({
    "lexicón": df["y"] == df["pred_lex"],
    "TF-IDF + RL": df["y"] == df["pred_rl"],
    "ambos fallan": (df["y"] != df["pred_lex"]) & (df["y"] != df["pred_rl"]),
    "y": df["y"],
}).groupby("y").mean().round(2)
aciertos.rename(columns={"lexicón": "exhaustividad lexicón",
                        "TF-IDF + RL": "exhaustividad TF-IDF + RL"})
```

exhaustividad lexicón exhaustividad TF-IDF + RL ambos fallan

y			
neg	0.39	0.69	0.22
neu	0.42	0.38	0.31
pos	0.63	0.84	0.06

```
df[(df["y"] == "neu") & (df["pred_lex"] != "neu") & (df["pred_rl"] != "neu")][
    ["calificacion", "pred_lex", "pred_rl", "texto"]].head(6)
```

	calificacion	pred_lex	pred_rl	texto
1	3	neg	pos	En general cumple lo que promete, ni mejor ni peor que otras tiendas de tecnologia que he probado.
3	3	neg	neg	El envio internacional tardo mas dias de los indicados originalmente.
41	3	pos	pos	El soporte responde en tiempos razonables, ni muy rapido ni muy lento comparado con otras tiendas.
46	3	pos	pos	La app movil a veces se cierra sola cuando cambio de pantalla muy rapido. Fuera de eso funciona bien para navegar el...
49	3	pos	pos	La variedad de categorias es buena aunque siento que podria ampliarse un poco mas en accesorios.
51	3	pos	pos	Las opciones de pago en cuotas varian bastante segun el banco, seria bueno mas claridad.

Discusión

¿Qué ocurre si «no» se elimina como palabra vacía?

Las listas de *stopwords* en español (NLTK, spaCy) incluyen «no». Probamos los dos enfoques sin esa palabra.

```
STOPWORDS_ES = ["el", "la", "los", "las", "de", "del", "y", "en", "un", "una",
                "que", "a", "se", "por", "con", "para", "mi", "me", "es",
                "fue", "lo", "al", "su", "no"]
```

```
variantes = {}
for nombre, stop in [("con «no»", [w for w in STOPWORDS_ES if w != "no"]),
                    ("sin «no»", STOPWORDS_ES)]:
    m = make_pipeline(
        TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True,
                        stop_words=stop),
        LogisticRegression(C=5, max_iter=2000, class_weight="balanced"))
    p = cross_val_predict(m, df["texto"], df["y"], cv=cv)
    variantes[f"TF-IDF + RL {nombre}"] = metricas(df["y"], p)

p_sin_neg = [clase_lexicon(puntaje(t, LEXICON, negadores=set())[1]) for t in
df["texto"]]
variantes["lexicón con negación"] = metricas(df["y"], df["pred_lex"])
variantes["lexicón sin negación"] = metricas(df["y"], p_sin_neg)
```

```
pd.DataFrame(variantes).T.round(3)
```

	exactitud	macro-F1	κ de Cohen
TF-IDF + RL con «no»	0.706	0.655	0.519
TF-IDF + RL sin «no»	0.698	0.647	0.503
lexicón con negación	0.522	0.490	0.257
lexicón sin negación	0.494	0.459	0.214

Preguntas para el grupo:

- Sin la regla de negación, «no me pidieron datos innecesarios» o «llegó sin ningún daño» suman como negativos. ¿Cuántos textos cambian de clase en el lexicón? ¿Por qué el TF-IDF es menos sensible a quitar «no»?
- La clase **neutra** es la más difícil para ambos enfoques: mezcla elogios y quejas en el mismo texto (3★). ¿Es un problema del modelo o de la etiqueta?
- Revisen los errores del lexicón: ¿cuántos se deben a palabras que no están en el diccionario y cuántos a reglas mal aplicadas?

```
# Espacio para experimentar: agreguen términos al lexicón y vuelvan a medir
LEXICON_V2 = dict(LEXICON)
LEXICON_V2.update({"demora": -1.5, "cancele": -1.2})

pred_v2 = [clase_lexicon(puntaje(t, LEXICON_V2)[1]) for t in df["texto"]]
pd.DataFrame({"lexicón": metricas(df["y"], df["pred_lex"]),
              "lexicón v2": metricas(df["y"], pred_v2)}).T.round(3)
```

	exactitud	macro-F1	κ de Cohen
lexicón	0.522	0.490	0.257
lexicón v2	0.527	0.495	0.263

Sesión de clase 07 · Práctica: Transformers locales desde Jupyter Notebook

Los modelos de la Parte 1 ven el texto como una **bolsa de palabras**: el orden se pierde. Un **Transformer** asigna a cada token un vector que depende de todos los demás (autoatención), así distingue «no llegó tarde» de «llegó tarde».

Todo corre **local, en CPU**: cada modelo se descarga una sola vez (cientos de MB) a la caché de Hugging Face y después funciona sin internet.

Modelo	Tarea	Salida
cardiffnlp/twitter-xlm-roberta-base-sentiment	sentimiento	negative · neutral · positive
MoritzLaurer/mDeBERTa-v3-base-mnli-xnli	zero-shot con NLI	etiquetas propias

Secciones: 1. El orden importa · 2. `pipeline` de sentimiento · 3. Reseñas de entrega · 4. Zero-shot con NLI · Discusión.

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import cohen_kappa_score, f1_score
from transformers import pipeline

pd.set_option("display.max_colwidth", 120)
```

1. Bolsa de palabras: el orden cambia el significado

Dos opiniones opuestas con exactamente las mismas palabras.

```
A = "El envío no llegó tarde, llegó bien"
B = "El envío no llegó bien, llegó tarde"

bow = CountVectorizer()
X = bow.fit_transform([A, B])
vectores = pd.DataFrame(X.toarray(), columns=bow.get_feature_names_out(),
                        index=["A", "B"])
print("¿x_A == x_B?", (vectores.loc["A"] == vectores.loc["B"]).all())
vectores
```

¿x_A == x_B? True

	bien	el	envío	llegó	no	tarde
A	1	1	1	2	1	1
B	1	1	1	2	1	1

Como $x_A = x_B$, **cualquier** modelo lineal sobre conteos (lexicón, TF-IDF + regresión logística, Naive Bayes) les asigna el mismo puntaje.

2. pipeline de sentimiento

`pipeline` junta el **tokenizador**, el **modelo** y la conversión de la salida en etiquetas. La primera ejecución descarga el modelo.

```
clf = pipeline("text-classification",
               model="cardiffnlp/twitter-xlm-roberta-base-sentiment")
clf([A, B])
```

Loading weights: 100%  201/201 [00:00<00:00, 22331.40it/s]

```
[{'label': 'positive', 'score': 0.5884841084480286},
 {'label': 'negative', 'score': 0.7565773129463196}]
```

El Transformer sí separa A (positivo) de B (negativo): «tarde» precedido por «no» y «tarde» al final de la frase producen vectores distintos.

Tokenización por sub-palabras

Las palabras raras se dividen en piezas frecuentes; así el vocabulario es finito y no hay palabras desconocidas. El `_` marca el inicio de una palabra (tokenizador SentencePiece de XLM-RoBERTa).

```
tokens = clf.tokenizer.tokenize("no me pidieron datos innecesarios")
print(len(tokens), "tokens:", tokens)
print("Vocabulario del modelo:", clf.tokenizer.vocab_size, "piezas")
```

```
8 tokens: ['_no', '_me', '_pidi', 'eron', '_datos', '_inne', 'ces', 'arios']
Vocabulario del modelo: 250002 piezas
```

¿Dónde falla una regla de palabras?

Los cuatro comentarios de la pregunta inicial de la clase. La regla «si contiene *no* o una palabra negativa → negativo» los marca a todos como negativos.

```
ejemplos = pd.DataFrame({
    "caso": ["A", "B", "C", "D"],
    "estrellas": [4, 4, 3, 2],
    "texto": [
        "El registro de cuenta fue rápido y no me pidieron datos innecesarios.",
        "El checkout como invitado es rápido si no quieres crear una cuenta.",
        "El diseño del sitio es agradable pero en modo oscuro algunos textos son difíciles de leer.",
        "Las fotos de un producto no coincidían del todo con el color que llegó.",
        "No es un problema grave, pero genera cierta desconfianza."
    ]
})
pred = clf(ejemplos["texto"].tolist())
ejemplos["modelo"] = [p["label"] for p in pred]
ejemplos["confianza"] = [round(p["score"], 2) for p in pred]
ejemplos
```

	caso	estrellas	texto	modelo	confianza
0	A	4	El registro de cuenta fue rápido y no me pidieron datos innecesarios.	neutral	0.56
1	B	4	El checkout como invitado es rápido si no quieres crear una cuenta.	neutral	0.75
2	C	3	El diseño del sitio es agradable pero en modo oscuro algunos textos son difíciles de leer.	negative	0.71
3	D	2	Las fotos de un producto no coincidían del todo con el color que llegó. No es un problema grave, pero genera cierta ...	negative	0.82

El modelo ya no marca A y B como negativos (los lee como neutros: no hay queja) y sí detecta la insatisfacción de D. En C el contraste con «pero» pesa más que el elogio, igual que en la regla del lexicón; el cliente puso 3★.

3. Reseñas de entrega

- `batch_size=16` procesa varios textos a la vez.
- `truncation=True` corta textos más largos que el máximo del modelo (512 tokens).
- `label` es la clase elegida y `score` la confianza del modelo en esa clase (0 a 1).

```
r = pd.read_csv("../data/resenas_entrega.csv")

pred = clf(r["texto"].tolist(), batch_size=16, truncation=True)
r["sentimiento"] = [p["label"] for p in pred]
r["confianza"] = [round(p["score"], 2) for p in pred]
r[["calificacion", "sentimiento", "confianza", "texto"]].head(8)
```

	calificacion	sentimiento	confianza	texto
0	1	negative	0.76	Recibi el pedido incompleto, faltaba el cable de carga original del OnePlus 12 dentro de la caja. Tuve que esperar u...
1	3	positive	0.53	El segundo cargador Belkin llego sin contratiempos, en el tiempo estimado y correctamente empacado.
2	5	positive	0.91	La segunda Switch Lite que regale llego perfecta y justo a tiempo para el cumpleaños que la necesitaba. Toda la expe...
3	5	positive	0.77	El segundo television TCL llego perfecto, bien empacado y sin ningun dano, muy distinto a la primera experiencia neg...
4	4	neutral	0.66	Me confirmaron por correo la garantia de un ano del cable Anker sin ningun problema.
5	5	neutral	0.51	Los audifonos Bose se emparejaron al instante con mi telefono, cero complicaciones.
6	2	negative	0.76	La instalacion de este segundo pedido de Family Hub se retraso otra vez, casi diez dias despues de la entrega del eq...
7	3	positive	0.65	La segunda camara instantanea llego completa y a tiempo, sin ningun inconveniente en esta ocasion.

¿Coincide con las estrellas?

Llevamos ambas cosas a tres clases: estrellas 1-2 → **neg**, 3 → **neu**, 4-5 → **pos**; y las etiquetas del modelo **negative/neutral/positive** a las mismas abreviaturas.

```
a3 = lambda s: "neg" if s <= 2 else ("neu" if s == 3 else "pos")
r["y"] = r["calificacion"].map(a3)
r["pred"] = r["sentimiento"].str[:3]

print(f"macro-F1 = {f1_score(r['y'], r['pred'], average='macro'):.3f}")
print(f"kappa = {cohen_kappa_score(r['y'], r['pred']):.3f}")
pd.crosstab(r["y"], r["pred"], rownames=["estrellas"], colnames=["modelo"])
```

macro-F1 = 0.552

kappa = 0.350

modelo neg neu pos

estrellas

	neg	neu	pos
neg	18	6	1
neu	9	9	3
pos	1	27	36

Sin haber visto **ni un ejemplo** de la tienda, el modelo ya supera claramente al azar ($\kappa > 0$). Pero fue entrenado con **tuits**: muchas reseñas de 4-5★ escritas en tono descriptivo («llegó en la fecha estimada») las lee como **neutras**. El error más frecuente es positivo → neutro, no positivo → negativo.

Probabilidades de las tres clases

Con **top_k=None** el pipeline devuelve el puntaje de **todas** las clases. Con eso armamos un puntaje continuo: P(positivo) – P(negativo), de –1 a +1.

```
todas = clf(r["texto"].tolist(), batch_size=16, truncation=True, top_k=None)
probs = pd.DataFrame([d["label"]: d["score"] for d in fila for fila in todas])
r["puntaje"] = (probs["positive"] - probs["negative"]).round(2)

r.groupby("calificacion")["puntaje"].agg(["count", "mean"]).round(2)
```

	count	mean
calificacion		
1	10	-0.58
2	15	-0.39
3	21	-0.18
4	29	0.37
5	35	0.58

El puntaje medio sube con las estrellas: el modelo ordena bien los textos aunque sus clases no coincidan exactamente con las del cliente.

4. Zero-shot con NLI: etiquetas propias sin entrenar

Un modelo de **inferencia textual (NLI)** decide si un texto *implica* una frase. El pipeline `zero-shot-classification` arma una hipótesis por etiqueta con `hypothesis_template` y elige la más implicada. Sirve para **aspectos** («¿de qué habla?») y también para polaridad.

```
zs = pipeline("zero-shot-classification",
              model="MoritzLaurer/mDeBERTa-v3-base-mnli-xnli")

zs("El envío tardó diez días",
  candidate_labels=["entrega", "pago", "sitio web", "soporte"],
  hypothesis_template="Este comentario trata sobre {}.")
```

```
/Users/ferris/.virtualenvs/mineria-web-2026-2-sspb/lib/python3.14/site-
packages/torch/jit/_script.py:1485: FutureWarning: `torch.jit.script` is not
supported in Python 3.14+ and may break. Please switch to `torch.compile` or
`torch.export`.
  warnings.warn(
```

Loading weights: 100%  202/202 [00:00<00:00, 6083.24it/s]

```
{'sequence': 'El envío tardó diez días',
 'labels': ['entrega', 'soporte', 'sitio web', 'pago'],
 'scores': [0.9988775849342346,
            0.0005281950579956174,
            0.0004162076802458614,
            0.0001779628946678713]}
```

```
texto = "El envío internacional tardó más días de los indicados originalmente."
res = zs(texto,
         candidate_labels=["satisfecho", "con una opinión neutral", "insatisfecho"],
         hypothesis_template="El cliente está {}.")
pd.DataFrame({"hipotesis": [f"El cliente está {l}." for l in res["labels"]],
              "probabilidad": [round(s, 2) for s in res["scores"]]})
```

	hipotesis	probabilidad
0	El cliente está insatisfecho.	0.49
1	El cliente está con una opinión neutral.	0.28
2	El cliente está satisfecho.	0.22

El cliente calificó este texto con **3★**: el modelo lo lee como insatisfacción. La etiqueta de referencia también tiene ruido.

Aspectos en las reseñas de entrega

```
ASPECTOS = ["tiempo de entrega", "estado del producto", "empaquete",
            "pedido incompleto", "atención al cliente"]

res = zs(r["texto"].tolist(), candidate_labels=ASPECTOS,
        hypothesis_template="Esta reseña trata sobre {}. ", batch_size=16)
```

```
r["aspecto"] = [x["labels"][0] for x in res]
r["aspecto_score"] = [round(x["scores"][0], 2) for x in res]

(r.groupby("aspecto")
 .agg(n=("texto", "size"), estrellas=("calificacion", "mean"),
      puntaje=("puntaje", "mean"))
 .sort_values("estrellas").round(2))
```

	n	estrellas	puntaje
aspecto			
pedido incompleto	5	2.20	-0.29
empaque	17	3.18	0.09
tiempo de entrega	26	3.38	0.04
estado del producto	53	3.87	0.23
atención al cliente	9	4.00	0.27

```
r.sort_values("aspecto_score", ascending=False).groupby("aspecto").head(1)[
 ["aspecto", "aspecto_score", "calificacion", "texto"]]
```

	aspecto	aspecto_score	calificacion	texto
87	estado del producto	1.00	4	El disco WD Blue no ha dado ningun problema desde que lo instale.
105	tiempo de entrega	1.00	2	El Samsung The Frame llego tres dias despues de la fecha estimada sin ninguna notificacion previa del retraso. Cuand...
24	atención al cliente	1.00	4	Resolvieron mi consulta sobre el hervidor Xiaomi mediante un video explicativo por correo.
65	empaque	0.99	4	Los JBL Tune 510BT llegaron bien empacados junto con el resto de mi pedido. El unico detalle menor fue que el manual...
61	pedido incompleto	0.80	3	El robot aspiradora llego completo aunque el manual venia en ingles a pesar de haberlo pedido en un sitio en espanol.

Discusión

- `cardiffnlp` fue entrenado con tuits y `nlptown` (Ejercicio 2) con reseñas de productos. ¿Cuál esperan que se adapte mejor a `resenas_entrega.csv` ?
- Cambien `hypothesis_template` por "El problema principal es {}." : ¿cambian los aspectos asignados?
- Escriban un comentario con ironía («Genial, tercera vez que me cobran doble») y pruébenlo en `clf` . ¿Lo detecta?

```
# Espacio para experimentar
clf(["Genial, tercera vez que me cobran doble",
     "Qué rapidez: solo tardaron tres semanas en responder mi reclamo"])
```

```
[{'label': 'positive', 'score': 0.798257052898407},  
 {'label': 'positive', 'score': 0.4163312613964081}]
```

La ironía sigue siendo difícil: el léxico positivo («genial», «rapidez») puede dominar aunque la intención sea negativa.

Sesión de clase 07 · Ejercicio 2: modelo preentrenado frente a zero-shot

Comparamos dos formas de clasificar las **110 reseñas de entrega**

(`data/resenas_entrega.csv`) **sin entrenar nada** con datos de la tienda:

- **Enfoque A — modelo preentrenado de reseñas:** `nlptown/bert-base-multilingual-uncased-sentiment` predice de 1 a 5 estrellas. Ya fue ajustado con reseñas de productos en seis idiomas.
- **Enfoque B — zero-shot con embeddings y ChromaDB:** escribimos tres frases prototipo por clase, las guardamos en una colección de Chroma y cada reseña va a la clase de su prototipo más parecido (coseno más alto).

Referencia: las estrellas del cliente (1–5) y su versión en 3 clases.

Métricas:

- **Macro-F1:** promedio del F1 de las 3 clases, cada una con igual peso.
- **κ de Cohen:** cuánto supera el modelo a responder al azar (0 = azar, 1 = perfecto).
- **MAE:** en promedio, ¿por cuántas estrellas se equivoca el modelo? (solo el enfoque A predice estrellas).

```
import chromadb
import matplotlib.pyplot as plt
import pandas as pd
from chromadb.utils import embedding_functions
from sklearn.metrics import (ConfusionMatrixDisplay, classification_report,
                             cohen_kappa_score, f1_score)
from transformers import pipeline

pd.set_option("display.max_colwidth", 120)
CLASES = ["neg", "neu", "pos"]
a3 = lambda s: "neg" if s <= 2 else ("neu" if s == 3 else "pos")
```

1. Referencia

```
r = pd.read_csv("../data/resenas_entrega.csv")
r["y"] = r["calificacion"].map(a3)

print(r.shape)
pd.DataFrame({"reseñas": r["calificacion"].value_counts().sort_index()})

(110, 8)
```

reseñas	
calificacion	
1	10
2	15
3	21
4	29
5	35

```
r["y"].value_counts()[CLASES]
```

```
y
neg    25
neu    21
pos    64
Name: count, dtype: int64
```

2. Enfoque A: modelo de reseñas (1 a 5 estrellas)

La etiqueta del modelo es un texto como "4 stars": su primer carácter es el número de estrellas.

```
clf = pipeline("text-classification",
               model="nlptown/bert-base-multilingual-uncased-sentiment")
pred = clf(r["texto"].tolist(), batch_size=16, truncation=True)
r["est_pred"] = [int(p["label"][0]) for p in pred]
r["pred_a"] = r["est_pred"].map(a3)

mae = (r["est_pred"] - r["calificacion"]).abs().mean()
print(f"MAE      = {mae:.2f} estrellas")
print(f"macro-F1 = {f1_score(r['y'], r['pred_a'], average='macro'):.3f}")
print(f"kappa    = {cohen_kappa_score(r['y'], r['pred_a']):.3f}")
```

Loading weights: 100%  201/201 [00:00<00:00, 5836.55it/s]

```
MAE      = 0.79 estrellas
macro-F1 = 0.693
kappa    = 0.554
```

Estrellas reales (filas) frente a estrellas predichas (columnas): los aciertos exactos están en la diagonal; los errores de una estrella, junto a ella.

```
pd.crosstab(r["calificacion"], r["est_pred"],
            rownames=["real"], colnames=["predicho"])
```

predicho	1	2	3	4	5
real					
1	9	0	0	1	0
2	7	3	3	2	0
3	3	1	12	3	2
4	4	1	1	12	11
5	4	2	2	4	23

3. Enfoque B: zero-shot con embeddings y ChromaDB

Tres frases prototipo por clase. Cada frase se guarda con su clase como **metadato**; con `n_results=1` Chroma devuelve el prototipo más cercano a cada reseña y su clase es la predicción.

Usamos el mismo modelo de embeddings de la sesión anterior: `paraphrase-multilingual-MiniLM-L12-v2`.

```

PROTOTIPOS = {
    "pos": ["El pedido llegó rápido y en perfecto estado.",
            "Estoy muy satisfecho con la compra y con la entrega.",
            "Excelente servicio, todo llegó completo y bien empacado."],
    "neu": ["La entrega fue normal, llegó en la fecha indicada.",
            "El producto llegó bien aunque hubo un detalle menor.",
            "Cumple con lo esperado, ni bueno ni malo."],
    "neg": ["El pedido llegó con retraso y tuve que reclamar.",
            "El producto llegó dañado o incompleto.",
            "Muy mala experiencia con la entrega, estoy insatisfecho."],
}

ef = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="paraphrase-multilingual-MiniLM-L12-v2")

cliente = chromadb.Client()
col_prototipos = cliente.get_or_create_collection(
    name="prototipos_sentimiento",
    embedding_function=ef,
    metadata={"hnsw:space": "cosine"},
)

ids, docs, metas = [], [], []
for clase, frases in PROTOTIPOS.items():
    for i, frase in enumerate(frases):
        ids.append(f"{clase}_{i}")
        docs.append(frase)
        metas.append({"clase": clase})
col_prototipos.upsert(ids=ids, documents=docs, metadatas=metas)
col_prototipos.count()

```

Loading weights: 100%  199/199 [00:00<00:00, 10850.68it/s]

```
res = col_prototipos.query(query_texts=r["texto"].tolist(), n_results=1)

r["pred_b"] = [m[0]["clase"] for m in res["metadatas"]]
r["prototipo"] = [d[0] for d in res["documents"]]
r["similitud"] = [round(1 - d[0], 3) for d in res["distances"]]

print(f"macro-F1 = {f1_score(r['y'], r['pred_b'], average='macro'):.3f}")
print(f"kappa = {cohen_kappa_score(r['y'], r['pred_b']):.3f}")
r[["calificacion", "pred_b", "similitud", "prototipo", "texto"]].head(6)
```

```
macro-F1 = 0.483
kappa = 0.284
```

	calificacion	pred_b	similitud	prototipo	texto
0	1	neg	0.472	El pedido llegó con retraso y tuve que reclamar.	Recibi el pedido incompleto, faltaba el cable de carga original del OnePlus 12 dentro de la caja. Tuve que esperar u...
1	3	pos	0.380	El pedido llegó rápido y en perfecto estado.	El segundo cargador Belkin llego sin contratiempos, en el tiempo estimado y correctamente empacado.
2	5	pos	0.398	Estoy muy satisfecho con la compra y con la entrega.	La segunda Switch Lite que regale llego perfecta y justo a tiempo para el cumpleaños que la necesitaba. Toda la expe...
3	5	neu	0.337	El producto llegó bien aunque hubo un detalle menor.	El segundo television TCL llego perfecto, bien empacado y sin ningun dano, muy distinto a la primera experiencia neg...
4	4	neu	0.483	La entrega fue normal, llegó en la fecha indicada.	Me confirmaron por correo la garantia de un ano del cable Anker sin ningun problema.
5	5	pos	0.358	El pedido llegó rápido y en perfecto estado.	Los audifonos Bose se emparejaron al instante con mi telefono, cero complicaciones.

4. Comparar

```
def metricas(y, pred):
    return {"macro-F1": f1_score(y, pred, average="macro"),
            "κ de Cohen": cohen_kappa_score(y, pred)}

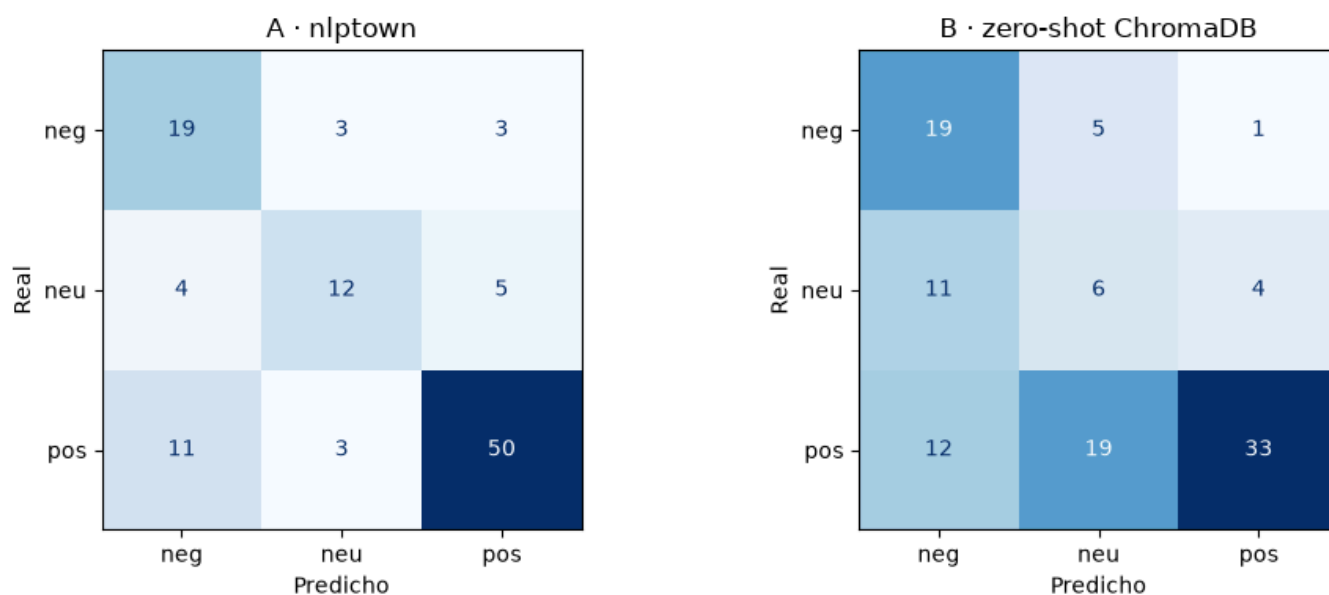
comparacion = pd.DataFrame({
    "A · nlptown (1-5★)": {**metricas(r["y"], r["pred_a"]), "MAE (estrellas)": mae},
    "B · zero-shot ChromaDB": {**metricas(r["y"], r["pred_b"]), "MAE (estrellas)":
None},
}).T.round(3)
comparacion
```

	macro-F1	κ de Cohen	MAE (estrellas)
A · nlptown (1-5★)	0.693	0.554	0.791
B · zero-shot ChromaDB	0.483	0.284	NaN

```

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for ax, col, titulo in [(axes[0], "pred_a", "A · nlptown"),
                        (axes[1], "pred_b", "B · zero-shot ChromaDB")]:
    ConfusionMatrixDisplay.from_predictions(
        r["y"], r[col], labels=CLASES, ax=ax, colorbar=False, cmap="Blues")
    ax.set_title(titulo)
    ax.set_xlabel("Predicho")
    ax.set_ylabel("Real")
plt.tight_layout()
plt.show()

```



```

for col, titulo in [("pred_a", "A · nlptown"), ("pred_b", "B · zero-shot ChromaDB")]:
    print(titulo)
    print(classification_report(r["y"], r[col], labels=CLASES, digits=3,
                                zero_division=0))

```

A · nlptown

	precision	recall	f1-score	support
neg	0.559	0.760	0.644	25
neu	0.667	0.571	0.615	21
pos	0.862	0.781	0.820	64
accuracy			0.736	110
macro avg	0.696	0.704	0.693	110
weighted avg	0.756	0.736	0.741	110

B · zero-shot ChromaDB

	precision	recall	f1-score	support
neg	0.452	0.760	0.567	25
neu	0.200	0.286	0.235	21
pos	0.868	0.516	0.647	64
accuracy			0.527	110
macro avg	0.507	0.520	0.483	110
weighted avg	0.646	0.527	0.550	110

Lectura: el zero-shot con prototipos encuentra casi la misma proporción de negativos que nlptown, pero se hunde en la clase **neutra** y manda muchas reseñas positivas a neutro o negativo: la similitud

coseno mide *de qué habla* el texto (entrega, retraso, empaque) más que *cómo lo valora*.

Los 10 desacuerdos más grandes (enfoque A)

Ordenados por la distancia en estrellas entre la predicción y la calificación del cliente. Antes de culpar al modelo, lean el texto: ¿la estrella del cliente refleja lo que escribió?

```
r["error_estrellas"] = (r["est_pred"] - r["calificacion"]).abs()
r.sort_values("error_estrellas", ascending=False).head(10)[
    ["calificacion", "est_pred", "pred_b", "texto"]]
```

	calificacion	est_pred	pred_b	texto
63	5	1	neu	La instalacion del Sony Bravia estuvo lista en menos de una hora gracias al tecnico.
38	5	1	pos	La webcam Logitech llego al dia siguiente de realizar el pedido. Todo llego completo, con caja original y sellada.
53	5	1	neg	El Galaxy Watch se sincronizo con mi telefono en segundos apenas lo active.
48	5	1	neg	Cuando tuve una duda sobre la garantia de la Acer, me respondieron el mismo dia.
15	1	4	neg	Recibi el pedido con los audifonos SteelSeries de un color distinto al solicitado, sin ninguna indicacion previa del...
54	4	1	pos	El cable Anker llego junto con otros productos de mi pedido, todo en un solo envio.
24	4	1	neg	Resolvieron mi consulta sobre el hervidor Xiaomi mediante un video explicativo por correo.
9	4	1	pos	El Amazfit se sincronizo sin problemas con la app apenas lo encendi.
103	5	2	neu	La Xbox se actualizo sola apenas la conecte a internet, lista para jugar en poco tiempo.
79	4	1	neg	El hervidor Xiaomi llego completo, solo el manual venia en ingles y no en espanol.

Casi todos son textos **cortos y descriptivos** («se sincronizó en segundos», «llegó al día siguiente») calificados con 4–5★: no contienen ninguna palabra de valoración y el modelo, entrenado con reseñas que sí opinan, no sabe qué hacer con ellos. El caso inverso (1★ predicho como 4★) es una queja sin ninguna palabra negativa explícita: «llegó de un color distinto al solicitado».

¿Dónde discrepan los dos enfoques?

```
pd.crosstab(r["pred_a"], r["pred_b"],
            rownames=["A · nlptown"], colnames=["B · zero-shot"])
```

B · zero-shot neg neu pos

A · nlptown

neg	23	5	6
neu	11	5	2
pos	8	20	30

```
print(f"κ entre A y B: {cohen_kappa_score(r['pred_a'], r['pred_b']):.3f}")
r[(r["pred_a"] != r["pred_b"]) & (r["pred_a"] != "neu") & (r["pred_b"] != "neu")][
    ["calificacion", "est_pred", "pred_b", "prototipo", "texto"]]
```

κ entre A y B: 0.279

	calificacion	est_pred	pred_b	prototipo	texto
9	4	1	pos	El pedido llegó rápido y en perfecto estado.	El Amazfit se sincronizo sin problemas con la app apenas lo encendi.
12	4	5	neg	El pedido llegó con retraso y tuve que reclamar.	Me explicaron por chat como registrar la garantia internacional de la Canon Rebel.
13	5	2	pos	Estoy muy satisfecho con la compra y con la entrega.	El Galaxy S24 Ultra ya venia con la mayoría del software actualizado desde fabrica.
15	1	4	neg	El pedido llegó con retraso y tuve que reclamar.	Recibi el pedido con los audifonos SteelSeries de un color distinto al solicitado, sin ninguna indicacion previa del...
28	5	5	neg	El producto llegó dañado o incompleto.	El equipo de soporte reemplazo rapido un cable defectuoso de mis audifonos Bose.
31	1	1	pos	El pedido llegó rápido y en perfecto estado.	El telefono llego usado, con marcas de uso en la pantalla que no correspondian a un equipo nuevo segun la orden de c...
33	4	4	neg	El producto llegó dañado o incompleto.	El disco WD Blue llego bien protegido contra golpes con plastico de burbuja.
38	5	1	pos	Excelente servicio, todo llegó completo y bien empacado.	La webcam Logitech llego al dia siguiente de realizar el pedido. Todo llego completo, con caja original y sellada.
44	4	5	neg	El pedido llegó con retraso y tuve que reclamar.	El soporte me ayudo por telefono a resolver un error de conexion del robot aspiradora.
54	4	1	pos	Excelente servicio, todo llegó completo y bien empacado.	El cable Anker llego junto con otros productos de mi pedido, todo en un solo envio.
59	5	5	neg	El pedido llegó con retraso y tuve que reclamar.	El parlante Marshall llego en menos de 48 horas, muy conforme con la rapidez.
85	2	4	neg	El pedido llegó con retraso y tuve que reclamar.	La instalacion de la Samsung Family Hub se coordino con dos semanas de demora desde la entrega del equipo. El tecnic...
89	3	4	neg	El pedido llegó con retraso y tuve que reclamar.	El Amazfit GTS 4 Mini llego con un dia de retraso frente a la fecha estimada.
96	3	2	pos	Excelente servicio, todo llegó completo y bien empacado.	Los audifonos SteelSeries llegaron completos aunque el seguimiento del envio se quedo congelado por dos dias sin act...

Discusión

- ¿Qué enfoque confunde más la clase **neutra**? Revisen la exhaustividad (*recall*) de **neu** en ambos reportes.
- El enfoque B depende por completo de las frases prototipo. Reescriban los prototipos neutros (por ejemplo, con frases que mezclen un elogio y una queja) y vuelvan a ejecutar: ¿sube el macro-F1?
- El MAE de A mide errores **ordinales**: equivocarse de 4★ a 5★ no es lo mismo que de 1★ a 5★. ¿Qué métrica usarían para comparar A con un modelo de tres clases?

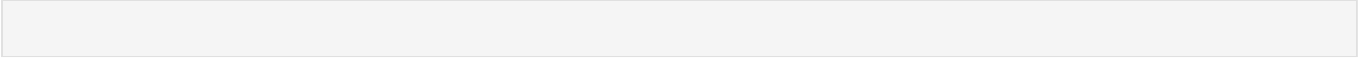
```
# Espacio para experimentar: prototipos neutros que mezclan elogio y queja
PROTOTIPOS_V2 = dict(PROTOTIPOS)
PROTOTIPOS_V2["neu"] = ["El producto llegó bien, pero la entrega tardó un poco más.",
                        "Buen producto, aunque el empaque podría mejorar.",
                        "Llegó a tiempo, aunque faltó información del seguimiento."]

col_v2 = cliente.get_or_create_collection(
    name="prototipos_sentimiento_v2", embedding_function=ef,
    metadata={"hnsw:space": "cosine"})
col_v2.upsert(
    ids=[f"{c}_{i}" for c, fs in PROTOTIPOS_V2.items() for i in range(len(fs))],
    documents=[f for fs in PROTOTIPOS_V2.values() for f in fs],
    metadatas=[{"clase": c} for c, fs in PROTOTIPOS_V2.items() for _ in fs])

res_v2 = col_v2.query(query_texts=r["texto"].tolist(), n_results=1)
pred_b_v2 = [m[0]["clase"] for m in res_v2["metadatas"]]
pd.DataFrame({"B · prototipos originales": metricas(r["y"], r["pred_b"]),
              "B · prototipos v2": metricas(r["y"], pred_b_v2)}).T.round(3)
```

	macro-F1	κ de Cohen
B · prototipos originales	0.483	0.284
B · prototipos v2	0.557	0.364

01_lexicon_vs_tfidf.ipynb



Sesión de clase 08 · Ejercicio 1: tópicos con LDA en comentarios.csv

El **análisis de tópicos** descubre, sin etiquetas previas, los temas de los que habla una colección de documentos. Produce dos salidas:

1. **Tópicos descritos por palabras** (ϕ): cada tópico es una distribución de probabilidad sobre el vocabulario.
2. **Mezcla de tópicos por documento** (θ): qué proporción de cada tópico hay en cada texto.

LDA (*Latent Dirichlet Allocation*, Blei, Ng y Jordan, 2003) estima ambas cosas a partir de una **bolsa de palabras**: solo ve cuántas veces aparece cada palabra en cada documento y qué palabras aparecen juntas.

Trabajamos con los **135 comentarios** de `data/comentarios.csv`.

Etapas: 1. Cargar · 2. Preprocesar · 3. Frecuencias y co-ocurrencia · 4. Entrenar LDA · 5. Interpretar · 6. Evaluar (coherencia NPMI y diversidad) · Discusión.

```
import itertools
import re
import unicodedata

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer

pd.set_option("display.max_colwidth", 120)

AZUL, ROJO, GRIS, TINTA = "#2a78d6", "#e34948", "#8a8985", "#52514e"
plt.rcParams.update({
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": GRIS, "axes.labelcolor": TINTA,
    "xtick.color": TINTA, "ytick.color": TINTA,
    "axes.grid": True, "grid.color": "#e6e5e1", "grid.linewidth": 0.8,
    "axes.axisbelow": True, "figure.dpi": 110,
})
```

1. Cargar

Columnas: `id`, `cliente_id`, `cliente_ciudad`, `cliente_pais`, `calificacion` (1–5★), `fecha` y `texto`.

```
df = pd.read_csv("../data/comentarios.csv")
print(df.shape)
```

```
df.sample(5, random_state=7)
```

```
(135, 7)
```

	id	cliente_id	cliente_ciudad	cliente_pais	calificacion	fecha	texto
134	419	5	Arequipa	Peru	1	2024-01-15	El sistema de puntos se reinicio sin explicacion despues de una actualizacion de la plataforma. Perdi el saldo acumu...
107	332	53	Santiago	Chile	2	2024-07-20	El chat de soporte demoro mas de media hora en responder un dia de alta demanda.
98	397	46	Valparaiso	Chile	1	2024-09-22	Recibi un correo de confirmacion pero el pedido nunca aparecio en mi historial de compras. Tuve que llamar por telef...
81	432	83	Cordoba	Argentina	2	2025-01-13	El diseno del sitio en el celular se ve desordenado en pantallas pequenas, algunos botones quedan superpuestos.
123	324	21	Puno	Peru	5	2024-03-30	Excelente variedad de marcas, encuentre opciones que no tenia en otras tiendas.

```
largo = df["texto"].str.split().str.len()
print(f"Palabras por comentario: mínimo {largo.min()}, mediana {largo.median():.0f}, "
      f"máximo {largo.max()}")
df["calificacion"].value_counts().sort_index().rename("comentarios").to_frame().T
```

Palabras por comentario: mínimo 8, mediana 15, máximo 32

calificacion	1	2	3	4	5
comentarios	17	25	31	30	32

Son **textos cortos** (unas 15 palabras): cada comentario ofrece muy pocas co-ocurrencias. Esto será importante al evaluar LDA.

2. Preprocesar

Cuatro pasos habituales:

1. **Normalizar:** minúsculas y sin tildes (los textos del archivo casi no las usan, así «demoró» y «demoro» quedan iguales).
2. **Quitar stopwords:** palabras funcionales («el», «de», «que») más palabras del **dominio** que aparecen en todos los temas («tienda», «producto», «compra»).

3. **Lematizar** («demoró» → demorar). Requiere spaCy u otro lematizador; aquí lo omitimos y lo dejamos como mejora.
4. **N-gramas**: unir términos que van juntos («alta demanda»).

`CountVectorizer` de scikit-learn solo trae *stopwords* en **inglés** (`stop_words="english"`), por eso definimos `STOPWORDS_ES`.

```
def normalizar(texto):
    """Minúsculas y sin tildes."""
    t = unicodedata.normalize("NFKD", texto.lower())
    return "".join(ch for ch in t if not unicodedata.combining(ch))

STOPWORDS_ES = sorted(set("""
a al algo algun alguna algunas alguno algunos ante antes aqui asi aun aunque
bajo bien cada casi como con contra cual cuando de del desde donde dos durante
e el ella ellas ellos en entre era eran es esa esas ese eso esos esta estaba
estaban estan estar este esto estos fue fueron ha habia han hasta hay he la las
le les lo los mas me mi mis mucho muy nada ni no nos o otra otras otro otros
para pero poco por porque que se ser si sin sobre solo su sus tambien tan tanto
te tiene todo todos tu tuve un una uno unos y ya yo vez veces mismo misma
son podria poder bastante
""").split()))

print(len(STOPWORDS_ES), "stopwords")
```

119 stopwords

Veamos cómo queda un comentario real: en **mayúsculas** las palabras que se conservan, en minúsculas las *stopwords* que se descartan.

```
ejemplo = df.loc[df["texto"].str.contains("media hora"), "texto"].iloc[0]
tokens = re.findall(r"\w+", normalizar(ejemplo))
print(ejemplo)
print(" ".join(t if t in STOPWORDS_ES else t.upper() for t in tokens))
```

El chat de soporte demoro mas de media hora en responder un dia de alta demanda.
el CHAT de SOPORTE DEMORO mas de MEDIA HORA en RESPONDER un DIA de ALTA DEMANDA

3. Frecuencias y co-ocurrencia

3.1 Términos más frecuentes

`CountVectorizer(min_df=2)` arma el vocabulario con los términos que aparecen en al menos 2 comentarios. Contamos en **cuántos comentarios** aparece cada término.

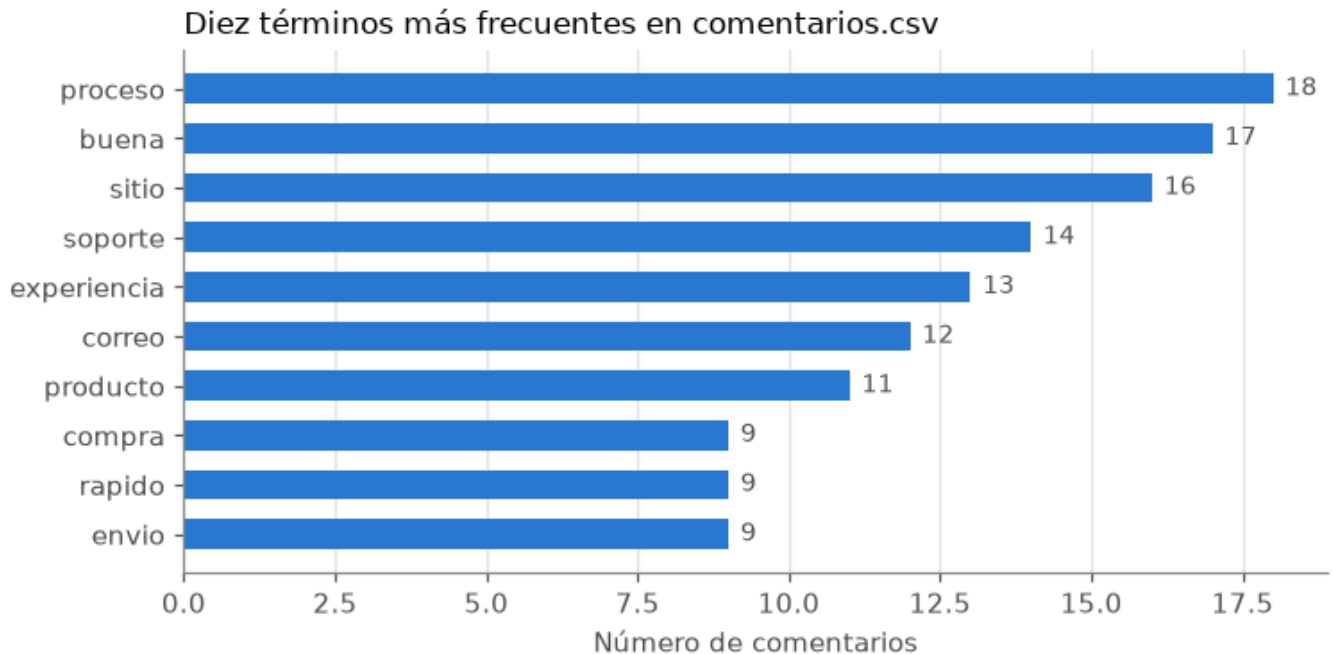
```
cv = CountVectorizer(preprocessor=normalizar, stop_words=STOPWORDS_ES, min_df=2)
X = cv.fit_transform(df["texto"])
vocab = cv.get_feature_names_out()
print(f"{X.shape[0]} comentarios × {X.shape[1]} términos distintos")

docs_por_termino = pd.Series(np.asarray((X > 0).sum(axis=0)).ravel(), index=vocab)
top10 = docs_por_termino.sort_values(ascending=False).head(10)

fig, ax = plt.subplots(figsize=(7, 3.6))
ax.barh(top10.index[::-1], top10.values[::-1], color=AZUL, height=0.6)
```

```
for i, v in enumerate(top10.values[::-1]):
    ax.text(v + 0.2, i, str(v), va="center", color=TINTA, fontsize=9)
ax.set_xlabel("Número de comentarios")
ax.set_title("Diez términos más frecuentes en comentarios.csv", loc="left",
            fontsize=11)
ax.grid(axis="y", visible=False)
plt.tight_layout()
plt.show()
```

135 comentarios × 220 términos distintos



Frecuente no es lo mismo que tema. Términos como «proceso», «buena» o «experiencia» aparecen en comentarios de pagos, de soporte y de envíos: no identifican ningún tema. Comprobémoslo leyendo algunos:

```
df.loc[df["texto"].map(normalizar).str.contains(r"\bproceso\b"), ["calificacion", "texto"]].head(6)
```

	calificacion	texto
0	5	El proceso de facturacion para empresa fue mas simple de lo que esperaba.
4	3	El proceso de devolucion fue sencillo de iniciar aunque tomo unos dias en confirmarse.
15	2	El proceso de cambio de producto por otro modelo fue mas lento de lo esperado.
29	3	El proceso de pago es estandar, similar a lo que ya conozco de otras tiendas en linea.
39	3	El proceso de canje de puntos podria explicarse mejor en la pagina de ayuda.
45	3	El proceso de registro es sencillo, aunque pide confirmar el correo un paso adicional que otros sitios no piden.

3.2 Matriz documento-término y co-ocurrencia

Cada fila es un documento, cada columna una palabra; la celda cuenta cuántas veces aparece. Con cinco documentos pequeños se ve la idea clave de LDA: **las palabras que aparecen juntas en los mismos documentos forman un tópico.**

```

mini = ["chat de soporte con mucha espera",
        "el soporte tardo, mucha espera",
        "cobro duplicado en la tarjeta",
        "reembolso del cobro a la tarjeta",
        "el chat no resolvió el reembolso"]
terminos = ["chat", "soporte", "espera", "cobro", "tarjeta", "reembolso"]

cv_mini = CountVectorizer(vocabulary=terminos)
M = cv_mini.fit_transform(mini)
pd.DataFrame(M.toarray(), columns=terminos, index=[f"D{i+1} «{d}»" for i, d in
enumerate(mini)])

```

	chat	soporte	espera	cobro	tarjeta	reembolso
D1 «chat de soporte con mucha espera»	1	1	1	0	0	0
D2 «el soporte tardo, mucha espera»	0	1	1	0	0	0
D3 «cobro duplicado en la tarjeta»	0	0	0	1	1	0
D4 «reembolso del cobro a la tarjeta»	0	0	0	1	1	1
D5 «el chat no resolvió el reembolso»	1	0	0	0	0	1

$M^T \cdot M$ cuenta en cuántos documentos aparece **cada par** de palabras (co-ocurrencia). Se forman dos bloques: {chat, soporte, espera} y {cobro, tarjeta, reembolso}. D5 mezcla palabras de ambos: **un documento puede pertenecer a varios tópicos a la vez**.

```

pd.DataFrame((M.T @ M).toarray(), index=terminos, columns=terminos)

```

	chat	soporte	espera	cobro	tarjeta	reembolso
chat	2	1	1	0	0	1
soporte	1	2	2	0	0	0
espera	1	2	2	0	0	0
cobro	0	0	0	2	2	1
tarjeta	0	0	0	2	2	1
reembolso	1	0	0	1	1	2

4. Entrenar LDA

LDA supone que cada documento se escribió así:

1. Elegir la mezcla de tópicos del documento: $\theta \sim \text{Dirichlet}(\alpha)$.
2. Para cada palabra, elegir un tópico: $z \sim \text{Multinomial}(\theta)$.
3. Elegir la palabra dentro de ese tópico: $w \sim \text{Multinomial}(\phi_z)$.

El modelo observa **solo las palabras** y estima θ y ϕ «al revés» (scikit-learn usa inferencia variacional).

Hiperparámetro	scikit-learn	Efecto
K	n_components	Pocos tópicos mezclan temas; demasiados los fragmentan
α	doc_topic_prior	Bajo: cada documento usa pocos tópicos
β	topic_word_prior	Bajo: tópicos definidos por pocas palabras dominantes

En textos cortos conviene un **α bajo**: cada comentario trata pocos temas.

```
K = 5
lda = LatentDirichletAllocation(n_components=K, doc_topic_prior=0.1,
                               topic_word_prior=0.01, random_state=42)
theta = lda.fit_transform(X) #  $\theta$ : documentos  $\times$  tópicos
phi = lda.components_ / lda.components_.sum(axis=1, keepdims=True) #  $\phi$ : tópicos  $\times$  palabras

print("\theta:", theta.shape, "  $\phi$ :", phi.shape)
print("Cada fila de  $\theta$  suma 1:", np.allclose(theta.sum(axis=1), 1))
```

θ : (135, 5) ϕ : (5, 220)
Cada fila de θ suma 1: True

5. Interpretar

5.1 Palabras con mayor peso en cada tópico (top-8 de ϕ)

```
def top_palabras(componentes, n=8):
    return [[vocab[i] for i in fila.argsort()[::-1][:n]] for fila in componentes]

df["topico_lda"] = theta.argmax(axis=1)
resumen = pd.DataFrame({
    "palabras con mayor peso": [", ".join(p for p in top_palabras(lda.components_[i])),
    "comentarios": df["topico_lda"].value_counts().sort_index(),
    "calif. promedio": df.groupby("topico_lda")["calificacion"].mean().round(1),
}).rename_axis("tópico")
resumen
```

	palabras con mayor peso	comentarios	calif. promedio
tópico			
0	experiencia, sitio, soporte, general, comprar, tiempo, chat, buena	31	3.4
1	buena, garantía, resolver, opciones, atención, accesorios, experiencia, conforme	23	3.8
2	proceso, pedido, pago, rápido, sitio, días, búsqueda, tres	37	3.2
3	puntos, soporte, proceso, correo, tarde, servicio, problema, sistema	22	3.0
4	proceso, producto, envío, tiendas, varias, precio, productos, comparado	22	2.9

¿Qué nombre le pondrían a cada tópico? Fíjense en cuántas palabras genéricas («proceso», «buena», «experiencia», «compra») se repiten entre tópicos.

5.2 Pesos de ϕ en un gráfico

```
fig, axes = plt.subplots(1, K, figsize=(14, 3.4), sharex=True)
for k, ax in enumerate(axes):
    orden = phi[k].argsort()[::-1][:8]
    ax.barh([vocab[i] for i in orden][::-1], phi[k][orden][::-1], color=AZUL,
height=0.6)
    ax.set_title(f"Tópico {k}", loc="left", fontsize=10)
    ax.grid(axis="y", visible=False)
    ax.tick_params(labelsize=8)
fig.supxlabel("Probabilidad de la palabra en el tópico ( $\phi$ )", fontsize=9, color=TINTA)
plt.tight_layout()
plt.show()
```



5.3 Un documento es una mezcla de tópicos (θ)

Tres comentarios con su mezcla estimada. El tópico «dominante» (`argmax`) es una simplificación: LDA reparte cada documento entre varios tópicos.

```
muestra = df.sample(3, random_state=3).index
pd.concat([df.loc[muestra, ["texto"]],
pd.DataFrame(theta[muestra], index=muestra,
columns=[f"T{k}" for k in range(K)].round(2)], axis=1)
```

	texto	T0	T1	T2	T3	T4
15	El proceso de cambio de producto por otro modelo fue mas lento de lo esperado.	0.02	0.02	0.02	0.02	0.94
73	El servicio de asistencia por correo respondio con una plantilla generica que no resolvía mi problema específico.	0.02	0.02	0.02	0.91	0.02
57	Un vendedor de soporte me dio informacion incorrecta sobre un plazo de entrega y tuve que reprogramar mis planes.	0.03	0.03	0.03	0.89	0.03

5.4 Efecto de α en la mezcla

Con $\alpha = 0.1$ la mayor parte de cada documento cae en un solo tópico; con $\alpha = 1$ las mezclas se vuelven más uniformes. Lo medimos con el **peso del tópico dominante** promedio.

```
for alfa in [0.1, 1.0]:
    t = LatentDirichletAllocation(n_components=K, doc_topic_prior=alfa,
topic_word_prior=0.01,
random_state=42).fit_transform(X)
```

```
print(f"α = {alfa:<4} peso promedio del tópico dominante = {t.max(axis=1).mean():.2f}")
```

α = 0.1 peso promedio del tópico dominante = 0.89

α = 1.0 peso promedio del tópico dominante = 0.44

6. Evaluar

6.1 Coherencia NPMI

¿Las palabras principales de un tópico **aparecen juntas** en los documentos?

$$\text{NPMI}(w_i, w_j) = \log[P(w_i, w_j) / (P(w_i) P(w_j))] / -\log P(w_i, w_j)$$

- **+1**: las dos palabras siempre aparecen juntas.
- **0**: aparecen juntas lo que se esperaría por azar.
- **-1**: nunca aparecen juntas.

Se promedia sobre todos los pares de las top-n palabras de cada tópico. Las probabilidades se estiman como proporción de documentos.

```
B = (X > 0).toarray()          # presencia (1) o ausencia (0) de cada término

def npmi_par(i, j, B=B):
    p_i, p_j = B[:, i].mean(), B[:, j].mean()
    p_ij = (B[:, i] & B[:, j]).mean()
    if p_ij == 0:
        return -1.0
    if p_ij == 1:
        return 1.0
    return np.log(p_ij / (p_i * p_j)) / -np.log(p_ij)

def coherencia(componentes, n=6):
    """NPMI promedio de cada tópico sobre los pares de sus top-n palabras."""
    return [np.mean([npmi_par(i, j)
                      for i, j in itertools.combinations(fila.argsort()[::-1][:n],
2)])
            for fila in componentes]

def diversidad(componentes, n=6):
    """Proporción de palabras únicas entre las top-n de todos los tópicos."""
    tops = [p for fila in top_palabras(componentes, n) for p in fila]
    return len(set(tops)) / len(tops)

resumen["NPMI"] = np.round(coherencia(lda.components_), 2)
print(f"Coherencia NPMI promedio: {np.mean(resumen['NPMI']):.2f}")
print(f"Diversidad (top-6): {diversidad(lda.components_):.2f}")
resumen
```

Coherencia NPMI promedio: -0.29

Diversidad (top-6): 0.87

tópico	palabras con mayor peso	comentarios	calif. promedio	NPMI
0	experiencia, sitio, soporte, general, comprar, tiempo, chat, buena	31	3.4	-0.42
1	buena, garantia, resolver, opciones, atencion, accesorios, experiencia, conforme	23	3.8	-0.28
2	proceso, pedido, pago, rapido, sitio, dias, busqueda, tres	37	3.2	-0.16
3	puntos, soporte, proceso, correo, tardó, servicio, problema, sistema	22	3.0	-0.38
4	proceso, producto, envío, tiendas, varias, precio, productos, comparado	22	2.9	-0.19

Una NPMI promedio **negativa** indica que las palabras de un mismo tópico rara vez aparecen juntas en un comentario. Con 135 textos cortos, LDA no logra separar los temas con claridad.

6.2 Coherencia según el número de tópicos K

Probamos K de 2 a 10. En corpus grandes la curva suele subir, estabilizarse y luego decaer; se elige K donde se estabiliza **y** los tópicos tienen sentido para el negocio.

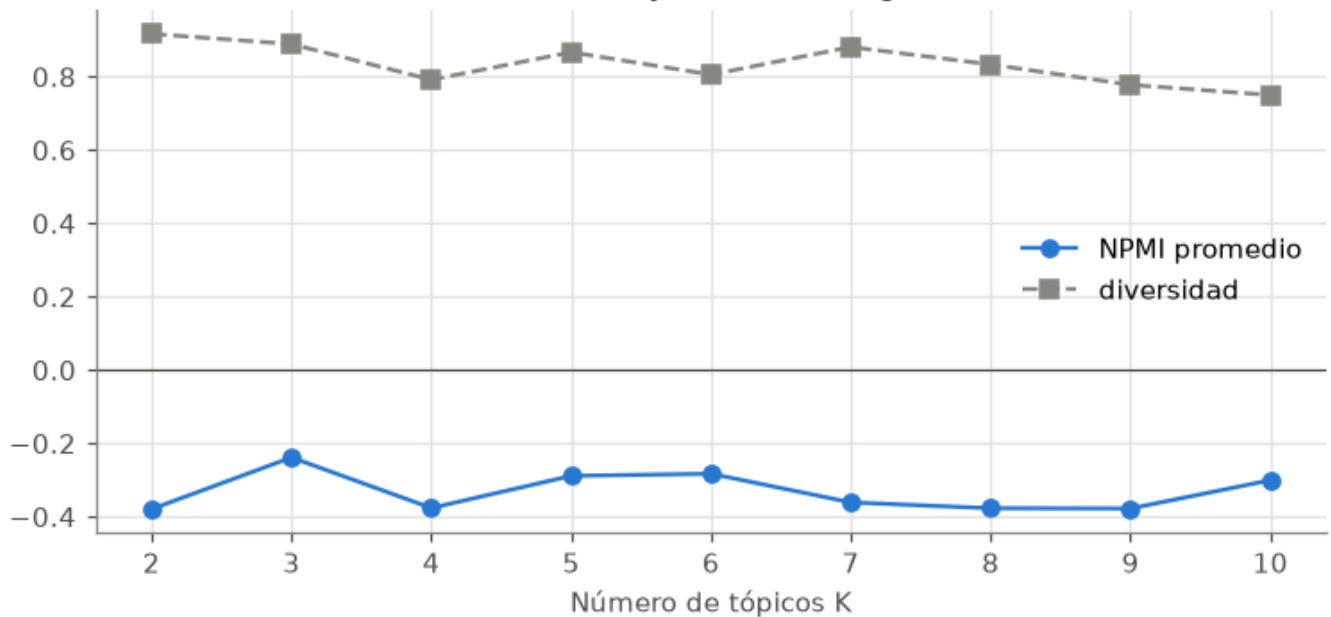
```

filas = []
for k in range(2, 11):
    m = LatentDirichletAllocation(n_components=k, doc_topic_prior=0.1,
                                  topic_word_prior=0.01, random_state=42).fit(X)
    filas.append({"K": k, "NPMI": np.mean(coherencia(m.components_)),
                  "diversidad": diversidad(m.components_)})
curva = pd.DataFrame(filas).set_index("K")

fig, ax = plt.subplots(figsize=(7, 3.6))
ax.plot(curva.index, curva["NPMI"], "o-", color=AZUL, label="NPMI promedio")
ax.plot(curva.index, curva["diversidad"], "s--", color=GRIS, label="diversidad")
ax.axhline(0, color=TINTA, linewidth=0.8)
ax.set_xlabel("Número de tópicos K")
ax.set_title("LDA en comentarios.csv: coherencia y diversidad según K", loc="left",
             fontsize=11)
ax.legend(frameon=False)
plt.tight_layout()
plt.show()
curva.round(2).T

```

LDA en comentarios.csv: coherencia y diversidad según K



K	2	3	4	5	6	7	8	9	10
NPMI	-0.38	-0.24	-0.38	-0.29	-0.28	-0.36	-0.38	-0.38	-0.30
diversidad	0.92	0.89	0.79	0.87	0.81	0.88	0.83	0.78	0.75

Discusión

¿Qué ocurre con K = 3 y con K = 8?

```
for k in [3, 8]:
    m = LatentDirichletAllocation(n_components=k, doc_topic_prior=0.1,
                                  topic_word_prior=0.01, random_state=42).fit(X)
    print(f"--- K = {k}")
    for i, palabras in enumerate(top_palabras(m.components_, 6)):
        print(f"  {i}: {' '.join(palabras)}")
```

```
--- K = 3
0: comprar, soporte, tiempo, chat, mejor, correo
1: buena, experiencia, soporte, sitio, general, busqueda
2: proceso, envio, sitio, pedido, pago, tiendas
--- K = 8
0: correo, general, mejor, minutos, compras, registro
1: buena, garantia, atencion, accesorios, producto, resolver
2: sitio, pago, rapido, proceso, tiendas, opciones
3: soporte, producto, puntos, error, descuento, facil
4: proceso, varias, tres, lento, precios, primera
5: pedido, proceso, correo, dias, comprar, electronica
6: tiempo, soporte, sitio, marca, tener, espera
7: experiencia, buena, compra, envio, general, tienda
```

¿Qué palabras agregarían a **STOPWORDS_ES** ?

Agreguemos palabras de **dominio** y de **valoración genérica** que no identifican temas, y usemos bigramas (`ngram_range=(1, 2)`). Comparen las palabras y la coherencia con el modelo original.

```

STOPWORDS_DOMINIO = STOPWORDS_ES + [
    "tienda", "tiendas", "producto", "productos", "compra", "comprar", "proceso",
    "experiencia", "buena", "general", "tiempo",
]

cv2 = CountVectorizer(preprocessor=normalizar, stop_words=STOPWORDS_DOMINIO,
                     min_df=2, ngram_range=(1, 2))
X2 = cv2.fit_transform(df["texto"])
vocab2 = cv2.get_feature_names_out()
B2 = (X2 > 0).toarray()
lda2 = LatentDirichletAllocation(n_components=K, doc_topic_prior=0.1,
                                topic_word_prior=0.01, random_state=42).fit(X2)

nmpi2 = [np.mean([nmpi_par(i, j, B2)
                  for i, j in itertools.combinations(f.argsort()[::-1][:6], 2)])
          for f in lda2.components_]
pd.DataFrame({
    "palabras con mayor peso": [", ".join(vocab2[i] for i in f.argsort()[::-1][:8])
                                for f in lda2.components_],
    "NPMI": np.round(nmpi2, 2),
}).rename_axis("tópico")

```

	palabras con mayor peso	NPMI
tópico		
0	sitio, pago, compras, resolver, opciones, puntos, pedido, diseno	-0.65
1	envio, despues, atencion, pago, modelo, correo, rapido, registro	-0.80
2	real, siempre, tecnologia, sitio, soporte, dias, checkout, dia	-0.40
3	soporte, correo, garantia, busqueda, minutos, mejor, rapido, deberian	-0.49
4	tres, catalogo, envio, funciona, distintos, decidir, recibi, soporte	-0.50

Preguntas para el grupo:

- ¿Mejoró la interpretabilidad al quitar palabras de dominio? ¿Y la NPMI? ¿Siempre van de la mano?
- Cambien `random_state` (p. ej., 0, 1, 7): ¿se mantienen los tópicos? ¿Qué dice eso sobre la estabilidad de LDA con 135 comentarios?
- LDA trata «demoro», «tardo» y «lento» como tres palabras distintas. ¿Qué representación vista en clases anteriores los pondría juntos? Esa es la idea de BERTopic (Ejercicio 2).

Espacio para experimentar: cambien K , α , β , la semilla o las stopwords

Sesión de clase 08 · Ejercicio 2: BERTopic paso a paso

BERTopic (Grootendorst, 2022) separa dos problemas:

1. **Agrupar documentos por su significado:** embeddings → UMAP → HDBSCAN.
2. **Describir cada grupo con palabras:** CountVectorizer → c-TF-IDF → representación (KeyBERT, MMR o un LLM).

Aspecto	LDA	BERTopic
Representación	Conteo de palabras	Embeddings de Transformers
Sinónimos	No los reconoce	«demoró» y «tardó» quedan cerca
Número de tópicos	K fijo	Lo determina HDBSCAN
Asignación	Mezcla de tópicos por documento	Un tópico por documento
Documentos atípicos	Todos se asignan	Tópico -1 para el ruido

Primero ejecutamos **cada módulo por separado** para ver qué hace, y luego los unimos en un `BERTopic`. Corpus: los 135 comentarios de `data/comentarios.csv`.

Etapas: 1. Embeddings · 2. UMAP · 3. HDBSCAN · 4. c-TF-IDF · 5. BERTopic completo · 6. Representaciones · 7. Ajuste (outliers y `min_cluster_size`) · Discusión.

```
import unicodedata
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from bertopic import BERTopic
from bertopic.representation import KeyBERTInspired, MaximalMarginalRelevance
from hdbscan import HDBSCAN
from sentence_transformers import SentenceTransformer
from sklearn.cluster import KMeans
from sklearn.datasets import make_moons
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from umap import UMAP

pd.set_option("display.max_colwidth", 120)
SALIDAS = Path("../salidas")
SALIDAS.mkdir(exist_ok=True)

AZUL, ROJO, GRIS, TINTA = "#2a78d6", "#e34948", "#8a8985", "#52514e"
plt.rcParams.update({
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": GRIS, "axes.labelcolor": TINTA,
    "xtick.color": TINTA, "ytick.color": TINTA,
```

```

"axes.grid": True, "grid.color": "#e6e5e1", "grid.linewidth": 0.8,
"axes.axisbelow": True, "figure.dpi": 110,
})

df = pd.read_csv("../data/comentarios.csv")
docs = df["texto"].tolist()
print(len(docs), "comentarios")

```

135 comentarios

1. Embeddings de los documentos

`paraphrase-multilingual-MiniLM-L12-v2` : 384 dimensiones, más de 50 idiomas, corre en CPU. **Buena práctica:** calcular los embeddings **una vez**, guardarlos (`embeddings.npy`) y pasarlos a `fit_transform(docs, embeddings)` para no recalcularlos en cada prueba.

```

st = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")

ruta_emb = SALIDAS / "embeddings.npy"
if ruta_emb.exists():
    embeddings = np.load(ruta_emb)
else:
    embeddings = st.encode(docs, show_progress_bar=True)
    np.save(ruta_emb, embeddings)
print(embeddings.shape)

```

Loading weights: 100%  199/199 [00:00<00:00, 10184.08it/s]
(135, 384)

Los embeddings acercan textos con **el mismo significado aunque no compartan palabras**: justo lo que LDA no podía hacer.

```

frases = ["El chat demoro media hora en responder",
          "El soporte tardo en contestar mi consulta",
          "Me cobraron dos veces en la tarjeta"]
sim = cosine_similarity(st.encode(frases))
pd.DataFrame(sim, index=frases, columns=["chat demoró", "soporte tardó", "cobraron dos veces"]).round(2)

```

	chat demoró	soporte tardó	cobraron dos veces
El chat demoro media hora en responder	1.00	0.34	0.24
El soporte tardo en contestar mi consulta	0.34	1.00	0.18
Me cobraron dos veces en la tarjeta	0.24	0.18	1.00

2. Reducción de dimensionalidad con UMAP

Hipótesis del manifold: los embeddings viven en 384 dimensiones, pero se concentran cerca de una superficie de dimensión mucho menor. En 384 dimensiones las distancias entre puntos tienden a parecerse (**maldición de la dimensionalidad**) y la densidad pierde contraste, lo que dificulta el clustering.

UMAP construye un grafo de k vecinos en 384 d y busca una representación de pocas dimensiones que preserve ese grafo.

Parámetro	Valor	Efecto
n_neighbors	10	Bajo: estructura local; alto: visión global
n_components	5	Dimensiones de salida para el clustering
min_dist	0.0	Puntos compactos que favorecen HDBSCAN
metric	"cosine"	La misma métrica de los embeddings
random_state	42	Resultados reproducibles

Hacemos **dos reducciones**: a 5 d para agrupar y a 2 d solo para graficar.

```
umap_5d = UMAP(n_neighbors=10, n_components=5, min_dist=0.0,
               metric="cosine", random_state=42)
red_5d = umap_5d.fit_transform(embeddings)

red_2d = UMAP(n_neighbors=10, n_components=2, min_dist=0.0,
              metric="cosine", random_state=42).fit_transform(embeddings)
print("5 d:", red_5d.shape, " 2 d:", red_2d.shape)
```

5 d: (135, 5) 2 d: (135, 2)

¿Por qué no agrupar directamente en 384 d?

Comparamos la dispersión de las distancias: cuanto más se parecen todas las distancias entre sí, menos contraste hay entre zonas densas y vacías. El **coeficiente de variación** (desviación ÷ media) lo resume.

```
from scipy.spatial.distance import pdist

for nombre, datos, metrica in [("384 d (embeddings)", embeddings, "cosine"),
                              ("5 d (UMAP)", red_5d, "euclidean")]:
    d = pdist(datos, metric=metrica)
    print(f"{nombre:<20} coeficiente de variación de las distancias = {d.std() / d.mean():.2f}")
```

384 d (embeddings) coeficiente de variación de las distancias = 0.21
5 d (UMAP) coeficiente de variación de las distancias = 0.53

3. Clustering con HDBSCAN

- **Agrupar por densidad:** un clúster es una región con muchos puntos cercanos.
- **Construye una jerarquía** de densidades y conserva los clústeres más estables.
- **Define el número de tópicos:** no se fija K.
- **Separa el ruido:** los documentos aislados reciben la etiqueta -1.

3.1 HDBSCAN frente a k-means

Con datos simulados de dos medias lunas más puntos aislados:

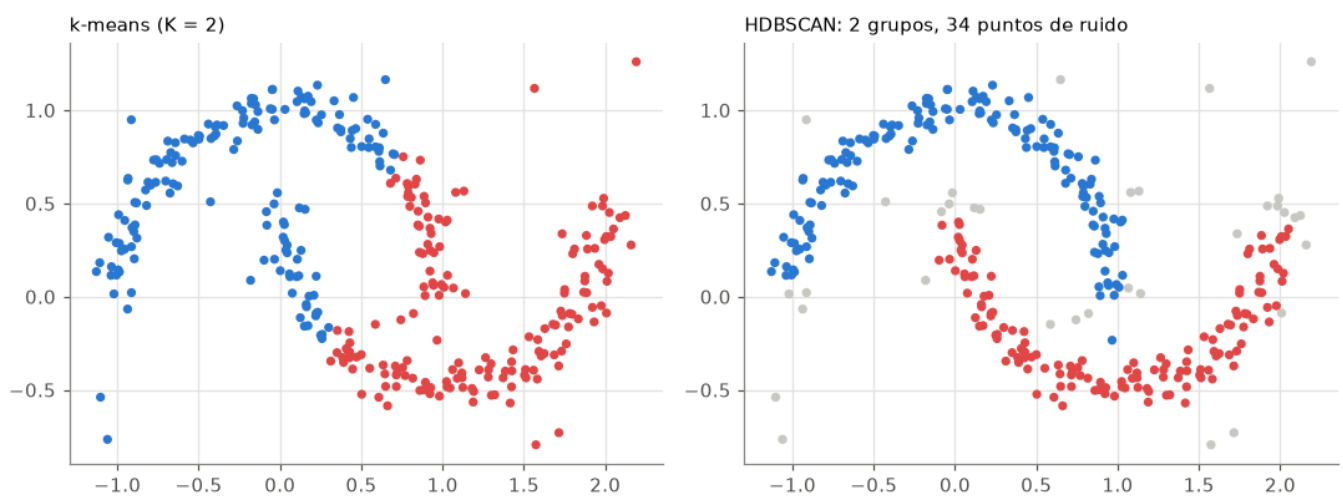
```

X_lunas, _ = make_moons(300, noise=0.06, random_state=42)
rng = np.random.default_rng(0)
X_lunas = np.vstack([X_lunas, rng.uniform([-1.2, -0.8], [2.2, 1.3], size=(25, 2))])

etq_km = KMeans(2, n_init=10, random_state=42).fit_predict(X_lunas)
etq_hd = HDBSCAN(min_cluster_size=15).fit_predict(X_lunas)

fig, axes = plt.subplots(1, 2, figsize=(10, 3.8))
paleta = np.array([AZUL, ROJO, "#3aa776", "#e0a030"])
for ax, etq, titulo in [(axes[0], etq_km, "k-means (K = 2)"),
                        (axes[1], etq_hd, f"HDBSCAN: {etq_hd.max() + 1} grupos, "
                                           f"{(etq_hd == -1).sum()} puntos de "
                                           "ruido")]:
    color = np.where(etq == -1, "#c9c8c4", paleta[np.clip(etq, 0, 3)])
    ax.scatter(X_lunas[:, 0], X_lunas[:, 1], c=color, s=14)
    ax.set_title(titulo, loc="left", fontsize=10)
plt.tight_layout()
plt.show()

```



k-means necesita K, supone grupos esféricos (corta las medias lunas) y asigna **todos** los puntos, incluso el ruido. HDBSCAN encuentra las dos formas y deja los puntos aislados en -1.

3.2 HDBSCAN sobre los comentarios

`min_cluster_size` es el tamaño mínimo de un tópico. Con 135 documentos conviene un valor entre 4 y 8. Probamos varios:

```

filas = []
for mcs in [4, 5, 6, 8, 10]:
    etq = HDBSCAN(min_cluster_size=mcs).fit_predict(red_5d)
    tamanos = np.bincount(etq[etq >= 0])
    filas.append({"min_cluster_size": mcs, "grupos": etq.max() + 1,
                  "ruido (-1)": (etq == -1).sum(), "grupo más grande": tamanos.max(),
                  "tamaños": sorted(tamanos.tolist(), reverse=True)})
pd.DataFrame(filas).set_index("min_cluster_size")

```

	grupos	ruido (-1)	grupo más grande	tamaños
min_cluster_size				
4	9	26	34	[34, 13, 12, 12, 11, 10, 7, 6, 4]
5	5	7	65	[65, 35, 12, 10, 6]
6	4	21	64	[64, 35, 8, 7]
8	3	27	65	[65, 34, 9]
10	2	21	66	[66, 48]

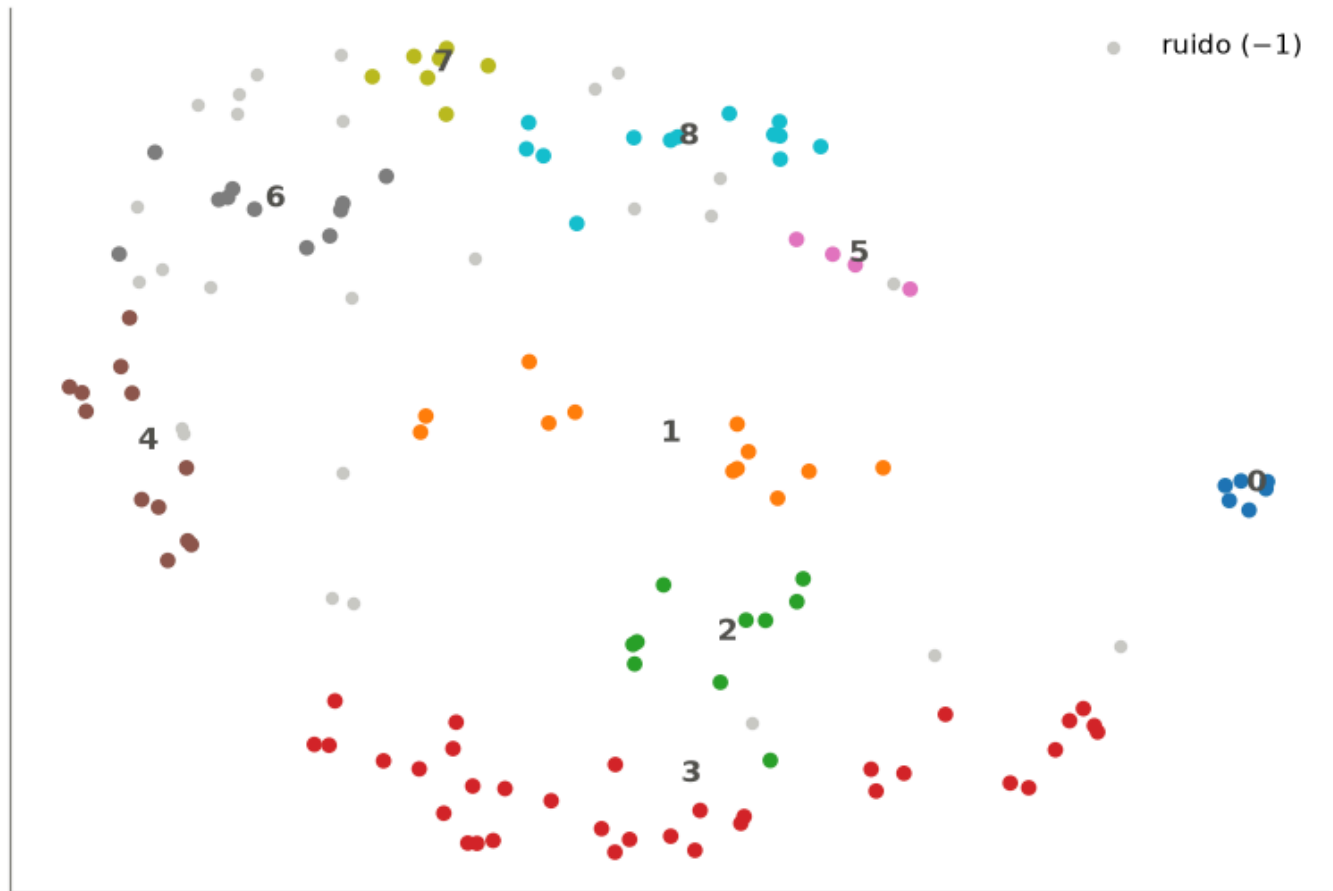
Valores altos producen **pocos tópicos muy grandes** que mezclan temas. Usamos `min_cluster_size=4`.

```
MIN_CLUSTER_SIZE = 4
etiquetas = HDBSCAN(min_cluster_size=MIN_CLUSTER_SIZE).fit_predict(red_5d)
print(f"{etiquetas.max() + 1} grupos y {(etiquetas == -1).sum()} documentos de ruido")

fig, ax = plt.subplots(figsize=(7, 5))
ruido = etiquetas == -1
ax.scatter(red_2d[ruido, 0], red_2d[ruido, 1], c="#c9c8c4", s=14, label="ruido (-1)")
sc = ax.scatter(red_2d[~ruido, 0], red_2d[~ruido, 1], c=etiquetas[~ruido],
                cmap="tab10", s=22)
for g in range(etiquetas.max() + 1):
    cx, cy = red_2d[etiquetas == g].mean(axis=0)
    ax.text(cx, cy, str(g), fontsize=11, weight="bold", color=TINTA)
ax.set_title("Comentarios en 2 d (UMAP) coloreados por grupo de HDBSCAN", loc="left",
            fontsize=11)
ax.legend(frameon=False, loc="best")
ax.set_xticks([]); ax.set_yticks([])
plt.tight_layout()
plt.show()
```

9 grupos y 26 documentos de ruido

Comentarios en 2 d (UMAP) coloreados por grupo de HDBSCAN



Antes de describir los grupos con palabras, leamos tres comentarios de cada uno: ¿qué tienen en común?

```
(df.assign(grupo=etiquetas)
 .groupby("grupo").head(3)
 .sort_values("grupo")[["grupo", "calificacion", "texto"]])
```

grupo	calificacion		texto
6	-1	4	El soporte me oriento bien sobre compatibilidad de accesorios con mi equipo.
16	-1	5	Me gusto poder guardar varias direcciones de envio para regalos.
15	-1	2	El proceso de cambio de producto por otro modelo fue mas lento de lo esperado.
81	0	2	El diseno del sitio en el celular se ve desordenado en pantallas pequenas, algunos botones quedan superpuestos.
46	0	3	La app movil a veces se cierra sola cuando cambio de pantalla muy rapido. Fuera de eso funciona bien para navegar el...
33	0	3	La app funciona correctamente la mayoria del tiempo, con alguna que otra pausa al cargar imagenes.
22	1	3	El diseno del sitio es agradable pero en modo oscuro algunos textos son dificiles de leer. Pequeños ajustes de contr...
19	1	4	Buena experiencia general, aunque el sitio podria cargar mas rapido en horas pico.
5	1	5	Muy buena atencion personalizada cuando pregunte por disponibilidad de un modelo agotado.
0	2	5	El proceso de facturacion para empresa fue mas simple de lo que esperaba.
4	2	3	El proceso de devolucion fue sencillo de iniciar aunque tomo unos dias en confirmarse.
9	2	3	El registro de reclamos deberia enviar actualizaciones automaticas por correo.
20	3	4	El tiempo de respuesta por WhatsApp fue mejor que por correo electronico.
17	3	3	El tiempo de respuesta del correo de soporte es aceptable, normalmente en menos de 24 horas.
3	3	3	El envio internacional tardo mas dias de los indicados originalmente.
7	4	3	El catalogo de marcas es decente aunque hay ciertos modelos que nunca llegan a tener stock.
24	4	3	La seccion de preguntas frecuentes no cubre todos los casos que tuve.
31	4	2	El filtro de precio no se actualiza correctamente al combinarlo con marca.
12	5	5	Muy satisfecho con la claridad de las politicas de garantia antes de comprar.
70	5	5	Muy conforme con la transparencia de las politicas de garantia antes de comprar. Todo quedo claro desde el principio...
80	5	5	El soporte por correo fue muy claro al explicarme las opciones de financiamiento.
11	6	4	El sitio recuerda mis busquedas anteriores y sugiere productos relacionados de forma acertada.
58	6	4	Me gusto poder comparar hasta tres productos a la vez antes de decidir. Esa funcion me ahorro bastante tiempo en la ...
35	6	4	El sitio permite comparar hasta tres productos a la vez, muy util para decidir.
1	7	3	En general cumple lo que promete, ni mejor ni peor que otras tiendas de tecnologia que he probado.

	grupo	calificacion	texto
14	7	5	Es mi tienda de tecnologia preferida desde hace varios meses, siempre encuentro lo que busco. La atencion al cliente...
2	7	4	Buena tienda en general, seguire comprando aqui mis proximos equipos.
8	8	5	La experiencia de compra en oferta de temporada fue muy fluida, sin caidas del sitio.
13	8	3	Es una tienda confiable en general, aunque los precios no siempre son los mas bajos del mercado.
10	8	3	La experiencia de compra es correcta, sin contratiempos importantes en mis ultimas ordenes.

4. c-TF-IDF: palabras que distinguen a cada grupo

Se unen todos los documentos de un grupo en **un solo documento por grupo** y se pondera cada término:

$$W(t, c) = tf(t, c) \times \log(1 + A / f(t))$$

- **tf(t, c)**: frecuencia del término t en el grupo c.
- **f(t)**: frecuencia de t en todos los grupos.
- **A**: promedio de palabras por grupo.

Una palabra **exclusiva y frecuente** en un grupo recibe peso alto; una palabra **genérica** («buena», «proceso») recibe peso bajo aunque sea frecuente.

Las *stopwords* se quitan **solo aquí**: los embeddings se calcularon con el texto completo.

```
def normalizar(texto):
    t = unicodedata.normalize("NFKD", texto.lower())
    return "".join(ch for ch in t if not unicodedata.combining(ch))

STOPWORDS_ES = sorted(set("""
a al algo algun alguna algunas alguno algunos ante antes aqui asi aun aunque
bajo bien cada casi como con contra cual cuando de del desde donde dos durante
e el ella ellas ellos en entre era eran es esa esas ese eso esos esta estaba
estaban estan estar este esto estos fue fueron ha habia han hasta hay he la las
le les lo los mas me mi mis mucho muy nada ni no nos o otra otras otro otros
para pero poco por porque que se ser si sin sobre solo su sus tambien tan tanto
te tiene todo todos tu tuve un una uno unos y ya yo vez veces mismo misma
son podria poder bastante
""").split()))

vectorizer = CountVectorizer(preprocessor=normalizar, stop_words=STOPWORDS_ES)
```

Calculamos c-TF-IDF **a mano** con la fórmula, para los grupos encontrados (sin el ruido):

```
grupos = sorted(g for g in set(etiquetas) if g != -1)
doc_por_grupo = [" ".join(np.array(docs)[etiquetas == g]) for g in grupos]

conteos = vectorizer.fit_transform(doc_por_grupo).toarray() # grupos x términos
```

```
terminos = vectorizer.get_feature_names_out()

tf = conteos
f = conteos.sum(axis=0) # frecuencia de cada término en todos los grupos
A = conteos.sum(axis=1).mean() # promedio de palabras por grupo
ctfidf = tf * np.log(1 + A / f)

print(f"A = {A:.1f} palabras por grupo")
pd.DataFrame({"grupo {g}": [terminos[i] for i in ctfidf[k].argsort()[::-1][:6]]
              for k, g in enumerate(grupos)})
```

A = 106.4 palabras por grupo

	grupo 0	grupo 1	grupo 2	grupo 3	grupo 4	grupo 5	grupo 6	grupo 7	grupo 8
0	app	sitio	registro	soporte	producto	claro	productos	tecnologia	compra
1	movil	buena	pago	correo	marcas	politicas	comparar	tienda	tienda
2	celular	experiencia	proceso	chat	stock	garantia	decidir	general	puntos
3	funciona	dificiles	cuenta	tardo	disponible	comprar	busqueda	buena	envio
4	rapido	oscuro	rapido	tiempo	variedad	financiamiento	revisar	preferida	pena
5	quedan	textos	simple	pedido	precio	explicarme	filtros	amplio	ultima

Comparemos en el grupo más grande el peso de una palabra exclusiva con el de una genérica:

```
g = int(np.argmax(np.bincount(etiquetas[etiquetas >= 0])))
fila = grupos.index(g)
palabras = [terminos[i] for i in ctfidf[fila].argsort()[::-1][:3]] + ["buena",
"experiencia", "general"]
pd.DataFrame([{"término": t, "tf en el grupo": tf[fila, j], "f en el corpus": f[j],
               "peso c-TF-IDF": round(ctfidf[fila, j], 1)}
              for t in palabras if t in terminos
              for j in [list(terminos).index(t)]]).set_index("término")
```

	tf en el grupo	f en el corpus	peso c-TF-IDF
término			
soporte	8	9	20.4
correo	7	9	17.9
chat	5	5	15.5
buena	0	13	0.0
experiencia	1	10	2.5
general	0	8	0.0

5. BERTopic completo

Unimos los módulos. Como fijamos la semilla de UMAP y pasamos los mismos embeddings, BERTopic reproduce los grupos anteriores (renumerados por tamaño: el tópico 0 es el más grande).

En `representation_model` pedimos **tres descripciones a la vez**: c-TF-IDF (`Main`), `KeyBERTInspired` y `MaximalMarginalRelevance` .

```
def crear_modelo(min_cluster_size=MIN_CLUSTER_SIZE):
    return BERTopic(
        embedding_model=st,
        umap_model=UMAP(n_neighbors=10, n_components=5, min_dist=0.0,
                        metric="cosine", random_state=42),
        hdbscan_model=HDBSCAN(min_cluster_size=min_cluster_size,
        prediction_data=True),
        vectorizer_model=CountVectorizer(preprocessor=normalizar,
        stop_words=STOPWORDS_ES),
        representation_model={"KeyBERT": KeyBERTInspired(),
                             "MMR": MaximalMarginalRelevance(diversity=0.3)},
    )

topic_model = crear_modelo()
temas, probs = topic_model.fit_transform(docs, embeddings)

info = topic_model.get_topic_info()
info["calif. promedio"] = info["Topic"].map(
    df.assign(t=temas).groupby("t")["calificacion"].mean().round(1))
info[["Topic", "Count", "calif. promedio", "Representation"]]
```

	Topic	Count	calif. promedio	Representation
0	-1	26	4.0	[producto, accesorios, soporte, buena, compatibilidad, reconoce, postventa, elegir, laptops, buscador]
1	0	34	1.9	[soporte, correo, chat, tardó, tiempo, pedido, proceso, días, semana, sistema]
2	1	13	4.1	[compra, tienda, puntos, pena, programa, última, envío, realmente, experiencia, compras]
3	2	12	3.8	[sitio, buena, experiencia, general, virtual, modo, oscuro, leer, difíciles, rápidas]
4	3	12	2.6	[marcas, stock, producto, disponible, variedad, precio, compras, tiendas, productos, marcado]
5	4	11	3.8	[productos, comparar, decidir, búsqueda, filtros, anteriores, revisar, tener, fácil, marca]
6	5	10	3.4	[pago, registro, proceso, cuenta, crear, esperaba, simple, tomo, rápido, tarjeta]
7	6	7	4.3	[tecnología, tienda, general, buena, amplio, preferida, catálogo, tiendas, volver, diferencia]
8	7	6	3.2	[app, móvil, celular, funciona, rápido, incluso, botones, carga, duplicadas, mayoría]
9	8	4	5.0	[políticas, claro, garantía, comprar, escondida, pequeña, letra, financiamiento, explicarme, quedo]

`get_document_info` da el tópico, la probabilidad y las palabras clave de **cada documento**:

```
topic_model.get_document_info(docs)[["Document", "Topic", "Probability",
"Representative_document"]].head(8)
```

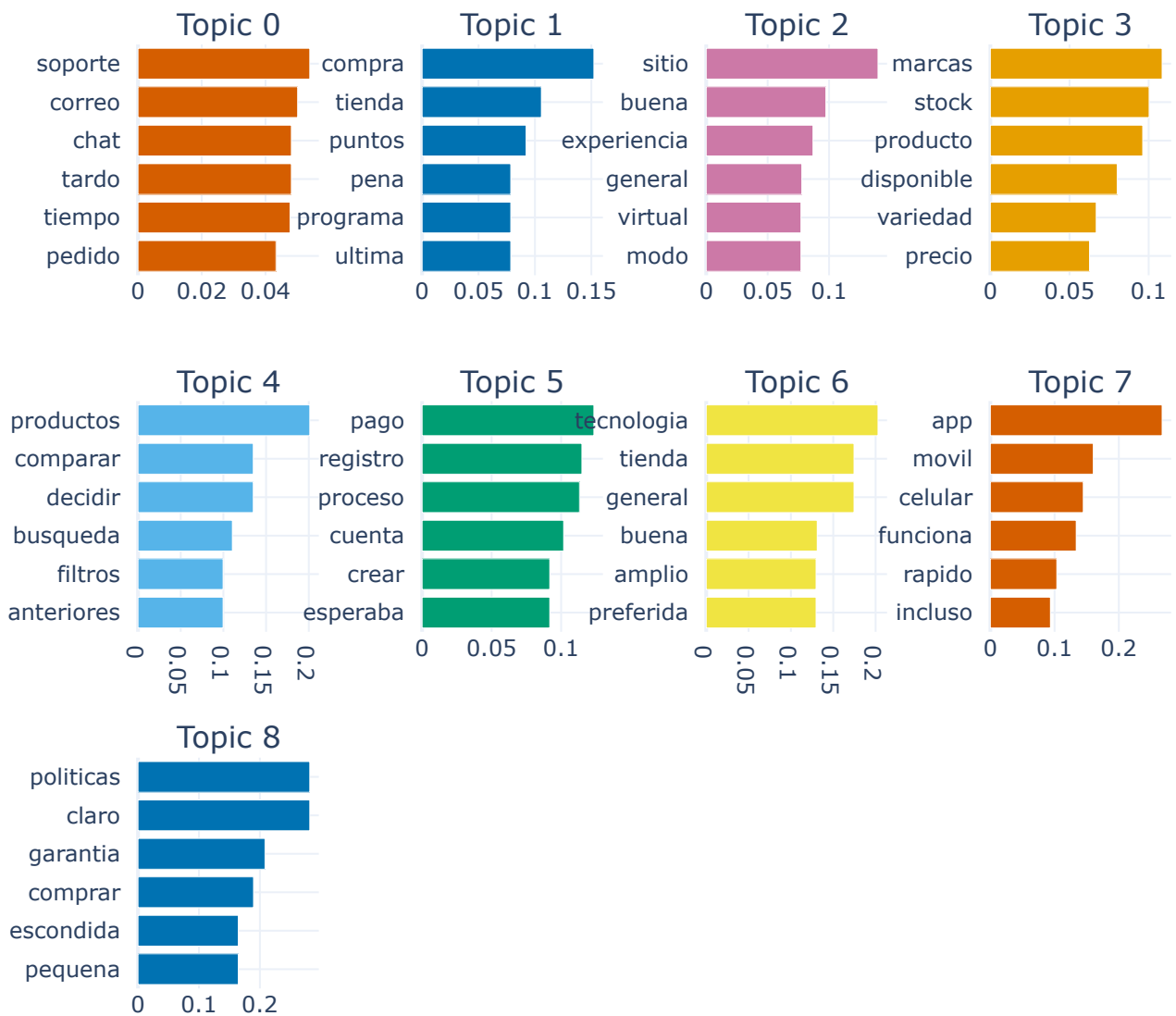
	Document	Topic	Probability	Representative_document
0	El proceso de facturacion para empresa fue mas simple de lo que esperaba.	5	1.000000	False
1	En general cumple lo que promete, ni mejor ni peor que otras tiendas de tecnologia que he probado.	6	1.000000	False
2	Buena tienda en general, seguire comprando aqui mis proximos equipos.	6	0.590634	False
3	El envio internacional tardo mas dias de los indicados originalmente.	0	1.000000	False
4	El proceso de devolucion fue sencillo de iniciar aunque tomo unos dias en confirmarse.	5	1.000000	False
5	Muy buena atencion personalizada cuando pregunte por disponibilidad de un modelo agotado.	2	0.761485	False
6	El soporte me oriento bien sobre compatibilidad de accesorios con mi equipo.	-1	0.000000	False
7	El catalogo de marcas es decente aunque hay ciertos modelos que nunca llegan a tener stock.	3	1.000000	True

Visualizaciones

Son gráficos interactivos de Plotly. Los guardamos también como HTML en `salidas/`.

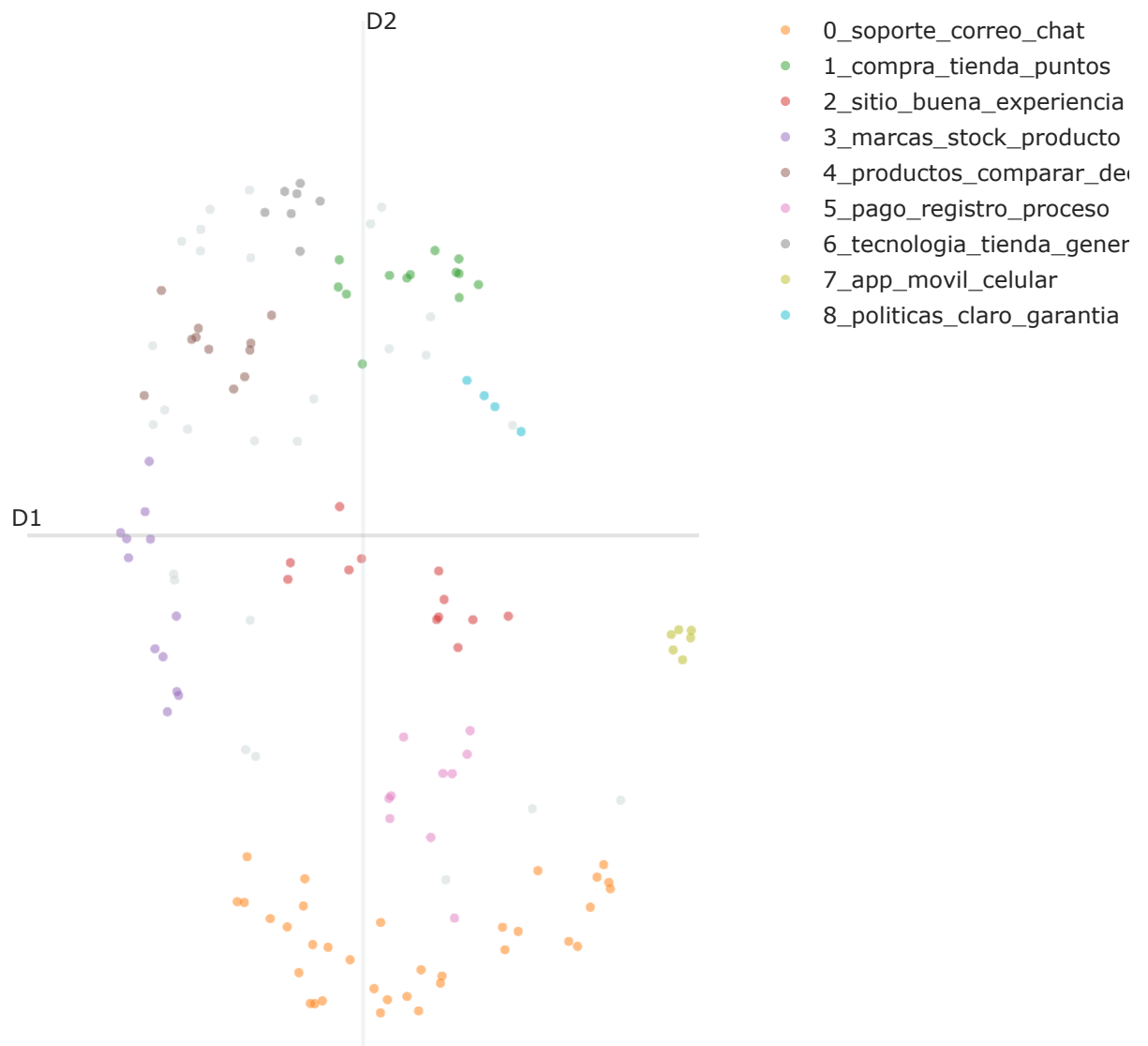
```
fig = topic_model.visualize_barchart(top_n_topics=10, n_words=6, width=230,
height=230)
fig.write_html(SALIDAS / "bertopic_barchart.html", include_plotlyjs="cdn")
fig.show()
```

Topic Word Scores



```
fig = topic_model.visualize_documents(docs, reduced_embeddings=red_2d,
hide_annotations=True)
fig.write_html(SALIDAS / "bertopic_documentos.html", include_plotlyjs="cdn")
fig.show()
```

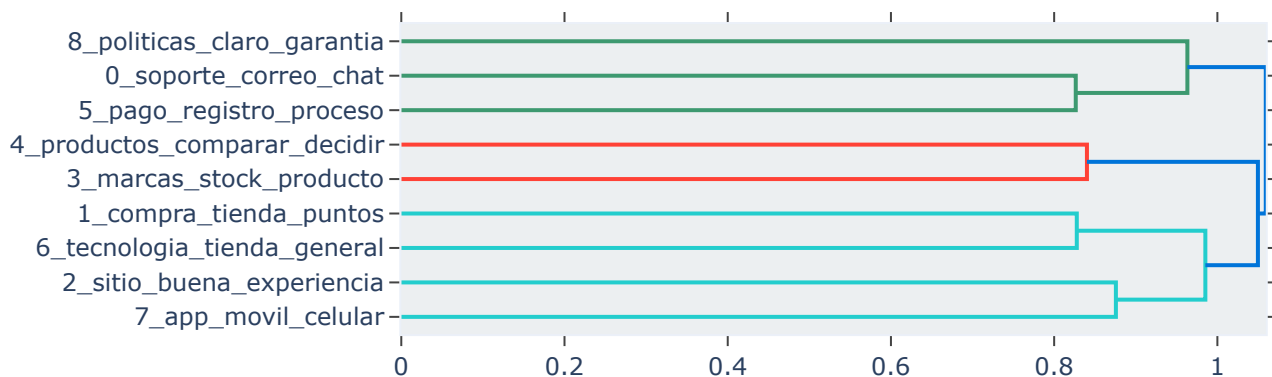
Documents and Topics



`visualize_hierarchy` muestra un dendrograma de los tópicos: ayuda a decidir cuáles fusionar.

```
fig = topic_model.visualize_hierarchy()
fig.write_html(SALIDAS / "bertopic_jerarquia.html", include_plotlyjs="cdn")
fig.show()
```

Hierarchical Clustering



6. Representaciones: c-TF-IDF, KeyBERTInspired y MMR

- **c-TF-IDF**: palabras con mayor peso.
- **KeyBERTInspired**: reordena las palabras por **similitud semántica** con los documentos representativos del tópico.
- **MMR (relevancia marginal máxima)**: cada nueva palabra debe ser relevante para el tópico **y distinta** de las ya elegidas:

$$\text{MMR} = \text{argmax} [(1 - d) \cdot \text{sim}(w, \text{tópico}) - d \cdot \text{máx sim}(w, \text{palabras elegidas})]$$

con $d = \text{diversity}$ (0 a 1). Elimina variantes redundantes («cobro», «cobraron», «cobros»).

```

reprs = info.set_index("Topic").drop(index=-1)
pd.DataFrame({
    "c-TF-IDF": reprs["Representation"].str[:6].str.join(", "),
    "KeyBERTInspired": reprs["KeyBERT"].str[:6].str.join(", "),
    "MMR (0.3)": reprs["MMR"].str[:6].str.join(", "),
}).rename_axis("tópico")

```

	c-TF-IDF	KeyBERTInspired	MMR (0.3)
tópico			
0	soporte, correo, chat, tardo, tiempo, pedido	correo, plazo, correos, checkout, retraso, cancele	chat, proceso, semana, sistema, error, menos
1	compra, tienda, puntos, pena, programa, ultima	compras, tiendas, compradores, comprando, tienda, compra	compra, tienda, experiencia, compras, tiendas, impuestos
2	sitio, buena, experiencia, general, virtual, modo	navegar, experiencia, virtual, pagina, leer, sitio	experiencia, virtual, oscuro, leer, dificiles, rapidas
3	marcas, stock, producto, disponible, variedad, precio	compre, compras, tiendas, comprar, compro, stock	stock, producto, variedad, compras, tiendas, desconfianza
4	productos, comparar, decidir, busqueda, filtros, anteriores	comparar, productos, compras, busquedas, comparacion, filtros	productos, comparar, busqueda, filtros, revisar, tres
5	pago, registro, proceso, cuenta, crear, esperaba	credito, checkout, tarjeta, facturacion, registro, pago	pago, registro, cuenta, tarjeta, datos, opciones
6	tecnologia, tienda, general, buena, amplio, preferida	tiendas, tienda, tecnologia, recomendaria, electronica, preferida	tecnologia, tienda, catalogo, tiendas, diferencia, inteligentes
7	app, movil, celular, funciona, rapido, incluso	app, navegador, pantalla, pantallas, celular, botones	app, movil, celular, funciona, botones, duplicadas
8	politicas, claro, garantia, comprar, escondida, pequena	transparencia, garantia, notificaciones, transparente, claridad, correo	politicas, garantia, escondida, financiamiento, transparencia, transparente

7. Ajuste en corpus pequeños

7.1 Documentos de ruido (tópico -1)

¿Qué comentarios quedaron como ruido?

```
df.assign(topico=temas).query("topico == -1")[["calificacion", "texto"]].head(10)
```

	calificacion	texto
6	4	El soporte me oriento bien sobre compatibilidad de accesorios con mi equipo.
15	2	El proceso de cambio de producto por otro modelo fue mas lento de lo esperado.
16	5	Me gusto poder guardar varias direcciones de envio para regalos.
18	4	Muy buena atencion cuando pregunte por la disponibilidad de un modelo agotado. Me avisaron apenas volvio a stock, ta...
21	3	El servicio postventa cumple lo basico, sin ser excepcional ni deficiente.
25	3	Las descripciones de producto son suficientes aunque podrian incluir mas fotos desde distintos angulos.
26	5	La comparacion de especificaciones tecnicas me ayudo muchisimo a elegir entre dos laptops similares. Es justo el tip...
27	5	El soporte tecnico me ayudo a elegir entre dos laptops segun mi uso real.
32	5	La confirmacion de pedido llega de inmediato al correo con todos los detalles.
36	5	Muy conforme con la rapidez para resolver un reclamo por un cargo duplicado.

`reduce_outliers` reasigna los documentos del tópico -1 al tópico más parecido. Con la estrategia "c-tf-idf" compara el c-TF-IDF de cada documento con el de los tópicos. Después hay que actualizar las representaciones con `update_topics` (pasando de nuevo el `vectorizer_model` ; si no, BERTopic usa uno sin *stopwords*).

```
nuevos = topic_model.reduce_outliers(docs, topicos, strategy="c-tf-idf")
print("Documentos en -1 antes:", (np.array(topicos) == -1).sum())
print("Documentos en -1 después:", (np.array(nuevos) == -1).sum())

pd.crosstab(pd.Series(topicos, name="antes"), pd.Series(nuevos,
name="después")).loc[[-1]]
```

Documentos en -1 antes: 26

Documentos en -1 después: 0

después 0 1 2 3 4 5 6 7 8

antes

-1 5 3 1 7 3 2 2 0 3

Comparamos la calificación promedio por tópico antes y después de reasignar:

```
modelo_sin_ruido = crear_modelo()
modelo_sin_ruido.fit(docs, embeddings)
modelo_sin_ruido.update_topics(
    docs, topics=nuevos,
    vectorizer_model=CountVectorizer(preprocessor=normalizar,
stop_words=STOPWORDS_ES),
    representation_model=KeyBERTInspired())

d = df.assign(antes=topicos, despues=nuevos)
pd.DataFrame({
    "comentarios antes": d["antes"].value_counts(),
    "comentarios después": d["despues"].value_counts(),
    "calif. antes": d.groupby("antes")["calificacion"].mean().round(2),
```

```

"calif. después": d.groupby("despues")["calificacion"].mean().round(2),
"palabras (KeyBERT, después)": pd.Series({t: ", ".join(w for w, _ in
modelo_sin_ruido.get_topic(t)[:5])
for t in set(nuevos)}),
}).rename_axis("tópico")

```

2026-10-06 20:45:55,140 – BERTopic – WARNING: Using a custom list of topic assignments may lead to errors if topic reduction techniques are used afterwards. Make sure that manually assigning topics is the last step in the pipeline. Note that topic embeddings will also be created through weightedc-TF-IDF embeddings instead of centroid embeddings.

	comentarios antes	comentarios después	calif. antes	calif. después	palabras (KeyBERT, después)
tópico					
-1	26	NaN	3.96	NaN	NaN
0	34	39.0	1.85	2.23	checkout, plazo, correo, correos, cancele
1	13	16.0	4.08	4.06	compras, tiendas, compradores, comprando, tienda
2	12	13.0	3.75	3.77	virtual, navegar, pagina, experiencia, leer
3	12	19.0	2.58	2.79	compre, compras, tiendas, comprar, compro
4	11	14.0	3.82	3.86	buscador, comparar, comparacion, busca, busquedas
5	10	12.0	3.40	3.33	checkout, registro, credito, facturacion, tarjeta
6	7	9.0	4.29	4.22	tiendas, tienda, recomendaria, comprando, comprar
7	6	6.0	3.17	3.17	app, navegador, pantalla, pantallas, celular
8	4	7.0	5.00	5.00	garantia, excelente, compre, comprar, claro

7.2 Variaciones de min_cluster_size

```

filas = []
for mcs in [4, 6, 10]:
    m = crear_modelo(mcs)
    t, _ = m.fit_transform(docs, embeddings)
    t = np.array(t)
    filas.append({"min_cluster_size": mcs, "tópicos": t.max() + 1, "ruido (-1)": (t
== -1).sum(),
                  "tópicos (top-4 palabras KeyBERT)":
                  " | ".join(", ".join(w for w, _ in m.get_topic(k)[:4]) for k in
range(t.max() + 1))})
pd.DataFrame(filas).set_index("min_cluster_size")

```

	tópicos	ruido (-1)	tópicos (top-4 palabras KeyBERT)
min_cluster_size			
4	9	26	soporte, correo, chat, tardo compra, tienda, puntos, pena sitio, buena, experiencia, general marcas, stock, pr...
6	4	21	producto, tienda, tiendas, productos correo, soporte, tiempo, chat proceso, cuenta, pago, rapido sitio, genera...
10	2	21	buena, producto, tienda, tiendas proceso, correo, soporte, cuenta

7.3 Fusionar tópicos

Si dos tópicos son redundantes, `merge_topics(docs, [[a, b]])` los une. Por ejemplo, los dos más cercanos según el dendrograma:

```
dist = 1 - cosine_similarity(topic_model.topic_embeddings_[1:]) # sin el -1
np.fill_diagonal(dist, np.inf)
a, b = np.unravel_index(dist.argmin(), dist.shape)
print(f"Tópicos más cercanos: {a} y {b}")
for k in (a, b):
    print(f" {k}: {' '.join(w for w, _ in topic_model.get_topic(k)[:6])}")
```

Tópicos más cercanos: 1 y 6

1: compra, tienda, puntos, pena, programa, ultima

6: tecnologia, tienda, general, buena, amplio, preferida

Discusión

- ¿Qué tópicos de BERTopic **no aparecían con LDA**? ¿Cuáles se ven más fáciles de nombrar?
- Observen la calificación promedio por tópico: hay un tópico de quejas con calificación muy baja. Los embeddings no solo agrupan por **tema**, también por **tono** (queja, elogio). ¿Es eso deseable en un análisis de tópicos?
- ¿Qué comentarios quedan como ruido y por qué? ¿Tiene sentido forzar su reasignación?
- ¿Cómo cambia el número de tópicos con `min_cluster_size`?
- KeyBERTInspired frente a MMR: ¿cuál describe mejor cada tópico?

```
# Espacio para experimentar: cambien n_neighbors, min_cluster_size, min_samples,
# diversity de MMR o agreguen ngram_range=(1, 2) al CountVectorizer
```

Sesión de clase 08 · Práctica: tópicos con LLM y variantes de BERTopic

Los LLM cumplen **tres roles** en el modelado de tópicos:

1. **Etiquetar tópicos:** reciben las palabras clave y documentos representativos de cada tópico y devuelven un nombre legible.
2. **Generar tópicos:** leen una muestra de documentos y proponen una lista de tópicos (TopicGPT).
3. **Asignar documentos:** con una lista de tópicos ya definida, clasifican cada documento (zero-shot).

Configuración frecuente: **BERTopic agrupa** de forma barata y reproducible y el **LLM solo nombra** cada tópico, con pocas llamadas (una por tópico, no una por documento).

Usamos un LLM **local**, `Qwen/Qwen2.5-0.5B-Instruct`, que corre en CPU: los comentarios de los clientes no salen de la computadora.

Etapas: 1. Modelo base · 2. Etiquetado con LLM · 3. LLM como juez · 4. BERTopic zero-shot · 5. BERTopic guiado · 6. Tópicos en el tiempo · 7. Tópicos por país · Discusión.

```
import random
import unicodedata
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
from bertopic import BERTopic
from bertopic.representation import KeyBERTInspired, MaximalMarginalRelevance
from hdbscan import HDBSCAN
from sentence_transformers import SentenceTransformer
from sklearn.feature_extraction.text import CountVectorizer
from transformers import pipeline
from transformers.utils import logging as hf_logging
from umap import UMAP

hf_logging.set_verbosity_error()
pd.set_option("display.max_colwidth", 120)
SALIDAS = Path("../salidas")
SALIDAS.mkdir(exist_ok=True)

AZUL, ROJO, GRIS, TINTA = "#2a78d6", "#e34948", "#8a8985", "#52514e"
plt.rcParams.update({
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": GRIS, "axes.labelcolor": TINTA,
    "xtick.color": TINTA, "ytick.color": TINTA,
    "axes.grid": True, "grid.color": "#e6e5e1", "grid.linewidth": 0.8,
    "axes.axisbelow": True, "figure.dpi": 110,
})
```

1. Modelo base

La misma configuración del Ejercicio 2: embeddings guardados en `salidas/embeddings.npy` (si no existen, se calculan), UMAP con semilla fija, HDBSCAN con `min_cluster_size=4`; c-TF-IDF como representación principal y KeyBERTInspired y MMR como representaciones adicionales.

```
df = pd.read_csv("../data/comentarios.csv")
docs = df["texto"].tolist()

st = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
ruta_emb = SALIDAS / "embeddings.npy"
if ruta_emb.exists():
    embeddings = np.load(ruta_emb)
else:
    embeddings = st.encode(docs, show_progress_bar=True)
    np.save(ruta_emb, embeddings)

def normalizar(texto):
    t = unicodedata.normalize("NFKD", texto.lower())
    return "".join(ch for ch in t if not unicodedata.combining(ch))

STOPWORDS_ES = sorted(set("""
a al algo algun alguna algunas alguno algunos ante antes aqui asi aun aunque
bajo bien cada casi como con contra cual cuando de del desde donde dos durante
e el ella ellas ellos en entre era eran es esa esas ese eso esos esta estaba
estaban estan estar este esto estos fue fueron ha habia han hasta hay he la las
le les lo los mas me mi mis mucho muy nada ni no nos o otra otras otro otros
para pero poco por porque que se ser si sin sobre solo su sus tambien tan tanto
te tiene todo todos tu tuve un una uno unos y ya yo vez veces mismo misma
son podria poder bastante
""").split()))

def crear_modelo(**variante):
    return BERTopic(
        embedding_model=st,
        umap_model=UMAP(n_neighbors=10, n_components=5, min_dist=0.0,
                        metric="cosine", random_state=42),
        hdbscan_model=HDBSCAN(min_cluster_size=4, prediction_data=True),
        vectorizer_model=CountVectorizer(preprocessor=normalizar,
        stop_words=STOPWORDS_ES),
        representation_model={"KeyBERT": KeyBERTInspired(),
                            "MMR": MaximalMarginalRelevance(diversity=0.3)},
        **variante,
    )

topic_model = crear_modelo()
temas, _ = topic_model.fit_transform(docs, embeddings)
df["tema"] = temas
topic_model.get_topic_info()[["Tema", "Count", "Representación", "KeyBERT"]]
```

	Topic	Count	Representation	KeyBERT
0	-1	26	[producto, accesorios, soporte, buena, compatibilidad, reconoce, postventa, elegir, laptops, buscador]	[buscador, compatibilidad, recomendados, laptops, busca, servidores, lenta, laptop, rapidez, busqueda]
1	0	34	[soporte, correo, chat, tardó, tiempo, pedido, proceso, días, semana, sistema]	[correo, plazo, correos, checkout, retraso, cancele, reembolso, semanas, mensaje, minutos]
2	1	13	[compra, tienda, puntos, pena, programa, última, envío, realmente, experiencia, compras]	[compras, tiendas, compradores, comprando, tienda, compra, descuento, comprar, mercado, precios]
3	2	12	[sitio, buena, experiencia, general, virtual, modo, oscuro, leer, difíciles, rápidas]	[navegar, experiencia, virtual, página, leer, sitio, difíciles, páginas, personalizada, disponibilidad]
4	3	12	[marcas, stock, producto, disponible, variedad, precio, compras, tiendas, productos, marcado]	[compre, compras, tiendas, comprar, compro, stock, productos, producto, precios, oferta]
5	4	11	[productos, comparar, decidir, busqueda, filtros, anteriores, revisar, tener, fácil, marca]	[comparar, productos, compras, búsquedas, comparación, filtros, comprarlos, busqueda, útil, revisar]
6	5	10	[pago, registro, proceso, cuenta, crear, esperaba, simple, tomo, rápido, tarjeta]	[crédito, checkout, tarjeta, facturación, registro, pago, cuenta, banco, verificación, datos]
7	6	7	[tecnología, tienda, general, buena, amplio, preferida, catálogo, tiendas, volver, diferencia]	[tiendas, tienda, tecnología, recomendaría, electrónica, preferida, comprando, comprar, catálogo, opciones]
8	7	6	[app, móvil, celular, funciona, rápido, incluso, botones, carga, duplicadas, mayoría]	[app, navegador, pantalla, pantallas, celular, botones, push, funciona, notificaciones, navegar]
9	8	4	[políticas, claro, garantía, comprar, escondida, pequeña, letra, financiamiento, explicarme, quedo]	[transparencia, garantía, notificaciones, transparente, claridad, correo, financiamiento, soporte, conforme, satisfe...

2. Etiquetado de tópicos con un LLM local

2.1 Plantilla del prompt

Por cada tópico enviamos sus **documentos representativos** y sus **palabras clave**, y pedimos una etiqueta corta con un formato fijo para poder extraerla del texto generado.

```
PROMPT = """Tengo un tópico con los siguientes documentos:
[DOCUMENTS]
El tópico se describe con estas palabras clave: [KEYWORDS]
```

```
Devuelve una etiqueta corta en español (máximo 6 palabras) con el formato:
tópico: <etiqueta>"""
```

```
SISTEMA = ("Eres un analista de la voz del cliente de una tienda virtual de "
"tecnología. Respondes solo con lo que se te pide.")
```

```
def armar_prompt(k, n_palabras=8):
    documentos = "\n".join(f"- {d}" for d in topic_model.get_representative_docs(k))
    palabras = ", ".join(w for w, _ in topic_model.get_topic(k)[:n_palabras])
    return PROMPT.replace("[DOCUMENTS]", documentos).replace("[KEYWORDS]", palabras)

print(armar_prompt(0))
```

Tengo un tópico con los siguientes documentos:

- Cancele un pedido y el reembolso tardo mas de diez dias habiles en aparecer en mi tarjeta. El soporte solo repetia que estaba en proceso sin dar una fecha concreta.
- El tiempo de espera del chat de soporte fue bastante largo un fin de semana. Entiendo que hay menos personal pero deberian avisar el tiempo estimado de espera.
- Me cobraron el envio dos veces por un error del sistema y tuve que escribir tres correos distintos para que me lo revirtieran. La solucion tardo mas de una semana en llegar.

El tópico se describe con estas palabras clave: soporte, correo, chat, tardo, tiempo, pedido, proceso, dias

Devuelve una etiqueta corta en español (máximo 6 palabras) con el formato: tópico: <etiqueta>

2.2 Cargar el modelo

`Qwen/Qwen2.5-0.5B-Instruct` (≈ 1 GB, se descarga una sola vez a la caché de Hugging Face).

Con `do_sample=False` la generación es **determinista**: la misma entrada produce siempre la misma etiqueta.

```
llm = pipeline("text-generation", model="Qwen/Qwen2.5-0.5B-Instruct",
               dtype=torch.float32, device="cpu")

def preguntar(texto, max_new_tokens=30):
    mensajes = [{"role": "system", "content": SISTEMA},
                {"role": "user", "content": texto}]
    salida = llm(mensajes, max_new_tokens=max_new_tokens, do_sample=False)
    return salida[0]["generated_text"][-1]["content"].strip()

respuesta = preguntar(armar_prompt(0))
print(respuesta)
```

config.json: 100%		659/659 [00:00<00:00, 207kB/s]
model.safetensors: reconstructing file: 100%		988MB / 988MB, 58.2MB/s
model.safetensors: downloading bytes:		854MB, 23.5MB/s
Loading weights: 100%		290/290 [00:00<00:00, 3315.95it/s]
generation_config.json: 100%		242/242 [00:00<00:00, 51.0kB/s]
tokenizer_config.json: 100%		7.30k/7.30k [00:00<00:00, 2.54MB/s]
vocab.json: 100%		2.78M/2.78M [00:00<00:00, 4.98MB/s]
merges.txt: 100%		1.67M/1.67M [00:00<00:00, 18.3MB/s]

tópico: Soporte y Envío

2.3 Etiquetar todos los tópicos

Extraemos el texto que sigue a «tópico:». Con `set_topic_labels` las etiquetas quedan guardadas en el modelo (sin volver a entrenarlo) y las visualizaciones las usan con `custom_labels=True`.

```
def extraer_etiqueta(respuesta):
    linea = respuesta.split("\n")[0]
    if ":" in linea:
        linea = linea.split(":", 1)[1]
    return linea.strip(" .«»\''*").capitalize()

etiquetas = {}
for k in sorted(set(topicos) - {-1}):
    etiquetas[k] = extraer_etiqueta(preguntar(armar_prompt(k)))
etiquetas[-1] = "Ruido (sin tópico)"

topic_model.set_topic_labels(etiquetas)
tabla = topic_model.get_topic_info()[["Topic", "Count", "CustomName",
"Representation"]]
tabla["Representation"] = tabla["Representation"].str[:5].str.join(", ")
tabla.to_csv(SALIDAS / "etiquetas_llm.csv", index=False)
tabla
```

	Topic	Count	CustomName	Representation
0	-1	26	Ruido (sin tópico)	producto, accesorios, soporte, buena, compatibilidad
1	0	34	Soporte y envío	soporte, correo, chat, tardo, tiempo
2	1	13	Compra y puntos	compra, tienda, puntos, pena, programa
3	2	12	Sitio - experiencia - general	sitio, buena, experiencia, general, virtual
4	3	12	Desafíos en el catálogo de marcas	marcas, stock, producto, disponible, variedad
5	4	11	Comparación de productos	productos, comparar, decidir, busqueda, filtros
6	5	10	Pago y cuenta	pago, registro, proceso, cuenta, crear
7	6	7	Tienda de tecnología	tecnologia, tienda, general, buena, amplio
8	7	6	Problemas con la aplicación mobile	app, movil, celular, funciona, rapido
9	8	4	Claro y transparente	politicas, claro, garantia, comprar, escondida

Revisen las etiquetas: ¿alguna **inventa** algo que no está en los documentos? Un modelo de 0.5 mil millones de parámetros es rápido y privado, pero menos preciso que un LLM grande; por eso conviene revisar siempre la salida.

```
fig = topic_model.visualize_barchart(top_n_topics=10, n_words=5, custom_labels=True,
width=260, height=230)
```

```
fig.write_html(SALIDAS / "temas_etiquetados.html", include_plotlyjs="cdn")
fig.show()
```

Topic Word Scores



2.4 Con un LLM externo

BERTopic integra LLM como modelos de representación. Con un proveedor externo el código sería:

```
import openai
from bertopic.representation import OpenAI
```

```
llm = OpenAI(openai.OpenAI(), model="gpt-4o-mini", prompt=PROMPT)
topic_model.update_topics(docs, representation_model=llm)
```

Implica **enviar los comentarios de los clientes a un tercero** y pagar por token. Alternativas locales con la misma interfaz: `LiteLLM(model="ollama/...")` o `LlamaCPP(...)`.

3. LLM como juez: prueba de la palabra intrusa

Una forma de evaluar la **interpretabilidad** de un tópico: se mezclan sus 5 palabras principales con una **intrusa** tomada de otro tópico. Si el tópico es coherente, un evaluador (persona o LLM) debería identificar la intrusa.

```
random.seed(42)
ids = sorted(set(topicos) - {-1})
palabras_de = {k: [w for w, _ in topic_model.get_topic(k)] for k in ids}

filas = []
for k in ids:
    otro = random.choice([j for j in ids if j != k])
    intrusa = next(w for w in palabras_de[otro] if w not in palabras_de[k])
    opciones = palabras_de[k][:5] + [intrusa]
    random.shuffle(opciones)
    pregunta = (f"¿Cuál de estas palabras NO pertenece al mismo tema que las demás? "
                f"{', '.join(opciones)}. Responde solo con la palabra.")
    elegida = normalizar(preguntar(pregunta, max_new_tokens=10)).strip(" .«»\''*")
    filas.append({"tópico": k, "palabras": ", ".join(opciones), "intrusa": intrusa,
                 "elegida por el LLM": elegida, "acierto": elegida == intrusa})

juez = pd.DataFrame(filas)
print(f"Aciertos del LLM: {juez['acierto'].sum()} de {len(juez)}")
juez
```

Aciertos del LLM: 1 de 9

	tópico	palabras	intrusa	elegida por el LLM	acierto
0	0	tardo, tiempo, sitio, correo, chat, soporte	sitio	soporte	False
1	1	pena, tienda, puntos, compra, programa, sitio	sitio	puntos	False
2	2	general, experiencia, virtual, buena, soporte, sitio	soporte	bueno	False
3	3	producto, stock, marcas, disponible, variedad, comparar	comparar	marcas	False
4	4	pago, decidir, filtros, busqueda, comparar, productos	pago	decision	False
5	5	crear, sitio, cuenta, pago, proceso, registro	sitio	cuenta	False
6	6	buena, tienda, tecnologia, amplio, compra, general	compra	amplio	False
7	7	rapido, movil, compra, celular, app, funciona	compra	compra	True
8	8	claro, comprar, garantia, escondida, politicas, marcas	marcas	politicas	False

Hagan la prueba ustedes mismos con la columna **palabras** , sin mirar la respuesta. ¿Aciertan más que el LLM?

Un modelo de 0.5 mil millones de parámetros sirve para **nombrar** tópicos, pero es un **juez** poco confiable: la prueba exige razonar sobre el significado de seis palabras sueltas sin contexto. Para usar un LLM como juez se necesita un modelo más grande, y aun así conviene contrastarlo con una muestra de juicios humanos. Los tópicos donde ni las personas encuentran la intrusa son los menos interpretables.

4. BERTopic zero-shot: asignar documentos a tópicos predefinidos

Con `zeroshot_topic_list` damos una lista de tópicos esperados. BERTopic compara el embedding de cada documento con el de cada nombre de tópico: si la similitud supera `zeroshot_min_similarity`, lo asigna a ese tópico; el resto de documentos se agrupa con el procedimiento normal y puede **descubrir tópicos nuevos**.

```
TOPICOS_ESPERADOS = [
    "Atención y soporte al cliente",
    "Pagos, cobros y reembolsos",
    "Envíos y seguimiento del pedido",
    "Estabilidad del sitio y la app",
    "Cuenta, registro y privacidad",
    "Promociones, cupones y puntos",
]

modelo_zs = crear_modelo(zeroshot_topic_list=TOPICOS_ESPERADOS,
                        zeroshot_min_similarity=0.3)
temas_zs, _ = modelo_zs.fit_transform(docs, embeddings)
modelo_zs.get_topic_info()[["Topic", "Count", "Name", "Representation"]]
```

	Topic	Count	Name	Representation
0	-1	19	-1_proceso_dificiles_intente_esperaba	[proceso, dificiles, intente, esperaba, modo, buscaba, devolucion, oscuro, textos, tomo]
1	0	15	Atención y soporte al cliente	[soporte, buena, atencion, chat, experiencia, garantia, usando, virtual, asistente, rapidas]
2	1	12	Pagos, cobros y reembolsos	[pago, seria, bueno, cuotas, compra, tarjeta, perdi, oferta, opciones, retiro]
3	2	30	Envíos y seguimiento del pedido	[correo, pedido, envio, soporte, tardo, recibi, tiempo, dias, cuenta, confirmar]
4	3	17	Estabilidad del sitio y la app	[sitio, funciona, app, general, comprar, movil, rapido, alta, celular, cargar]
5	4	6	Cuenta, registro y privacidad	[datos, cuenta, registro, proceso, bloqueo, perfil, demasiados, corto, corrigieran, eliminar]
6	5	15	Promociones, cupones y puntos	[puntos, buena, variedad, general, tiendas, tienda, ofertas, pena, programa, marcas]
7	6	11	6_producto_stock_disponible_fotos	[producto, stock, disponible, fotos, compras, grave, compro, cierta, combinarlo, ciertos]
8	7	4	7_similares_tecnologia_activar_facilmente	[similares, tecnologia, activar, facilmente, pueda, necesite, muchisimo, laptop, gusta, invertir]
9	8	6	8_compra_experiencia_comparado_precio	[compra, experiencia, comparado, precio, tiendas, envio, ultimas, caidas, ordenes, normal]

Leamos dos documentos de cada tópico predefinido: ¿están bien asignados?

```
nombres_zs = modelo_zs.get_topic_info().set_index("Topic")["Name"]
(df.assign(topico_zs=temas_zs)
 .query("topico_zs >= 0")
 .groupby("topico_zs").head(2)
 .assign(nombre=lambda d: d["topico_zs"].map(nombres_zs))
 .sort_values("topico_zs")[["nombre", "calificacion", "texto"]])
```

	nombre	calificacion	texto
5	Atención y soporte al cliente	5	Muy buena atencion personalizada cuando pregunte por disponibilidad de un modelo agotado.
6	Atención y soporte al cliente	4	El soporte me oriento bien sobre compatibilidad de accesorios con mi equipo.
42	Pagos, cobros y reembolsos	4	Buena seleccion de opciones de pago, use la tarjeta de credito sin ningun problema. El unico detalle fue la demora d...
29	Pagos, cobros y reembolsos	3	El proceso de pago es estandar, similar a lo que ya conozco de otras tiendas en linea.
3	Envíos y seguimiento del pedido	3	El envio internacional tardo mas dias de los indicados originalmente.
9	Envíos y seguimiento del pedido	3	El registro de reclamos deberia enviar actualizaciones automaticas por correo.
13	Estabilidad del sitio y la app	3	Es una tienda confiable en general, aunque los precios no siempre son los mas bajos del mercado.
11	Estabilidad del sitio y la app	4	El sitio recuerda mis busquedas anteriores y sugiere productos relacionados de forma acertada.
70	Cuenta, registro y privacidad	5	Muy conforme con la transparencia de las politicas de garantia antes de comprar. Todo quedo claro desde el principio...
56	Cuenta, registro y privacidad	2	La factura electronica llego con datos incorrectos y tuve que pedir que la corrigieran.
1	Promociones, cupones y puntos	3	En general cumple lo que promete, ni mejor ni peor que otras tiendas de tecnologia que he probado.
2	Promociones, cupones y puntos	4	Buena tienda en general, seguire comprando aqui mis proximos equipos.
15	6_producto_stock_disponible_fotos	2	El proceso de cambio de producto por otro modelo fue mas lento de lo esperado.
7	6_producto_stock_disponible_fotos	3	El catalogo de marcas es decente aunque hay ciertos modelos que nunca llegan a tener stock.
26	7_similares_tecnologia_activar_facilmente	5	La comparacion de especificaciones tecnicas me ayudo muchisimo a elegir entre dos laptops similares. Es justo el tip...
43	7_similares_tecnologia_activar_facilmente	5	El servicio postventa fue excelente cuando necesite activar la garantia de mi laptop.

	nombre	calificacion	texto
8	8_compra_experiencia_comparado_precio	5	La experiencia de compra en oferta de temporada fue muy fluida, sin caidas del sitio.
10	8_compra_experiencia_comparado_precio	3	La experiencia de compra es correcta, sin contratiempos importantes en mis ultimas ordenes.

5. BERTopic guiado

Con `seed_topic_list` damos **palabras semilla** para orientar el modelo hacia temas esperados. A diferencia del zero-shot, no fija los tópicos: solo acerca los embeddings de los documentos que contienen esas palabras.

```
SEMILLAS = [
    ["chat", "soporte", "whatsapp", "agente", "espera"],
    ["cobro", "tarjeta", "reembolso", "cuotas", "factura"],
    ["envio", "seguimiento", "pedido", "retiro", "entrega"],
]

modelo_guiado = crear_modelo(seed_topic_list=SEMILLAS)
temas_guiado, _ = modelo_guiado.fit_transform(docs, embeddings)
modelo_guiado.get_topic_info()[["Topic", "Count", "Representation"]]
```

	Topic	Count	Representation
0	-1	9	[cupon, oferta, checkout, seguimiento, cuotas, llame, duplicado, vigencia, exigian, expiro]
1	0	68	[producto, buena, tiendas, tienda, productos, compra, busqueda, comprar, experiencia, envio]
2	1	26	[correo, pedido, tardo, proceso, envio, soporte, error, sistema, factura, datos]
3	2	11	[sitio, experiencia, buena, general, modo, dificiles, textos, usando, leer, virtual]
4	3	8	[chat, soporte, tiempo, espera, menos, whatsapp, minutos, respuesta, correo, semana]
5	4	7	[proceso, tarjeta, pago, esperaba, tomo, simple, registro, datos, dias, cuenta]
6	5	6	[app, movil, celular, funciona, rapido, incluso, botones, carga, duplicadas, mayoria]

6. Tópicos en el tiempo

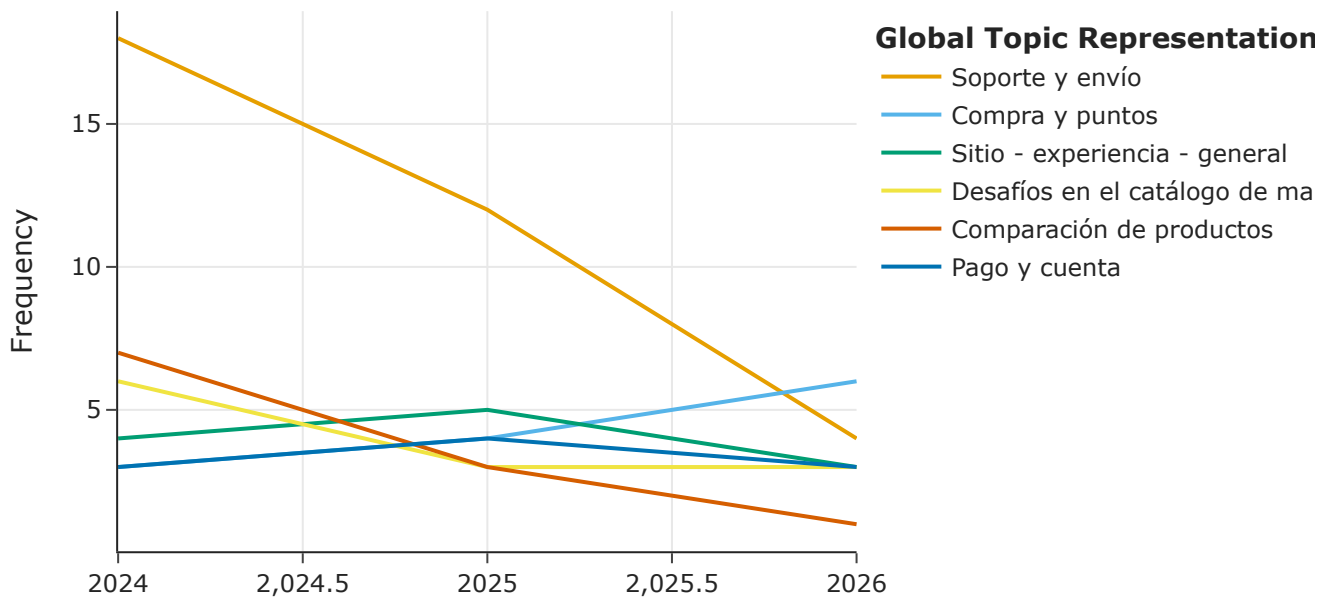
`topics_over_time(docs, marcas_de_tiempo)` recalcula el c-TF-IDF de cada tópico en cada período: muestra cuántos documentos tiene y **con qué palabras** se describe en cada momento. Usamos el año. *2026 incluye solo enero a agosto.*

```
anios = pd.to_datetime(df["fecha"]).dt.year.tolist()
en_el_tiempo = topic_model.topics_over_time(docs, anios)
en_el_tiempo.query("Topic >= 0").head(10)
```

	Topic	Words	Frequency	Timestamp
1	0	chat, soporte, sistema, tardo, tres	18	2024
2	1	puntos, programa, compra, compras, pena	3	2024
3	2	sitio, pocos, confunde, extendida, formas	4	2024
4	3	marcas, producto, variedad, stock, disponible	6	2024
5	4	productos, filtros, revisar, tener, busqueda	7	2024
6	5	pago, rapido, proceso, tomo, esperaba	3	2024
7	6	preferida, tecnologia, general, tienda, buena	1	2024
8	7	app, movil, celular, carga, incluso	3	2024
9	8	etapa, transparente, notificaciones, claras, recibi	1	2024
11	0	pedido, correo, datos, tardo, pedi	12	2025

```
fig = topic_model.visualize_topics_over_time(en_el_tiempo, top_n_topics=6,
custom_labels=True)
fig.write_html(SALIDAS / "temas_en_el_tiempo.html", include_plotlyjs="cdn")
fig.show()
```

Topics over Time



Con tan pocos comentarios por año conviene mirar también el **porcentaje**:

```
df["etiqueta"] = df["topico"].map(etiquetas)
df["anio"] = anios
pct_anio = pd.crosstab(df["etiqueta"], df["anio"],
normalize="columns").mul(100).round(0)
pct_anio.loc[pct_anio.mean(axis=1).sort_values(ascending=False).index]
```

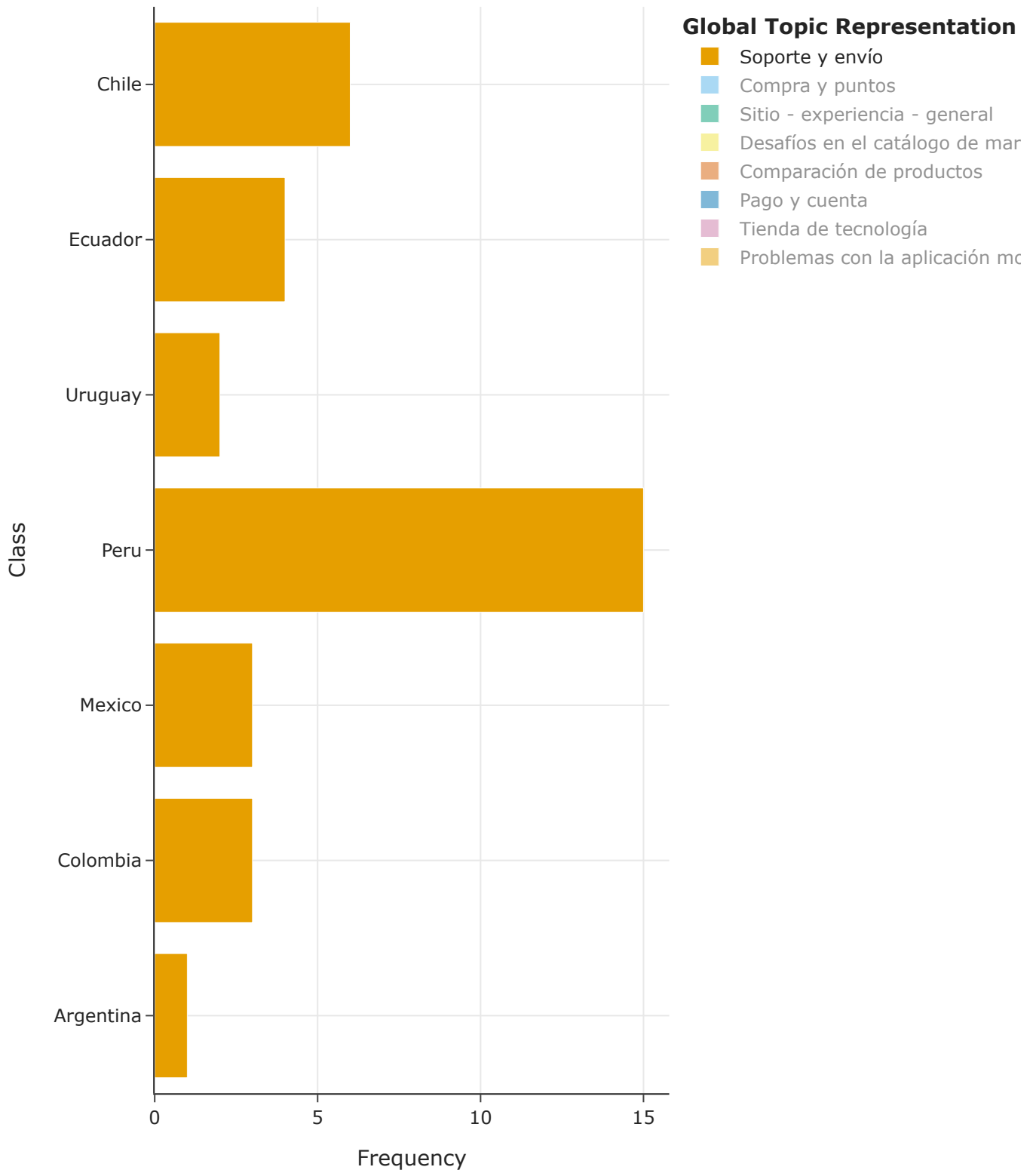
	anio	2024	2025	2026
etiqueta				
Soporte y envío		35.0	23.0	12.0
Ruido (sin tópico)		10.0	25.0	25.0
Compra y puntos		6.0	8.0	19.0
Desafíos en el catálogo de marcas		12.0	6.0	9.0
Sitio - experiencia - general		8.0	10.0	9.0
Comparación de productos		14.0	6.0	3.0
Pago y cuenta		6.0	8.0	9.0
Tienda de tecnología		2.0	6.0	9.0
Problemas con la aplicación mobile		6.0	6.0	0.0
Claro y transparente		2.0	4.0	3.0

7. Tópicos por país

`topics_per_class(docs, classes=países)` compara las palabras y el volumen de cada tópico entre grupos de clientes.

```
por_pais = topic_model.topics_per_class(docs, classes=df["cliente_pais"].tolist())
fig = topic_model.visualize_topics_per_class(por_pais, top_n_topics=8,
custom_labels=True)
fig.write_html(SALIDAS / "topicos_por_pais.html", include_plotlyjs="cdn")
fig.show()
```

Topics per Class



Perú concentra 57 de los 135 comentarios. Comparamos Perú con el resto de países en porcentaje:

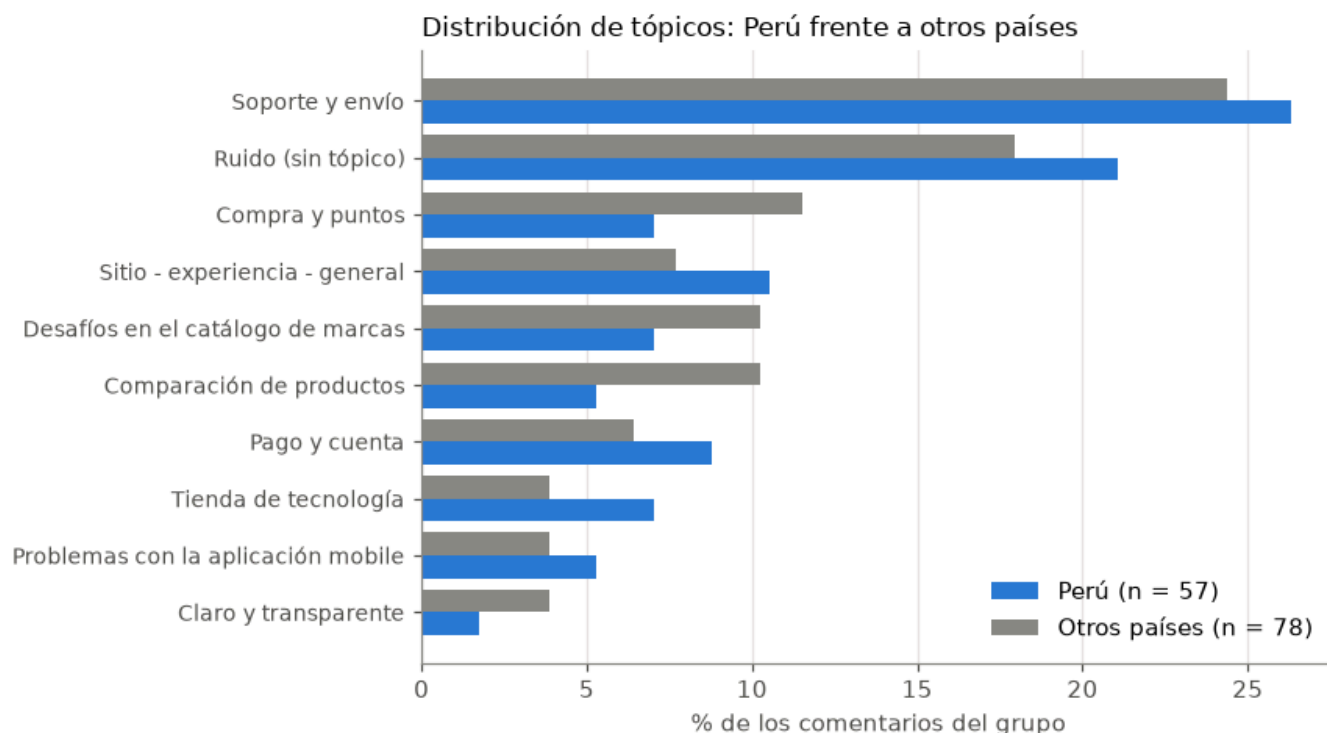
```
grupo = np.where(df["cliente_pais"] == "Peru", "Perú", "Otros países")
pct_pais = pd.crosstab(df["etiqueta"], grupo, normalize="columns").mul(100)

pct_pais = pct_pais.loc[pct_pais.mean(axis=1).sort_values().index]
```

```

fig, ax = plt.subplots(figsize=(8, 4.5))
y = np.arange(len(pct_pais))
ax.barh(y - 0.2, pct_pais["Perú"], height=0.4, color=AZUL, label=f"Perú (n = {(grupo == 'Perú').sum()})")
ax.barh(y + 0.2, pct_pais["Otros países"], height=0.4, color=GRIS,
        label=f"Otros países (n = {(grupo == 'Otros países').sum()})")
ax.set_yticks(y, pct_pais.index, fontsize=9)
ax.set_xlabel("% de los comentarios del grupo")
ax.set_title("Distribución de tópicos: Perú frente a otros países", loc="left",
             fontsize=11)
ax.legend(frameon=False, loc="lower right")
ax.grid(axis="y", visible=False)
plt.tight_layout()
plt.show()

```



Discusión

- ¿Las etiquetas del LLM coinciden con los nombres que ustedes pondrían? ¿Qué cambiarían en el prompt (más documentos, más palabras, ejemplos)?
- Riesgos de usar un LLM: **costo** por documento, resultados que cambian entre ejecuciones (si se muestrea), **tópicos inventados** que no están en los datos y **envío de datos de clientes** a un servicio externo. ¿Cuáles evita un modelo local y cuáles no?
- Zero-shot frente a guiado: ¿cuál usarían si la empresa ya tiene una taxonomía de reclamos? ¿Y si quiere descubrir temas nuevos?
- Con 57 comentarios de Perú y 8 de Uruguay, ¿qué diferencias entre países son confiables?

Espacio para experimentar: cambien el PROMPT, la lista de tópicos esperados, # zeroshot_min_similarity o las palabras semilla

Sesión de clase 08 · Laboratorio: polarización de sentimiento por tópico

Contexto: la tienda quiere saber **qué temas dividen a sus clientes**. Para cada tópico calcularemos el sentimiento neto y la polarización, y con ellos una matriz de priorización.

comentarios.csv → BERTopic → pysentimiento → tabla por tópico → gráficos
135 textos tópico por POS · NEU · NEG n, %POS, %NEG,
barras apiladas y comentario neto,
polarización matriz de priorización

Datos: data/comentarios.csv — 135 comentarios · 96 clientes · 7 países · calificación 1–5★ · fechas de 2024-01 a 2026-08.

Etapas: A. Tópicos · B. Sentimiento · C. Validación · D. Polarización · E. Análisis · Preguntas de reflexión · Reto opcional.

Las tablas (.csv) y gráficos (.html y .png) se guardan en salidas/ .

```
import unicodedata
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from bertopic import BERTopic
from bertopic.representation import KeyBERTInspired, MaximalMarginalRelevance
from hdbscan import HDBSCAN
from pysentimiento import create_analyzer
from sentence_transformers import SentenceTransformer
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import (ConfusionMatrixDisplay, classification_report,
                             cohen_kappa_score, f1_score)
from umap import UMAP

pd.set_option("display.max_colwidth", 110)
SALIDAS = Path("../salidas")
SALIDAS.mkdir(exist_ok=True)

CLASES = ["NEG", "NEU", "POS"]
a3 = lambda s: "NEG" if s <= 2 else ("NEU" if s == 3 else "POS")

# Paleta: azul = positivo, rojo = negativo, gris = neutro/referencias
AZUL, ROJO, GRIS, TINTA = "#2a78d6", "#e34948", "#8a8985", "#52514e"
COLOR = {"POS": AZUL, "NEU": "#c9c8c4", "NEG": ROJO}
plt.rcParams.update({
    "axes.spines.top": False, "axes.spines.right": False,
```

```

"axes.edgecolor": GRIS, "axes.labelcolor": TINTA,
"xtick.color": TINTA, "ytick.color": TINTA,
"axes.grid": True, "grid.color": "#e6e5e1", "grid.linewidth": 0.8,
"axes.axisbelow": True, "figure.dpi": 110,
})

df = pd.read_csv("../data/comentarios.csv")
docs = df["texto"].tolist()
print(df.shape)
df["calificacion"].value_counts().sort_index().rename("comentarios").to_frame().T

```

(135, 7)					
calificacion	1	2	3	4	5
comentarios	17	25	31	30	32

Etapa A — Tópicos

A.1 Ajustar BERTopic

Misma configuración del Ejercicio 2: MiniLM multilingüe, UMAP a 5 d con semilla fija, HDBSCAN con `min_cluster_size=4` y `stopwords` en español solo para describir los tópicos.

```
st = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
ruta_emb = SALIDAS / "embeddings.npy"
if ruta_emb.exists():
    embeddings = np.load(ruta_emb)
else:
    embeddings = st.encode(docs, show_progress_bar=True)
    np.save(ruta_emb, embeddings)

def normalizar(texto):
    t = unicodedata.normalize("NFKD", texto.lower())
    return "".join(ch for ch in t if not unicodedata.combining(ch))

STOPWORDS_ES = sorted(set("""
a al algo algun alguna algunas alguno algunos ante antes aqui asi aun aunque
bajo bien cada casi como con contra cual cuando de del desde donde dos durante
e el ella ellas ellos en entre era eran es esa esas ese eso esos esta estaba
estaban estan estar este esto estos fue fueron ha habia han hasta hay he la las
le les lo los mas me mi mis mucho muy nada ni no nos o otra otras otro otros
para pero poco por porque que se ser si sin sobre solo su sus tambien tan tanto
te tiene todo todos tu tuve un una uno unos y ya yo vez veces mismo misma
son podria poder bastante
""").split()))

def vectorizador():
    return CountVectorizer(preprocessor=normalizar, stop_words=STOPWORDS_ES)

topic_model = BERTopic(
    embedding_model=st,
    umap_model=UMAP(n_neighbors=10, n_components=5, min_dist=0.0,
                    metric="cosine", random_state=42),
```

```

hdbscan_model=HDBSCAN(min_cluster_size=4, prediction_data=True),
vectorizer_model=vectorizador(),
representation_model={"KeyBERT": KeyBERTInspired(),
                      "MMR": MaximalMarginalRelevance(diversity=0.3)},
)
temas_orig, _ = topic_model.fit_transform(docs, embeddings)
temas_orig = np.array(temas_orig)
print(f"{temas_orig.max() + 1} tópicos y {(temas_orig == -1).sum()} comentarios de ruido (-1)")

```

Loading weights: 100%  199/199 [00:00<00:00, 8597.90it/s]

9 tópicos y 26 comentarios de ruido (-1)

A.2 Reducir outliers

Reasignamos los comentarios del tópico -1 al tópico más parecido (estrategia "c-tf-idf") y actualizamos las palabras de cada tópico. Guardamos ambas asignaciones: la original la usaremos en la pregunta de reflexión 3.

```

temas = np.array(topic_model.reduce_outliers(docs, temas_orig, strategy="c-tf-idf"))
topic_model.update_topics(docs, topics=temas.tolist(),
vectorizer_model=vectorizador(),
                      representation_model={"KeyBERT": KeyBERTInspired(),
                      "MMR":
MaximalMarginalRelevance(diversity=0.3)})
df["tema_orig"] = temas_orig
df["tema"] = temas
print("Comentarios en -1 después de reasignar:", (temas == -1).sum())

```

2026-10-06 20:47:41,075 - BERTopic - WARNING: Using a custom list of topic assignments may lead to errors if topic reduction techniques are used afterwards. Make sure that manually assigning topics is the last step in the pipeline. Note that topic embeddings will also be created through weightedc-TF-IDF embeddings instead of centroid embeddings.

Comentarios en -1 después de reasignar: 0

A.3 Nombrar los tópicos

Leemos las palabras clave y tres comentarios de cada tópico y escribimos un nombre corto. Los nombres de `NOMBRES` corresponden a los tópicos que se obtienen con esta configuración; **si sus tópicos salen distintos** (otra versión de las librerías u otro equipo), revisen la tabla y ajusten el diccionario. Un tópico sin nombre se muestra con sus tres palabras principales.

```

info = topic_model.get_topic_info().set_index("Topic")
revision = pd.DataFrame({
    "comentarios": info["Count"],
    "c-TF-IDF": info["Representation"].str[:6].str.join(", "),
    "KeyBERT": info["KeyBERT"].str[:6].str.join(", "),
    "ejemplo": [df.loc[df["tema"] == k, "texto"].iloc[0] for k in info.index],
}).rename_axis("tópico")
revision

```

tópico	comentarios	c-TF-IDF	KeyBERT	ejemplo
0	39	soporte, correo, pedido, chat, tardó, tiempo	checkout, plazo, correo, correos, cancele, mensaje	El envío internacional tardó más días de los indicados originalmente.
1	16	compra, tienda, envío, puntos, experiencia, tiendas	compras, tiendas, compradores, comprando, tienda, compra	La experiencia de compra en oferta de temporada fue muy fluida, sin caídas del sitio.
2	13	sitio, buena, atención, experiencia, general, virtual	virtual, navegar, página, experiencia, leer, difíciles	Muy buena atención personalizada cuando pregunte por disponibilidad de un modelo agotado.
3	19	producto, marcas, stock, variedad, quería, disponible	compre, compras, tiendas, comprar, compro, oferta	El catálogo de marcas es decente aunque hay ciertos modelos que nunca llegan a tener stock.
4	14	productos, búsqueda, decidir, comparar, anteriores, filtros	buscador, comparar, comparación, busca, búsquedas, productos	El sitio recuerda mis búsquedas anteriores y sugiere productos relacionados de forma acertada.
5	12	registro, proceso, pago, rápido, cuenta, tomo	checkout, registro, crédito, facturación, tarjeta, cuenta	El proceso de facturación para empresa fue más simple de lo que esperaba.
6	9	tecnología, general, tienda, buena, seguir, próximos	tiendas, tienda, recomendaría, comprando, comprar, preferida	En general cumple lo que promete, ni mejor ni peor que otras tiendas de tecnología que he probado.
7	6	app, móvil, celular, funciona, rápido, incluso	app, navegador, pantalla, pantallas, celular, botones	La app funciona correctamente la mayoría del tiempo, con alguna que otra pausa al cargar imágenes.
8	7	conforme, garantía, claro, políticas, principio, comprar	garantía, excelente, compre, comprar, claro, principio	Muy satisfecho con la claridad de las políticas de garantía antes de comprar.

```

NOMBRES = {
    0: "Reclamos de soporte, cobros y demoras",
    1: "Experiencia de compra y puntos",
    2: "Diseño y navegación del sitio",
    3: "Catálogo, stock y precios",
    4: "Búsqueda, filtros y comparación",
    5: "Pagos, registro y facturación",
    6: "Recomendación de la tienda",
    7: "App móvil",
    8: "Garantías y políticas claras",
}

def nombre(k):
    if k == -1:
        return "Ruido (sin tópico)"
    return NOMBRES.get(k, f"Tópico {k}: " + ", ".join(w for w, _ in

```

```
topic_model.get_topic(k)[:3]))
```

```
topic_model.set_topic_labels({k: nombre(k) for k in info.index})
df["nombre"] = df["topico"].map(nombre)
revision.assign(nombre=[nombre(k) for k in revision.index])[["nombre", "comentarios",
"c-TF-IDF"]]
```

	nombre	comentarios	c-TF-IDF
tópico			
0	Reclamos de soporte, cobros y demoras	39	soporte, correo, pedido, chat, tardo, tiempo
1	Experiencia de compra y puntos	16	compra, tienda, envio, puntos, experiencia, tiendas
2	Diseño y navegación del sitio	13	sitio, buena, atencion, experiencia, general, virtual
3	Catálogo, stock y precios	19	producto, marcas, stock, variedad, queria, disponible
4	Búsqueda, filtros y comparación	14	productos, busqueda, decidir, comparar, anteriores, filtros
5	Pagos, registro y facturación	12	registro, proceso, pago, rapido, cuenta, tomo
6	Recomendación de la tienda	9	tecnologia, general, tienda, buena, seguire, proximos
7	App móvil	6	app, movil, celular, funciona, rapido, incluso
8	Garantías y políticas claras	7	conforme, garantia, claro, politicas, principio, comprar

Etapas B — Sentimiento con pysentimiento

`pysentimiento` usa **RoBERTuito**, un modelo entrenado con textos en español; devuelve **POS**, **NEU** o **NEG** con sus probabilidades. Corre localmente (la primera vez descarga el modelo).

```
analizador = create_analyzer(task="sentiment", lang="es")
predicciones = analizador.predict(docs)

df["sent"] = [p.output for p in predicciones]
df["confianza"] = [max(p.probas.values()) for p in predicciones]
for c in CLASES:
    df[f"p_{c}"] = [p.probas[c] for p in predicciones]

df[["calificacion", "sent", "confianza", "p_NEG", "p_NEU", "p_POS",
"texto"]].head(8).round(2)
```

config.json: 100%  925/925 [00:00<00:00, 186kB/s]

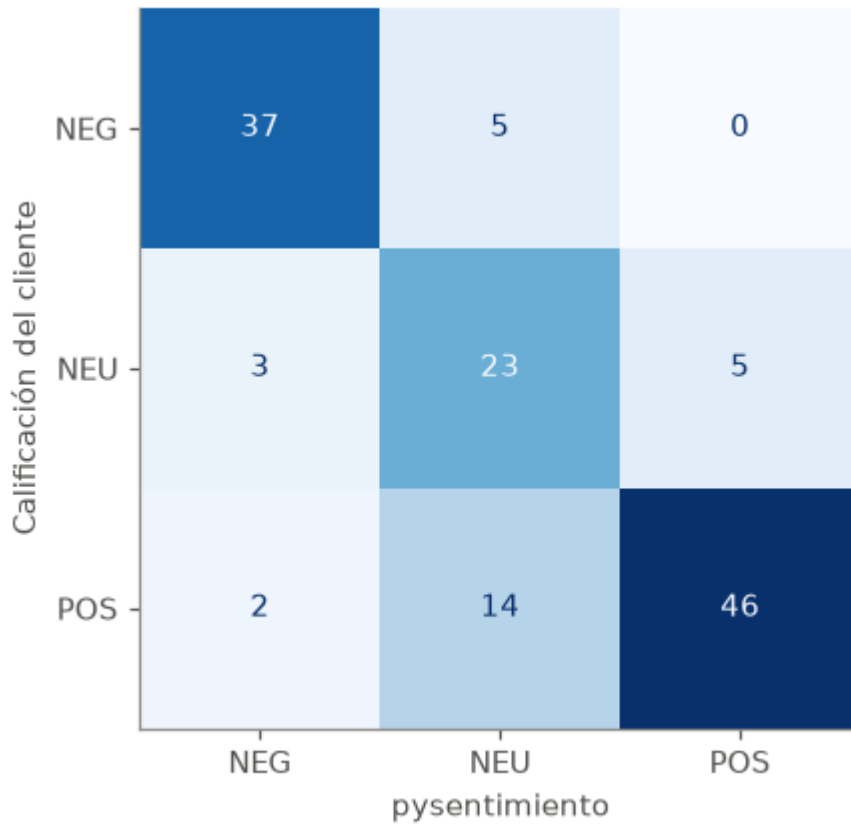
model.safetensors: reconstructing file: 100%  435MB / 435MB, 26.3MB/s

model.safetensors: downloading bytes:  412MB, 15.6MB/s

Loading weights: 100%  201/201 [00:00<00:00, 12911.88it/s]

	precision	recall	f1-score	support
NEG	0.881	0.881	0.881	42
NEU	0.548	0.742	0.630	31
POS	0.902	0.742	0.814	62
accuracy			0.785	135
macro avg	0.777	0.788	0.775	135
weighted avg	0.814	0.785	0.793	135

κ de Cohen = 0.674



¿En qué comentarios falla?

Los errores más graves son los que confunden **positivo con negativo**. Los ordenamos por la confianza del modelo:

```
grave = ((df["y"] == "POS") & (df["sent"] == "NEG")) | ((df["y"] == "NEG") &
(df["sent"] == "POS"))
print(f"Errores graves (POS ↔ NEG): {grave.sum()}")
errores = df[df["y"] != df["sent"]].assign(grave=grave)
errores.sort_values(["grave", "confianza"], ascending=False).head(10)[
["calificacion", "y", "sent", "confianza", "texto"]].round(2)
```

Errores graves (POS ↔ NEG): 2

	calificacion	y	sent	confianza	texto
132	4	POS	NEG	0.66	Me parece justo el precio de envio comparado con otras tiendas de electronica.
47	4	POS	NEG	0.51	El checkout como invitado es rapido si no quieres crear una cuenta.
76	3	NEU	NEG	0.90	El diseno del sitio en modo oscuro tiene algunos textos dificiles de leer.
10	3	NEU	POS	0.89	La experiencia de compra es correcta, sin contratiempos importantes en mis ultimas ordenes.
72	4	POS	NEU	0.86	La opcion de retiro en tienda me ahorro el costo de envio en mi ultima compra.
32	5	POS	NEU	0.86	La confirmacion de pedido llega de inmediato al correo con todos los detalles.
27	5	POS	NEU	0.86	El soporte tecnico me ayudo a elegir entre dos laptops segun mi uso real.
112	3	NEU	NEG	0.82	El sitio se cayo un momento durante una oferta especial, tuve que reintentar la compra.
24	3	NEU	NEG	0.81	La seccion de preguntas frecuentes no cubre todos los casos que tuve.
79	4	POS	NEU	0.78	Es facil guardar productos en una lista de deseos para comprarlos despues.

Revisen los errores: ¿son fallas del modelo, comentarios **mixtos** («buena experiencia, aunque...») o calificaciones que no coinciden con el texto?

Etapas D — Sentimiento neto y polarización por tópico

- **Sentimiento neto** = %POS – %NEG, de –1 (todos negativos) a +1 (todos positivos).
- **Polarización** = $2 \times \min(\%POS, \%NEG)$, de 0 (opiniones homogéneas) a 1 (clientes divididos en dos mitades).

El mismo neto puede esconder lecturas distintas: 44 % POS / 44 % NEG y 10 % POS / 10 % NEG tienen ambos neto 0, pero el primero tiene polarización 0.88 (clientes divididos) y el segundo 0.20 (clientes indiferentes).

```
def tabla_por_topico(datos, col_topico="nombre", col_sent="sent"):
    t = pd.crosstab(datos[col_topico], datos[col_sent], normalize="index").reindex(
        columns=CLASES, fill_value=0)
    t.columns = [f"%{c}" for c in CLASES]
    t.insert(0, "n", datos[col_topico].value_counts())
    t["neto"] = t["%POS"] - t["%NEG"]
    t["polarizacion"] = 2 * np.minimum(t["%POS"], t["%NEG"])
    t["calif. promedio"] = datos.groupby(col_topico)["calificacion"].mean()
    return t.sort_values("polarizacion", ascending=False)
```

```
por_topico = tabla_por_topico(df)
```

```
por_topico.round(2).to_csv(SALIDAS / "polarizacion_por_topico.csv")
por_topico.round(2)
```

	n	%NEG	%NEU	%POS	neto	polarizacion	calif. promedio
nombre							
Catálogo, stock y precios	19	0.53	0.21	0.26	-0.26	0.53	2.79
App móvil	6	0.17	0.67	0.17	0.00	0.33	3.17
Pagos, registro y facturación	12	0.17	0.58	0.25	0.08	0.33	3.33
Diseño y navegación del sitio	13	0.15	0.38	0.46	0.31	0.31	3.77
Reclamos de soporte, cobros y demoras	39	0.64	0.28	0.08	-0.56	0.15	2.23
Búsqueda, filtros y comparación	14	0.07	0.21	0.71	0.64	0.14	3.86
Experiencia de compra y puntos	16	0.06	0.31	0.62	0.56	0.12	4.06
Garantías y políticas claras	7	0.00	0.14	0.86	0.86	0.00	5.00
Recomendación de la tienda	9	0.00	0.22	0.78	0.78	0.00	4.22

Calculamos lo mismo con las **estrellas** en lugar del modelo, para comparar:

```
por_topico_estrellas = tabla_por_topico(df, col_sent="y")
pd.DataFrame({
    "neto (modelo)": por_topico["neto"],
    "neto (estrellas)": por_topico_estrellas["neto"],
    "polarización (modelo)": por_topico["polarizacion"],
    "polarización (estrellas)": por_topico_estrellas["polarizacion"],
}).round(2)
```

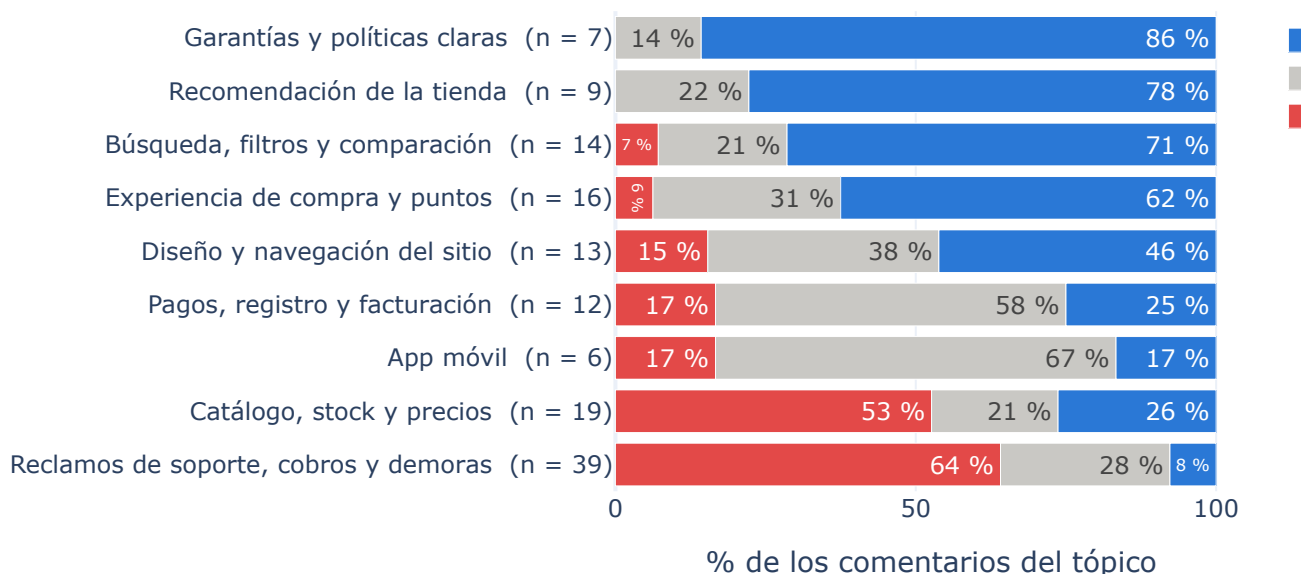
	neto (modelo)	neto (estrellas)	polarización (modelo)	polarización (estrellas)
nombre				
App móvil	0.00	0.00	0.33	0.67
Búsqueda, filtros y comparación	0.64	0.71	0.14	0.14
Catálogo, stock y precios	-0.26	-0.21	0.53	0.53
Diseño y navegación del sitio	0.31	0.46	0.31	0.15
Experiencia de compra y puntos	0.56	0.62	0.12	0.12
Garantías y políticas claras	0.86	1.00	0.00	0.00
Pagos, registro y facturación	0.08	0.33	0.33	0.17
Reclamos de soporte, cobros y demoras	-0.56	-0.51	0.15	0.36
Recomendación de la tienda	0.78	0.78	0.00	0.00

Etapas E — Análisis

E.1 Barras apiladas por tópico

```
orden = por_topico.sort_values("neto").index
fig = go.Figure()
for c in CLASES:
    fig.add_bar(y=orden, x=por_topico.loc[orden, f"%{c}"] * 100, name=c,
orientation="h",
                marker_color=COLOR[c],
                text=(por_topico.loc[orden, f"%{c}"] *
100).round(0).astype(int).astype(str) + " %",
                textposition="inside")
fig.update_layout(
    barmode="stack", template="plotly_white", height=420, width=900,
    title="Sentimiento por tópico (pysentimiento), ordenado por sentimiento neto",
    xaxis_title="% de los comentarios del tópico", legend_title_text="",
    yaxis={"ticktext": [f"{t} (n = {por_topico.loc[t, 'n']})" for t in orden],
"tickvals": list(orden)})
fig.write_html(SALIDAS / "sentimiento_por_topico.html", include_plotlyjs="cdn")
fig.show()
```

Sentimiento por tópico (pysentimiento), ordenado por sentimiento neto



E.2 Matriz de priorización: volumen y sentimiento

Cada tópico se ubica por su **volumen** (eje x) y su **sentimiento neto** (eje y); el tamaño del punto es la polarización. Los cortes son la mediana del volumen y el neto 0:

	Pocos comentarios	Muchos comentarios
Neto positivo	Mantener	Fortalezas

```

corte_n = por_topico["n"].median()

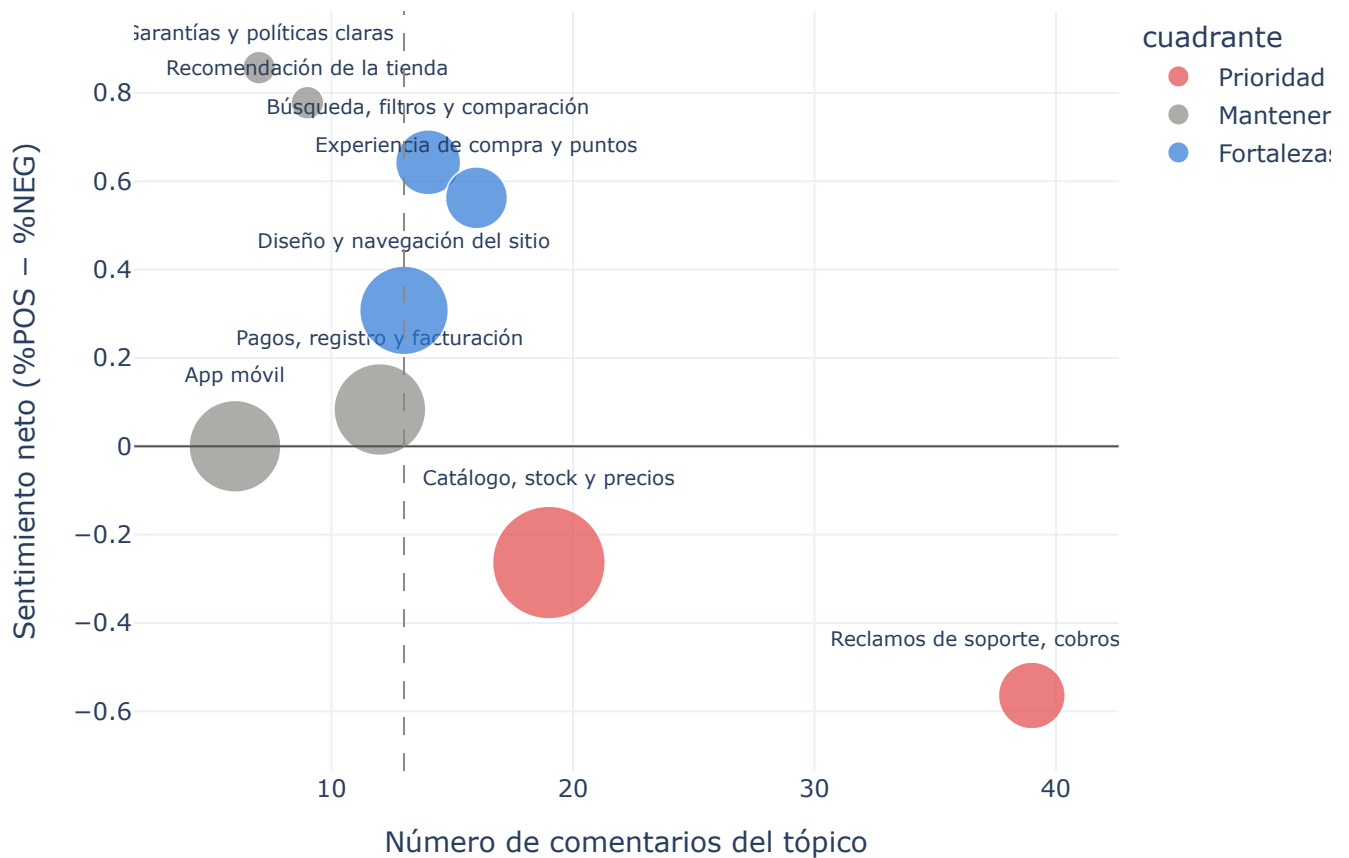
def cuadrante(fila):
    if fila["neto"] < 0:
        return "Prioridad alta" if fila["n"] >= corte_n else "Vigilar"
    return "Fortalezas" if fila["n"] >= corte_n else "Mantener"

matriz = por_topico.assign(cuadrante=por_topico.apply(cuadrante,
axis=1)).reset_index(
    names="tópico")
matriz.round(2).to_csv(SALIDAS / "matriz_priorizacion.csv", index=False)

fig = px.scatter(matriz, x="n", y="neto", size=matriz["polarizacion"] + 0.05,
text="tópico",
                    color="cuadrante", size_max=40, template="plotly_white",
                    color_discrete_map={"Prioridad alta": ROJO, "Vigilar": "#e0a030",
                    "Fortalezas": AZUL, "Mantener": GRIS},
                    hover_data={"polarizacion": ":.2f", "calif. promedio": ":.1f"})
fig.add_hline(y=0, line_color=TINTA, line_width=1)
fig.add_vline(x=corte_n, line_color=GRIS, line_dash="dash", line_width=1)
fig.update_traces(textposition="top center", textfont_size=10)
fig.update_layout(height=520, width=900,
                    title="Matriz de priorización (tamaño = polarización)",
                    xaxis_title="Número de comentarios del tópico",
                    yaxis_title="Sentimiento neto (%POS - %NEG)")
fig.write_html(SALIDAS / "matriz_priorizacion.html", include_plotlyjs="cdn")
fig.show()

```

Matriz de priorización (tamaño = polarización)



```
matriz.sort_values(["cuadrante", "neto"])[["cuadrante", "tópico", "n", "neto",
"polarizacion"]].round(2)
```

	cuadrante	tópico	n	neto	polarizacion
3	Fortalezas	Diseño y navegación del sitio	13	0.31	0.31
6	Fortalezas	Experiencia de compra y puntos	16	0.56	0.12
5	Fortalezas	Búsqueda, filtros y comparación	14	0.64	0.14
1	Mantener	App móvil	6	0.00	0.33
2	Mantener	Pagos, registro y facturación	12	0.08	0.33
8	Mantener	Recomendación de la tienda	9	0.78	0.00
7	Mantener	Garantías y políticas claras	7	0.86	0.00
4	Prioridad alta	Reclamos de soporte, cobros y demoras	39	-0.56	0.15
0	Prioridad alta	Catálogo, stock y precios	19	-0.26	0.53

E.3 Comentarios del tópico más polarizado

Para recomendar una acción hay que leer **qué dicen** los clientes en cada extremo.

```
mas_polarizado = por_topico["polarizacion"].idxmax()
print(f"Tópico más polarizado: {mas_polarizado}")
```

```
sub = df[df["nombre"] == mas_polarizado]
pd.concat([sub[sub["sent"] == "POS"].nlargest(3, "p_POS"),
          sub[sub["sent"] == "NEG"].nlargest(3, "p_NEG")])[["sent", "calificacion",
"texto"]]
```

Tópico más polarizado: Catálogo, stock y precios

	sent	calificacion	texto
84	POS	4	Buena seleccion de accesorios recomendados junto al producto principal.
123	POS	5	Excelente variedad de marcas, encuentre opciones que no tenia en otras tiendas.
49	POS	3	La variedad de categorias es buena aunque siento que podria ampliarse un poco mas en accesorios.
65	NEG	2	Las recomendaciones de productos no tienen mucho sentido con lo que realmente compro, parecen genericas.
121	NEG	2	El precio de un producto cambio de un dia para otro sin ninguna explicacion clara. Senti que perdi la oport...
114	NEG	2	La pagina se puso lenta durante una oferta de temporada y perdi el producto que queria comprar. Deberian r...

Preguntas de reflexión

1. ¿Coincide el sentimiento del modelo con la calificación? ¿En qué comentarios falla?

Ver la Etapa C: matriz de confusión, macro-F1, κ de Cohen y la lista de errores graves.

2. ¿Qué tópico es el más polarizado y qué acción recomendarían a la tienda?

Ver la Etapa E.3.

3. ¿Cómo cambian los resultados al reasignar los comentarios del tópico -1?

Repetimos la tabla con la asignación **original** (con el ruido aparte) y la comparamos con la reasignada:

```
df["nombre_orig"] = df["topico_orig"].map(nombre)
por_topico_orig = tabla_por_topico(df, col_topico="nombre_orig")

comparacion = pd.DataFrame({
    "n (con -1)": por_topico_orig["n"],
    "n (reasignado)": por_topico["n"],
    "neto (con -1)": por_topico_orig["neto"],
    "neto (reasignado)": por_topico["neto"],
    "polarización (con -1)": por_topico_orig["polarizacion"],
    "polarización (reasignado)": por_topico["polarizacion"],
}).round(2)
comparacion.to_csv(SALIDAS / "comparacion_reasignacion.csv")
comparacion
```

	n (con -1)	n (reasignado)	neto (con -1)	neto (reasignado)	polarización (con -1)	polarización (reasignado)
App móvil	6	6.0	0.00	0.00	0.33	0.33
Búsqueda, filtros y comparación	11	14.0	0.73	0.64	0.00	0.14
Catálogo, stock y precios	12	19.0	-0.50	-0.26	0.33	0.53
Diseño y navegación del sitio	12	13.0	0.33	0.31	0.33	0.31
Experiencia de compra y puntos	13	16.0	0.69	0.56	0.00	0.12
Garantías y políticas claras	4	7.0	0.75	0.86	0.00	0.00
Pagos, registro y facturación	10	12.0	0.10	0.08	0.40	0.33
Reclamos de soporte, cobros y demoras	34	39.0	-0.71	-0.56	0.06	0.15
Recomendación de la tienda	7	9.0	0.86	0.78	0.00	0.00
Ruido (sin tópico)	26	NaN	0.31	NaN	0.31	NaN

¿Qué tópicos cambian más? ¿Los comentarios de ruido tenían un sentimiento distinto del resto?

```
df.groupby(df["topico_orig"] == -1)
["sent"].value_counts(normalize=True).unstack().round(2).rename(
    index={True: "ruido (-1)", False: "con tópico"})
```

sent	NEG	NEU	POS
topico_orig			
con tópico	0.35	0.29	0.36
ruido (-1)	0.15	0.38	0.46

Reto opcional: polarización en Perú frente a otros países

`topics_per_class` compara las palabras de cada tópico por grupo; aquí calculamos además el sentimiento neto y la polarización por tópico y grupo. Con pocos comentarios por celda, mostrar siempre **n**.

```
df["grupo"] = np.where(df["cliente_pais"] == "Peru", "Perú", "Otros países")

filas = []
for (t, g), sub in df.groupby(["nombre", "grupo"]):
    pos, neg = (sub["sent"] == "POS").mean(), (sub["sent"] == "NEG").mean()
    filas.append({"tópico": t, "grupo": g, "n": len(sub), "neto": pos - neg,
```

```

        "polarizacion": 2 * min(pos, neg)})
reto = pd.DataFrame(filas).pivot(index="tópico", columns="grupo",
                                values=["n", "neto", "polarizacion"])
reto.round(2)

```

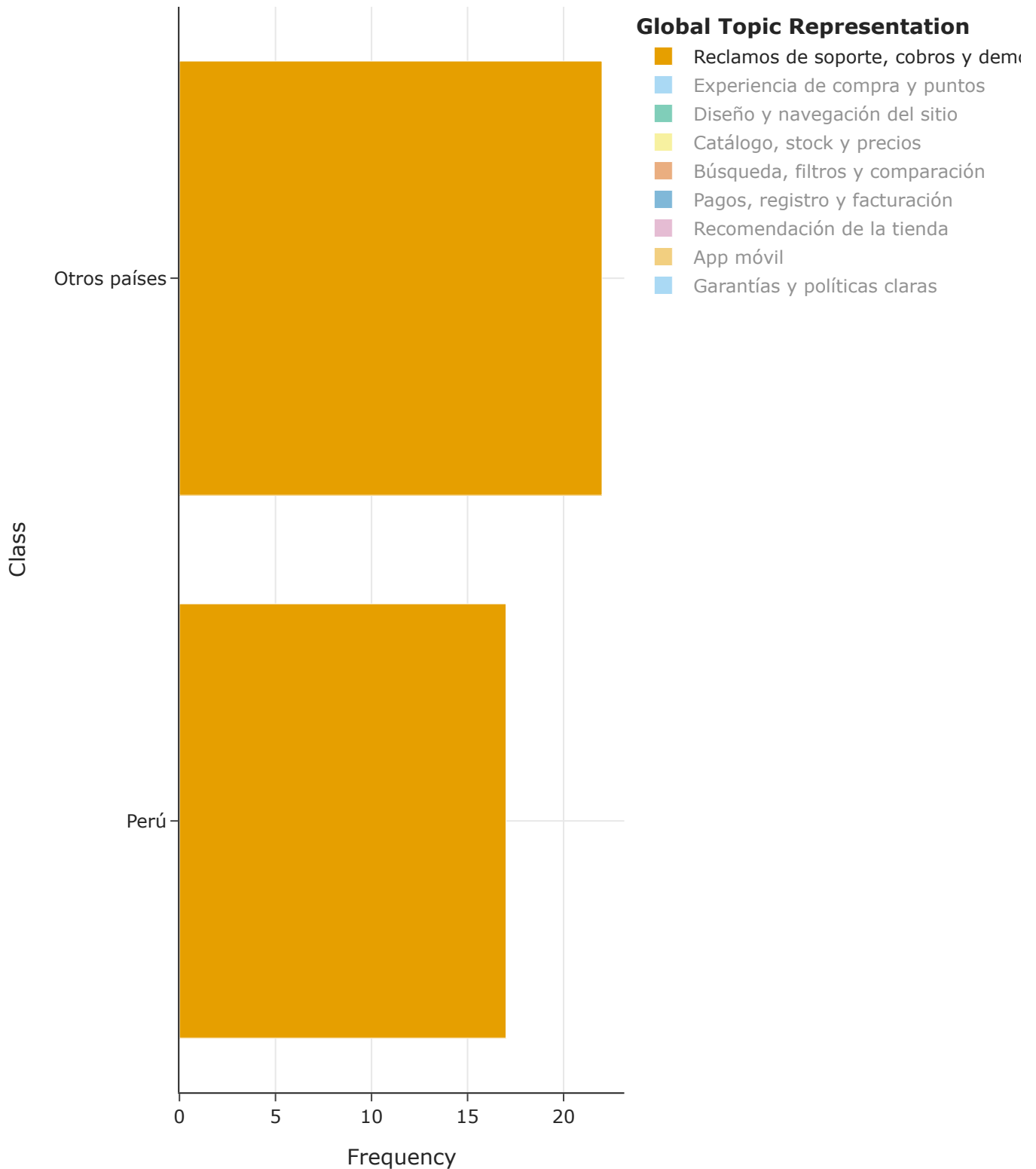
tópico	grupo	n		neto		polarizacion	
		Otros países	Perú	Otros países	Perú	Otros países	Perú
	App móvil	3.0	3.0	-0.33	0.33	0.00	0.00
	Búsqueda, filtros y comparación	9.0	5.0	0.67	0.60	0.00	0.40
	Catálogo, stock y precios	13.0	6.0	-0.46	0.17	0.31	0.67
	Diseño y navegación del sitio	7.0	6.0	0.29	0.33	0.29	0.33
	Experiencia de compra y puntos	10.0	6.0	0.70	0.33	0.00	0.33
	Garantías y políticas claras	5.0	2.0	0.80	1.00	0.00	0.00
	Pagos, registro y facturación	5.0	7.0	-0.20	0.29	0.00	0.29
	Reclamos de soporte, cobros y demoras	22.0	17.0	-0.50	-0.65	0.18	0.12
	Recomendación de la tienda	4.0	5.0	0.75	0.80	0.00	0.00

```

por_clase = topic_model.topics_per_class(docs, classes=df["grupo"].tolist())
fig = topic_model.visualize_topics_per_class(por_clase, custom_labels=True)
fig.write_html(SALIDAS / "temas_por_grupo.html", include_plotlyjs="cdn")
fig.show()

```

Topics per Class



Entregable

- Este notebook ejecutado de principio a fin.
- `salidas/polarizacion_por_topico.csv` y `salidas/matriz_priorizacion.csv`.

- `salidas/sentimiento_por_topico.html` y `salidas/matriz_priorizacion.html`.
- Respuestas a las tres preguntas de reflexión (en la celda siguiente).

Respuestas del grupo

- 1.
- 2.
- 3.

Sesión de clase 08 · Reportes cruzados: tópicos, sentimiento, clientes y ubicación

Los tópicos y el sentimiento se vuelven útiles para el negocio cuando se **cruzan** con otras tablas: ¿de qué se quejan en cada país?, ¿qué categorías de producto generan reclamos de entrega?, ¿qué clientes valiosos están insatisfechos y con qué tema?

Datos (data/):

- `comentarios.csv` — 135 comentarios sobre la tienda (con país y ciudad).
- `resenas_entrega.csv` — 110 reseñas de entrega de un producto comprado.
- `productos.csv` — 90 productos con categoría y precio.
- `clientes.csv` — 120 clientes de 7 países (**datos personales**: DNI, nombre, correo).

Etapas: A. Unir y seudonimizar · B. Tópicos en 245 textos · C. Sentimiento · D. Por ubicación · E. Por categoría de producto · F. Por cliente · G. En el tiempo · Hallazgos.

Regla de este notebook: con 245 textos muchas combinaciones tienen 2 o 3 casos. Toda cifra se muestra con su **n**, y no se calculan indicadores para grupos con menos de `N_MIN` textos.

```
import hashlib
import unicodedata
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from bertopic import BERTopic
from hdbscan import HDBSCAN
from pysentimiento import create_analyzer
from sentence_transformers import SentenceTransformer
from sklearn.feature_extraction.text import CountVectorizer
from umap import UMAP

pd.set_option("display.max_colwidth", 100)
SALIDAS = Path("../salidas")
SALIDAS.mkdir(exist_ok=True)
N_MIN = 5

AZUL, ROJO, GRIS, TINTA = "#2a78d6", "#e34948", "#8a8985", "#52514e"
plt.rcParams.update({
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": GRIS, "axes.labelcolor": TINTA,
    "xtick.color": TINTA, "ytick.color": TINTA,
    "axes.grid": True, "grid.color": "#e6e5e1", "grid.linewidth": 0.8,
    "axes.axisbelow": True, "figure.dpi": 110,
})
```

Etapas A — Unir y seudonimizar

A.1 Seudonimizar `clientes.csv`

Reemplazamos DNI, nombre y correo por un **código**: un *hash* SHA-256 del DNI con una **sal** secreta. El mismo cliente siempre recibe el mismo código, pero no se puede recuperar el DNI a partir de él (Ley N.º 29733 de Protección de Datos Personales). `cliente_id` es una clave interna de la tienda: la conservamos para unir tablas.

```
SAL = "mineria-web-s08" # en producción: fuera del código (variable de entorno)

clientes = pd.read_csv("../data/clientes.csv")
clientes["codigo"] = ["C-" + hashlib.sha256(f"{SAL}{dni}".encode()).hexdigest()[:8]
                      for dni in clientes["dni"]]
clientes = clientes.drop(columns=["dni", "nombre", "apellidos", "email"])
clientes.head()
```

	id	ciudad	pais	codigo
0	1	Lima	Peru	C-10d2959b
1	2	Lima	Peru	C-acefa649
2	3	Lima	Peru	C-27baaf45
3	4	Arequipa	Peru	C-0d5215e6
4	5	Arequipa	Peru	C-7ae40488

A.2 Un solo corpus de 245 textos

`resenas_entrega.csv` no tiene país ni ciudad, y `comentarios.csv` no tiene producto. Con dos **JOIN** cada texto hereda:

- de `clientes.csv` (por `cliente_id`): código, país y ciudad;
- de `productos.csv` (por `producto_id`, solo reseñas): categoría y precio.

`how="left"` conserva todos los textos aunque no encuentren pareja: así detectamos claves huérfanas.

```
c = pd.read_csv("../data/comentarios.csv")
r = pd.read_csv("../data/resenas_entrega.csv")
productos = pd.read_csv("../data/productos.csv")

textos = pd.concat([
    c[["cliente_id", "calificacion", "fecha", "texto"]].assign(fuente="comentario"),
    r[["cliente_id", "producto_id", "calificacion", "fecha",
      "texto"]].assign(fuente="reseña"),
], ignore_index=True)

textos = (textos
          .merge(clientes[["id", "codigo", "pais", "ciudad"]], left_on="cliente_id",
                right_on="id", how="left").drop(columns="id")
          .merge(productos[["id", "categoria", "precio"]], left_on="producto_id",
                right_on="id", how="left").drop(columns="id"))
textos["fecha"] = pd.to_datetime(textos["fecha"])
```

```
print("Textos:", len(textos), " sin país:", textos["pais"].isna().sum(),  
      " reseñas sin categoría:", textos.loc[textos["fuente"] == "reseña",  
      "categoria"].isna().sum())  
textos.sample(5, random_state=1)
```

Textos: 245 sin país: 0 reseñas sin categoría: 0

cliente_id	calificacion	fecha	texto	fuelle	producto_id	codigo	pais	c
67	14	4	2025-04-26 Buena variedad de opciones de envio, elegi la mas economica sin problema.	comentario	NaN	C-d1e65463	Peru	
244	84	5	2024-01-07 Mi Acer Aspire 5 llego en dos dias habiles, muy bien empacada y sin ningun golpe.	reseña	18.0	C-584205f3	Argentina	F
207	85	3	2026-06-01 El robot aspiradora funciona bien pero el filtro se ensucia mas rapido de lo que esperaba.	reseña	73.0	C-b872a31e	Argentina	Me
221	29	3	2024-03-14 La camara instantanea llego a tiempo, el empaque estaba en buen estado aunque algo genérico para...	reseña	69.0	C-cabbdcbd	Peru	
90	58	2	2024-11-17 Las fotos de un producto no coincidían del todo con el color que llego. No es un problema grave ...	comentario	NaN	C-eb6f7feb	Colombia	M

```
pd.crosstab(textos["pais"], textos["fuente"], margins=True, margins_name="Total")
```

fuelle	comentario	reseña	Total
pais			
Argentina	15	18	33
Chile	18	9	27
Colombia	15	8	23
Ecuador	13	12	25
Mexico	9	10	19
Peru	57	46	103
Uruguay	8	7	15
Total	135	110	245

Etapla B — Tópicos en los 245 textos

La misma configuración de los ejercicios, ahora con más textos: `min_cluster_size=5`. Reasignamos el ruido (–1) para que todos los textos entren en los cruces. Cada tópiclo se nombra **automáticamente** con sus tres palabras principales; pueden reemplazarlas por un nombre escrito en `NOMBRES`.

```
docs = textos["texto"].tolist()
st = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
embeddings = st.encode(docs, show_progress_bar=True)

def normalizar(texto):
    t = unicodedata.normalize("NFKD", texto.lower())
    return "".join(ch for ch in t if not unicodedata.combining(ch))

STOPWORDS_ES = sorted(set("""
a al algo algun alguna algunas alguno algunos ante antes aqui asi aun aunque
bajo bien cada casi como con contra cual cuando de del desde donde dos durante
e el ella ellas ellos en entre era eran es esa esas ese eso esos esta estaba
estaban estan estar este esto estos fue fueron ha habia han hasta hay he la las
le les lo los mas me mi mis mucho muy nada ni no nos o otra otras otro otros
para pero poco por porque que se ser si sin sobre solo su sus tambien tan tanto
te tiene todo todos tu tuve un una uno unos y ya yo vez veces mismo misma
son podria poder bastante
""").split()))

def vectorizador():
    return CountVectorizer(preprocessor=normalizar, stop_words=STOPWORDS_ES)

topic_model = BERTopic(
    embedding_model=st,
    umap_model=UMAP(n_neighbors=10, n_components=5, min_dist=0.0,
                    metric="cosine", random_state=42),
    hdbscan_model=HDBSCAN(min_cluster_size=5, prediction_data=True),
    vectorizer_model=vectorizador(),
```

```

)
temas, _ = topic_model.fit_transform(docs, embeddings)
temas = topic_model.reduce_outliers(docs, temas, strategy="c-tf-idf")
topic_model.update_topics(docs, topics=temas, vectorizer_model=vectorizador())

NOMBRES = {} # p. ej., {0: "Entregas tardías"}

def nombre(k):
    return NOMBRES.get(k, f"{k} · " + ", ".join(w for w, _ in
topic_model.get_topic(k)[:3]))

textos["topico"] = [nombre(k) for k in temas]
print(textos["topico"].nunique(), "tópicos")

```

Loading weights: 100%  199/199 [00:00<00:00, 14311.59it/s]

Batches: 100%  8/8 [00:00<00:00, 27.85it/s]

2026-10-06 20:49:11,494 - BERTopic - WARNING: Using a custom list of topic assignments may lead to errors if topic reduction techniques are used afterwards. Make sure that manually assigning topics is the last step in the pipeline. Note that topic embeddings will also be created through weightedc-TF-IDF embeddings instead of centroid embeddings.

9 tópicos

B.1 ¿De qué fuente viene cada tópico?

Los comentarios hablan de la **tienda** (sitio, soporte, pagos) y las reseñas de la **entrega y el producto**. Un tópico que mezcla ambas fuentes es un tema transversal.

```

por_fuente = pd.crosstab(textos["topico"], textos["fuente"])
por_fuente["% reseñas"] = (100 * por_fuente["reseña"] /
por_fuente.sum(axis=1)).round(0)
por_fuente.sort_values("% reseñas")

```

fuente	comentario	reseña	% reseñas
topico			
4 · stock, producto, avisaron	14	0	0.0
7 · sitio, pagina, general	10	0	0.0
1 · buena, tienda, experiencia	48	2	4.0
0 · soporte, correo, proceso	50	19	28.0
3 · pantalla, funciona, app	6	18	75.0
6 · galaxy, s24, z50	3	13	81.0
5 · llego, ningun, acer	3	19	86.0
2 · caja, cable, pedido	1	29	97.0
8 · xiaomi, hervidor, tv	0	10	100.0

Etapa C — Sentimiento

`pysentimiento` (RoBERTuito, español) clasifica cada texto en POS, NEU o NEG. Con eso definimos, para cualquier grupo:

- **neto** = %POS – %NEG (de -1 a +1);
- **polarización** = $2 \times \min(\%POS, \%NEG)$ (de 0 a 1).

```
analizador = create_analyzer(task="sentiment", lang="es")
textos["sent"] = [p.output for p in analizador.predict(docs)]

def indicadores(grupo):
    pos, neg = (grupo["sent"] == "POS").mean(), (grupo["sent"] == "NEG").mean()
    return pd.Series({"n": len(grupo), "%NEG": neg, "neto": pos - neg,
                     "polarizacion": 2 * min(pos, neg),
                     "calif. promedio": grupo["calificacion"].mean()})

por_topico = textos.groupby("topico").apply(indicadores, include_groups=False)
por_topico["n"] = por_topico["n"].astype(int)
por_topico.sort_values("neto").round(2)
```

Loading weights: 100%  201/201 [00:00<00:00, 9917.13it/s]

Map: 100%  245/245 [00:00<00:00, 20853.21 examples/s]

	n	%NEG	neto	polarizacion	calif. promedio
topico					
4 · stock, producto, avisaron	14	0.64	-0.57	0.14	2.57
0 · soporte, correo, proceso	69	0.49	-0.36	0.26	2.61
3 · pantalla, funciona, app	24	0.29	-0.17	0.25	2.88
7 · sitio, pagina, general	10	0.40	-0.10	0.60	3.00
2 · caja, cable, pedido	30	0.27	0.00	0.53	3.33
8 · xiaomi, hervidor, tv	10	0.00	0.30	0.00	4.10
1 · buena, tienda, experiencia	50	0.02	0.70	0.04	4.16
5 · llego, ningun, acer	22	0.00	0.73	0.00	4.36
6 · galaxy, s24, z50	16	0.00	0.88	0.00	4.62

Etapas D — Por ubicación

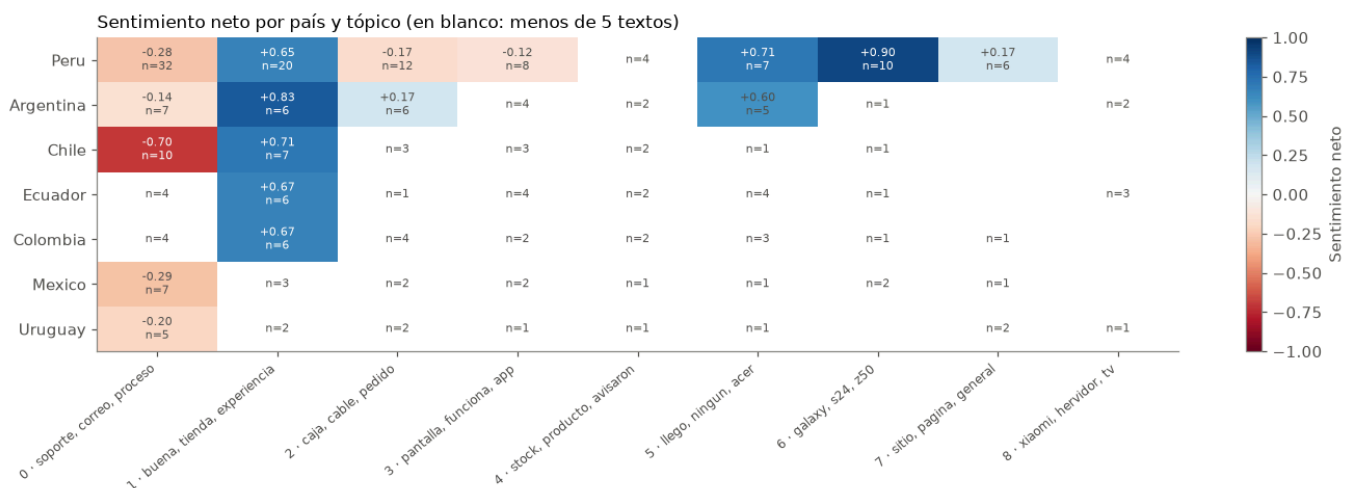
D.1 Mapa de calor país × tópico

Cada celda es el sentimiento neto del tópico en ese país. Las celdas con menos de `N_MIN` textos quedan en blanco: un solo texto más cambiaría la cifra en 20 puntos o más.

```
celdas = textos.groupby(["pais", "topico"]).apply(indicadores, include_groups=False)
neto = celdas["neto"].unstack()
n = celdas["n"].unstack().fillna(0).astype(int)
neto_visible = neto.where(n >= N_MIN)
```

```
orden_paises = textos["pais"].value_counts().index
neto_visible, n = neto_visible.loc[orden_paises], n.loc[orden_paises]

fig, ax = plt.subplots(figsize=(min(1.1 * neto.shape[1] + 3, 16), 4.5))
im = ax.imshow(neto_visible, cmap="RdBu", vmin=-1, vmax=1, aspect="auto")
for i in range(n.shape[0]):
    for j in range(n.shape[1]):
        v, k = neto_visible.iat[i, j], n.iat[i, j]
        if k:
            ax.text(j, i, f"{v:+.2f}\nn={k}" if pd.notna(v) else f"n={k}",
                    ha="center", va="center", fontsize=7,
                    color="white" if pd.notna(v) and abs(v) > 0.6 else TINTA)
ax.set_xticks(range(n.shape[1]), n.columns, rotation=40, ha="right", fontsize=8)
ax.set_yticks(range(n.shape[0]), n.index)
ax.grid(False)
fig.colorbar(im, ax=ax, label="Sentimiento neto")
ax.set_title(f"Sentimiento neto por país y tópico (en blanco: menos de {N_MIN} textos)",
             loc="left", fontsize=11)
plt.tight_layout()
plt.savefig(SALIDAS / "neto_pais_topico.png")
plt.show()
```



D.2 Resumen por país: ¿cuál es el tema más negativo de cada uno?

```
def peor_topico(grupo):
    t = grupo.groupby("topico").apply(indicadores, include_groups=False)
    t = t[t["n"] >= 3]
    if t.empty:
        return pd.Series({"tópico más negativo": "-", "neto del tópico": np.nan, "n del tópico": 0})
    k = t["neto"].idxmin()
    return pd.Series({"tópico más negativo": k, "neto del tópico": t.loc[k, "neto"],
                     "n del tópico": int(t.loc[k, "n"])})

resumen_pais = textos.groupby("pais").apply(indicadores, include_groups=False)
resumen_pais = resumen_pais.join(textos.groupby("pais").apply(peor_topico,
include_groups=False))
resumen_pais["n"] = resumen_pais["n"].astype(int)
resumen_pais = resumen_pais.sort_values("neto")
```

```
resumen_pais.round(2).to_csv(SALIDAS / "resumen_por_pais.csv")
resumen_pais.round(2)
```

	n	%NEG	neto	polarizacion	calif. promedio	tópico más negativo	neto del tópico	n del tópico
pais								
Uruguay	15	0.40	-0.13	0.53	2.93	0 · soporte, correo, proceso	-0.20	5
Chile	27	0.30	0.04	0.59	3.15	0 · soporte, correo, proceso	-0.70	10
Ecuador	25	0.32	0.08	0.64	3.16	0 · soporte, correo, proceso	-0.75	4
Mexico	19	0.26	0.11	0.53	3.74	0 · soporte, correo, proceso	-0.29	7
Peru	103	0.26	0.16	0.52	3.51	0 · soporte, correo, proceso	-0.28	32
Argentina	33	0.15	0.18	0.30	3.52	3 · pantalla, funciona, app	-0.50	4
Colombia	23	0.17	0.22	0.35	3.35	0 · soporte, correo, proceso	-0.50	4

D.3 Ciudades con suficientes textos

Perú concentra la mitad de los clientes; bajamos a nivel de ciudad solo donde hay al menos $2 \times N_MIN$ textos.

```
por_ciudad = textos.groupby(["pais", "ciudad"]).apply(indicadores,
include_groups=False)
por_ciudad = por_ciudad[por_ciudad["n"] >= 2 * N_MIN]
por_ciudad["n"] = por_ciudad["n"].astype(int)
por_ciudad.sort_values("neto").round(2)
```

		n	%NEG	neto	polarizacion	calif. promedio
pais	ciudad					
Chile	Santiago	13	0.46	-0.31	0.31	2.85
Argentina	Mendoza	10	0.30	0.00	0.60	3.50
Mexico	Ciudad de Mexico	10	0.30	0.00	0.60	3.30
Peru	Trujillo	11	0.27	0.09	0.55	3.55
	Chiclayo	10	0.20	0.10	0.40	3.50
	Lima	27	0.19	0.30	0.37	3.81
	Cusco	12	0.25	0.33	0.50	3.42

Etapas E — Por categoría de producto

Solo las **reseñas** tienen producto. ¿Qué categorías concentran las quejas y de qué tema son?

```
resenas = textos[textos["fuente"] == "reseña"]
por_categoria = resenas.groupby("categoria").apply(indicadores, include_groups=False)
por_categoria["n"] = por_categoria["n"].astype(int)
por_categoria["tópico más frecuente"] = resenas.groupby("categoria")["topico"].agg(
    lambda s: s.value_counts().index[0])
por_categoria = por_categoria.sort_values("neto")
por_categoria.round(2).to_csv(SALIDAS / "resumen_por_categoria.csv")
por_categoria.round(2)
```

	n	%NEG	neto	polarizacion	calif. promedio	tópico más frecuente
categoria						
Electrodomesticos Inteligentes	12	0.17	-0.17	0.00	3.33	8 · xiaomi, hervidor, tv
Smartphones	13	0.31	0.08	0.62	3.00	3 · pantalla, funciona, app
Wearables y Fitness	12	0.17	0.08	0.33	3.50	3 · pantalla, funciona, app
Audio	13	0.38	0.15	0.77	4.00	2 · caja, cable, pedido
Accesorios y Componentes	12	0.08	0.17	0.17	3.50	2 · caja, cable, pedido
Televisores y Video	13	0.23	0.31	0.46	3.77	0 · soporte, correo, proceso
Gaming y Consolas	12	0.08	0.33	0.17	4.00	5 · llego, ningun, acer
Fotografia y Video	11	0.09	0.36	0.18	3.45	0 · soporte, correo, proceso
Laptops y Computadoras	12	0.17	0.42	0.33	3.67	5 · llego, ningun, acer

```
quejas = resenas[resenas["sent"] == "NEG"]
pd.crosstab(quejas["categoria"], quejas["topico"], margins=True,
margins_name="Total")
```

tema	0 · soporte, correo, proceso	2 · caja, cable, pedido	3 · pantalla, funciona, app	Total
tema				
Accesorios y Componentes	0	1	0	1
Audio	2	3	0	5
Electrodomesticos Inteligentes	1	0	1	2
Fotografia y Video	1	0	0	1
Gaming y Consolas	1	0	0	1
Laptops y Computadoras	1	0	1	2
Smartphones	0	2	2	4
Televisores y Video	1	1	1	3
Wearables y Fitness	0	1	1	2
Total	7	8	6	21

Etapas F — Por cliente

F.1 Perfil de cada cliente

Por código seudonimizado: cuántos textos escribió, de cuántos tópicos distintos, su sentimiento neto, su tópico principal, en qué tópicos se quejó y el **valor comprado**. El valor solo se conoce por las reseñas (cada una corresponde a un producto comprado): es una **aproximación** que subestima a los clientes que compran sin reseñar.

```
def perfil(grupo):
    neg = grupo[grupo["sent"] == "NEG"]
    return pd.Series({
        "pais": grupo["pais"].iloc[0],
        "textos": len(grupo),
        "temas distintos": grupo["tema"].nunique(),
        "neto": (grupo["sent"] == "POS").mean() - (grupo["sent"] == "NEG").mean(),
        "tema principal": grupo["tema"].value_counts().index[0],
        "quejas": len(neg),
        "temas con queja": " | ".join(sorted(neg["tema"].unique())),
        "valor comprado": grupo["precio"].sum(),
    })
```

```
perfiles = textos.groupby("codigo").apply(perfil, include_groups=False)
perfiles.to_csv(SALIDAS / "perfil_clientes_temas.csv")
print(f"{len(perfiles)} clientes con al menos un texto de {len(clientes)}")
perfiles.sort_values("textos", ascending=False).head(8).round(2)
```

105 clientes con al menos un texto de 120

codigo	pais	textos	temas distintos	neto	tema principal	quejas	temas con queja	valor comprado
C-10d2959b	Peru	11	6	0.73	6 · galaxy, s24, z50	0		48790.0
C-82db662b	Peru	6	4	0.17	0 · soporte, correo, proceso	2	0 · soporte, correo, proceso 7 · sitio, pagina, general	5256.0
C-b872a31e	Argentina	6	6	0.17	0 · soporte, correo, proceso	1	3 · pantalla, funciona, app	15495.0
C-8e080668	Peru	6	5	0.50	1 · buena, tienda, experiencia	1	0 · soporte, correo, proceso	19196.0
C-0796c15c	Ecuador	5	3	0.60	1 · buena, tienda, experiencia	0		2897.0
C-0aa896eb	Colombia	5	4	0.00	2 · caja, cable, pedido	1	0 · soporte, correo, proceso	3657.0
C-fc81f112	Peru	5	4	0.60	1 · buena, tienda, experiencia	0		10997.0
C-2bafd967	Mexico	4	3	0.00	0 · soporte, correo, proceso	1	0 · soporte, correo, proceso	1348.0

F.2 Clientes con quejas en varios tópicos

Un cliente que se queja de **dos o más temas distintos** tiene una mala experiencia general, no un incidente aislado.

```
multi_queja = perfiles[perfiles["temas con queja"].str.count(r"\|") >= 1]
print(f"{len(multi_queja)} clientes con quejas en 2 o más tópicos")
multi_queja.sort_values("quejas", ascending=False)[
    ["pais", "textos", "quejas", "temas con queja", "neto"]].round(2)
```

11 clientes con quejas en 2 o más tópicos

	pais	textos	quejas	temas	temas con queja	neto
	codigo					
C-12a37385	Ecuador	2	2	3 · pantalla, funciona, app 4 · stock, producto, avisaron		-1.00
C-26aebf92	Ecuador	2	2	0 · soporte, correo, proceso 2 · caja, cable, pedido		-1.00
C-5fc403c9	Mexico	2	2	0 · soporte, correo, proceso 4 · stock, producto, avisaron		-1.00
C-82db662b	Peru	6	2	0 · soporte, correo, proceso 7 · sitio, pagina, general		0.17
C-89867460	Argentina	2	2	0 · soporte, correo, proceso 4 · stock, producto, avisaron		-1.00
C-9ed9a09e	Chile	2	2	0 · soporte, correo, proceso 2 · caja, cable, pedido		-1.00
C-a0b81c35	Peru	3	2	0 · soporte, correo, proceso 3 · pantalla, funciona, app		-0.67
C-d273ca98	Uruguay	2	2	4 · stock, producto, avisaron 7 · sitio, pagina, general		-1.00
C-dc41ad07	Ecuador	3	2	3 · pantalla, funciona, app 4 · stock, producto, avisaron		-0.33
C-fdc673ae	Peru	3	2	0 · soporte, correo, proceso 3 · pantalla, funciona, app		-0.33
C-ff4dc28e	Peru	3	2	0 · soporte, correo, proceso 4 · stock, producto, avisaron		-0.67

F.3 Clientes valiosos insatisfechos, por tópico

Valiosos = valor comprado en el **25 % superior** (entre quienes tienen reseñas); insatisfechos = neto < 0. ¿Por qué temas los estamos perdiendo?

```
con_compras = perfiles[perfiles["valor comprado"] > 0]
corte_valor = con_compras["valor comprado"].quantile(0.75)
en_riesgo = con_compras[(con_compras["valor comprado"] >= corte_valor) &
(con_compras["neto"] < 0)]

print(f"Corte de valor (percentil 75): {corte_valor:,.0f}")
print(f"{len(en_riesgo)} clientes valiosos insatisfechos; "
      f"valor comprado en juego: {en_riesgo['valor comprado'].sum():,.0f}")
en_riesgo.sort_values("valor comprado", ascending=False)[
    ["pais", "valor comprado", "quejas", "temas con queja"]].round(2)
```

Corte de valor (percentil 75): 3,548
4 clientes valiosos insatisfechos; valor comprado en juego: 21,394

codigo	pais	valor comprado	quejas	temas_riesgo	temas_valiosos con queja
C-7463e8ba	Colombia	6999.0	1	0 · soporte, correo, proceso	
C-a0b81c35	Peru	5999.0	2	0 · soporte, correo, proceso 3 · pantalla, funciona, app	
C-9ed9a09e	Chile	4798.0	2	0 · soporte, correo, proceso 2 · caja, cable, pedido	
C-92ba1db0	Mexico	3598.0	1		2 · caja, cable, pedido

```
temas_riesgo = (en_riesgo["temas_riesgo"].str.split(r" \| ").explode()
                 .replace("", np.nan).dropna().value_counts().rename("clientes
valiosos con queja"))
temas_riesgo.to_frame()
```

clientes valiosos con queja

temas_valiosos con queja	clientes valiosos con queja
0 · soporte, correo, proceso	3
2 · caja, cable, pedido	2
3 · pantalla, funciona, app	1

F.4 Clientes silenciosos

Los clientes sin ningún texto no son «neutros»: su opinión es **desconocida**.

```
silenciosos = clientes[~clientes["codigo"].isin(perfiles.index)]
print(f"{len(silenciosos)} de {len(clientes)} clientes no escribieron ningún texto")
silenciosos.groupby("pais").size().rename("silenciosos").to_frame().T
```

15 de 120 clientes no escribieron ningún texto

pais	Argentina	Chile	Colombia	Ecuador	Mexico	Peru	Uruguay
silenciosos	1	2	3	4	1	2	2

Etapas G — En el tiempo

Sentimiento neto por trimestre para los cuatro tópicos con más textos. Los puntos con menos de **N_MIN** textos no se dibujan.

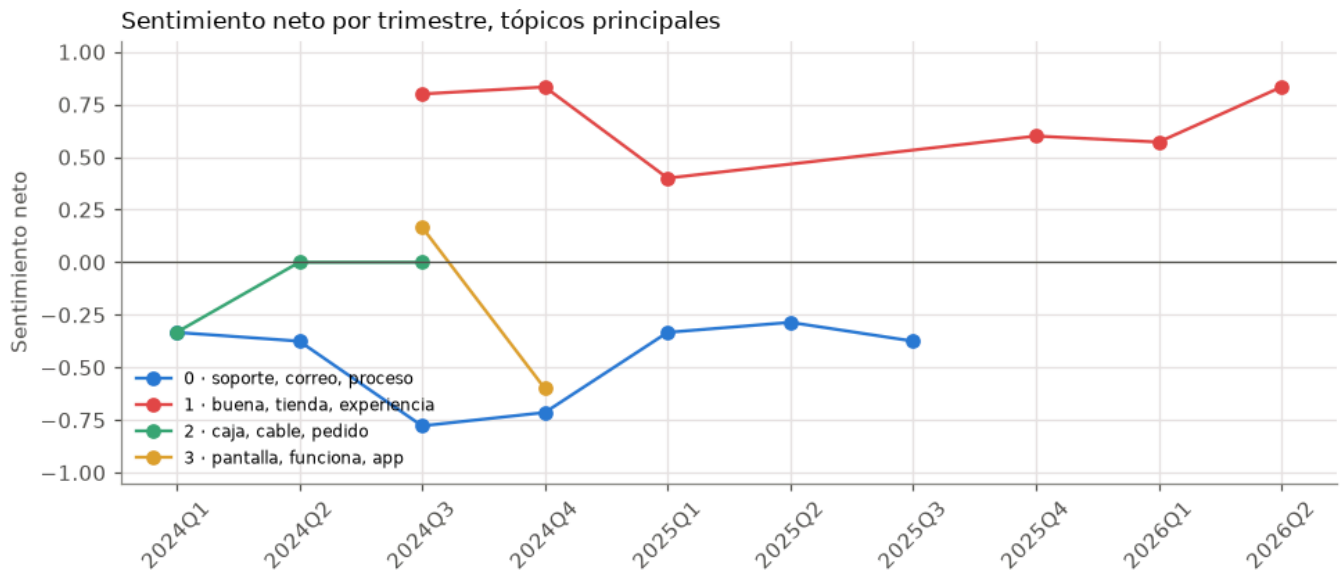
```
textos["trimestre"] = textos["fecha"].dt.to_period("Q")
principales = textos["tema"].value_counts().index[:4]

fig, ax = plt.subplots(figsize=(9, 4))
paleta = [AZUL, ROJO, "#3aa776", "#e0a030"]
for color, t in zip(paleta, principales):
    serie = (textos[textos["tema"] == t].groupby("trimestre"))
```

```

        .apply(indicadores, include_groups=False))
    serie = serie[serie["n"] >= N_MIN]
    ax.plot(serie.index.astype(str), serie["neto"], "o-", color=color, label=t)
    ax.axhline(0, color=TINTA, linewidth=0.8)
    ax.set_ylim(-1.05, 1.05)
    ax.set_ylabel("Sentimiento neto")
    ax.set_title("Sentimiento neto por trimestre, tópicos principales", loc="left",
    fontsize=11)
    ax.legend(frameon=False, fontsize=8, loc="lower left")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.savefig(SALIDAS / "neto_trimestral_topicos.png")
    plt.show()

```



Hallazgos

Respondan con las tablas y gráficos de este notebook:

1. ¿Qué país tiene el sentimiento más bajo y **de qué tema** se queja? ¿Hay suficientes textos para afirmarlo?
2. ¿Qué categoría de producto genera más quejas de entrega? ¿Qué acción recomendarían?
3. ¿Cuántos clientes valiosos están insatisfechos y cuál es el tema que más se repite entre ellos? ¿Por qué el «valor comprado» subestima a algunos clientes?
4. ¿Qué tópicos son **transversales** (aparecen en comentarios y en reseñas)?
5. Limitaciones: tamaño de muestra, clientes silenciosos, errores del modelo de sentimiento y tópicos que cambian con la semilla.

Respuestas del grupo

- 1.
- 2.
- 3.
- 4.
- 5.